MODULE 00 · EXAM STRATEGY - ALL DOMAINS

How this course works

What the exam is, what changed in May 2026, and how to get the most out of the next fourteen hours.

DOMAIN All domains COURSE RUNTIME 14h 15m MODULE 00 of 36

admin@sthithapragna.com

Almost nobody fails this exam because they have never heard of a feature. They fail because three options all look correct, and only one respects **a constraint in the second sentence**.

The exam at a glance

From the official exam guide, version dated 4 May 2026

- **45 scored questions, 90 minutes.** Unscored trial questions are also included, unmarked, with extra time allowed for them.
- **Multiple choice, four options.** One answer per question.
- **USD 200** plus local tax. Online proctored or at a test centre. No test aids of any kind.
- **No prerequisite.** Databricks recommends course attendance and six months of hands-on use.
- **The pass mark is not published.** Ignore any site that quotes one as fact.

> ON THE EXAM
>
> Valid for two years. To recertify you take whatever the live exam is at that time, not the one you sat.

The seven domains

DOMAIN	WEIGHT	MODULES
Databricks Intelligence Platform	6%	01-02
Data Ingestion and Loading	21%	03-08
Data Transformation and Modeling	22%	09-15
Working with Lakeflow Jobs	16%	16-19
Implementing CI/CD	10%	20-22
Troubleshooting, Monitoring and Optimisation	10%	23-26
Governance and Security	15%	27-30

ON THE EXAM

Module minutes track these weights, bar optimisation, which runs long on purpose.

Where the fourteen hours go

PART OF THE COURSE	EXAM WEIGHT	RUNTIME
Domain 3 - Transform	22%	160 min
Domain 2 - Ingest	21%	147 min
Domain 4 - Jobs	16%	86 min
Domain 7 - Govern	15%	80 min
Domain 6 - Troubleshoot	10%	80 min
Domain 5 - CI/CD	10%	61 min
Domain 1 - Platform	6%	51 min
Revision and 3 practice exams	-	176 min

ON THE EXAM

Teaching time tracks the exam's own weighting. Domains 2 and 3 are 43% of the paper and get the most minutes.

A new version of this exam went live on **4 May 2026**. Most material in circulation, including almost every question bank, describes the version before it.

Four renames you will meet under two different names

WHAT IT WAS CALLED	WHAT IT IS CALLED NOW	WHEN
Databricks Asset Bundles	Declarative Automation Bundles	16 Mar 2026
Databricks Repos	Databricks Git Folders	2025
Delta Live Tables, or DLT	Lakeflow Spark Declarative Pipelines	2025-2026
Databricks Workflows	Lakeflow Jobs	2025

ON THE EXAM

All four renames are cosmetic. No code changes, no migration, and the DAB acronym survived the bundles rename intact.

What the old material teaches, and what the exam now asks

OVER-REPRESENTED IN OLDER BANKS

Still useful, no longer weighted

- Pipeline expectation syntax and ON VIOLATION
- The LIVE keyword and CREATE STREAMING LIVE TABLE
- Delta Sharing, shares and recipients
- Databricks Connect
- Dashboards and alerts as their own topic

BARELY PRESENT IN OLDER BANKS

A fifth of the current exam

- CI/CD, bundles, Git folders and the CLI
- Cluster startup failures, library conflicts, memory
- Unity Catalog ABAC and governed tags
- The DENY privilege
- Lakeflow Connect managed and standard connectors

WHY THIS MATTERS

Roughly one question in five now comes from the right-hand column. Older question banks will not prepare you for any of them.

How this course is built

- **Every scenario is original.** Nothing is copied from a question bank. Banks are recycled, often leaked, and memorising them trains recognition rather than reasoning.
- **Every explanation traces to Databricks documentation**, checked against the live docs rather than recalled.
- **Wrong answers get explained too.** Understanding why a distractor fails transfers to questions you have never seen.
- **Dates on anything that changed.** Where the exam and current reality differ, you get both.

> ON THE EXAM
>
> A question bank was used here, but only to measure which decisions get tested and how often. None of its questions appear.

The shape of a teaching module

01

The idea

One statement the whole module turns on.

02

The mechanics

How the feature works, and its real limits.

03

The tiebreaker

The rule that separates two options that both look right.

04

Scenarios

Four to eight worked decisions, option by option.

ON THE EXAM

The tiebreaker slides are the highest-value minutes in the course. They are what the exam is actually testing.

How to use the scenarios

This convention is the difference between watching and learning

- **Every scenario is two slides.** The situation and question, then the full walkthrough.
- **Pause on the first one and commit to an answer.** Say it aloud or write it down.
- **Watching a walkthrough without answering first teaches almost nothing.** Every explanation sounds obvious once you have heard it.
- **When you are wrong, find the sentence you missed** rather than memorising the answer.

> ON THE EXAM
>
> Being wrong on a scenario and understanding why is worth more than being right on three.

The last six modules

- **31 Confusion pairs.** Every pair of features that sound alike, side by side. Almost no new content, which is the point.
- **32 The naming migration.** Every rename with its date, so old study material stops being confusing.
- **33 How the exam is written.** Question anatomy, qualifier words, how to spend ninety minutes.
- **34, 35, 36 Practice exams.** Three papers of 25 questions, mixed across domains as the real exam mixes them.

> ON THE EXAM
>
> 75 practice questions against a real paper of 45, weighted to the official domain percentages across the whole set. About two hours of the course.

Who this is for, and what to bring

- **Some SQL.** Joins, grouping, subqueries. You do not need to be fluent.
- **Enough Python to read it.** The exam shows PySpark; it does not ask you to write it from nothing.
- **Git as vocabulary.** Branch, commit, push, pull request. Not deep Git.
- **Not required:** machine learning, cluster administration, or prior Databricks certification.

> ON THE EXAM
>
> Databricks recommends six months of hands-on use. If you have none, expect to spend time in a free trial workspace alongside this course.

A study route that works

01

Watch in order

Modules build vocabulary. Later ones assume earlier ones.

02

Take exam 1

Under time. Note which domain the misses fall in.

03

Rewatch one domain

Only the weak one. Not the whole course.

04

Exams 2 and 3

Spaced a few days apart, not back to back.

05

Module 31 last

Confusion pairs the day before you sit.

ON THE EXAM

Which domain your misses cluster in tells you far more than the score does.

Three ways revision goes wrong

- **Rewatching what you already know.** It feels productive because the scores stay high, and it moves nothing.
- **Grinding a question bank.** You end up learning that bank. The exam will not show it to you.
- **Trusting a single source on anything that has a date.** Weights, names and features have all moved recently.
- **The fix for all three is the same.** Practise retrieving under time, then chase the misses.

> ON THE EXAM
>
> If a study session felt comfortable, it probably did not teach you anything.

Nothing on this exam is hard. Choosing between four things that all work, under time pressure, **is** hard. That is what the next fourteen hours are for.

MODULE 01 · DOMAIN 1 - DATABRICKS INTELLIGENCE PLATFORM

The lakehouse and the platform

Control plane and compute plane, Delta Lake and the transaction log, and the Unity Catalog object model.

DOMAIN 1 of 7 EXAM WEIGHT 6% MODULE 01 of 36

admin@sthithapragna.com

A lakehouse is not a data lake with better file formats. It is a data lake that has been given **a transaction log**.

Why the two older architectures both fell short

DATA WAREHOUSE

Reliable but closed

- Transactions, schema enforcement, fast SQL
- Proprietary storage format
- Expensive at scale
- Poor fit for unstructured data and ML

DATA LAKE

Open but unreliable

- Cheap object storage, any file type
- No transactions, so partial writes happen
- No schema enforcement, so quality drifts
- Reader sees half a job's output

THE PROBLEM THE LAKEHOUSE SOLVES

Most organisations ran both and copied data between them. Two systems, two copies, and a nightly window where the two disagreed.

What the lakehouse adds

- **Open storage.** Parquet files in your own cloud object storage. Any engine can read them.
- **ACID transactions.** A write either commits or it does not. Readers never see a half-finished job.
- **Schema enforcement and evolution.** Bad data is rejected at write time rather than discovered later.
- **One copy, both workloads.** BI and machine learning read the same tables. No nightly sync.

> ON THE EXAM
>
> Every one of these is delivered by Delta Lake, not by the platform around it. That distinction gets tested.

Two planes, and what lives in each

CONTROL PLANE	COMPUTE PLANE
The web application and workspace UI	Clusters and SQL warehouses
Notebook source and query definitions	Your actual table data
Job and pipeline configuration	Driver and executor processes
Cluster management and the REST API	The work of running your query
Managed by Databricks	Where processing happens

ON THE EXAM

The control plane holds the backend services Databricks manages. The compute plane is where your data is processed.

Classic compute plane or serverless compute plane?

CLASSIC COMPUTE PLANE

In your cloud account

- Resources run in your own AWS, Azure or GCP account
- In your own network, which you can configure
- Slower to start; you wait for VMs
- Choose when you need network control or compliance isolation

SERVERLESS COMPUTE PLANE

In the Databricks account

- Resources run in a Databricks-managed compute layer
- Always in the same cloud region as your workspace
- Starts in seconds; no VMs to provision
- Choose for speed and for not managing infrastructure

THE TIEBREAKER

Both are compute planes. Serverless does not move work into the control plane; it moves it into a different compute plane.

Two consequences worth remembering

- **Classic compute has no open inbound ports and no public IP addresses.** Clusters reach the control plane through an outbound HTTPS tunnel.
- **This is called secure cluster connectivity**, and it is the default on all new workspaces.
- **Serverless runs in the same region as your workspace**, so processing does not cross a regional boundary.
- **Notebook code lives in the control plane.** That surprises people who assume everything sits in their own account.

> ON THE EXAM
>
> Data residency questions turn on the compute plane. Code and configuration questions turn on the control plane.

A Delta table is ordinary Parquet files, plus a folder called _delta_log that says which of those files currently count.

What happens when you write

01

Write files

New Parquet files are written to storage. Nothing yet reads them.

02

Commit

A numbered JSON file is added to _delta_log listing what changed.

03

Visible

That commit is the moment the write becomes visible to readers.

04

Checkpoint

Every 10 commits by default, a Parquet snapshot of the log is written.

WHY ATOMICITY WORKS

Files exist before the commit but do not count. That is why a failed job leaves no partial data visible.

What the log gives you

- **Atomic writes.** Readers see the state before your job or the state after it, never during.
- **Time travel.** Every version is still described in the log, so you can query the table as it was.
- **Audit history. DESCRIBE HISTORY** shows every operation, who ran it and when.
- **Data skipping.** The log stores per-file min and max statistics, so irrelevant files are never opened.

> ON THE EXAM
>
> Time travel is bounded by file retention. VACUUM deletes old files and travel past that point stops working.

Delta table or plain Parquet files?

PLAIN PARQUET

A pile of files

- No transactions; partial writes are visible
- No history and no rollback
- Row-level updates mean rewriting whole files by hand
- Schema is whatever each file happens to contain

DELTA TABLE

Files plus a log

- ACID commits, so writes are all or nothing
- Time travel, DESCRIBE HISTORY and RESTORE
- MERGE, UPDATE and DELETE on rows
- Schema enforced on write, and evolvable on purpose

THE TIEBREAKER

Delta is the default table format on Databricks. If a question describes reliability, history or row-level changes, the answer involves Delta.

The Unity Catalog namespace

01

Metastore

Top-level container. One per cloud region.

02

Catalog

First level of the name. Often an environment or a business unit.

03

Schema

Second level. A project, a use case, or a team's area.

04

Table or volume

Third level. Also views, functions and models.

SAY IT OUT LOUD

Three-level namespace: catalog.schema.table. The metastore is above it and is never part of the name you write.

What sits where

LEVEL	WHAT IT CONTAINS
Metastore	Catalogs, plus storage credentials, external locations, connections, shares, recipients and providers
Catalog	Schemas, and nothing else directly
Schema	Tables, views, volumes, functions and models
Table or volume	Managed by Unity Catalog, or external to it

ON THE EXAM

Storage credentials and external locations hang off the metastore, not off a catalog. That placement is examinable.

Managed or external

Introduced here; the full treatment is module 27

- **Managed.** Unity Catalog controls both governance and the underlying files, including their lifecycle.
- **Drop a managed table and the data files are deleted too.**
- **External.** Unity Catalog governs access; you keep control of the files and where they live.
- **Drop an external table and only the registration goes.** The files remain untouched.

> ON THE EXAM
>
> Both tables and volumes come in managed and external forms. Databricks recommends managed unless you have a reason not to.

DBFS is deprecated

Both the root and mounts

- **DBFS root and DBFS mounts are deprecated.** New accounts are provisioned without access to either.
- **The DBFS root is readable by every user in the workspace.** There is no way to restrict it.
- **Mounts bypass Unity Catalog entirely.** A different access model with different credentials.
- **Use instead:** Unity Catalog volumes, external locations, or workspace files.

> ON THE EXAM
>
> The name survives in one place. dbfs:/ is still an optional scheme for addressing a Unity Catalog volume.

hive_metastore or Unity Catalog?

HIVE_METASTORE

The legacy catalog

- Scoped to one workspace only
- Managed tables land on the DBFS root
- Legacy table access control lists
- Appears as a catalog for backward compatibility

UNITY CATALOG

The current model

- Shared across workspaces in a region
- Managed tables use secure managed storage
- GRANT, REVOKE, DENY and ABAC policies
- Lineage, audit and search across everything

THE TIEBREAKER

If a question mentions sharing data between workspaces, or governing it centrally, the legacy metastore cannot do it.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Where does the web application run	Control plane
Where is my data processed	Compute plane, classic or serverless
Partial writes, rollback, or history	Delta Lake and the transaction log
Query the table as it was last Tuesday	Time travel, if VACUUM has not run
Share governed data across workspaces	Unity Catalog, never hive_metastore
Files on a path, governed	A Unity Catalog volume, never a DBFS mount

ON THE EXAM

Six sentences. If you can map each to its answer without hesitating, this module has done its job.

> A bank's security review asks the data platform team to confirm that no customer data leaves the bank's own cloud account. The team uses classic compute for all production jobs, and stores every table in buckets the bank owns.
>
> The reviewer then asks a second question. She wants to know where the source code of the notebooks and the definitions of the scheduled jobs are stored.

How should the team answer the second question?

A. Notebook source and job definitions are stored in the workspace storage bucket in the bank's own cloud account

B. Notebook source and job definitions are stored in the Databricks control plane

C. Notebook source is stored on the cluster driver node and is discarded when the cluster terminates

D. Notebook source and job definitions are stored in the serverless compute plane

◆ PAUSE AND DECIDE

How should the team answer the second question?

A In the workspace storage bucket in the bank's account

Tempting, because the first answer genuinely was 'in our account'. But this is where the two questions diverge. Data sits in the bank's storage; code does not.

B In the Databricks control plane

Correct. Notebook source, query definitions and job configuration are all backend services that Databricks manages in the control plane.

C On the driver node, discarded on termination

Confuses execution with storage. Code is sent to the driver to run, but it is not stored there, or nothing would survive a restart.

D In the serverless compute plane

The team uses classic compute, so no serverless compute plane is even involved. Compute planes process data; they do not store definitions.

THE SIGNAL

The reviewer asked two questions that sound like one. Data residency and code residency have different answers.

> An engineer joins a company and finds that a nightly job writes a customer file containing names and account numbers to a path on the DBFS root. A second job reads it back the next morning.
>
> It has worked reliably for two years. The engineer raises it with her manager, who asks what is actually wrong with it, given that it has never failed.

What is the strongest reason to change this?

A. The DBFS root is slower than cloud object storage, so the job takes longer than it needs to

B. Every user in the workspace can read anything on the DBFS root, and this file contains account numbers

C. The DBFS root has a size limit that this file will eventually exceed

D. Files on the DBFS root are deleted automatically after ninety days

◆ PAUSE AND DECIDE

What is the strongest reason to change this?

A. It is slower than cloud object storage

The DBFS root is backed by cloud object storage, so there is no meaningful performance gap. And the manager's question is about risk, not speed.

B. Every workspace user can read it, and it holds account numbers

Correct. The DBFS root is readable by all workspace users with no way to restrict it. Two years without failure says nothing about exposure.

C. It will exceed a size limit

Invented. There is no such limit, and a fabricated constraint is a common distractor shape.

D. Files are deleted after ninety days

Also invented. Nothing on the DBFS root expires on a schedule.

THE SIGNAL

'It has never failed' is about reliability. The risk here is disclosure, which produces no failures at all until it does.

> A nightly job overwrote a 400 million row Delta table with the output of a transformation that had a bug in it. The table now contains roughly 12 million rows, all of them wrong. The error was noticed nine hours later.
>
> The table is written to daily and has never had VACUUM run against it. The team needs the previous day's data back and wants the fastest correct route.

What should the team do?

A. Re-run the upstream pipeline from raw source data to rebuild the table

B. Restore the storage bucket from the cloud provider's most recent snapshot

C. Use RESTORE to return the table to the version before the bad write

D. Recover the deleted Parquet files from the storage bucket and re-register them

◆ PAUSE AND DECIDE

What should the team do?

A Re-run the upstream pipeline from raw

It would work, and that is what makes it tempting. But it reprocesses 400 million rows to undo a change the log can reverse by pointing at an earlier version.

B Restore the bucket from a cloud snapshot

Blunt and risky. It rolls back everything in that bucket, including other tables, to whenever the snapshot was taken.

C RESTORE to the version before the bad write

Correct. The old files still exist because VACUUM has never run, so this is a metadata operation. No data is reprocessed or moved.

D Recover deleted Parquet files and re-register them

Nothing was deleted. An overwrite writes new files and marks the old ones as removed in the log; they remain on disk until VACUUM.

THE SIGNAL

'VACUUM has never run' is the sentence that decides this. It guarantees the previous version's files are still there.

A team receives a daily delivery of PDF contracts into cloud object storage. A Python library that does not use Spark reads them by file path and extracts text.

The company has adopted Unity Catalog and requires that access to every data asset is granted centrally and appears in audit logs. The PDFs must stay in the existing bucket, because a separate legal system also writes to it.

How should the PDFs be made available?

A Mount the bucket to DBFS so the library can read the files by path

B Create an external location for the bucket, then an external volume over it

C Create a managed volume and copy the PDFs into it each day

D Register the PDFs as an external Delta table so they can be governed

◆ PAUSE AND DECIDE

How should the PDFs be made available?

A Mount the bucket to DBFS

It would give the library path access, which is why it tempts. But mounts are deprecated and bypass Unity Catalog, so the central grant and audit requirement fails.

B External location, then an external volume

Correct. Volumes give governed path-based access for non-tabular files, and external keeps the PDFs in the existing bucket the legal system also writes to.

C Managed volume, copying files in daily

Governed and path-addressable, but it duplicates every PDF and adds a job that can fail. The requirement was to leave them where they are.

D External Delta table

A Delta table describes rows and columns. PDFs are not tabular, and a table gives no path for a non-Spark library to open.

THE SIGNAL

Two constraints, each eliminating a different option. 'Reads them by file path' rules out a table. 'Must stay in the existing bucket' rules out managed.

Delta Lake gives you **reliability**. Unity Catalog gives you **governance**. The planes tell you **where things run**. Every later module sits on one of those three.

MODULE 02 · DOMAIN 1 - DATABRICKS INTELLIGENCE PLATFORM

Compute, and what it costs

All-purpose against jobs compute, the three SQL warehouse types, access modes, and the levers that actually move the bill.

DOMAIN 1 of 7 EXAM WEIGHT 6% MODULE 02 of 36

admin@sthithapragna.com

Databricks sends you **two bills**. One from Databricks for DBUs, one from your cloud provider for the machines. The compute type you pick changes the first one by two to three times.

The compute types

TYPE	WHAT IT IS FOR	LIFECYCLE
All-purpose	Interactive notebooks and development	Runs until stopped or idle timeout
Jobs compute	Scheduled and automated workloads	Starts with the job, terminates when it ends
SQL warehouse	SQL queries, dashboards and BI tools	Auto-stops after an idle period
Serverless	Any of the above, managed by Databricks	Starts in seconds, no idle cost
Instance pool	Warm VMs that cut start-up time	Cloud cost while idle, no DBUs

ON THE EXAM

Jobs compute ends when the work ends. All-purpose ends when someone remembers.

All-purpose or jobs compute?

ALL-PURPOSE

Built for people

- Attach a notebook and run cells interactively
- Shared across a team, so jobs compete for resources
- Auto-termination is tied to inactivity, not to work finishing
- The more expensive DBU rate

JOBS COMPUTE

Built for schedules

- Created for the run, destroyed after it
- Isolated, so no competition from other users
- Terminates when the job completes, always
- Roughly two to three times cheaper per DBU

THE TIEBREAKER

Databricks explicitly recommends against running jobs on all-purpose compute. It is the most common avoidable cost on the platform.

Why Databricks recommends against jobs on all-purpose

Four reasons, straight from the documentation

- **Different billing rate.** All-purpose is billed at a higher DBU rate than jobs compute for identical work.
- **Termination is tied to inactivity, not completion.** The cluster idles on after your job has finished.
- **Shared across teams.** Scheduled jobs get increased latency from competing for resources.
- **Tuning advice does not transfer.** Settings good for ad-hoc interactive work are wrong for batch.
- **The one accepted exception:** iteratively developing or testing a new job, where jobs-compute start-up makes each cycle tedious.

> ON THE EXAM
>
> Development is a legitimate reason. 'The cluster was already running' is not.

Serverless or classic compute?

CLASSIC

You own the cluster

- Runs in your cloud account and network
- Minutes to start, unless you use a pool
- You choose node types, versions and libraries
- Idle time bills until termination fires

SERVERLESS

Databricks owns the cluster

- Runs in the Databricks account, same region
- Starts in seconds; nothing to provision
- Fewer knobs, and fewer ways to misconfigure
- No idle cost, because there is no idle

THE TIEBREAKER

Bursty short work favours serverless. Long steady runs, or a need for specific node types, favour classic.

The three SQL warehouse types

	CLASSIC	PRO	SERVERLESS
Photon	Yes	Yes	Yes
Predictive IO	No	Yes	Yes
Intelligent Workload Management	No	No	Yes
Compute runs in	Your account	Your account	Databricks account
Typical start-up	~4 minutes	~4 minutes	Seconds

ON THE EXAM

Serverless is the only type whose compute layer sits in the Databricks account, and the only one that starts in seconds.

Sizing and stopping a warehouse

- **Size is about one query's speed.** A bigger warehouse makes an individual query faster by giving it more workers.
- **Scaling is about many queries at once.** More clusters handle more concurrent users, not faster single queries.
- **Sizing up to fix a queue is the classic mistake.** Queuing is a concurrency problem, so scale out instead.
- **Auto-stop is the idle-cost lever.** Serverless defaults to stopping after ten minutes idle and can be set lower.

> ON THE EXAM
>
> A busy dashboard keeps a warehouse warm on its own. Aggressive auto-stop costs nothing when traffic is steady.

Access modes were renamed. **Single user** is now **dedicated**. **Shared** is now **standard**. Older study material uses the old names throughout.

The three access modes

MODE	FORMERLY	UNITY CATALOG	USE IT WHEN
Standard	Shared	Yes, fully enforced	Almost always. The recommended default.
Dedicated	Single user or assigned	Yes	Standard cannot do what you need
No isolation shared	Unchanged	No	Legacy hive_metastore work only

ON THE EXAM

Only standard and dedicated can access Unity Catalog data. If a question needs UC governance, no isolation shared is always wrong.

When dedicated is actually required

- **Databricks Runtime ML**, including GPU workloads. Not supported on standard access mode.
- **The RDD API**, rather than DataFrames or SQL.
- **R workloads.**
- **Dedicated can be assigned to a group**, not only a person, from Databricks Runtime 15.4 upwards.

> ON THE EXAM
>
> On a group-dedicated resource, a user's permissions down-scope to the group's. The group must own the privileges, not the individual.

Photon

- **A vectorised query engine written in C++**, replacing parts of the JVM execution path. Your code does not change.
- **Helps most on** large scans, aggregations, joins and writes to Delta and Parquet.
- **Helps least on** queries already finishing in a couple of seconds, and on non-Spark Python running in a UDF.
- **Costs a higher DBU rate**, and can still be cheaper overall, because billing is by time and the job finishes sooner.

> ON THE EXAM
>
> Photon is enabled by default on all SQL warehouse types. On clusters it is a checkbox.

What a cluster is made of

01

Driver

Holds the notebook state, plans work, collects results.

02

Executors

Worker nodes that do the actual processing.

03

Cores

Each executor runs tasks in parallel, one per core.

04

Tasks

A partition of data processed by one core.

WHY THIS ASYMMETRY MATTERS

One driver, many executors. Lose an executor and Spark re-runs its tasks. Lose the driver and the whole cluster dies.

Fixed size or autoscaling?

FIXED SIZE

Predictable

- Same number of workers throughout
- No time lost adding or removing nodes
- Predictable cost per run
- Good when the workload is steady and known

AUTOSCALING

Elastic

- Workers added when tasks queue, removed when idle
- You set a minimum and a maximum
- Good for variable or unpredictable load
- Scaling itself takes time, so it suits longer runs

THE TRAP

Autoscaling is not free. Adding nodes takes minutes, so on a short job the scale-up finishes about when the job does.

Three settings that cost or save real money

- **Auto-termination.** Shuts an idle all-purpose cluster down. Defaults are generous, commonly 120 minutes.
- **Spot or preemptible instances.** Spare cloud capacity at a large discount, reclaimable at any moment.
- **Instance pools.** Warm VMs that clusters borrow, cutting start-up from minutes to seconds.
- **Never put the driver on spot.** Losing an executor costs a retry. Losing the driver kills the job.

> ON THE EXAM
>
> Pools bill you for idle VMs at the cloud provider's rate but consume no DBUs while idle. They trade a small standing cost for fast starts.

Databricks Runtime

- **A versioned image** containing Spark, Delta Lake, Python, Java, Scala, R and preinstalled libraries.
- **LTS versions are long-term support.** Databricks recommends the latest LTS for production.
- **ML variants** add machine learning libraries and GPU support, and require dedicated access mode.
- **Features have version floors.** Group-dedicated compute needs 15.4 or above, for example.

> ON THE EXAM
>
> When a question says a feature 'is not available', check the runtime version before concluding the approach is wrong.

Cluster policies

- **Admin-defined rules** constraining what compute a user may create.
- **Can force an access mode**, so nobody creates compute that bypasses Unity Catalog.
- **Can cap node counts and types**, and require auto-termination below a limit.
- **Can enforce tags**, which is how cost gets attributed to a team or project.

> ON THE EXAM
>
> Policies answer questions about preventing a mistake, rather than detecting it afterwards.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
A scheduled pipeline on a shared interactive cluster	Move it to jobs compute
First dashboard query each morning is slow	Classic or pro warehouse cold start, ~4 minutes
Queries queue when many analysts arrive	Scale out to more clusters, not up a size
Needs Databricks Runtime ML, GPU, RDD or R	Dedicated access mode
Governed access to Unity Catalog data	Standard or dedicated, never no isolation shared
Cutting cost on a critical job	Spot for workers, on-demand for the driver

ON THE EXAM

Six lines. Each one is a whole family of exam questions compressed into a sentence.

A team runs five nightly ETL pipelines. All five are scheduled against a shared all-purpose cluster that the team also uses for development during the day. The cluster has auto-termination set to 120 minutes.

The pipelines take about 95 minutes of processing in total, spread across the night. Finance has asked the team to reduce the cost of this workload.

Which change reduces cost the most?

A. Reduce auto-termination on the all-purpose cluster from 120 minutes to 30 minutes

B. Enable autoscaling on the all-purpose cluster so it shrinks between pipeline runs

C. Schedule the five pipelines against jobs compute instead

D. Enable Photon on the all-purpose cluster so the pipelines finish sooner

◆ PAUSE AND DECIDE

Which change reduces cost the most?

A. Cut auto-termination to 30 minutes

Genuinely helps, and it is the trap. It shortens idle windows but leaves every minute of actual work billing at the all-purpose rate.

B. Enable autoscaling between runs

Shrinks the cluster during gaps, but the driver keeps running and the rate is unchanged. A partial fix to half the problem.

C. Move the pipelines to jobs compute

Correct. It fixes both halves at once: the DBU rate drops two to three times, and each cluster terminates when its run ends rather than idling.

D. Enable Photon

May shorten runtime, but Photon costs a higher DBU rate and the workload still sits on the wrong compute type. Optimising the wrong thing.

THE SIGNAL

Two costs are in play: the rate and the idle time. A, B and D each address one. Only C addresses both.

> Analysts report that the first dashboard load each morning takes around four minutes, after which everything is fast for the rest of the day. The dashboard runs on a classic SQL warehouse with auto-stop set to 30 minutes.
>
> A colleague suggests increasing the warehouse size from Small to Large to fix the slow morning load.

What is actually happening, and what should the team do?

A The warehouse is undersized. Increase it to Large as suggested

B The warehouse is cold-starting. Switch to a serverless SQL warehouse

C The Delta tables need OPTIMIZE, which is why the first query is slow

D The result cache is empty each morning. Increase auto-stop to 12 hours

◆ PAUSE AND DECIDE

What is actually happening, and what should the team do?

A Increase the size to Large

A bigger warehouse still takes about four minutes to start. You would pay more for every hour of the day and the morning wait would not move.

B Switch to serverless

Correct. Four minutes is the documented cold-start for classic and pro warehouses. Serverless starts in seconds, which removes the wait entirely.

C Run OPTIMIZE on the tables

File layout affects every query equally. It would not produce fast queries all afternoon and one slow query at nine.

D Keep it running for 12 hours

It would work, by paying for an idle warehouse all night. It also misreads the cause: this is start-up time, not cache warming.

THE SIGNAL

'Four minutes, then fast all day' is a signature. It is the shape of a cold start, not of a sizing or layout problem.

> A data science team needs to train models using Databricks Runtime ML on GPU instances. The workspace is governed by Unity Catalog, and the platform team requires that all data access is enforced through it.
>
> Four people need to use the same compute resource. Any tables they create must be owned by the team rather than by one individual.

How should the compute be configured?

A Standard access mode, which supports multiple users and Unity Catalog

B No isolation shared access mode, so all four can use one resource

C Dedicated access mode assigned to a group containing the four users

D Four separate dedicated resources, one assigned to each user

◆ PAUSE AND DECIDE

How should the compute be configured?

A Standard access mode

Normally the right default, and that is the trap. Standard does not support Databricks Runtime ML or GPU workloads, so it cannot run this at all.

B No isolation shared

Unity Catalog permissions do not apply on this mode, which breaks the platform team's stated requirement outright.

C Dedicated, assigned to a group

Correct. Dedicated supports Runtime ML and GPUs, group assignment lets all four share it, and objects created are owned by the group.

D Four individual dedicated resources

It would run the workload, but four GPU clusters instead of one, and each person would own their own tables. Both requirements missed.

THE SIGNAL

Two constraints pull in opposite directions: Runtime ML forces dedicated, sharing suggests standard. Group-dedicated is what resolves them.

> A regulatory report must be produced by 06:00 every day. The job takes about 90 minutes on a 12-worker cluster. To cut cost, an engineer configures the cluster to use spot instances for every node, including the driver.
>
> Over the following fortnight the job fails three times. Each failure happens partway through, and the run has to be restarted from the beginning.

What should the engineer change?

A. Remove spot instances entirely and use on-demand for all nodes

B. Keep spot for the workers but move the driver to an on-demand instance

C. Increase the cluster from 12 workers to 24 so the job finishes before reclamation

D. Add an instance pool so reclaimed nodes are replaced immediately

◆ PAUSE AND DECIDE

What should the engineer change?

A On-demand for everything

It fixes the failures and throws away all the saving. The question asks what to change, and this changes more than the problem requires.

B Spot workers, on-demand driver

Correct. Losing a spot worker costs a retry of its tasks. Losing the spot driver kills the run, which is exactly the failure described.

C Double the workers

Reclamation is not a race against the clock; it can happen at any moment. Twice the nodes is twice the exposure, not less.

D Add an instance pool

Pools speed up start-up. They do not stop the cloud provider reclaiming a spot instance, and they do not protect the driver.

THE SIGNAL

Spot is safe where work can be retried and unsafe where it cannot. That maps exactly onto executors and the driver.

A team has forty small jobs that each run for about two minutes, triggered irregularly through the day. They currently run on classic jobs compute, which takes roughly four minutes to start each time.

The team's bill shows far more compute time than the work should require, and users complain that short jobs feel slow.

What is the best change?

A Move the jobs to serverless compute

B Move the jobs onto a shared all-purpose cluster that stays running all day

C Increase the cluster size so each two-minute job finishes faster

D Combine all forty jobs into one job that runs four times a day

◆ PAUSE AND DECIDE

What is the best change?

A Move to serverless compute

Correct. Start-up drops from four minutes to seconds, so the overhead that dominates a two-minute job disappears, and there is no idle cost.

B A shared all-purpose cluster all day

It removes start-up and replaces it with a cluster billing all day at the higher all-purpose rate. One form of waste traded for a worse one.

C Increase the cluster size

The two minutes of work is not the problem; the four minutes of start-up is. A bigger cluster starts just as slowly and costs more.

D Combine into one job run four times daily

It cuts start-ups from forty to four, but the jobs are triggered irregularly. Batching destroys the timing the triggers exist to provide.

THE SIGNAL

When start-up time exceeds run time, the overhead is the workload. Fix the overhead, not the work.

Compute questions almost always reduce to two things: **are you paying the right rate**, and **is anything billing while doing nothing**.

MODULE 03 · DOMAIN 2 - DATA INGESTION AND LOADING

Ingestion patterns and where data lands

Batch, streaming and incremental loading, the medallion layers, file formats, and a map of the tools that follow.

DOMAIN 2 of 7 EXAM WEIGHT 21% MODULE 03 of 36

admin@sthithapragna.com

Almost every ingestion question reduces to one thing: **how does the system know what it has already processed?**

The three ingestion patterns

PATTERN	WHAT IT DOES	TYPICAL LATENCY
Batch	Reads a defined set of data, all at once	Hours or a day
Incremental	Reads only what is new since last time	Minutes to hours
Streaming	Processes records continuously as they arrive	Seconds

ON THE EXAM

The exam names all three. In practice most Databricks ingestion is incremental, and the tooling is built around it.

Where batch and streaming actually differ

BATCH

You define the data

- The job knows the full set of input before it starts
- Runs, finishes, exits
- Re-running reprocesses the same data again
- No memory between runs unless you build one

STREAMING

The data is unbounded

- New data may arrive at any time; there is no 'all of it'
- A checkpoint records progress so the next run resumes
- Re-running continues rather than repeats
- Can be triggered to stop once caught up

THE TIEBREAKER

The real difference is not speed. It is whether progress is remembered. Streaming keeps a checkpoint; batch does not.

What incremental actually requires

- **A record of what has been processed.** A checkpoint, or a list of loaded files, kept somewhere durable.
- **Idempotency.** Running the same load twice must not double the data.
- **A way to discover new input** without listing everything every time, once volumes get large.
- **Not a filter on a date column.** That is a manual workaround, and it breaks on late-arriving data.

> ON THE EXAM
>
> If an option achieves incremental behaviour by filtering on yesterday's date, look for the option that tracks state instead.

The shape of a Databricks pipeline

01

Source

Files, databases, SaaS applications, message buses.

02

Bronze

Raw, as it arrived, appended. Nothing thrown away.

03

Silver

Cleaned, typed, deduplicated, joined. Trustworthy.

04

Gold

Aggregated and shaped for a specific consumer.

LEARN THE LAYER NAMES

This is the medallion architecture. It is the vocabulary of domain three, and the exam uses the layer names without defining them.

What each layer is for

LAYER	CONTAINS	WRITTEN HOW
Bronze	Raw source data, plus ingestion metadata	Append only
Silver	Cleaned, typed, deduplicated, conformed	Merge or overwrite
Gold	Business-level aggregates and metrics	Recomputed or merged

ON THE EXAM

Bronze is append-only on purpose. It is your ability to rebuild everything downstream when a transformation turns out to be wrong.

Three rules that follow from the layers

- **Do not clean on the way into bronze.** Cleaning belongs in the bronze-to-silver step, where it can be re-run.
- **Silver is the layer most consumers should use.** It is trustworthy and still detailed.
- **Gold is allowed to be duplicated.** Several gold tables from one silver table is normal, not a design flaw.
- **Layers are a convention, not a feature.** There is no CREATE BRONZE TABLE. They are ordinary tables you have named.

> ON THE EXAM
>
> A question describing 'raw data with quality problems' is describing bronze, and the fix belongs downstream of it.

File formats you will meet

FORMAT	SHAPE	WHERE IT TURNS UP
CSV	Row-based text, no types	Exports from older systems, manual uploads
JSON	Text, nested, self-describing	APIs, event streams, application logs
Parquet	Columnar, compressed, typed	Analytics interchange, the basis of Delta
Avro	Row-based binary, schema attached	Message buses, Kafka payloads
ORC	Columnar, typed	Legacy Hadoop estates
Delta	Parquet plus a transaction log	Every table you create on Databricks

ON THE EXAM

Columnar formats let a query read only the columns it needs. Row-based formats make it read whole records.

Reading files

- **spark.read.format("csv").option(...).load(path)** is the general shape for a batch read.
- **Options carry the details:** header, delimiter, inferSchema, multiLine for JSON.
- **inferSchema costs a pass over the data** to work out types, which is slow on large inputs.
- **Supplying a schema is faster and safer.** It also stops types changing silently between runs.

> ON THE EXAM
>
> A read with no schema and no inference gives every column as STRING. That is the cause behind a whole family of questions.

Local files and small uploads

- **Unity Catalog volumes** are the governed home for file data. Grants apply, and access appears in audit logs.
- **The upload UI** can create a table directly from a small CSV or JSON file.
- **Workspace files** sit beside your notebooks and suit small reference files and code, not data at scale.
- **Not the DBFS root.** Deprecated, and readable by everyone in the workspace.

> ON THE EXAM
>
> The exam guide explicitly lists local files as an ingestion source. The governed answer is a volume.

Land in storage first, or write straight to a table?

LAND IN CLOUD STORAGE

A raw landing zone

- Original files are kept exactly as delivered
- Ingestion can be re-run after a bug is found
- Needed when the source pushes files to you
- Adds a step and a place for files to pile up

WRITE DIRECTLY TO A TABLE

Straight into Unity Catalog

- Fewer moving parts
- Governed and queryable immediately
- Nothing to re-read if the load logic was wrong
- Suits API and database pulls where the source can be re-queried

THE TIEBREAKER

The question is whether you can go back to the source. If you cannot, keep the files.

The ingestion tools, and which module covers each

TOOL	BEST FOR	MODULE
COPY INTO	Files in cloud storage, modest volumes, SQL-first teams	04
Auto Loader	Files in cloud storage at scale, schema drift, continuous arrival	05
Lakeflow Connect managed	SaaS and database sources, fully handled for you	06
Lakeflow Connect standard	Sources needing more control than managed allows	06
JDBC, ODBC and REST	Databases and APIs pulled from a notebook	07
Partner connectors	Sources Databricks does not cover natively	08

ON THE EXAM

Module 08 is entirely about choosing between these. It carries eight scenarios rather than four.

Three things that go wrong at ingestion

- **Duplicates.** A load re-runs, or a file is delivered twice, and there is nothing tracking what was seen.
- **Schema drift.** The source adds a column, and the pipeline either fails or silently drops it.
- **Small files.** Frequent tiny writes produce thousands of small files, and reads slow to a crawl.
- **Each has a named owner:** checkpoints for duplicates, schema evolution for drift, OPTIMIZE and liquid clustering for small files.

> ON THE EXAM
>
> Recognising which of the three a scenario describes usually names the answer before you have read the options.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Files keep arriving and must not be reprocessed	Incremental, with tracked state
Re-running the load creates duplicates	Nothing is tracking processed files
Keep the raw data exactly as delivered	Bronze, append only
A BI team needs one specific aggregate	Gold, built for that consumer
Governed access to uploaded files	A Unity Catalog volume
Every column arrived as a string	No schema supplied and no inference

ON THE EXAM

Six lines. Each maps a phrase you will actually see in a question to the concept behind it.

> A supplier drops CSV files into a cloud storage folder several times a day. The times are irregular. A nightly job runs spark.read on the whole folder and overwrites a Delta table with the result.
>
> The folder now holds four years of files. The job takes over three hours and is getting slower each month. The team wants the table to stay current without the job growing indefinitely.

What is the underlying problem?

A The job overwrites rather than appends, so it loses history each night

B The job reprocesses every file every night, because nothing records what was already loaded

C The cluster is undersized for four years of CSV data

D CSV is the wrong format and the files should be converted to Parquet

◆ PAUSE AND DECIDE

What is the underlying problem?

A Overwrite loses history

It does overwrite, but it rebuilds from all four years each night, so nothing is actually lost. A real observation that is not the cause.

B It reprocesses everything, because nothing records what was loaded

Correct. This is a batch job pretending to be incremental. The runtime grows with total history rather than with new data.

C The cluster is undersized

A bigger cluster makes the same wasteful work finish faster, and the runtime still grows every month. It treats the symptom.

D CSV should be Parquet

Parquet would read faster, but the job would still reprocess four years of it nightly. Same growth curve, shallower slope.

THE SIGNAL

'Getting slower each month' is the signature. Runtime scaling with accumulated history rather than with new data means no state is being tracked.

> A team ingests order events from an API into a bronze table. During ingestion they drop records with a null customer_id, cast the amount column to decimal, and remove three columns nobody uses.
>
> Six months later they discover the cast was wrong: amounts in one currency were being divided by 100. They need to correct the historical figures.

What is the design error?

A They should have used a streaming read rather than a batch read

B They transformed data on the way into bronze, so the original values no longer exist

C They should have partitioned the bronze table by currency

D They should have written to silver rather than bronze

◆ PAUSE AND DECIDE

What is the design error?

A. Should have used streaming

Batch against streaming is unrelated to this failure. Either one would have destroyed the original values just the same.

B. They transformed on the way into bronze

Correct. Bronze exists to hold the source as delivered. Cleaning during ingestion removed the only copy of the true amounts.

C. Should have partitioned by currency

Partitioning affects query performance. It does nothing to preserve values that were altered before they were ever written.

D. Should have written to silver

Backwards. The table is named bronze but behaves like silver. Renaming it would not bring back the discarded data.

THE SIGNAL

Three transformations happened at ingestion: a filter, a cast and a projection. Each one discarded something bronze was supposed to keep.

> An analyst uploads a 40-row CSV of regional sales targets that she maintains by hand. Three pipelines need to join against it, and the platform team requires that all data access is governed and auditable.
>
> She currently emails the file to an engineer, who puts a copy on the DBFS root each quarter.

What should the team do instead?

A Keep it on the DBFS root but restrict access with a cluster policy

B Have her upload it to a Unity Catalog volume, or create a table from it via the upload UI

C Ingest it with Auto Loader so it is tracked incrementally

D Store it as a workspace file next to the pipeline notebooks

◆ PAUSE AND DECIDE

What should the team do instead?

A DBFS root with a cluster policy

Cluster policies constrain compute configuration, not file access. The DBFS root stays readable by every workspace user.

B A Unity Catalog volume, or a table via the upload UI

Correct. Both are governed, auditable, and let the analyst maintain the file herself without an engineer in the loop.

C Auto Loader

Machinery for continuous arrival at scale, aimed at a 40-row file replaced quarterly. It solves a problem that does not exist here.

D A workspace file

Reasonable for small reference data, but it is not a Unity Catalog securable, so it does not satisfy the governance requirement.

THE SIGNAL

Two requirements: governed and auditable, and the analyst should not need an engineer. Only B satisfies both.

A team is migrating from an on-premises warehouse. They must load five years of history once, roughly 2 TB in Parquet files, and then keep a daily delta of about 3 GB flowing afterwards.

An engineer proposes building one pipeline that handles both, using the same incremental tool for the history and the daily load.

How should the team think about this?

A The proposal is wrong; historical backfill and ongoing ingestion always need separate tools

B The proposal is reasonable; an incremental tool processes the backlog first, then continues with daily files

C The proposal is wrong; 2 TB is too large to load into a Delta table

D The proposal is reasonable, but only if the history is loaded as a stream with a seconds-level trigger

◆ PAUSE AND DECIDE

How should the team think about this?

A Backfill and ongoing always need separate tools

The word 'always' is the giveaway. An incremental tool pointed at a directory treats existing files as its first batch, then continues from the checkpoint.

B Reasonable; it processes the backlog then continues

Correct. The initial run consumes what is already there, records progress, and subsequent runs pick up only new files.

C 2 TB is too large for a Delta table

Invented limit. Delta tables routinely hold far more than this.

D Only as a stream with a seconds-level trigger

A seconds-level trigger on a daily 3 GB delta means a cluster running continuously to do nothing most of the time.

THE SIGNAL

Backfill and steady state are the same problem to an incremental tool. The first run is simply a very large batch.

Every tool in the next five modules is answering the same question in a different way: **what have I already seen, and how do I know?**

MODULE 04 · DOMAIN 2 - DATA INGESTION AND LOADING

COPY INTO

The idempotent SQL loader: syntax, the files it remembers, schema evolution, and the one behaviour every question turns on.

DOMAIN 2 of 7 EXAM WEIGHT 21% MODULE 04 of 36

admin@sthithapragna.com

COPY INTO keeps a record of every file it has loaded, and skips them next time. That is the whole feature. **Everything else is detail.**

The anatomy of the statement

CLAUSE	WHAT IT DOES
`COPY INTO my_table`	The target Delta table. It must already exist.
`FROM '/Volumes/...'`	Source location: S3, ADLS, ABFS, GCS or a UC volume
`FILEFORMAT = CSV`	The format of the source files
`FORMAT_OPTIONS (...)`	Passed to the Spark reader for that format
`COPY_OPTIONS (...)`	Controls COPY INTO itself

ON THE EXAM

Two option blocks that look alike and do different jobs. FORMAT_OPTIONS is about reading files. COPY_OPTIONS is about the command.

What idempotency actually guarantees

- **Files already loaded are skipped.** The record lives with the table, not in your scheduler.
- **This holds even if the file has been modified since it was loaded.** Same filename, still skipped.
- **The operation is retryable.** A failed run can simply be run again.
- **The same statement is safe on a schedule.** No date filter, no bookkeeping of your own.

> ON THE EXAM
>
> That second point is the single most testable detail about COPY INTO. Tracking is by file, not by content.

Default behaviour, and force

DEFAULT

force = false

- Previously loaded files are skipped
- Safe to schedule; safe to retry
- Corrections to an already-loaded file are ignored
- This is what you want almost always

COPY_OPTIONS ('FORCE' = 'TRUE')

Idempotency disabled

- Every matching file is loaded, seen before or not
- Use with FILES or PATTERN to target a subset
- On an unfiltered directory, it reloads everything
- Never leave it on in a scheduled job

THE TRAP

force does not mean 'try harder'. It means 'forget what you have already loaded'.

mergeSchema, in two places

- **In FORMAT_OPTIONS:** merge the schemas of the source files with each other while reading them.
- **In COPY_OPTIONS:** evolve the target table's schema to accept the incoming data.
- **Both default to false.** Without the COPY_OPTIONS one, a new source column causes a failure rather than a new column.
- **Schema evolution questions need the COPY_OPTIONS form.** That is the one that changes the table.

> ON THE EXAM
>
> Same word, two blocks, two jobs. FORMAT_OPTIONS reconciles files with each other; COPY_OPTIONS reconciles them with the table.

Loading into a schemaless placeholder table

STEP	STATEMENT
Create an empty shell	CREATE TABLE IF NOT EXISTS my_table;
Load and infer	COPY INTO my_table FROM '/path'
	FILEFORMAT = JSON
	FORMAT_OPTIONS ('mergeSchema' = 'true')
	COPY_OPTIONS ('mergeSchema' = 'true');

ON THE EXAM

Available from Databricks Runtime 11.3 LTS. The empty table is unusable until COPY INTO has populated it: INSERT INTO and MERGE INTO are not supported on it.

Choosing which files to load

OPTION	BEHAVIOUR	LIMIT
`FILES`	An explicit list of file names	1000 files maximum
`PATTERN`	A glob pattern matching the source directory	No fixed limit
`Neither`	Everything in the source location	
`Both together`	Not allowed	Mutually exclusive

ON THE EXAM

FILES and PATTERN cannot be used in the same statement. If an option shows both, it is wrong on syntax alone.

VALIDATE

- **Runs the load without writing anything**, so you can see what would happen.
- **The default validates all of the data** that would be loaded.
- **VALIDATE 15 ROWS** limits it to a given number of rows.
- **Returns a preview** of up to 50 rows.

> ON THE EXAM
>
> Useful for a first load against unfamiliar files, where you want to see the parse result before committing it to a table.

BY POSITION

- **Matches source columns to target columns by ordinal position** rather than by name.
- **Type casting of matched columns happens automatically.**
- **Only supported for headerless CSV.** FILEFORMAT must be CSV, with headers set to false.
- **The alternative is naming columns explicitly** in the COPY INTO clause.

> ON THE EXAM
>
> A narrow feature with a hard constraint. If the format is not headerless CSV, BY POSITION is not available.

What COPY INTO can read

FORMATS	SOURCES
CSV, JSON, XML	Amazon S3
Avro, ORC, Parquet	ADLS and ABFS
Text and binary files	Google Cloud Storage
	Unity Catalog volumes

ON THE EXAM

Note what is absent: message buses. COPY INTO reads files from object storage. Kafka and Kinesis are Structured Streaming's territory.

COPY INTO or Auto Loader?

COPY INTO

Thousands of files

- A single SQL statement, schedulable
- Directory listing only, to discover files
- Easy to reload a chosen subset with FILES or PATTERN
- No rescued data column; schema must be settable

AUTO LOADER

Millions of files

- A streaming source, cloudFiles
- Directory listing or file notification
- Harder to reprocess a specific subset
- Richer schema inference, evolution and rescued data

THE TIEBREAKER

Databricks' own line: thousands of files over time, use COPY INTO. Millions or more, use Auto Loader.

Four things COPY INTO cannot do

- **No rescued data column.** Records that fail to parse are not captured to one side for you.
- **Directory listing only.** No file notification mode, so discovery cost grows with directory size.
- **No message buses.** Files in object storage only.
- **It will not re-read a modified file.** Same name, already loaded, skipped.

> ON THE EXAM
>
> Each of these is a reason a question steers you to Auto Loader, streaming tables, or Structured Streaming instead.

A currency note

Worth knowing, because the documentation has started to shift

- **Databricks now recommends streaming tables** for SQL users wanting scalable, robust file ingestion.
- **Some documentation pages now label it 'COPY INTO (legacy)'.**
- **It remains fully supported**, and the exam guide names it as an explicit objective.
- **Learn it as taught here.** Streaming tables appear in module 13.

> ON THE EXAM
>
> Do not read 'legacy' as 'wrong answer'. The current exam guide names COPY INTO by name in Domain 2.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Scheduled nightly, no duplicates appear	Idempotency, on by default
A corrected file was re-uploaded but ignored	Same filename, already loaded, skipped
The table must gain a new source column	mergeSchema in COPY_OPTIONS
Reload one specific batch of files	FILES or PATTERN, with force
Millions of files over time	Auto Loader, not COPY INTO
Check the parse before writing anything	VALIDATE

ON THE EXAM

Six lines. The second one is the detail most candidates get wrong.

A nightly job runs COPY INTO against a folder of daily CSV exports. On Tuesday the team discovers that Monday's file, sales_2026_03_16.csv, contained corrupted amounts. The supplier corrects the file and re-uploads it under the same name.

The team re-runs the COPY INTO statement. The corrected amounts do not appear in the table, and the run reports that no files were loaded.

Why did nothing load?

A The re-uploaded file has a modification timestamp that is out of order, so it was skipped

B COPY INTO tracks files by name and had already loaded that file, so it skipped it despite the change

C The target table's schema no longer matches the corrected file

D COPY INTO only loads files created after the most recent successful load

◆ PAUSE AND DECIDE

Why did nothing load?

A Out-of-order modification timestamp

Plausible-sounding, but timestamps are not the mechanism. The file was skipped because of its identity, not its age.

B Already loaded, so skipped despite the modification

Correct. The documentation is explicit: already-loaded files are skipped even if they have been modified since loading.

C Schema mismatch

A schema mismatch produces an error, not a quiet report that zero files were loaded. The symptom does not fit.

D Only loads files created after the last load

Describes a timestamp watermark, which is how people imagine COPY INTO works. It tracks a set of files instead.

THE SIGNAL

Fix it with COPY_OPTIONS ('force' = 'true') plus FILES limited to that one file. Forcing the whole directory would reload everything.

> An IoT platform writes one small JSON file per device per minute into cloud storage. There are 9,000 devices. The directory already holds over 200 million files and grows by roughly 13 million a month.
>
> A team proposes a nightly COPY INTO statement against the directory to load the previous day's readings into bronze.

What is the strongest objection?

A COPY INTO cannot read JSON, so the format is wrong

B COPY INTO discovers files by directory listing, which does not scale to hundreds of millions of files

C COPY INTO would create duplicates because the files arrive every minute

D COPY INTO cannot write to a bronze table because bronze must be append-only

◆ PAUSE AND DECIDE

What is the strongest objection?

A COPY INTO cannot read JSON

False. JSON is one of its eight supported formats, alongside CSV, XML, Avro, ORC, Parquet, text and binary.

B Directory listing does not scale to this many files

Correct. COPY INTO only discovers files by listing. Databricks draws the line at thousands of files; this is 200 million.

C It would create duplicates

Backwards. Idempotency is on by default, so arrival frequency does not cause duplicates. This is the opposite of the truth.

D Cannot write to an append-only bronze table

Invented restriction. COPY INTO appends, which is precisely what a bronze table wants.

THE SIGNAL

Two numbers decide it: thousands of files is COPY INTO's range, millions is Auto Loader's. This scenario is 200 million.

> An engineer notices that some days a supplier delivers files late, after the nightly load has run. Worried that data is being missed, she edits the scheduled statement to add COPY_OPTIONS ('force' = 'true').
>
> The next morning the table contains four times as many rows as expected. The row count roughly matches the total number of records ever delivered.

What happened?

A force made COPY INTO retry failed files, and the retries were written twice

B force disabled idempotency, so the entire directory was reloaded on top of the existing data

C The late-arriving files had duplicate contents, and force preserved them

D force switched the write mode from append to overwrite, doubling the data

◆ PAUSE AND DECIDE

What happened?

A force retried failed files

force has nothing to do with retries. COPY INTO is retryable by default precisely because idempotency is on.

B Idempotency disabled, so the whole directory reloaded

Correct. force means load everything matching the source, seen before or not. Unfiltered, that is the entire history.

C Late files had duplicate contents

Blames the data. The row count matching everything ever delivered points at the command, not at a few late files.

D force switched append to overwrite

Overwrite would replace the data, giving roughly the expected count. The table grew instead, so it appended.

THE SIGNAL

The engineer's actual problem was real. Late files are picked up on the next run anyway, because they have not been loaded yet.

A team runs an Auto Loader stream that continuously ingests files from a landing directory into a bronze table. A supplier reports that 40 files delivered last week were malformed, and re-uploads corrected versions under new names.

The team needs those 40 corrected files loaded, without stopping the running stream and without reprocessing anything else.

What is the cleanest approach?

A Stop the stream, delete its checkpoint, and restart it to reprocess the directory

B Run COPY INTO with a FILES list naming the 40 corrected files, alongside the running stream

C Run COPY INTO with force = true against the whole landing directory

D Wait for the Auto Loader stream to detect the corrected files on its next pass

◆ PAUSE AND DECIDE

What is the cleanest approach?

A Delete the checkpoint and restart

Deleting the checkpoint discards all record of what was processed, so the stream reingests the entire directory. Duplicates on a large scale.

B COPY INTO with a FILES list, alongside the stream

Correct. Databricks documents exactly this: COPY INTO can reload a chosen subset while an Auto Loader stream runs simultaneously.

C COPY INTO with force on the whole directory

Loads all 40, and everything else as well. It meets one requirement and violates the other.

D Wait for Auto Loader to find them

Re-read the setup: the corrections were uploaded under new names, so Auto Loader will find them. But the question asks for the cleanest approach to a stated need, and waiting is not an approach.

THE SIGNAL

COPY INTO's weakness at scale is its strength here. FILES gives surgical control that a stream's checkpoint cannot.

COPY INTO remembers **files**, not contents, and not timestamps. Nearly every question about it is testing whether you know which.

MODULE 05 · DOMAIN 2 - DATA INGESTION AND LOADING

Auto Loader in depth

cloudFiles, checkpoints, file discovery modes, schema inference and evolution, and the rescued data column.

DOMAIN 2 of 7 EXAM WEIGHT 21% MODULE 05 of 36

admin@sthithapragna.com

Auto Loader is a **Structured Streaming source over a directory**. Everything it does follows from that, including the things people find surprising.

The minimal shape

LINE	PURPOSE
`spark.readStream`	It is a stream, not a batch read
`.format("cloudFiles")`	The Auto Loader source
`.option("cloudFiles.format", "json")`	Format of the source files. Required.
`.option("cloudFiles.schemaLocation", path)`	Where the inferred schema is stored
`.load(source_path)`	The directory to watch
`.writeStream.option("checkpointLocation", p)`	Where progress is recorded

ON THE EXAM

Two locations, two jobs. schemaLocation holds the schema; checkpointLocation holds progress. They may share a directory.

The checkpoint is the memory

- **It records which files have been processed**, so a restart resumes rather than repeats.
- **Exactly-once processing** follows from it. This is the guarantee, not a best effort.
- **Delete the checkpoint and the stream reingests everything.** There is no memory anywhere else.
- **Every source location needs its own checkpoint**, even when several streams write to the same target table.

> ON THE EXAM
>
> Setting schemaLocation enables schema inference and evolution. Without it, you get neither.

Auto Loader or a plain streaming file read?

SPARK.READSTREAM.FORMAT("JSON")

The generic file source

- Lists the whole directory on every micro-batch
- Cost grows with directory size, not with new data
- No schema evolution handling
- No rescued data column

FORMAT("CLOUDFILES")

Auto Loader

- Optimised discovery, and a notification option
- Cost tracks new files, not total files
- Schema inference, hints and evolution modes
- Rescued data column included by default

THE TIEBREAKER

Both are streams with checkpoints. Auto Loader adds efficient discovery and the whole schema apparatus on top.

How Auto Loader finds new files

DIRECTORY LISTING

Ask the storage what is there

- The default mode
- No extra permissions beyond reading the data
- Cost grows with the size of the directory
- Fine for modest volumes and simple setups

FILE NOTIFICATION

Let the storage tell you

- Subscribes to file events from the cloud provider
- Scales to millions of files an hour
- Needs permission to create or use cloud resources
- Databricks recommends it for most workloads

WORTH KNOWING

You can switch between the two across stream restarts and still keep exactly-once processing guarantees.

Classic file notification mode

- **Set cloudFiles.useNotifications to true** and Auto Loader sets up the notification and queue services itself.
- **It creates cloud resources** — an SNS topic and SQS queue on AWS, equivalents on Azure and GCP.
- **Each stream manages its own queue.** Many streams on one bucket means many parallel notification pipelines.
- **Notification systems do not guarantee 100% delivery**, which is what cloudFiles.backfillInterval exists for.

> ON THE EXAM
>
> backfillInterval triggers an asynchronous backfill at a set interval, so anything a missed notification skipped still gets processed.

File notification with file events

The managed option, and the current recommendation

- **Enable file events on the external location**, then set cloudFiles.useManagedFileEvents to true.
- **One managed service, shared.** A single topic and queue serve many consumers instead of one each.
- **The first run does a full directory listing** to synchronise, then reads from the file events cache.
- **Run the stream at least every seven days.** Otherwise the stored read position expires and a full listing is needed again.

> ON THE EXAM
>
> Databricks recommends migrating from directory listing to file notification with file events for most workloads.

The three discovery configurations

CONFIGURATION	OPTION	WHEN
Directory listing	(the default)	Modest volumes, no setup, quick to start
Classic file notification	cloudFiles.useNotifications	High volume, per-stream queues
File events	cloudFiles.useManagedFileEvents	High volume, managed and shared. Recommended.

ON THE EXAM

cloudFiles.useIncrementalListing is deprecated and defaults to false from DBR 17.3 LTS. Ignore it in older study material.

Schema **inference** happens once, when the stream first runs. Schema **evolution** is the ongoing policy for what happens when the data changes afterwards.

Schema inference

- **Set cloudFiles.schemaLocation** and Auto Loader infers a schema on first run and stores it there.
- **For JSON and CSV, columns are inferred as strings by default.** This is deliberate, and it surprises everyone.
- **Why:** a column that looks like an integer in the first files may contain text later. String never loses data.
- **The schema location may be the same directory as the checkpoint.** In Lakeflow pipelines both are managed for you.

> ON THE EXAM
>
> 'Why did everything arrive as STRING?' is one of the most reliably asked questions about Auto Loader.

Two ways to get real types

- **cloudFiles.inferColumnTypes = true.** Infer actual types rather than defaulting to string. Default is false.
- **cloudFiles.schemaHints.** Name specific columns and their types, and let the rest be inferred.
- **Hints are surgical.** Fix the three columns you care about without committing on the other forty.
- **A hint still permits addNewColumns**, whereas supplying a full schema does not.

> ON THE EXAM
>
> Prefer hints when you know some columns and not others. Prefer inferColumnTypes when you trust the data broadly.

The rescued data column captures anything that did not fit — wrong types, unexpected fields, casing mismatches — as JSON, **instead of dropping it**.

The four schema evolution modes

MODE	BEHAVIOUR WHEN A NEW COLUMN APPEARS
`addNewColumns`	Stream fails. The new column is added to the schema. Existing column types are unchanged.
`rescue`	Schema never evolves and the stream never fails. New columns are recorded in the rescued data column.
`failOnNewColumns`	Stream fails, and will not restart until the schema is updated or the offending file removed.
`none`	Schema is not evolved and new columns are ignored. Not rescued unless the option is set. Stream does not fail.

ON THE EXAM

Learn all four rows. Questions describe the behaviour and ask for the mode.

The defaults, which are not what you would guess

- addNewColumns is the default when you do not provide a schema.
- none is the default when you do provide a schema.
- addNewColumns is not allowed at all when a full schema is provided.
- It does work if you supply your schema as a schema hint instead.

> ON THE EXAM
>
> Two different defaults depending on whether you supplied a schema. That asymmetry is exactly the sort of thing an exam plants.

What 'the stream fails' actually means

- **Auto Loader detects the new column and stops with an UnknownFieldException.**
- **Before it stops, it updates the schema location** with the new column merged onto the end.
- **On restart, the evolved schema is used** and processing continues normally.
- **Databricks recommends running Auto Loader streams from Lakeflow Jobs** configured to restart automatically.

> ON THE EXAM
>
> With addNewColumns the failure is a feature, not a fault. One deliberate stop, then the pipeline carries on with the wider schema.

Two more evolution behaviours

- **A renamed column is treated as a new column.** The old one is filled with NULL for new rows.
- **A dropped column is a soft delete.** New rows get NULL; existing data is untouched.
- **addNewColumnsWithTypeWidening** also evolves on compatible type changes, from Databricks Runtime 16.4.
- **Widening is lossless only:** int to long, float to double, and similar promotions.

> ON THE EXAM
>
> Type widening is newer than most study material. It is a fifth mode name you may not have seen before.

Triggers: how often the stream runs

- **Trigger.AvailableNow** processes everything available, then stops. Batch behaviour with streaming bookkeeping.
- **Trigger with a processing time** runs micro-batches on an interval, leaving the cluster up between them.
- **Continuous processing** means a cluster that never stops, and a bill that never stops either.
- **AvailableNow on a schedule is the common production pattern** for data that arrives in bursts.

> ON THE EXAM
>
> AvailableNow gives you a checkpoint's exactly-once guarantee with a batch job's cost profile. That combination is why it dominates.

Three options that decide what gets picked up

- **cloudFiles.includeExistingFiles.** Default true. Whether files already present are processed, or only new arrivals.
- **It is evaluated only on the first start.** Changing it after the stream has run has no effect at all.
- **cloudFiles.backfillInterval.** Periodic asynchronous backfill, insuring against missed notifications.
- **cloudFiles.allowOverwrites.** Default false. Whether a changed file is reprocessed.

> ON THE EXAM
>
> Like COPY INTO, Auto Loader will not reprocess a modified file by default. Same principle, different mechanism.

Auto Loader without writing Python

- **Inside Lakeflow Spark Declarative Pipelines**, schema location and checkpoints are managed for you.
- **As a streaming table in Databricks SQL**, which wraps Auto Loader in a SQL declaration.
- **Via read_files in SQL**, the table-valued function over files in storage.
- **Same engine underneath.** The options you have learnt still apply, just expressed differently.

> ON THE EXAM
>
> A SQL-first team is not shut out of Auto Loader. Streaming tables are Databricks' recommended route for them.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Every JSON column arrived as STRING	Default inference. Use inferColumnTypes or schemaHints.
The pipeline must never stop, but capture new fields	rescue
A new column must be added automatically	addNewColumns, with a job that restarts
Schema drift must halt the pipeline immediately	failOnNewColumns
Millions of files an hour	File notification, ideally with file events
Data arrives in bursts a few times a day	Trigger.AvailableNow on a schedule
Duplicates appeared after a restart	The checkpoint was deleted or moved

ON THE EXAM

Seven lines. If you can answer all seven without hesitating, you have the most valuable module in domain two.

A team sets up Auto Loader against a directory of JSON event files. They set cloudFiles.format and cloudFiles.schemaLocation, and start the stream. It runs without error.

An analyst then reports that she cannot do arithmetic on the quantity or unit_price columns. Inspecting the bronze table, every single column has type STRING, including several that are obviously numeric.

What happened, and what is the most appropriate fix?

A The JSON files are malformed; the source system must be corrected

B Auto Loader infers JSON columns as STRING by default; set cloudFiles.inferColumnTypes or supply schemaHints

C The schemaLocation was not writable, so Auto Loader fell back to a default all-STRING schema

D The stream needs cloudFiles.schemaEvolutionMode set to addNewColumns to detect types

◆ PAUSE AND DECIDE

What happened, and what is the most appropriate fix?

A. The JSON files are malformed

The stream ran without error and produced every expected column. Malformed input would have surfaced in the rescued data column instead.

B. Default inference; use inferColumnTypes or schemaHints

Correct. For JSON and CSV, columns are inferred as STRING unless cloudFiles.inferColumnTypes is set to true or hints are supplied.

C. schemaLocation was not writable

Invented failure mode. An unwritable schema location produces an error, not a silent fallback.

D. Set schemaEvolutionMode to addNewColumns

Confuses evolution with inference. Evolution governs what happens when data changes later; it has no bearing on initial types.

THE SIGNAL

The stream ran without error. That is the clue: nothing failed, the default simply is not what the team expected.

> An Auto Loader stream has run reliably for eight months. On Sunday night it stopped with an UnknownFieldException. Investigating on Monday, an engineer finds that the source system began sending a new field, loyalty_tier, on Sunday evening.
>
> He also notices that the schema stored in the schema location already contains loyalty_tier, even though the stream failed.

What is going on?

A. The schema location has been corrupted, and should be deleted so the schema can be re-inferred

B. This is addNewColumns behaving as designed: it updated the schema, then stopped. Restarting will resume with the wider schema

C. Someone manually edited the schema location, which caused the stream to fail

D. The mode must be failOnNewColumns, so the schema must be updated by hand before restarting

◆ PAUSE AND DECIDE

What is going on?

A Corrupted; delete the schema location

Actively harmful. Deleting it discards the evolved schema, and if the checkpoint goes with it the stream reingests everything.

B addNewColumns working as designed; restart resumes

Correct. It updates the schema location with the new column merged on, then stops. The restart picks up the evolved schema.

C Someone edited the schema by hand

Invents a human. Auto Loader wrote that entry itself, which is exactly the documented sequence.

D failOnNewColumns, needing a manual schema update

Under failOnNewColumns the schema would not have been updated. The presence of loyalty_tier rules this mode out.

THE SIGNAL

The updated schema is the evidence. failOnNewColumns would have failed without writing anything; addNewColumns writes, then stops.

A payments team ingests transaction files from twelve partner banks. Partners occasionally add fields without warning. The pipeline feeds a regulatory report and must not stop.

The compliance team requires that any unexpected field is retained and reviewable, not discarded, but must never enter the reported dataset without approval.

Which schema evolution mode fits?

A addNewColumns

B failOnNewColumns

C rescue

D none

◆ PAUSE AND DECIDE

Which schema evolution mode fits?

A addNewColumns

Fails the stream on every new field, breaking the must-not-stop requirement. It also admits the field to the dataset without approval.

B failOnNewColumns

Stops the pipeline and will not restart until a human intervenes. The strictest possible violation of must-not-stop.

C rescue

Correct. The schema never evolves and the stream never fails. Unexpected fields land in the rescued data column, retained and reviewable.

D none

The stream does not fail, but new columns are ignored and not rescued unless separately configured. Retention fails.

THE SIGNAL

Three requirements: never stop, never discard, never auto-admit. Only rescue satisfies all three at once.

> A logistics platform lands roughly 4 million small files per hour into one S3 prefix. An Auto Loader stream in directory listing mode is falling further behind each day, and cloud costs for LIST operations have become the largest line on the bill.
>
> The team has permission to enable file events on the external location.

What should they change?

A. Increase the cluster size so the stream can process files faster

B. Switch to file notification mode using file events

C. Set cloudFiles.includeExistingFiles to false to skip the backlog

D. Replace Auto Loader with a scheduled COPY INTO statement

◆ PAUSE AND DECIDE

What should they change?

A Increase the cluster size

The bottleneck is discovery, not processing. More executors do not reduce the number of LIST calls against the prefix.

B File notification using file events

Correct. Notification removes listing entirely, scales to millions of files an hour, and is Databricks' recommendation over directory listing.

C includeExistingFiles = false

Evaluated only on first start, so it does nothing to a running stream. It also abandons the backlog rather than processing it.

D Scheduled COPY INTO

COPY INTO uses directory listing only, so it has the same bottleneck in a less capable tool. Strictly backwards.

THE SIGNAL

LIST costs being the largest line on the bill names the bottleneck outright. The fix has to stop the listing.

> Two Auto Loader streams ingest into the same bronze table: one from a European partner's directory, one from an American partner's. To keep configuration simple, an engineer points both streams at the same checkpointLocation.
>
> Records begin going missing. Neither stream reports an error.

What is wrong?

A Two streams cannot write to the same Delta table concurrently

B Each source location requires its own checkpoint; sharing one means the streams overwrite each other's progress

C The schemaLocation must differ, but sharing a checkpointLocation is supported

D The streams need cloudFiles.allowOverwrites set to true

◆ PAUSE AND DECIDE

What is wrong?

A Two streams cannot write to one Delta table

They can. Delta handles concurrent appends, and multiple sources feeding one bronze table is a normal pattern.

B Each source location needs its own checkpoint

Correct. Databricks states this explicitly: if more than one source location loads into the same target, each workload needs a separate streaming checkpoint.

C Only schemaLocation must differ

Half right, which makes it dangerous. Both need to be separate, and the checkpoint is the one causing lost records here.

D Set allowOverwrites to true

Controls whether a modified file is reprocessed. Unrelated to two streams sharing progress state.

THE SIGNAL

Silent data loss with no error is the signature of shared state. Each stream believes the other's progress is its own.

Auto Loader is a stream. The checkpoint is its memory, the schema location is its expectations, and the evolution mode is **what it does when reality disagrees**.

MODULE 06 · DOMAIN 2 - DATA INGESTION AND LOADING

Lakeflow Connect

Managed and standard connectors, the three-layer stack, ingestion pipelines and gateways, and the rule for choosing a layer.

DOMAIN 2 of 7 EXAM WEIGHT 21% MODULE 06 of 36

admin@sthithapragna.com

Lakeflow Connect is not one tool. It is an **umbrella over the whole ingestion stack**, and questions about it are almost always asking which layer of that stack you should use.

The connector types

TYPE	WHAT IT INGESTS FROM
Database (CDC)	Relational databases using change data capture: MySQL, PostgreSQL, SQL Server
Query-based	Databases, by querying the source directly, with no CDC configuration
SaaS	Enterprise applications: Salesforce, Workday, HubSpot, Jira and others
File source	Enterprise file services such as SharePoint and Google Drive
Streaming	Message buses and event streaming sources
Community and custom	Sources with no built-in connector

ON THE EXAM

Note that database sources have two routes: CDC-based and query-based. They have different architectures and different costs.

The three layers of the ingestion stack

01

Structured Streaming

Most customisable. Full control, full Spark APIs, you build everything.

02

Lakeflow pipelines

Declarative. Orchestration, monitoring, quality and error handling are managed.

03

Managed connectors

Most automated. Auth, CDC, retries and schema evolution all handled.

LEARN THIS RULE

Databricks' rule: start with the most managed layer. If it does not satisfy your requirements, drop down to the next.

Why the rule runs downward, not upward

- **Each layer is built on the one below it.** Managed connectors run on pipelines, which run on Structured Streaming.
- **So moving down never adds capability you lose by moving up.** It adds work you would otherwise not do.
- **Starting at the bottom means rebuilding solved problems:** auth, retries, schema drift, incremental state.
- **The only good reason to drop down is a requirement the layer above cannot meet.**

> ON THE EXAM
>
> In an exam, the most managed option that meets every stated requirement is usually correct. Extra control you were not asked for is a cost.

Managed or standard connectors?

MANAGED CONNECTORS

Fully handled

- Source-specific auth, CDC and edge cases handled
- Governed by Unity Catalog, on serverless compute
- Automatic retries with exponential backoff
- Limited to the sources Databricks has built

STANDARD CONNECTORS

More control, more work

- Broader source support
- Choose your own level: Structured Streaming or pipelines
- You configure discovery, schema handling and state
- Auto Loader is the canonical standard connector

THE TIEBREAKER

Standard connectors trade automation for broader source support and customisation. That trade is the whole distinction.

Anatomy of a managed connector

COMPONENT	WHAT IT IS
Connection	A Unity Catalog securable holding the source's authentication details
Ingestion pipeline	Reads the source and writes the results. Runs on serverless compute by default.
Destination tables	Streaming tables that the pipeline writes into
Ingestion gateway	Database CDC connectors only. Captures changes continuously.
Staging storage	Database CDC connectors only. Holds captured changes.

ON THE EXAM

SaaS, file, query-based and streaming connectors use the first three. Only database CDC connectors add a gateway and staging.

Two ways to ingest from a database

DATABASE CONNECTOR

Change data capture

- An ingestion gateway runs continuously as its own job
- Staging storage holds the captured changes
- Captures every change, including deletes
- The gateway is continuous, so not fully serverless

QUERY-BASED CONNECTOR

Query the source on a schedule

- No gateway, no staging storage
- The pipeline queries the source directly
- Simpler and cheaper to run
- Sees the state at query time, not every change between runs

THE TIEBREAKER

CDC catches everything that happened. Query-based catches what is there when it looks. That difference decides which you need.

How incremental ingestion works here

- **The first run ingests everything selected** from the source, and starts tracking changes in parallel.
- **Every later run uses that change tracking** to ingest only what changed since the last run.
- **The approach depends on the source.** SQL Server can use both change tracking and CDC; Salesforce picks a cursor column.
- **Some sources and tables do not support incremental ingestion yet.** Those fall back to a full load each run.

> ON THE EXAM
>
> Without a valid cursor or change-tracking mechanism, the connector performs a full load every run. That is the cause behind a whole family of cost complaints.

Orchestration

- **Add a schedule to a pipeline and Lakeflow Connect creates a job for it automatically.**
- **The ingestion pipeline becomes a task inside that job**, and you can add further tasks around it.
- **A pipeline can have more than one schedule**, each getting its own job.
- **For database connectors the gateway runs in its own separate job**, as a continuous task.

> ON THE EXAM
>
> That last point explains the cost profile. The pipeline may run four times a day, but the gateway never stops.

Failure recovery

- **Failures retry automatically with exponential backoff.** Most transient problems resolve without anyone noticing.
- **Some errors need you**, most commonly expired credentials.
- **The connector stores the last cursor position**, so it resumes from there rather than losing data.
- **A full refresh can erase connector-generated history.** Treat it as destructive, not as a reset button.

> ON THE EXAM
>
> Automatic recovery is a large part of what managed connectors are for. It is also why 'we can just write a notebook' underestimates the work.

Networking

- **SaaS connectors reach out to the source's APIs**, and are automatically compatible with serverless egress controls.
- **Cloud databases** can connect over Private Link, or the gateway can be deployed inside a peered VNet or VPC.
- **On-premises databases** connect through services such as AWS Direct Connect or Azure ExpressRoute.
- **Networking is usually the real constraint** on whether a managed connector is viable, rather than the connector itself.

> ON THE EXAM
>
> When a scenario mentions an on-premises source behind a corporate firewall, the network path is the thing to check first.

Standard connectors, by level of control

SOURCE	MORE CUSTOMISATION	MORE AUTOMATION
Cloud object storage	Auto Loader with Structured Streaming	Auto Loader with Databricks SQL
Apache Kafka	Structured Streaming with Kafka source	Databricks SQL with Kafka source
Google Pub/Sub	Structured Streaming with Pub/Sub source	Lakeflow pipelines with Pub/Sub source
SFTP servers	Python or SQL	Not available

ON THE EXAM

The same source appears at several levels. Choosing a standard connector means choosing a source and a level of control.

The SQL route

- **CREATE STREAMING TABLE** is the SQL syntax for incremental ingestion from cloud object storage.
- **Databricks calls it the recommended alternative to COPY INTO** for SQL users.
- **It wraps Auto Loader**, so all the schema and discovery behaviour from module 5 still applies.
- **Schema location and checkpoints are managed for you** when it runs inside a Lakeflow pipeline.

> ON THE EXAM
>
> This is the same recommendation module 04 flagged, seen from the other side. It does not make COPY INTO a wrong answer.

Deployment and connections

- **Ingestion pipelines deploy with Declarative Automation Bundles**, so they get source control, review and CI/CD.
- **Connections can be created programmatically** via the Connections API, the CLI, or a bundle pre-deploy script.
- **Except where browser-based OAuth is the only option.** Those need an interactive sign-in and cannot be automated.
- **Affected connectors include** Jira, HubSpot, Slack, Zendesk Support and several advertising platforms.

> ON THE EXAM
>
> A fully automated deployment can still be blocked by one connector that requires a human to click through a sign-in.

Two risks the documentation states plainly

- **Managed connectors depend on external services** that Databricks does not control.
- **Databricks may discontinue a connector** if changes to that external service make it impractical to maintain.
- **Connectors sit in various release states.** Check whether one is generally available before designing around it.
- **Monitoring is provided:** event logs, cluster logs, pipeline health and data quality metrics, plus system.billing.usage for cost.

> ON THE EXAM
>
> For database connectors specifically, gateway progress can be monitored in real time through event logs.

Lakeflow Connect or Lakehouse Federation?

LAKEFLOW CONNECT

Ingest a copy

- Reads the source and writes data into Delta tables
- The data now lives in your lakehouse
- Queries do not touch the source system
- Freshness depends on the pipeline schedule

LAKEHOUSE FEDERATION

Query in place

- Registers the source as a foreign catalog
- No copy is made; data stays where it is
- Every query reaches out to the source system
- Always current, but load lands on the source

WORTH DISTINGUISHING

Federation is not a Domain 2 objective on the current exam, but the distinction is a clean confusion pair worth holding.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Salesforce, Workday, ServiceNow or similar	A managed SaaS connector
Every change to the database, including deletes	A database CDC connector, with a gateway
Periodic snapshot of database tables, cheaply	A query-based connector, no gateway
Files in cloud object storage	Auto Loader, a standard connector
Kafka, Kinesis or a message bus	Structured Streaming or a streaming connector
A source with no built-in connector	Community, then custom
The source must not be copied	Lakehouse Federation, not ingestion

ON THE EXAM

Seven lines. Most Lakeflow Connect questions are answered by naming the source type correctly.

A marketing team needs Salesforce opportunity and account data in the lakehouse, refreshed several times a day. Nobody on the team writes Python. The platform team requires that credentials are governed and that the destination tables appear in Unity Catalog with lineage.

An engineer offers to write a notebook that calls the Salesforce REST API and handles pagination, retries and token refresh himself.

How should the team proceed?

A Accept the notebook, since it gives full control over the API calls

B Use the managed Salesforce connector, which handles auth, incremental reads and retries

C Use Auto Loader pointed at a directory the notebook exports to

D Use Lakehouse Federation to query Salesforce in place

◆ PAUSE AND DECIDE

How should the team proceed?

A **Accept the notebook**

It rebuilds auth, pagination, retries and incremental state by hand, and nobody on the team can maintain it. Control nobody asked for.

B **The managed Salesforce connector**

Correct. A managed connector exists for exactly this source. Credentials live in a UC connection, and destination tables are governed streaming tables.

C **Auto Loader over an exported directory**

Keeps the fragile notebook and adds a second hop. Auto Loader reads files, so something still has to produce them.

D **Lakehouse Federation**

Queries in place rather than ingesting, so every downstream query hits Salesforce. The team asked for data in the lakehouse.

THE SIGNAL

Start at the most managed layer. A managed connector exists for this source and meets every stated requirement, so nothing justifies dropping down.

A finance team ingests from an on-premises SQL Server using a managed database connector. The ingestion pipeline is scheduled four times a day. Reviewing costs, an analyst is puzzled: the pipeline runs for about twelve minutes each time, but compute charges look continuous.

She asks whether the pipeline has been misconfigured to run continuously.

What should she be told?

A The pipeline is misconfigured; it should be changed from continuous to triggered mode

B The ingestion gateway runs continuously in its own job to capture changes, separate from the scheduled pipeline

C Serverless compute bills continuously once enabled, regardless of pipeline activity

D The destination streaming tables keep a cluster alive to serve queries

◆ PAUSE AND DECIDE

What should she be told?

A. Change the pipeline to triggered mode

The pipeline is already triggered, four times a day. Changing a setting that is already correct will not touch the continuous cost.

B. The gateway runs continuously in its own job

Correct. Database CDC connectors deploy a gateway as a continuous task in a separate job, because change capture cannot be intermittent.

C. Serverless bills continuously once enabled

False, and the reverse of how serverless works. It bills for what runs, which is why it suits bursty workloads.

D. Streaming tables keep a cluster alive

A streaming table is a table. It does not hold compute open to serve queries.

THE SIGNAL

The cost is not a fault. Continuous change capture is the architecture, and it is the price of catching every change including deletes.

A team needs to ingest events from a self-hosted Apache Kafka cluster into bronze. They need custom deserialisation of a proprietary binary payload, and they must apply a per-record routing rule before anything is written.

A colleague argues that Databricks recommends starting with the most managed layer, so they should look for a managed connector first.

What is the right conclusion?

A. The colleague is right; the rule always means choosing the most managed option available

B. The rule applies, but custom deserialisation and per-record routing are requirements the managed layer cannot meet, so drop down

C. The rule does not apply to message buses, which always require Structured Streaming

D. Kafka cannot be ingested by Databricks; the data must first be written to object storage

◆ PAUSE AND DECIDE

What is the right conclusion?

A. Always choose the most managed option

Drops the second half of the rule. Databricks says start at the most managed layer and drop down when it does not satisfy your requirements.

B. The rule applies, but these requirements force a drop down

Correct. Custom deserialisation and per-record routing need code, which is exactly the situation the rule's second clause describes.

C. The rule does not apply to message buses

It does. Kafka appears at several levels, including Lakeflow pipelines and Databricks SQL. The rule is not suspended by source type.

D. Kafka cannot be ingested directly

False. Kafka is a supported source at multiple levels of the stack.

THE SIGNAL

The rule has two clauses. Candidates who memorise only the first one get this wrong in exactly the way the colleague does.

> A managed connector ingests eleven tables from a SaaS application. Ten of them refresh in under a minute each run. The eleventh takes forty minutes every time and its row count in the destination is identical after every run.
>
> The pipeline is configured identically for all eleven tables.

What is the most likely explanation?

A That table is much larger, so it simply takes longer

B That table has no valid cursor or change-tracking mechanism, so the connector performs a full load every run

C The ingestion pipeline needs a separate schedule for that table

D The destination table needs OPTIMIZE to speed up the writes

◆ PAUSE AND DECIDE

What is the most likely explanation?

A It is simply a larger table

Size alone would make the first run slow. Incremental runs afterwards should be proportional to what changed, not to total size.

B No valid cursor, so a full load every run

Correct. Some sources and tables do not support incremental ingestion. Without a cursor, the connector reloads everything each time.

C It needs its own schedule

Scheduling changes when the work happens, not how much work there is. Forty minutes would still be forty minutes.

D Run OPTIMIZE on the destination

File layout affects reads. It does not explain why this one table reprocesses its whole contents on every run.

THE SIGNAL

'Identical row count after every run' is the clue. Consistent full reloads look identical; genuine incremental loads do not.

Start at the most managed layer. Drop down only when a stated requirement forces it. **Both halves of that sentence are examinable.**

MODULE 07 · DOMAIN 2 - DATA INGESTION AND LOADING

JDBC, REST and nested data

Pulling from databases and APIs in a notebook, and working with JSON, structs, arrays and the VARIANT type.

DOMAIN 2 of 7 EXAM WEIGHT 21% MODULE 07 of 36

admin@sthithapragna.com

JDBC and REST pulls are **escape hatches**. They are the right answer when no connector covers your source, and the wrong answer when one does.

Reading over JDBC

OPTION	PURPOSE
`url`	The JDBC URL, in the form jdbc:subprotocol:subname
`dbtable`	The table to read. Or use query for a SQL statement.
`user and password`	Credentials. Read these from a secret scope.
`partitionColumn`	A numeric, date or timestamp column to split on
`lowerBound and upperBound`	The range used to calculate the split
`numPartitions`	How many parallel reads to issue

> ON THE EXAM
>
> The same options work from SQL via CREATE TEMPORARY VIEW … USING JDBC OPTIONS (…).

Credentials

- **Never write a password into a notebook.** It ends up in Git, in the workspace, and in anyone's screen share.
- **Use a secret scope** and read the value with dbutils.secrets.get at runtime.
- **Secrets are redacted in notebook output**, so a printed secret shows as REDACTED rather than the value.
- **Better still, use a Unity Catalog connection** where one exists, so credentials are a governed securable.

> ON THE EXAM
>
> A hardcoded credential is a wrong answer even when the option would otherwise work. Treat it as disqualifying.

Without **partitionColumn**, Spark reads the entire table through a **single connection on one executor** — no matter what numPartitions says.

How the four partitioning options work together

- **partitionColumn** must be a numeric, date or timestamp column, ideally evenly distributed.
- **lowerBound and upperBound** define the range Spark divides into stripes.
- **numPartitions** is how many stripes, and therefore how many concurrent connections.
- **Do not set numPartitions very large.** Hundreds of concurrent connections can overwhelm the source database.

> ON THE EXAM
>
> Spark generates one WHERE clause per stripe and issues them concurrently. A skewed partition column produces skewed reads.

lowerBound and upperBound decide the **partition stride only**. They do not filter. Every row in the table is still returned.

Other JDBC tuning

- **fetchSize** controls how many rows are pulled per round trip. Defaults can be tiny.
- **Oracle's default fetchSize is 10**, which means an enormous number of round trips on a large table.
- **Use the query option** to filter and aggregate on the source, so less data crosses the network.
- **Select only the columns you need.** Wide tables are often slow for this reason alone.

> ON THE EXAM
>
> The cheapest optimisation is always reading less. Push filtering to the source before you tune parallelism.

Pulling from a REST API

- **An ordinary Python HTTP call runs on the driver only.** The executors are not involved at all.
- **Pagination, rate limits, retries and token refresh are all yours to handle.**
- **The usual pattern:** fetch pages on the driver, write the raw responses to a volume, then process them with Spark.
- **Schedule it with Lakeflow Jobs**, which gives you retries, alerting and a run history for free.

> ON THE EXAM
>
> Writing raw responses to storage first means a parsing bug can be fixed without re-calling the API. Same logic as bronze.

Where should the pulled data land?

INTO CLOUD STORAGE OR A VOLUME

Land the raw response

- Original payloads kept exactly as returned
- Re-parse after a bug without re-calling the API
- Auto Loader can then pick them up incrementally
- An extra hop and files to manage

STRAIGHT INTO A GOVERNED TABLE

Write direct

- Fewer moving parts
- Governed and queryable immediately
- Nothing to re-read if parsing was wrong
- Fine when the source can simply be re-queried

ON THE EXAM

The exam guide names both routes explicitly. The deciding question is whether the source can be asked again.

Three ways to get at a database

APPROACH	WHAT IT DOES	CHOOSE WHEN
Managed connector	Ingests a copy, handles CDC and auth	A connector exists for your source
JDBC in a notebook	You issue the query and write the result	No connector fits, or you need a specific query
Lakehouse Federation	Queries in place, no copy made	The data must not be copied

ON THE EXAM

These are not interchangeable. One copies with automation, one copies with effort, one does not copy at all.

Nested data is not a problem to be eliminated. It is a **shape**, and Spark has a type for each part of it.

The three complex types

TYPE	WHAT IT HOLDS	HOW YOU GET IN
STRUCT	Named fields of fixed, known types	Dot notation: address.city
ARRAY	An ordered list of same-typed elements	Index, or explode into rows
MAP	Key-value pairs, keys all one type	Key lookup: attributes['colour']

ON THE EXAM

A struct is one row with more columns inside it. An array is many values in one row, which is why it usually needs exploding.

Structs: dot notation

- **customer.address.city** reaches through nested structs, however deep.
- **Row count is unchanged.** You are selecting a column that happened to live inside another column.
- **Select customer.* to promote every field** of a struct to a top-level column in one go.
- **Field names are case-sensitive in some contexts**, notably when navigating VARIANT paths.

> ON THE EXAM
>
> If a scenario is only about reaching a nested field, dot notation is enough. No explode, no from_json.

Arrays: explode

- **explode** turns each element of an array into its own row, repeating the other columns.
- **It changes the row count.** An order with four line items becomes four rows.
- **explode_outer keeps rows whose array is empty or null**, giving them a null instead of dropping them.
- **Use size to count elements** without exploding, when you only need the count.

> ON THE EXAM
>
> Plain explode silently drops rows with an empty array. That is a quiet data-loss bug and a good distractor.

When JSON arrives as a string

- **from_json** parses a JSON string column into a struct, given a schema.
- **schema_of_json** infers that schema from a sample value, which helps you write the first version.
- **The colon operator** extracts directly from a JSON string without declaring a schema: payload:user.name
- **from_json does not support schema evolution.** A new field needs the schema updated by hand.

> ON THE EXAM
>
> The colon operator is quick and needs no schema. from_json is stricter and gives you real typed columns.

VARIANT stores semi-structured data natively. Databricks now recommends it over storing JSON as strings.

Working with VARIANT

OPERATION	SYNTAX
Create from a JSON string	parse_json(json_string_column)
Query a field	column_name:field_name
Query a subfield	column_name:field_name.subfield_name
Query an array element	column_name:array_name[0]
Cast to a type	column_name:field::INT
Inspect the shape	schema_of_variant(column_name)

ON THE EXAM

Note the pattern: colon to enter the variant, dot to go deeper, brackets for array position. Paths are matched case-sensitively.

VARIANT limitations

- **VARIANT columns cannot be clustering keys, partition columns, or Z-order keys.**
- **They cannot be used for comparisons, grouping, ordering or set operations.**
- **So extract frequently queried fields into ordinary typed columns**, and keep the variant alongside for the rest.
- **Iceberg v2 does not support VARIANT columns.** Iceberg v3 does.

> ON THE EXAM
>
> That first limitation decides scenarios. If a column must be a clustering key, it cannot be VARIANT, whatever else recommends it.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
JDBC read uses one core despite numPartitions	partitionColumn is missing; all four are needed
Bounds were set but every row came back	lowerBound and upperBound set stride, not a filter
A password appears in the notebook	Wrong answer. Use a secret scope.
One row per array element	explode, or explode_outer to keep empties
Reach a nested field	Dot notation. Nothing more is needed.
A JSON string column, no schema declared	The colon operator, or parse_json to VARIANT
The column must be a clustering key	It cannot be VARIANT

ON THE EXAM

Seven lines covering both halves of the module. The second and the last are the two most commonly missed.

> An engineer reads a 90 million row table from PostgreSQL over JDBC into bronze. The read takes just over four hours. She sets numPartitions to 24 and reruns it. The runtime is unchanged.
>
> The Spark UI shows one task in the stage, running for the whole four hours, on a cluster with 32 cores available.

What is wrong?

A numPartitions is capped by the source database's connection limit

B partitionColumn, lowerBound and upperBound are missing, so numPartitions has no effect

C The cluster needs more executors before numPartitions takes effect

D fetchSize must be increased before parallelism is possible

◆ PAUSE AND DECIDE

What is wrong?

A Capped by the connection limit

A connection cap would reduce parallelism, not eliminate it. You would see several tasks, not exactly one.

B partitionColumn and the bounds are missing

Correct. All four options are required together. Without a partition column Spark cannot generate ranges, so it opens a single connection.

C More executors are needed

32 cores are already idle. Adding capacity to a job that uses one core changes nothing.

D fetchSize must be raised first

fetchSize affects rows per round trip on a connection. It does not create additional connections.

THE SIGNAL

Exactly one task is the giveaway. A partly-working configuration gives you a few tasks; a missing partitionColumn gives you precisely one.

A team reads an orders table over JDBC. They only want orders with an order_id above 9,000,000, which is roughly the last month. They set partitionColumn to order_id, lowerBound to 9000000, upperBound to 9400000, and numPartitions to 8.

The resulting DataFrame contains all 9.4 million orders, not the 400,000 they expected. One partition is enormous.

What happened?

A lowerBound and upperBound determine the partition stride only; they do not filter rows

B The bounds were applied, but Spark re-read the excluded rows to rebalance the partitions

C order_id is not evenly distributed, so the bounds were ignored

D numPartitions must equal the number of rows divided by the bound range for filtering to apply

◆ PAUSE AND DECIDE

What happened?

A The bounds set stride, not a filter

Correct. The documentation is explicit: the bounds decide the partition stride only. All rows in the table are partitioned and returned.

B Spark re-read excluded rows to rebalance

Invents a rebalancing step. Nothing was ever excluded, so there was nothing to re-read.

C order_id is unevenly distributed

Distribution affects how evenly stripes fill. It does not cause 9 million extra rows to appear.

D numPartitions must match a ratio

Fabricated rule. No arithmetic relationship between numPartitions and the bounds turns them into a filter.

THE SIGNAL

To filter, put a WHERE clause in the query option. The bounds are about how to split, never about what to include.

A bronze table holds one row per order. Each row has an items column that is an array of structs, each with sku, quantity and price. Silver needs one row per item, carrying the order_id and order_date alongside.

About 3% of orders are cancelled before any item is added, so their items array is empty. Finance needs those cancelled orders to remain visible in silver.

Which approach is correct?

A Select items.sku, items.quantity and items.price using dot notation

B Use explode on items, then dot notation on the resulting struct

C Use explode_outer on items, then dot notation on the resulting struct

D Use from_json on items with a declared schema

◆ PAUSE AND DECIDE

Which approach is correct?

A Dot notation on the array

Dot notation does not change the row count, so you would still have one row per order, now holding arrays of skus. The requirement was one row per item.

B explode, then dot notation

Right shape, wrong on the second requirement. Plain explode silently drops the 3% of orders with empty arrays, and finance needs those.

C explode_outer, then dot notation

Correct. It produces one row per item and keeps rows whose array is empty, giving them null item fields.

D from_json with a schema

items is already an array of structs, not a JSON string. There is nothing left to parse.

THE SIGNAL

Two requirements again. 'One row per item' rules out A and D. 'Cancelled orders must remain visible' is what separates B from C.

An events table ingests JSON payloads whose shape varies widely between event types and changes often. A team stores the payload in a VARIANT column, which suits it well.

The table has grown large, and queries almost always filter on event_type, which currently lives only inside the payload. The team wants to use event_type as a clustering key.

What should they do?

A Set the VARIANT column as the clustering key, since event_type lives inside it

B Extract event_type into its own typed column and cluster on that, keeping the VARIANT alongside

C Convert the whole payload to a struct so it can be clustered

D Store the payload as a JSON string instead, which supports clustering

◆ PAUSE AND DECIDE

What should they do?

A Cluster on the VARIANT column

Not permitted. VARIANT columns cannot be clustering keys, partition columns or Z-order keys.

B Extract event_type to a typed column, keep the VARIANT

Correct. This is Databricks' own recommendation: pull frequently queried fields into ordinary columns and keep the variant for the rest.

C Convert everything to a struct

Throws away the flexibility that made VARIANT the right choice. A struct needs a declared schema, and the shape changes often.

D Store as a JSON string

A JSON string is what VARIANT was recommended over. It also does not make the nested field clusterable.

THE SIGNAL

The hybrid is the intended design, not a compromise. Typed columns for what you query, VARIANT for what you might.

When you pull data yourself, **you own everything the connector would have done**. When data arrives nested, name the shape first and the operation follows.

MODULE 08 · DOMAIN 2 - DATA INGESTION AND LOADING

Choosing an ingestion method

The decision sequence, the four criteria the exam guide names, and eight worked choices.

DOMAIN 2 of 7 EXAM WEIGHT 21% MODULE 08 of 36

admin@sthithapragna.com

Name the **source type** first. It eliminates most options before you have thought about anything else. The four criteria then break the tie.

admin@sthithapragna.com

The decision sequence

01

Name the source

Files, database, SaaS, message bus, or API.

02

Does a managed connector exist?

If yes, and it meets the requirements, stop here.

03

Check volume and frequency

These separate COPY INTO from Auto Loader, and batch from streaming.

04

Check governance

Unity Catalog, credentials, audit. Often eliminates an otherwise good option.

05

Drop a layer only if forced

Custom code is the answer when a requirement demands it, not by default.

WORK IT IN THIS ORDER

Five steps in order. Most exam questions are decided by step one or step two.

The four criteria, named in the exam guide

CRITERION	WHAT IT DECIDES
Data volume	Thousands of files means COPY INTO. Millions means Auto Loader.
Ingestion frequency	Daily batch, bursts through the day, or continuous seconds-level.
Data types	Files, tables, semi-structured payloads, or unstructured documents.
Governance needs	Unity Catalog grants, credential handling, audit and lineage.

ON THE EXAM

These are the guide's own words. When a scenario states a number or a frequency, it is handing you one of these criteria.

Criterion one: data volume

- **Thousands of files over time:** COPY INTO is appropriate and simpler.
- **Millions or more:** Auto Loader, because discovery cost is what breaks first.
- **Row count is not the criterion. File count is.** A billion rows in twelve Parquet files is not a volume problem.
- **Above millions per hour:** Auto Loader with file notification rather than directory listing.

> ON THE EXAM
>
> The distinction that catches people: many small files is a discovery problem, while a lot of data is a processing problem. Different fixes.

Criterion two: ingestion frequency

- **Once or a few times a day:** a scheduled job, with COPY INTO or Auto Loader using AvailableNow.
- **Bursts through the day at unpredictable times:** a file arrival trigger, or serverless to avoid idle cost.
- **Continuous, seconds matter:** a genuine stream, with a cluster that stays up.
- **Continuous costs continuously.** Only choose it when latency is a stated requirement, not an aspiration.

> ON THE EXAM
>
> 'Near real time' in a scenario usually means minutes. Read the stated latency before assuming a continuous stream is required.

Criterion three: data types

- **Structured files** in object storage: COPY INTO or Auto Loader.
- **Semi-structured JSON with drift:** Auto Loader, for schema evolution and the rescued data column.
- **Unstructured documents** such as PDFs or images: a volume, and possibly a file source connector.
- **Message bus payloads:** Structured Streaming or a streaming connector. Never Auto Loader.

> ON THE EXAM
>
> Auto Loader reads files from object storage. It does not read Kafka. That single fact settles a whole family of questions.

Criterion four: governance

- **A hardcoded credential is disqualifying**, whatever else the option does well.
- **A DBFS mount bypasses Unity Catalog entirely.** Deprecated, and wrong wherever governance is mentioned.
- **Destination tables must be Unity Catalog governed** for lineage and audit to work.
- **Managed connectors store credentials as a UC connection**, which is a governed securable rather than a string.

> ON THE EXAM
>
> Governance is the criterion most likely to eliminate a technically correct answer. Check it last, but do check it.

The whole of domain 2 on one slide

METHOD	SOURCE	BEST WHEN
COPY INTO	Files in object storage	Thousands of files, SQL-first, stable schema
Auto Loader	Files in object storage	Millions of files, schema drift, continuous arrival
Streaming tables	Files, in SQL	The recommended SQL route; wraps Auto Loader
Managed connector	SaaS, databases, enterprise files	A connector exists for your source
Streaming connector	Message buses	Kafka, Pub/Sub, Kinesis and similar
JDBC or REST in a notebook	Databases and APIs	No connector fits, or a specific query is needed
Partner connector	Long tail of SaaS sources	Databricks has no connector but a partner does
Lakehouse Federation	Databases	No copy should exist at all

ON THE EXAM

Eight methods. Read the middle column first — the source type alone eliminates most rows.

Partner connectors

- **Partner Connect is a portal** of validated data, analytics and AI tools that integrate with Databricks.
- **Fivetran is the canonical ingestion partner**, offering connectors to a very long list of SaaS sources.
- **They connect through a SQL warehouse or cluster**, and write into Delta tables in your workspace.
- **The reason to use one is coverage.** A partner may support a source Databricks does not.

> ON THE EXAM
>
> The exam guide names partner connectors as one of the options to prioritise between. Coverage is what they add.

Instant eliminations

Patterns that make an option wrong before you evaluate it properly

- **Auto Loader or COPY INTO against a message bus.** Wrong source type.
- **A password written into a notebook.** Governance failure.
- **A DBFS mount or the DBFS root.** Deprecated and ungoverned.
- **Deleting a checkpoint to reprocess.** Causes mass duplication.
- **An invented limit**, such as a maximum table size or a fixed retention period.

> ON THE EXAM
>
> Learning to eliminate is faster than learning to select. Two eliminations often leave one plausible answer and one obvious one.

Two ways to get it wrong

OVER-ENGINEERING

Machinery nobody needed

- A streaming pipeline for a quarterly reference file
- File notification for a directory of forty files
- A custom connector where a managed one exists
- Costs money and adds things that can break

UNDER-ENGINEERING

Something that will not hold

- A plain directory read for a growing feed
- No incremental state, so duplicates on re-run
- No schema handling for a source that drifts
- Works today, fails as volume or time increases

BOTH ARE TESTED

The exam tests both. An option can be wrong for doing too much just as easily as for doing too little.

Tiebreakers for the whole domain

WHEN A QUESTION SAYS	REACH FOR
Thousands of files, SQL team	COPY INTO
Millions of files, or schema drift	Auto Loader
Salesforce, Workday, SQL Server	A managed connector
Kafka, Kinesis, Pub/Sub	Structured Streaming or a streaming connector
A niche SaaS app Databricks does not cover	A partner connector
A query no connector can express	JDBC in a notebook
The data must not be copied	Lakehouse Federation
A 40-row file maintained by hand	A volume. No machinery at all.

ON THE EXAM

Eight lines for eight methods. Now eight scenarios to practise applying them under pressure.

A finance team receives about 30 CSV files each night from three banks, into a Unity Catalog volume. The schema has not changed in four years and is contractually fixed. The team writes SQL and has no Python experience.

They need the files loaded into a bronze table each morning, without reloading what was already loaded.

Which method fits best?

A. Auto Loader with file notification mode and schema evolution

B. COPY INTO on a scheduled job

C. A nightly spark.read over the directory, overwriting the table

D. A managed connector for each of the three banks

◆ PAUSE AND DECIDE

Which method fits best?

A Auto Loader with file notification

Every part of it is aimed at a problem this team does not have. Thirty files a night is not a discovery problem, and the schema is contractually fixed.

B COPY INTO on a schedule

Correct. Thousands of files is COPY INTO's range and this is thirty a night. It is one SQL statement, idempotent by default, and the team writes SQL.

C spark.read and overwrite nightly

No incremental state, so runtime grows with history forever. This is the module 3 anti-pattern.

D A managed connector per bank

The banks deliver files. Managed connectors read SaaS applications and databases, not a directory someone drops CSVs into.

THE SIGNAL

Three details point the same way: 30 files, a fixed schema, and a SQL-only team. Every one of them favours the simpler tool.

A logistics company needs data from a niche freight-tracking SaaS platform. Databricks has no managed connector for it. The platform has a documented REST API with OAuth, pagination and rate limits, and changes its API roughly twice a year.

The data team is two people. Their manager asks for the lowest-maintenance option that still lands governed tables.

What should they do?

A. Write a Python notebook that calls the REST API and handles OAuth, pagination and rate limits

B. Check Partner Connect for a partner connector that supports the platform

C. Use Auto Loader against the platform's API endpoint

D. Use Lakehouse Federation to query the platform in place

◆ PAUSE AND DECIDE

What should they do?

A Write the REST notebook

Viable, and the fallback if B fails. But an API that changes twice a year means recurring breakage for a two-person team. Highest maintenance, not lowest.

B Check Partner Connect first

Correct. Partner connectors exist precisely for sources Databricks has not covered, and a partner absorbs the API changes rather than your two engineers.

C Auto Loader against the API

Wrong source type. Auto Loader reads files in object storage; it cannot call a REST endpoint.

D Lakehouse Federation

Federation connects to databases as foreign catalogs, not to arbitrary SaaS REST APIs.

THE SIGNAL

'No managed connector exists' does not mean 'write it yourself'. Partner connectors sit between those two, and the guide names them.

> A retailer publishes clickstream events to an Apache Kafka topic at around 40,000 messages per second. They need those events in a bronze Delta table with a latency of under a minute.
>
> An engineer suggests configuring Auto Loader with file notification mode, since it is designed for high-volume ingestion.

How should the team respond?

A Agree; Auto Loader with file notification handles this volume comfortably

B Auto Loader reads files from object storage and cannot read Kafka; use Structured Streaming or a streaming connector

C Agree, but use directory listing mode instead for lower latency

D Use COPY INTO with a short schedule to poll the topic

◆ PAUSE AND DECIDE

How should the team respond?

A. Auto Loader handles this volume

Volume is not the objection. Auto Loader's source is a directory in object storage, and Kafka is not one.

B. Wrong source type; use Structured Streaming or a streaming connector

Correct. Message buses need a streaming source. Auto Loader's cloudFiles source reads files, not topics.

C. Directory listing for lower latency

Accepts the same false premise and then argues about a setting. There is no directory to list.

D. COPY INTO polling the topic

COPY INTO reads files from object storage. It has no mechanism for subscribing to a topic.

THE SIGNAL

The engineer's reasoning is about volume, which is a real strength of Auto Loader. It is irrelevant here, because the source type rules it out first.

A media company lands roughly 900,000 JSON files a day into cloud storage from twelve partner feeds. The partners add fields without notice, several times a year. The pipeline supports a live dashboard and must not stop.

The current process is a COPY INTO statement scheduled hourly. It has begun failing whenever a partner adds a field.

What should replace it?

A. COPY INTO with mergeSchema set to true in COPY_OPTIONS

B. Auto Loader with schemaEvolutionMode set to rescue

C. Auto Loader with schemaEvolutionMode set to failOnNewColumns

D. A managed connector for each of the twelve partners

◆ PAUSE AND DECIDE

What should replace it?

A COPY INTO with mergeSchema

Fixes the schema failure but leaves 900,000 files a day being discovered by directory listing. Solves one problem, keeps the bigger one.

B Auto Loader with rescue mode

Correct. Auto Loader handles the file volume, and rescue keeps the stream running while capturing new fields for later review.

C Auto Loader with failOnNewColumns

Right tool, wrong mode. It stops the pipeline on every new field, which is exactly what the must-not-stop requirement forbids.

D Twelve managed connectors

The partners deliver files to object storage. Managed connectors read SaaS applications and databases, not partner file drops.

THE SIGNAL

Two requirements, and A satisfies only one. Volume points to Auto Loader; 'must not stop' then picks the mode.

> An insurer needs policy changes from an on-premises SQL Server, including deletes, so that a fraud model sees the full change history. The database sits behind a corporate firewall with no public route.
>
> The platform team has an ExpressRoute link already in place and wants credentials governed rather than stored in code.

What is the right approach?

A. A JDBC notebook reading the table nightly, with credentials in a secret scope

B. A managed database connector using CDC, connecting over the existing ExpressRoute link

C. A query-based connector querying the table on a schedule

D. Lakehouse Federation, so no copy is made

◆ PAUSE AND DECIDE

What is the right approach?

A Nightly JDBC with secret scope

Governance is handled, but a nightly snapshot cannot see deletes or intermediate changes. It shows the state at query time only.

B Managed CDC connector over ExpressRoute

Correct. CDC captures every change including deletes, the connection is a governed UC securable, and ExpressRoute is a documented path for on-premises sources.

C Query-based connector on a schedule

Governed and simpler, but it has the same blind spot as A. Querying sees current state, not the changes between runs.

D Lakehouse Federation

Makes no copy, so there is no history to give the fraud model, and every query would traverse the firewall to a production database.

THE SIGNAL

'Including deletes' is the deciding phrase. Only change data capture sees a row that no longer exists.

A team is decommissioning an old analytics platform. They must load 4 TB of historical Parquet, sitting in about 1,100 files in cloud storage, into a Delta table. This is a one-off migration. Nothing further will ever be written to that location.

An architect proposes building an Auto Loader stream with a checkpoint and file notification mode.

What is the best response?

A. Agree; Auto Loader is the standard tool for ingesting files from cloud storage

B. A one-off load of 1,100 files needs neither a checkpoint nor notification; COPY INTO or a batch read is sufficient

C. Use Auto Loader but with directory listing rather than file notification

D. Split the files across twelve separate Auto Loader streams to parallelise

◆ PAUSE AND DECIDE

What is the best response?

A. Agree, Auto Loader is standard

Standard for ongoing ingestion. A checkpoint tracks progress across runs, and there will never be a second run.

B. COPY INTO or a batch read is sufficient

Correct. 1,100 files is well inside COPY INTO's range, and the incremental machinery has no future to serve.

C. Auto Loader with directory listing

Better than file notification here, but still builds a checkpoint and a stream for something that happens exactly once.

D. Twelve parallel streams

Twelve checkpoints instead of one, for a load that is not discovery-bound. It multiplies the over-engineering.

THE SIGNAL

4 TB sounds large, and it is not the criterion. 1,100 files is the number that matters, and the load happens once.

> A hospital's analysts need to join patient demographics, held in a clinical PostgreSQL system, against treatment outcomes already in the lakehouse. The clinical system is subject to a data protection agreement that prohibits copying patient records outside it.
>
> Query volumes would be low, a few dozen analyst queries a day.

What should the team use?

A. A managed PostgreSQL CDC connector into a governed bronze table

B. A JDBC notebook writing demographics into a restricted catalog

C. Lakehouse Federation, registering PostgreSQL as a foreign catalog

D. A query-based connector on a nightly schedule

◆ PAUSE AND DECIDE

What should the team use?

A Managed CDC connector into bronze

Copies the records. Governed and well built, and still a copy, which the agreement prohibits.

B JDBC into a restricted catalog

Restricting access to a copy does not stop it being a copy. Same violation, less automation.

C Lakehouse Federation as a foreign catalog

Correct. Federation queries in place and makes no copy, and low query volume means the load on the clinical system stays acceptable.

D Query-based connector nightly

Query-based describes how the source is read, not whether a copy results. It still writes records into destination tables.

THE SIGNAL

Three of the four options ingest. The constraint is not about governance quality; it is that a copy must not exist at all.

A team has an ingestion pipeline pulling from Salesforce, built and working in their development workspace using the managed connector. It was configured entirely through the UI.

They now need the same pipeline in test and production workspaces, with different destination catalogs per environment. It must be promoted through code review rather than rebuilt by hand.

How should they proceed?

A Repeat the UI configuration in each workspace, carefully matching the settings

B Define the pipeline in a Declarative Automation Bundle with per-target variables, and deploy with the CLI

C Export the pipeline as JSON and import it into each workspace manually

D Use one shared pipeline in development and grant the other workspaces access to its output

◆ PAUSE AND DECIDE

How should they proceed?

A Repeat the UI configuration

Works once, drifts immediately, and satisfies neither the code review requirement nor reproducibility.

B A bundle with per-target variables, deployed by CLI

Correct. Bundles are the documented way to deploy ingestion pipelines across dev, test and prod with source control and CI/CD.

C Export and import JSON manually

Closer, but the manual step is the problem. Nothing is under review, and the per-environment differences are still hand-edited.

D One shared pipeline in development

Production would depend on a development workspace. It also does not give test and prod their own destination catalogs.

THE SIGNAL

Ingestion pipelines are deployable assets. The Lakeflow Connect docs name Declarative Automation Bundles for exactly this.

Name the source. Take the most managed option that meets every stated requirement. **Let the stated requirements, not your instincts, force you down a layer.**

admin@sthithapragna.com

MODULE 09 · DOMAIN 3 - DATA TRANSFORMATION AND MODELING

PySpark and Spark SQL for ETL

Lazy evaluation, the core transformations, writing tables, views, and how to read code under exam conditions.

DOMAIN 3 of 7 EXAM WEIGHT 22% MODULE 09 of 36

admin@sthithapragna.com

PySpark and Spark SQL are **the same engine**. Both compile to the same plan, and neither is faster. The exam expects you to read both.

Two front ends, one engine

PYSPARK DATAFRAME API

Method chaining

- df.filter(…).groupBy(…).agg(…)
- Composable in Python; build chains programmatically
- Errors surface as Python exceptions
- Suits complex conditional logic

SPARK SQL

Declarative statements

- SELECT … FROM … WHERE … GROUP BY
- Familiar to analysts and BI teams
- Runs anywhere SQL runs, including warehouses
- Suits set-based transformations

THEY INTEROPERATE

You can move between them freely. createOrReplaceTempView exposes a DataFrame to SQL; spark.sql returns a DataFrame.

Transformations are **lazy**. Nothing runs until an action asks for a result. Spark spends that time optimising the whole chain at once.

Transformations and actions

TRANSFORMATIONS (LAZY)	ACTIONS (TRIGGER EXECUTION)
`select, filter, where`	show, display
`withColumn, withColumnRenamed`	collect, take, first
`join, union`	count
`groupBy with agg`	write, saveAsTable
`orderBy, distinct, dropDuplicates`	toPandas

ON THE EXAM

If a cell returns instantly and does nothing visible, you wrote a transformation. If it takes two minutes, you called an action.

Two consequences of laziness

- **Errors appear late.** A typo in a column name may not raise until the cell that calls an action.
- **Re-using a DataFrame re-computes it**, unless you cache. Calling three actions runs the chain three times.
- **Filters get pushed down**, so filtering after a join is often as fast as filtering before it.
- **Use explain to see the plan** Spark actually intends to run, rather than the one you wrote.

> ON THE EXAM
>
> An error surfacing in a cell you did not change usually means the mistake is in an earlier, lazy cell.

Narrow and wide transformations

- **Narrow:** each output partition depends on one input partition. select, filter, withColumn.
- **Wide:** output partitions depend on many input partitions. join, groupBy, distinct, orderBy.
- **Wide transformations shuffle**, moving data across the network between executors.
- **Shuffles are where jobs get slow.** Domain 6 is largely about diagnosing them.

> ON THE EXAM
>
> A stage boundary in the Spark UI is a shuffle. Counting the wide transformations in a chain predicts the number of stages.

The core transformations

OPERATION	WHAT IT DOES
`select("a", "b")`	Choose columns. Also renames, with alias.
`filter(...) / where(...)`	Keep rows matching a condition. Identical methods.
`withColumn("c", expr)`	Add a column, or replace one of the same name.
`withColumnRenamed("a", "b")`	Rename a column, leaving the data alone.
`drop("a")`	Remove a column.
`distinct() / dropDuplicates([...])`	Remove duplicate rows, optionally on given columns.

ON THE EXAM

filter and where are the same method with two names. Both exist so that SQL people and Python people each find one they expect.

Referring to columns

- **col("name")** refers to a column. Needed when you build expressions rather than name columns.
- **lit(5)** wraps a literal value, so Spark treats it as a constant rather than a column name.
- **expr("a + b * 2")** takes a SQL expression as a string, inside the DataFrame API.
- **A bare string is a column name.** Passing 5 where a column is expected is a common error.

> ON THE EXAM
>
> lit exists because Spark cannot tell a constant from a column name otherwise. It is the answer to 'why does this say column not found'.

withColumn or select?

WITHCOLUMN

Add or replace one column

- Keeps every existing column
- Replaces in place if the name already exists
- Readable for one or two additions
- Chaining many of them builds a large plan

SELECT

State the full output

- Only the columns you name survive
- Several derived columns in one step
- Clearer when the output shape changes a lot
- Easy to drop a column by forgetting it

WORTH KNOWING

withColumn with an existing name silently replaces that column. That is either exactly what you wanted or a quiet bug.

Write modes

MODE	BEHAVIOUR IF THE TARGET ALREADY EXISTS
`append`	Add these rows to the existing data
`overwrite`	Replace the existing data entirely
`error or errorifexists`	Throw an exception. This is the default.
`ignore`	Do nothing at all, silently

> ON THE EXAM
>
> The default is to fail. That is deliberate: a write that silently destroys or duplicates data would be a worse default.

saveAsTable and insertInto are not the same

- **saveAsTable matches columns by name.** The order in your DataFrame does not have to match the table.
- **insertInto matches columns by position.** Names are ignored entirely.
- **So insertInto with reordered columns writes data into the wrong columns**, silently, if the types happen to be compatible.
- **With overwrite mode, saveAsTable does not require the schema to match** the existing table.

> ON THE EXAM
>
> This is a genuine production hazard. Two string columns swapped by insertInto produce no error at all.

CTAS or create then insert?

CREATE TABLE AS SELECT

One statement

- Schema is inferred from the query result
- Table and data created together
- Cannot specify column comments or constraints inline
- Common for derived silver and gold tables

CREATE TABLE, THEN INSERT

Two statements

- Schema declared explicitly, with types you choose
- Constraints, comments and properties can be set
- Empty table exists before any data lands
- Better when the contract matters

THE TIEBREAKER

CTAS is quicker and takes the query's types. Declaring the schema is how you stop those types drifting when the query changes.

Three things called views

KIND	SCOPE	LIVES UNTIL
Temporary view	The SparkSession that created it	The session ends
Global temporary view	All sessions in the cluster, via global_temp	The cluster terminates
View (persisted)	Registered in Unity Catalog	It is dropped

ON THE EXAM

Only the third survives a cluster restart, and only the third can be granted to another user.

Views hold a query, not data

- **A view stores a query definition.** Every time you read it, the query runs again.
- **So a view is always current**, because it reads the underlying table at query time.
- **And a view is not faster.** An expensive query is expensive every time it is read.
- **A materialised view stores results**, and is the answer when the same expensive query runs repeatedly.

> ON THE EXAM
>
> View, table and materialised view differ in what is stored. Module 13 works through the choice properly.

Common table expressions

- **WITH name AS (SELECT ...)** names a subquery so the main statement reads clearly.
- **Several CTEs can be chained**, each referring to the ones before it.
- **A CTE is not stored and not reused.** Referencing it twice may compute it twice.
- **They exist for readability**, turning a nested pile of subqueries into named steps.

> ON THE EXAM
>
> A CTE is scoped to the single statement it appears in. It is not a temporary view and does not outlive the query.

DDL and DML

- **DDL defines structure:** CREATE, ALTER, DROP, TRUNCATE.
- **DML changes data:** INSERT, UPDATE, DELETE, MERGE.
- **CREATE OR REPLACE TABLE is DDL** and replaces the whole object, including its schema.
- **INSERT OVERWRITE is DML** and replaces the data while keeping the existing schema.

> WORTH DISTINGUISHING
>
> That last pair is the classic confusion. One can change the schema; the other cannot.

Reading code under exam conditions

- **Read the last line first.** What is this chain producing, and where is it going?
- **Check the order of filter and aggregate.** Filtering before or after a groupBy gives different answers.
- **Watch for a column being replaced** by a withColumn using an existing name.
- **Look for a missing action.** A chain that is never written or displayed does nothing at all.

> ON THE EXAM
>
> Most code questions differ from their distractors in one line. Find that line rather than reading all four options fully.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
The cell returned instantly and did nothing	A transformation. No action was called.
The error names a cell that was not changed	A lazy transformation earlier in the chain
Columns ended up in the wrong order	insertInto matches by position; saveAsTable by name
Another notebook cannot see the view	A temporary view is session-scoped
The view must survive a cluster restart	A persisted view in Unity Catalog
The same expensive query runs all day	A materialised view, not a view
The write failed because the table exists	Default mode is error. Choose append or overwrite.

ON THE EXAM

Seven lines. The third is the one that causes silent data corruption rather than an error.

> An engineer writes a notebook cell containing a chain: read the bronze table, filter to the last 30 days, add two derived columns, and drop three columns. The cell completes in under a second.
>
> She concludes the transformation is extremely fast and moves on. The next morning the silver table is empty.

What went wrong?

A The filter removed all rows, because the date comparison used the wrong type

B The chain is all transformations, so nothing executed and nothing was written

C The bronze table was empty when the cell ran

D Spark cached the result but did not persist it to the table

◆ PAUSE AND DECIDE

What went wrong?

A The filter removed all rows

A filter that matches nothing still takes time to run. Sub-second completion means the filter never executed at all.

B All transformations, so nothing ran or was written

Correct. read, filter, withColumn and drop are all lazy. With no action, Spark built a plan and did nothing else.

C Bronze was empty at the time

Possible in principle, but it does not explain the sub-second completion, and the chain never wrote anywhere regardless.

D Spark cached but did not persist

cache is itself lazy, and nothing here requested caching. Inventing a mechanism that was never invoked.

THE SIGNAL

'Completes in under a second' is the tell. Real work on a real table takes real time, so instant completion means no work happened.

> A silver table has columns in the order: customer_id, given_name, family_name, city. A daily job builds a DataFrame with columns in the order customer_id, family_name, given_name, city and writes it with insertInto.
>
> The job succeeds every night with no errors. Weeks later, support reports that customer names are being displayed back to front.

What is the cause?

A insertInto matches columns by position, so the two name columns were swapped

B The source system began sending names in the wrong order

C The table schema was altered to reorder the columns

D insertInto requires an explicit column list, and omitting it randomises the order

◆ PAUSE AND DECIDE

What is the cause?

A insertInto matches by position

Correct. Unlike saveAsTable, insertInto ignores column names entirely. Both columns are strings, so nothing errors and the values land swapped.

B The source changed the order

Blames the source, but the scenario tells you the DataFrame's column order directly. The mismatch is stated, not hypothetical.

C The table was altered

Would produce the same symptom, but the scenario already supplies a sufficient cause. Do not invent a second one.

D Omitting a column list randomises order

Fabricated. Positional matching is deterministic, which is exactly why it fails consistently rather than randomly.

THE SIGNAL

No error, for weeks, with a real mismatch stated in the setup. Silent wrongness of that kind points at positional matching.

> An analyst builds a complex filtered and joined dataset in a notebook and registers it with createOrReplaceTempView. She tells a colleague to query it from his own notebook.
>
> He reports that the view does not exist. She can see it perfectly well from hers. The next morning she cannot see it either.

What is happening, and what should they use?

A. A permissions problem; he needs SELECT granted on the view

B. A temporary view is scoped to one SparkSession and dies with it; they need a persisted view or a table

C. He needs to query it as global_temp.view_name

D. The view must be refreshed each morning with REFRESH TABLE

◆ PAUSE AND DECIDE

What is happening, and what should they use?

A Grant him SELECT

There is nothing to grant. A temporary view is not a Unity Catalog securable, so no permission would make it visible to him.

B Session-scoped; use a persisted view or table

Correct. It explains both symptoms at once: invisible to his session, and gone from hers after a restart.

C Query it as global_temp.view_name

That is the syntax for a global temporary view, which this is not. It would also still die with the cluster.

D REFRESH TABLE each morning

Refresh updates cached metadata for something that exists. It cannot recreate an object that no longer exists.

THE SIGNAL

Two symptoms, one cause. Any answer explaining only the colleague's problem leaves the overnight disappearance unexplained.

A gold table is defined with CREATE TABLE AS SELECT over a silver table. Six months later a colleague changes the silver query so that a total_value column, previously computed as an integer, is now computed with a decimal division.

The gold table has been rebuilt nightly with CREATE OR REPLACE TABLE. A downstream BI report that summed total_value now shows different numbers.

What is the underlying design weakness?

A CREATE OR REPLACE TABLE should have been INSERT OVERWRITE

B CTAS takes its column types from the query, so a change to the query silently changed the table's schema

C The BI tool cached the old schema and needs refreshing

D Gold tables should never be rebuilt from silver

◆ PAUSE AND DECIDE

What is the underlying design weakness?

A Should have used INSERT OVERWRITE

Close, and worth understanding. INSERT OVERWRITE keeps the existing schema, so it would have failed loudly instead of changing types silently.

B CTAS inherits types from the query

Correct. The schema was never declared, so it is whatever the query produces, and changing the query changed the contract.

C The BI tool cached the schema

The numbers changed because the values changed. A stale cache would show old numbers, not new ones.

D Gold should not be rebuilt from silver

Rebuilding gold from silver is the normal medallion pattern. Nothing about it caused this.

THE SIGNAL

A and B describe the same failure from two angles. B names the weakness; A names the mechanism that would have caught it.

Nothing runs until an action asks for a result. **Everything else in this domain is built on that.**

MODULE 10 · DOMAIN 3 - DATA TRANSFORMATION AND MODELING

Cleaning bronze into silver

Handling nulls, standardising types, ANSI mode, malformed records, and writing silver so it can be rebuilt safely.

DOMAIN 3 of 7 EXAM WEIGHT 22% MODULE 10 of 36

admin@sthithapragna.com

Cleaning is not tidying. It is **deciding what a bad value means** and encoding that decision, so nobody downstream has to guess.

The bronze to silver pass

01

Read bronze

As it arrived. Nothing was cleaned on the way in.

02

Standardise types

Strings become dates, decimals, integers.

03

Handle nulls

Drop, fill, or keep. Each is a decision.

04

Normalise values

Trim, case, codes, units.

05

Deduplicate and write

One row per real-world thing, into silver.

WHY THE ORDER MATTERS

Every step is repeatable because bronze still holds the original. That is the whole reason bronze is append-only.

From **Databricks Runtime 17.0**, ANSI mode is on by default. A failed cast now **throws an error** rather than returning null.

The same cast, two behaviours

EXPRESSION	ANSI ON (DEFAULT FROM DBR 17)	ANSI OFF (LEGACY)
`CAST('a' AS INT)`	CAST_INVALID_INPUT error	null
`CAST(2147483648L AS INT)`	CAST_OVERFLOW error	-2147483648, wrapped
`try_cast('a' AS INT)`	null	null

ON THE EXAM

try_cast behaves identically in both modes. That is what makes it the safe choice when null is genuinely the meaning you want.

How to think about the change

- **Each new error is a row that was previously being corrupted silently.** The failure is information.
- **Disabling ANSI mode to make errors go away** restores the silent corruption. It is the wrong instinct.
- **Switch to try_cast where null is the correct meaning**, per expression rather than globally.
- **Widen the type where overflow is the issue**, for example INT to BIGINT.

> ON THE EXAM
>
> spark.sql.storeAssignmentPolicy also defaults to ANSI, and is independent. It rejects invalid casts during table insertion.

A null is not zero and not an empty string. It means **unknown**, and Spark's functions treat it that way consistently.

Working with nulls

OPERATION	WHAT IT DOES
`isNull() / isNotNull()`	Test for null. Never use = NULL, which is always null.
`df.na.drop() / dropna()`	Remove rows containing nulls
`df.na.fill(0) / fillna(0)`	Replace nulls with a value
`coalesce(a, b, c)`	First non-null of several columns
`nullif(a, b)`	Return null when two values are equal
`ifnull(a, b) / nvl(a, b)`	Return b when a is null

ON THE EXAM

Equality never matches null. WHERE col = NULL returns no rows, always, and it is a silent bug rather than an error.

dropna is more configurable than people expect

- **how='any'** drops a row if any column is null. This is the default.
- **how='all'** drops a row only if every column is null.
- **subset=['a','b']** considers only those columns, ignoring nulls elsewhere.
- **thresh=3** keeps rows with at least that many non-null values, and overrides how.

> ON THE EXAM
>
> The default drops a row if any single column is null. On a wide table with optional fields, that can discard almost everything.

How nulls change your numbers

- **Aggregates skip nulls.** avg of 10, 20 and null is 15, not 10.
- **count(*) counts rows. count(col) counts non-null values.** They differ whenever nulls exist.
- **An inner join drops rows with a null key**, because null never equals null.
- **So filling nulls with zero changes averages**, while leaving them changes counts. Neither is automatically right.

> ON THE EXAM
>
> This is the most commonly missed behaviour in the whole domain. Decide what the null means before choosing.

Standardising types

- **Dates:** to_date and to_timestamp with an explicit format string. Never rely on the default parse.
- **Money: use DECIMAL with an explicit precision and scale**, not DOUBLE. Floating point loses pennies.
- **Booleans arriving as text** may be Y, yes, true, 1 or T. Map them explicitly.
- **Watch for placeholder values:** 1900-01-01, -1, 'N/A', 9999. Convert these to null with nullif.

> ON THE EXAM
>
> A cast that succeeds is not a cast that is correct. A date parsed with the wrong format can produce a valid but wrong date.

Standardising strings

- **trim** removes leading and trailing whitespace, which otherwise breaks joins and grouping.
- **upper or lower** makes case consistent, so 'UK' and 'uk' stop being two countries.
- **regexp_replace** strips or rewrites patterns, such as removing punctuation from phone numbers.
- **An empty string is not null.** Convert it deliberately if that is what you mean.

> ON THE EXAM
>
> Trailing whitespace is invisible in output and fatal to a join. It is worth trimming key columns as a matter of habit.

Malformed records at read time

MODE	BEHAVIOUR
PERMISSIVE	Sets unparseable fields to null and keeps the row. The default.
DROPMALFORMED	Silently discards rows that cannot be parsed
FAILFAST	Throws an exception on the first malformed record

ON THE EXAM

With PERMISSIVE, set columnNameOfCorruptRecord to capture the raw text of bad rows rather than losing it. badRecordsPath writes bad records to files.

How should silver be written?

OVERWRITE

Rebuild the whole table

- Simple, and always consistent with bronze
- Re-runs are naturally idempotent
- Cost grows with total history, not new data
- Fine while the table is small

MERGE INTO

Apply changes

- Updates matched rows, inserts new ones
- Cost tracks what changed
- Handles late-arriving corrections properly
- Needs a reliable key to match on

THE TIEBREAKER

Append is the third option, and the dangerous one. It is only safe if you can guarantee the same row never arrives twice.

Making silver rebuildable

- **Aim for idempotency:** running the job twice should leave the table in the same state as running it once.
- **MERGE on a business key is idempotent.** The second run matches and updates rather than duplicating.
- **Append is not idempotent** unless something upstream guarantees each row arrives once.
- **Carry ingestion metadata through:** source file, load timestamp. It makes a bad batch traceable.

> ON THE EXAM
>
> The test to apply: if this job runs twice by accident, does anything break? If yes, the design is fragile.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Casts started failing after a runtime upgrade	ANSI mode, on by default from DBR 17
A bad value should become null, not an error	try_cast, which behaves the same in both modes
The average changed after filling nulls	Aggregates skip nulls; zeros participate
count(*) and count(col) disagree	That column contains nulls
Rows vanished after an inner join	Null keys never match, not even each other
Almost every row was dropped by dropna	Default is how='any'. Use subset.
Malformed rows became rows of nulls	PERMISSIVE mode, the read default
The job must be safe to re-run	MERGE on a key, or overwrite. Not append.

ON THE EXAM

Eight lines. The first is the newest and the third is the most commonly missed.

> A silver pipeline has run nightly for two years without incident. It casts a quantity column from string to integer. After the platform team upgrades the cluster from Databricks Runtime 15.4 LTS to 17.3 LTS, the job fails with CAST_INVALID_INPUT.
>
> An engineer proposes setting spark.sql.ansi.enabled to false so the job runs again tonight.

What should the team do?

A. Set spark.sql.ansi.enabled to false; the pipeline worked for two years and the data has not changed

B. Recognise that these rows were previously becoming silent nulls, and decide deliberately: try_cast where null is correct, or reject the rows

C. Roll back to Databricks Runtime 15.4 LTS permanently

D. Change the column to DOUBLE, which accepts a wider range of inputs

◆ PAUSE AND DECIDE

What should the team do?

A Disable ANSI mode

Makes the error go away and restores the silent corruption underneath it. Defensible as a temporary unblock, wrong as the answer.

B Treat the errors as information and decide per expression

Correct. The rows were always bad; ANSI mode simply revealed them. try_cast where null is the right meaning, rejection where it is not.

C Roll back the runtime permanently

Freezes the platform to preserve a bug, and forfeits every later improvement and security fix.

D Use DOUBLE instead

Misreads the error. CAST_INVALID_INPUT means the text is not a number at all, so a wider numeric type parses it no better.

THE SIGNAL

'Ran for two years without incident' is the false comfort. It ran for two years turning bad values into nulls without telling anyone.

> A silver table has a delivery_rating column, populated only when a customer chose to leave one. About 40% of rows have a null there. A dashboard reports the average rating as 4.2.
>
> An engineer, tidying the pipeline, adds fillna(0) across all numeric columns 'to avoid nulls downstream'. The dashboard now reports 2.5, and nobody can explain the drop.

What happened?

A fillna corrupted the ratings column by overwriting real values

B Averages skip nulls, so the 40% were previously excluded; filling them with 0 made them count as zero ratings

C The dashboard is caching the old value and needs refreshing

D fillna changed the column type, causing an incorrect calculation

◆ PAUSE AND DECIDE

What happened?

A fillna overwrote real values

fillna only replaces nulls. Existing ratings are untouched, which is why the answer lies in what the nulls became.

B Averages skip nulls; zeros now participate

Correct. 40% of rows moved from being excluded from the average to being counted as ratings of zero, which drags the mean down.

C The dashboard is caching

A cache would show the old number, not a new and lower one. The value changed because the data changed.

D fillna changed the column type

Filling an integer column with 0 does not change its type, and a type change would not produce a plausible but wrong average.

THE SIGNAL

A null rating means 'did not rate'. A zero means 'rated it zero'. The engineer changed the meaning of 40% of the data.

> A bronze customer table has 40 columns. Twelve are genuinely optional: a second address line, a middle name, a mobile number and similar. Two are essential: customer_id and email.
>
> A cleaning step calls df.dropna() with no arguments. The silver table ends up with 6% of the expected rows.

What should the call be instead?

A df.dropna(how='all')

B df.dropna(subset=['customer_id', 'email'])

C df.dropna(thresh=40)

D df.fillna('') followed by df.dropna()

◆ PAUSE AND DECIDE

What should the call be instead?

A how='all'

Only drops rows where every one of the 40 columns is null. That is almost no rows, so bad records with a missing customer_id survive.

B subset=['customer_id', 'email']

Correct. It drops rows missing something essential and ignores nulls in the twelve genuinely optional columns.

C thresh=40

Requires all 40 columns to be non-null, which is stricter than the default and would keep even fewer rows.

D fillna('') then dropna()

Keeps every row by making optional nulls into empty strings, and also disguises missing customer_ids as empty strings.

THE SIGNAL

The default is how='any' across all 40 columns. With twelve optional fields, almost every real customer has at least one null.

> A silver table is built by reading the last 24 hours of bronze, cleaning it, and appending to silver. Late one evening a job fails halfway and is retried by the on-call engineer. It succeeds.
>
> The next day, revenue figures are roughly 40% too high for that date. The bronze table is correct.

What is the design problem?

A. The retry duplicated rows because appending is not idempotent

B. The cleaning logic double-counted currency conversions

C. Bronze contained duplicates that the cleaning step failed to remove

D. The failed run left an uncommitted transaction that was counted twice

◆ PAUSE AND DECIDE

What is the design problem?

A Append is not idempotent, so the retry duplicated rows

Correct. The first run appended part of the window before failing; the retry appended the whole window again. The overlap is duplicated.

B Currency conversion double-counted

Would affect every day, not just the one that was retried. The symptom is tied to the retry.

C Bronze contained duplicates

The scenario states bronze is correct. Do not argue with a fact the question supplies.

D An uncommitted transaction was counted twice

Delta writes are atomic. An incomplete write commits nothing, so there is no partial transaction to count.

THE SIGNAL

Delta's atomicity means the failed run's committed batches survive. Atomic per write is not the same as atomic per job.

Every cleaning step changes a meaning. **Filling a null, dropping a row and casting a value are all decisions**, and the exam tests whether you know which one you made.

MODULE 11 · DOMAIN 3 - DATA TRANSFORMATION AND MODELING

Joins and set operations

Join types, multiple keys, broadcast against sort-merge, and why union does not do what SQL people expect.

DOMAIN 3 of 7 EXAM WEIGHT 22% MODULE 11 of 36

admin@sthithapragna.com

A join question is almost never about syntax. It is about **which rows survive**, and **how many of each**.

The join types

TYPE	WHICH ROWS SURVIVE
`inner`	Only rows with a match on both sides. The default.
`left / left_outer`	All left rows. Unmatched right columns become null.
`right / right_outer`	All right rows. Unmatched left columns become null.
`full / full_outer`	All rows from both sides, nulls where unmatched.
`left_semi`	Left rows that have a match. No right columns are added.
`left_anti`	Left rows that have no match. No right columns are added.
`cross`	Every left row paired with every right row.

ON THE EXAM

Inner is the default when you do not specify how. That default silently discards unmatched rows.

Semi and anti joins are filters

- **They add no columns.** The output has exactly the left side's schema.
- **left_semi answers 'which of these exist over there?'** Like an IN or EXISTS clause.
- **left_anti answers 'which of these are missing?'** Like NOT IN or NOT EXISTS.
- **They never multiply rows**, even when the right side has duplicate keys. An inner join would.

> ON THE EXAM
>
> left_anti is the clean way to find orphaned records, and it is far cheaper than a left join followed by a null filter.

Joining on keys

- **on="customer_id"** joins on a shared column name and keeps only one copy of it.
- **on=["customer_id", "region"]** joins on multiple keys, all of which must match.
- **on=(df.id == other.id)** uses an expression, and keeps both columns, giving you two columns of the same name.
- **Use the string form when names match.** It avoids the ambiguous column problem entirely.

> ON THE EXAM
>
> Two columns with the same name produce AMBIGUOUS_REFERENCE later, often in a cell far from the join that caused it.

An inner join **silently drops every row with a null key**, because null never equals null. Not even another null.

Joins can multiply rows

- **If a key appears twice on the right, each matching left row is duplicated.**
- **Two on the left and three on the right gives six rows** for that key. This is fan-out.
- **Check for uniqueness before joining:** count rows against count distinct on the key.
- **Deduplicate the lookup side first**, or use left_semi if you only need existence.

> ON THE EXAM
>
> A row count that grows after an inner join is the classic symptom. Inner joins are usually expected to shrink the data, not expand it.

Spark's choice is between **moving the data to the join** or **moving the join to the data**. That choice dominates performance.

Broadcast or sort-merge?

BROADCAST HASH JOIN

Copy the small side everywhere

- No shuffle of the large table at all
- Dramatically faster when one side is small
- The small side is collected on the driver first
- Fails if the small side is not actually small

SORT-MERGE JOIN

Shuffle both sides

- Works at any size, on both sides
- Both sides shuffled and sorted by key
- The default when broadcast is not viable
- Cost dominated by the shuffle

THE TIEBREAKER

Broadcast is the classic answer for a large fact table joined to a small dimension table. That pattern is worth recognising on sight.

Two thresholds, not one

- **spark.sql.autoBroadcastJoinThreshold** is the static, plan-time threshold. Default 10 MB.
- **spark.databricks.adaptive.autoBroadcastJoinThreshold** is the AQE runtime threshold on Databricks. Default 30 MB.
- **Set either to -1 to disable** automatic broadcasting on that path.
- **Confusing the two is the common mistake.** Note the different namespaces.

> ON THE EXAM
>
> The exam guide names spark.sql.autoBroadcastJoinThreshold specifically. Module 15 revisits it as a tuning parameter.

When broadcasting goes wrong

- **The driver collects the small side first.** An oversized broadcast is a driver out-of-memory error, not an executor one.
- **Spark cannot broadcast a relation larger than 8 GB**, whatever you configure.
- **Size estimates can be wrong**, especially after filters and transformations.
- **Broadcast only supports some join types:** inner, cross, left outer, right outer, left semi and left anti.

> ON THE EXAM
>
> A lookup table that quietly grows past the threshold stops being broadcast, and the job gets slower for no visible reason.

Forcing the choice

- **broadcast(df) in PySpark**, or a /*+ BROADCAST(t) */ hint in SQL, forces the planner's hand.
- **A hint overrides the size threshold**, but not the 8 GB hard limit or the join type restrictions.
- **Hint both sides and Spark broadcasts the smaller**, based on statistics.
- **ANALYZE TABLE ... COMPUTE STATISTICS** improves the planner's estimates, which often fixes the problem without a hint.

> ON THE EXAM
>
> When hints collide, the priority is fixed: BROADCAST beats MERGE, which beats SHUFFLE_HASH, which beats SHUFFLE_REPLICATE_NL.

Adaptive query execution

- **AQE re-plans the query mid-flight**, using real sizes measured after a shuffle rather than estimates.
- **It can switch a sort-merge join to a broadcast join** at runtime, once it sees a side is actually small.
- **It also coalesces small shuffle partitions** and splits skewed ones.
- **On by default on Databricks** since Databricks Runtime 7.3.

> ON THE EXAM
>
> AQE is why a plan you saw in EXPLAIN may not be the plan that ran. Check the Spark UI for what actually happened.

In SQL, **UNION** removes duplicates. In the DataFrame API, **union() does not** — and neither does unionAll.

Four ways to combine rows

OPERATION	DEDUPLICATES?	MATCHES COLUMNS BY
SQL UNION	Yes	Position
SQL UNION ALL	No	Position
DataFrame union()	No	Position
DataFrame unionAll()	No — alias of union()	Position
DataFrame unionByName()	No	Name

ON THE EXAM

Only SQL UNION deduplicates. For the DataFrame equivalent, follow union() with distinct().

unionByName

- **Matches columns by name**, so the two DataFrames may have them in different orders.
- **allowMissingColumns=True** permits different sets of columns, filling the gaps with null.
- **Use it when combining sources** that describe the same thing with columns in different orders.
- **Plain union with mismatched order silently interleaves values**, exactly like insertInto does.

> ON THE EXAM
>
> Same hazard, second appearance. Positional matching is fast and unforgiving; name matching is safe and explicit.

The other set operations

OPERATION	RESULT
INTERSECT	Rows appearing in both, deduplicated
INTERSECT ALL	Rows appearing in both, duplicates retained
EXCEPT / MINUS	Rows in the first but not the second, deduplicated
EXCEPT ALL	As above, duplicates retained

ON THE EXAM

EXCEPT compares whole rows. left_anti compares only join keys. That difference decides which one a scenario needs.

Cross joins

- **Every row on the left paired with every row on the right.** 10,000 by 10,000 is 100 million rows.
- **Legitimate uses exist:** generating a date spine, or every product-store combination.
- **An accidental cross join** is usually a join condition that was forgotten or never matched.
- **The symptom is a job that never finishes**, or a row count with far too many zeros.

> ON THE EXAM
>
> Spark requires the intent to be explicit, via crossJoin or the cross join type, so that an accident is harder to commit.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Rows vanished after joining	Inner join with unmatched or null keys
Row count grew after an inner join	Fan-out. The key is not unique on one side.
Which of these have no matching record	left_anti join
Large fact table joined to a small lookup	Broadcast join
The job slowed down with no code change	A lookup table grew past 10 MB
Driver out of memory during a join	An oversized broadcast
Duplicates appeared after union in PySpark	union() does not deduplicate. Add distinct().
Columns are in a different order	unionByName, not union

ON THE EXAM

Eight lines. The last two are where SQL habits mislead you most.

A silver orders table has 2.4 million rows. It is joined to a customer_addresses table on customer_id with an inner join, to attach the city to each order.

The result has 3.1 million rows. Total revenue calculated from the joined table is about 29% higher than the same figure calculated from orders alone.

What is the cause?

A The join should have been a left join, which would have kept the row count correct

B Some customers have more than one address, so those orders are duplicated by the join

C The revenue column is being summed twice because it appears on both sides

D Null customer_ids in orders were matched to multiple address rows

◆ PAUSE AND DECIDE

What is the cause?

A. Should have been a left join

A left join would keep unmatched orders, which addresses shrinkage. This is growth. It would produce even more rows, not fewer.

B. Customers with multiple addresses duplicate their orders

Correct. Fan-out. A customer with three addresses turns one order into three rows, and revenue is counted three times.

C. Revenue appears on both sides

An addresses table would not carry revenue. And a duplicated column would raise an ambiguous reference, not inflate a sum.

D. Null customer_ids matched multiple rows

Backwards. Null keys never match anything, so those rows would be dropped by an inner join, reducing the count.

THE SIGNAL

An inner join that grows the data is the signature. Inner joins discard unmatched rows, so growth can only come from duplication.

A team is migrating a SQL pipeline to PySpark. The original statement combined three regional tables with UNION. The engineer translates it faithfully as df_uk.union(df_fr).union(df_de).

The PySpark output has more rows than the SQL version. Investigation shows that a small number of customers trade in two regions and appear in two source tables.

What is the fix?

A Use unionAll instead, which handles deduplication correctly

B DataFrame union does not deduplicate; append .distinct() to match SQL UNION

C Use unionByName, which removes duplicate rows while matching on column names

D The source tables must be deduplicated individually before the union

◆ PAUSE AND DECIDE

What is the fix?

A. Use unionAll instead

unionAll is an alias for union. Identical behaviour, so this changes nothing at all.

B. union does not deduplicate; add .distinct()

Correct. The docs state it performs the union with no automatic deduplication; distinct() is the documented way to get SQL UNION behaviour.

C. unionByName removes duplicates

unionByName matches columns by name. It does not deduplicate either, so duplicates across sources survive.

D. Deduplicate each source first

The duplicates are across sources, not within them. Each table is internally correct; the same customer appears in two of them.

THE SIGNAL

The engineer translated the word faithfully and the behaviour changed. UNION and union() are false friends across the two APIs.

A nightly job joins a 900 GB transactions table to a product_catalogue lookup, using an inner join on product_id. It has completed in about 14 minutes for a year.

Over the last month it has slowed to nearly two hours. No code has changed. The Spark UI now shows a large shuffle on both sides where previously there was none.

What is the most likely explanation?

A. The transactions table has grown, so the shuffle takes longer

B. The product catalogue grew past the broadcast threshold, so Spark switched to a sort-merge join

C. Adaptive query execution was disabled by a platform change

D. The product_id column developed nulls, forcing a different join strategy

◆ PAUSE AND DECIDE

What is the most likely explanation?

A. The transactions table grew

Gradual growth does not create a shuffle that was not there before. The shuffle is new, which points at a change of strategy.

B. The lookup grew past the threshold, so Spark switched to sort-merge

Correct. Below 10 MB it was broadcast with no shuffle. Once past the threshold, broadcast stops and both sides shuffle.

C. AQE was disabled

Possible, and it would not by itself turn a broadcast into a shuffle if the lookup were still small enough to broadcast at plan time.

D. Nulls appeared in product_id

Nulls affect which rows match, not which strategy is chosen. They would change the row count, not introduce a shuffle.

THE SIGNAL

'A shuffle where previously there was none' is the deciding observation. That is a strategy change, not a volume change.

> A finance team needs a list of customers who have never placed an order, so they can be removed from a mailing list. The customers table has 8 million rows; the orders table has 200 million.
>
> An engineer writes a left join from customers to orders, then filters where order_id is null, then selects distinct customer_id.

What is the better approach?

A The approach is correct and cannot be improved

B Use a left_anti join on customer_id, which returns exactly the unmatched customers

C Use EXCEPT between the customers table and the orders table

D Use an inner join and filter for rows where order_id is null

◆ PAUSE AND DECIDE

What is the better approach?

A — It is correct and cannot be improved

It is correct, which makes it tempting. But it materialises one row per customer per order before discarding almost all of them, then deduplicates.

B — left_anti join on customer_id

Correct. It returns unmatched left rows directly, adds no columns, never fans out, and needs no distinct afterwards.

C — EXCEPT between the two tables

EXCEPT compares whole rows, and these two tables have different schemas. It is not a key-based comparison.

D — Inner join then filter for null order_id

An inner join only keeps matched rows, so no row can have a null order_id. The result is always empty.

THE SIGNAL

A left join plus a null filter plus distinct is the long way round. left_anti expresses the same intent in one operation.

Ask which rows survive and how many of each. Then ask whether Spark is **moving the data or moving the join**.

MODULE 12 · DOMAIN 3 - DATA TRANSFORMATION AND MODELING

Reshaping, deduplication and aggregation

Columns, rows and filters; distinct against dropDuplicates; and the aggregate functions the exam names.

DOMAIN 3 of 7 EXAM WEIGHT 22% MODULE 12 of 36

admin@sthithapragna.com

Every operation in this module either changes the **columns**, the **rows**, or the **grain**. Knowing which one tells you what the output looks like.

What each operation changes

OPERATION	COLUMNS	ROWS
`withColumn, drop, withColumnRenamed`	Changed	Unchanged
`select`	Changed	Unchanged
`filter, where`	Unchanged	Fewer
`distinct, dropDuplicates`	Unchanged	Fewer
`explode`	Changed	More, or fewer
`groupBy with agg`	Changed	Fewer, one per group
`join`	Changed	More or fewer
`pivot`	Changed	Fewer

ON THE EXAM

Note explode: it usually increases the row count, but it silently removes rows whose array is empty.

Columns: adding, dropping, renaming

- **withColumn("name", expr)** adds a column, or replaces one of the same name.
- **drop("a", "b")** removes columns, and does not error if a name is absent.
- **withColumnRenamed("old", "new")** renames one column.
- **select with alias** renames several at once, and drops anything you did not list.

> ON THE EXAM
>
> Chaining many withColumn calls builds a large plan. For several derived columns at once, one select is usually cleaner.

Splitting a column

- **split(col, delimiter)** returns an **array**, not several columns.
- **Take a position with an index:** split(name, ' ')[0] gives the first element.
- **Or explode it** if each element should become its own row.
- **Indexing past the end gives null**, not an error. A missing middle name simply becomes null.

> ON THE EXAM
>
> Splitting is a two-step idea: produce an array, then decide whether you want positions as columns or elements as rows.

Filters

- **Multiple conditions need brackets:** (col('a') > 5) & (col('b') == 'x'). Python precedence bites otherwise.
- **Use & and | and ~**, not the Python words and, or, not.
- **isin([...]) for a list**, between(a, b) for a range, like and rlike for patterns.
- **Filtering on a null never matches.** Use isNull, and remember filters silently drop null rows.

> ON THE EXAM
>
> A filter on a column containing nulls quietly discards those rows, because a comparison with null is null, not false.

Exploding arrays

- **explode** gives one row per element, repeating the other columns.
- **explode drops rows whose array is empty or null.** Silently.
- **explode_outer keeps them**, with null in the exploded column.
- **posexplode also returns the position**, which matters when the order of elements is meaningful.

> ON THE EXAM
>
> Explode changes the grain. After it, one row no longer means what it meant before, and any existing aggregate is now wrong.

Pivot

- **groupBy(...).pivot("column").agg(...)** turns distinct values of a column into columns.
- **Supply the list of values** if you know it. Otherwise Spark must scan the data to find them first.
- **Without that list it is an extra job**, and a column with many distinct values produces an unusable table.
- **The reverse is stack**, turning columns back into rows.

> ON THE EXAM
>
> Pivot is a reporting shape, so it usually belongs in gold rather than silver. Wide tables are awkward to maintain.

distinct compares whole rows. **dropDuplicates** can compare only the columns you name. That difference decides most deduplication questions.

distinct or dropDuplicates?

DISTINCT()

Whole rows

- Every column must match for two rows to be duplicates
- Cannot be told to ignore a column
- Fails to dedupe when a timestamp or file name differs
- Same as dropDuplicates() with no arguments

DROPDUPLICATES(SUBSET)

Named columns

- Duplicates defined by the columns you name
- Ignores differences in every other column
- Works when rows carry ingestion metadata
- Keeps one row per key — but which one?

THE TIEBREAKER

That last question is the trap, and it gets its own slide.

dropDuplicates with a subset keeps **an arbitrary row** from each group. Not the newest, not the first. Whichever one Spark happened to see.

Deterministic deduplication

STEP	WHAT IT DOES
`Window.partitionBy("order_id")`	Group the rows that are candidates for each other
`.orderBy(col("updated_at").desc())`	Define which one wins. Newest first, here.
`row_number().over(window)`	Number the rows within each group, 1 for the winner
`filter(col("rn") == 1)`	Keep only the winner

ON THE EXAM

Four lines, and the result is reproducible. This is the pattern to reach for whenever which duplicate you keep matters.

Two other ways to deduplicate

- **MERGE INTO on the key** deduplicates on write. A second arrival updates rather than inserting.
- **dropDuplicates on a stream needs a watermark**, or state grows forever waiting for duplicates.
- **Aggregating can deduplicate implicitly**, since grouping by the key collapses duplicates by definition.
- **Deduplicating late is expensive.** Silver is the right place, not gold.

> ON THE EXAM
>
> MERGE is the idempotency answer from module 10 and the deduplication answer here. It is the same mechanism.

Aggregation changes the grain. One row stops meaning one transaction and starts meaning **one group**.

The aggregate functions the exam names

FUNCTION	WHAT IT RETURNS
`count("*")`	Number of rows in the group
`count("col")`	Number of non-null values in that column
`count_distinct("col")`	Exact number of distinct values. Expensive at scale.
`approx_count_distinct("col")`	Estimated distinct count. Far cheaper.
`mean or avg`	Arithmetic mean, ignoring nulls
`sum, min, max`	As expected, all ignoring nulls

ON THE EXAM

The exam guide names count, approximate count distinct, mean and summary specifically. Learn those four properly.

approx_count_distinct

- **Estimates the distinct count** using a sketch, rather than tracking every value seen.
- **The rsd parameter is the maximum relative standard deviation.** Default 0.05, meaning about 5%.
- **Below rsd 0.01 it is more efficient to use count_distinct** instead. The docs say so directly.
- **Use it when the trend matters more than the exact figure**, such as daily active users on a dashboard.

> ON THE EXAM
>
> An estimate within 5% is usually fine for a trend line and completely unacceptable for a regulatory count. The requirement decides.

describe and summary

- **describe()** gives count, mean, standard deviation, min and max for numeric columns.
- **summary()** gives those plus the 25th, 50th and 75th percentiles by default.
- **summary accepts arguments**, so you can request specific statistics.
- **Both are for exploration**, giving a quick profile of unfamiliar data.

> ON THE EXAM
>
> summary is the richer of the two. Its quartiles are what reveal skew, which describe's mean and max cannot show you.

groupBy and agg

- **groupBy("a").agg(sum("x").alias("total"), count("*").alias("n"))** is the general shape.
- **Always alias your aggregates.** Otherwise you get column names like sum(x).
- **Group by several columns** by passing several names.
- **Only grouped or aggregated columns can be selected.** Anything else is ambiguous, and Spark says so.

> ON THE EXAM
>
> Aliasing is not cosmetic. A downstream query referring to a column called sum(x) is fragile and awkward to write.

Filter before or after aggregating?

FILTER FIRST

WHERE, in SQL terms

- Removes rows before the calculation
- The aggregate is computed on fewer rows
- Changes the value of every aggregate
- Also faster, since less data is grouped

FILTER AFTER

HAVING, in SQL terms

- Removes groups after the calculation
- The aggregate is computed on everything
- Changes which groups appear, not their values
- Needed when the condition is about the aggregate

THE TIEBREAKER

'Customers whose orders over £100 total more than £1000' filters first. 'Customers whose total exceeds £1000' filters after.

When you need the detail as well

- **groupBy collapses rows.** You get one row per group and the detail is gone.
- **A window function keeps every row** and adds the aggregate alongside it.
- **sum("x").over(Window.partitionBy("customer"))** gives each row its customer's total.
- **Use a window when you need row and group together**, such as each order's share of the customer's spend.

> ON THE EXAM
>
> Same aggregate, different grain. groupBy answers 'what is the total per customer'; a window answers 'what is this row's share of it'.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
distinct did not remove the duplicates	The rows differ in some column. Use dropDuplicates(subset).
A different row is kept each run	dropDuplicates picks arbitrarily. Use row_number in a window.
One row per array element	explode, or explode_outer to keep empty arrays
Split a column into parts	split returns an array. Index it, or explode it.
Exact distinct count is too slow	approx_count_distinct, rsd default 0.05
Quartiles or a data profile	summary, which is richer than describe
Groups whose total exceeds a threshold	Filter after the aggregation
Each row's share of its group's total	A window function, not groupBy

ON THE EXAM

Eight lines. The first two are a pair, and they are the most valuable in the module.

A bronze orders table receives the same order twice whenever an upstream retry fires. Each bronze row carries _source_file and _ingested_at columns added at load time.

An engineer adds .distinct() to the silver build to remove the duplicates. The duplicate orders are still there.

Why did distinct not work?

A distinct only works on a single column and must be given one

B The rows are not identical: _source_file and _ingested_at differ, so distinct sees two different rows

C distinct is lazy and needs an action before it takes effect

D distinct only removes adjacent duplicate rows, so the data must be sorted first

◆ PAUSE AND DECIDE

Why did distinct not work?

A distinct needs a column argument

distinct takes no arguments and compares whole rows. That is precisely the problem, but this describes the API incorrectly.

B The ingestion metadata differs, so the rows are not identical

Correct. distinct compares every column. Two loads of the same order differ in _source_file and _ingested_at, so they are genuinely different rows.

C distinct is lazy

It is lazy, and that is irrelevant. The silver table was written, so an action ran and the transformation executed.

D Only removes adjacent duplicates

Describes the Unix uniq command, not Spark. distinct is a global operation and needs no sorting.

THE SIGNAL

The ingestion metadata that makes bronze traceable is exactly what makes distinct useless on it. Both things are true at once.

> Following the fix, the engineer uses dropDuplicates(['order_id']). The duplicates are gone. Weeks later, an analyst notices that the delivery address on some orders changes between one daily run and the next, even though nothing upstream changed.
>
> Where an order was received twice, the two bronze rows differ: the second correction included an updated address.

What is happening, and what is the fix?

A dropDuplicates keeps an arbitrary row per key; use row_number over a window ordered by _ingested_at desc

B dropDuplicates keeps the first row by file order; sort the DataFrame before calling it

C The bronze table is being rewritten between runs

D dropDuplicates should be given all columns so it picks the complete row

◆ PAUSE AND DECIDE

What is happening, and what is the fix?

A Arbitrary row; use row_number over a window

Correct. dropDuplicates gives no guarantee about which row survives. The window pattern states the rule explicitly and is reproducible.

B It keeps the first row; sort beforehand

There is no first-row guarantee to rely on, and sorting before dropDuplicates does not make the choice deterministic.

C Bronze is being rewritten

Bronze is append-only and the scenario says nothing upstream changed. It invents a cause when a sufficient one is present.

D Pass all columns

That makes it equivalent to distinct, which returns to scenario 1's problem: both rows survive because they differ.

THE SIGNAL

Non-determinism is the symptom to recognise. The same input producing different output means something is choosing arbitrarily.

> A team needs a report of customers whose total spend across the year exceeds £10,000. An engineer writes: filter total_spend greater than 10000, then groupBy customer, then sum the amounts.
>
> The report returns far fewer customers than the business expects, and the totals shown are lower than the customers' actual annual spend.

What is wrong?

A The filter is applied to individual orders before grouping, so only customers with a single order over £10,000 appear

B sum ignores nulls, so some order amounts were excluded

C groupBy should have been a window function

D The filter should use >= rather than >

◆ PAUSE AND DECIDE

What is wrong?

A The filter runs before grouping, on individual orders

Correct. Filtering first removes rows, so only large single orders survive to be grouped, and each customer's total is computed from those alone.

B sum ignores nulls

True of sum, and it would not produce this pattern. It also does not explain why fewer customers appear.

C Should have been a window function

A window would keep every row, which is not what a customer-level report wants. The problem is the filter's position, not the aggregate.

D Use >= instead of >

An off-by-one on the boundary would change the result by a handful of customers, not produce systematically lower totals.

THE SIGNAL

Two symptoms, one cause: fewer customers AND lower totals. Filtering after grouping would change which customers appear but not their totals.

> A dashboard shows daily unique visitors across 400 million events. It uses count_distinct on visitor_id, grouped by day. The query takes eleven minutes and the dashboard times out.
>
> The dashboard is used to spot week-on-week trends. A separate regulatory report also counts unique visitors, and must be exact.

What should the team do?

A. Use approx_count_distinct for both, since it is far faster

B. Use approx_count_distinct for the dashboard and keep count_distinct for the regulatory report

C. Use approx_count_distinct with rsd set to 0.001 for both, giving speed and accuracy

D. Keep count_distinct for both and increase the SQL warehouse size

◆ PAUSE AND DECIDE

What should the team do?

A Approximate for both

Fast, and it makes the regulatory figure an estimate. 'Must be exact' is a stated requirement, not a preference.

B Approximate for the dashboard, exact for the report

Correct. The requirements differ, so the answers differ. A 5% estimate is fine for a trend and unacceptable for a regulatory count.

C rsd 0.001 for both

The docs note that below rsd 0.01 it is more efficient to use count_distinct. At 0.001 you pay for precision and lose the speed advantage.

D Exact for both, bigger warehouse

Buys time on the dashboard at permanent cost, and the exact count still scales badly as events grow.

THE SIGNAL

Two consumers, two requirements. Nothing says one query must serve both, and the exam rewards noticing when it does not.

Ask what one row means before the operation, and what it means after. **Every question in this module is answered by that.**

MODULE 13 · DOMAIN 3 - DATA TRANSFORMATION AND MODELING

Gold layer objects

Tables, views, materialised views and streaming tables — what each stores, when each computes, and how to choose.

DOMAIN 3 of 7 EXAM WEIGHT 22% MODULE 13 of 36

admin@sthithapragna.com

The four objects differ on two axes: **is the result stored**, and **when is it computed**. Everything else follows.

The four objects

OBJECT	STORES	COMPUTED	FRESHNESS
Table	Rows	When written	As of the last write
View	Only the query	At every query	Always current
Materialised view	The query result	At refresh	As of the last refresh
Streaming table	Rows, appended	As data arrives	Continuously, or per refresh

ON THE EXAM

Read the last two columns together. A view is always current and always pays for it. A materialised view is fast and can be stale.

Table

- **The default and the simplest.** Rows written by a job, read by anyone.
- **Refresh is your responsibility**, through whatever job builds it.
- **Full control over the write pattern:** overwrite, merge, or append.
- **Choose it when the build logic is complex**, or when the write needs to be a MERGE.

> ON THE EXAM
>
> A gold table built by a scheduled job is not old-fashioned. It is the right answer whenever you need control over how rows are written.

View

- **Stores a query, not data.** Zero storage cost.
- **Always current**, because it reads the base tables at query time.
- **Every reader pays the full cost.** A ninety-second query takes ninety seconds for everybody, every time.
- **Good for lightweight logic**: a filter, a rename, a simple join, or hiding columns.

> ON THE EXAM
>
> A view is also a governance tool. It can expose a subset of a table to a group who have no access to the table itself.

A materialised view stores the **results** of a query and updates them as the sources change, either on a schedule or automatically.

What a materialised view needs

- **Backed by a serverless pipeline.** The workspace must support serverless pipelines.
- **A Unity Catalog-enabled pro or serverless SQL warehouse.** Classic will not do.
- **You must be in the workspace that created it** to refresh it.
- **Privileges:** SELECT on base tables, USE CATALOG and USE SCHEMA on both source and target, plus CREATE MATERIALIZED VIEW.

> ON THE EXAM
>
> Creating a materialised view in Databricks SQL silently creates a serverless pipeline to maintain it. That is where the refresh cost appears.

Refreshing

STATEMENT	EFFECT
`REFRESH MATERIALIZED VIEW mv`	Refresh, incrementally where possible
`REFRESH MATERIALIZED VIEW mv FULL`	Force a complete recomputation
`REFRESH ... ASYNC`	Return immediately rather than blocking
`A SCHEDULE clause`	Refresh automatically on a cron schedule

ON THE EXAM

REFRESH is synchronous by default, so the statement blocks until the refresh finishes. ASYNC returns straight away.

Incremental refresh

- **Detects changes in the sources** and recomputes only the affected results, rather than the whole query.
- **Requires serverless compute.**
- **From Delta tables, the source tables must have row tracking enabled.** Without it, refreshes are full.
- **Not every query can be refreshed incrementally.** Where it cannot, Spark falls back to a full recomputation.

> ON THE EXAM
>
> Row tracking on the sources is the requirement people miss. A materialised view that silently recomputes in full is usually missing it.

What supports incremental refresh

- **Delta tables**, both Unity Catalog managed and external.
- **Other materialised views.**
- **Streaming tables**, including the targets of AUTO CDC INTO operations.
- **Unity Catalog managed Iceberg tables**, v2 and v3.

> ON THE EXAM
>
> Sources outside this list force a full refresh every time, whatever else you configure.

A streaming table processes each input record **exactly once**. That is the property that decides when you need one.

Do not define a materialised view against a source if records **should only be processed once**. Use a streaming table instead.

Why that rule exists

- **A materialised view may recompute its query from scratch.** That means re-reading the whole source.
- **If the source no longer holds the history**, a full recompute silently produces a smaller answer.
- **A streaming table never re-reads.** It appends what is new and remembers, via a checkpoint.
- **So streaming tables suit ingestion**, and materialised views suit aggregation over data that persists.

> ON THE EXAM
>
> Kafka retains messages for days. Rebuild a materialised view over it after the retention window and the older data is simply gone.

Materialised view or streaming table?

MATERIALISED VIEW

A stored answer

- Result of an aggregation or join, kept up to date
- Can be recomputed from source at any time
- Refreshed on a schedule or automatically
- Wrong when the source does not retain history

STREAMING TABLE

An append-only accumulation

- Each source record processed exactly once
- Never re-reads what it has already consumed
- Suits ingestion from files or message buses
- Wrong when you need a full aggregate recomputed

THE TIEBREAKER

Ask whether the source can be read again. If it cannot, or should not be, that is a streaming table.

Materialised view limitations

- **No identity columns or surrogate keys.**
- **Refreshable only from the workspace that created it.**
- **Source tables must be registered in Unity Catalog** as managed or external Delta tables.
- **Queryable from a SQL warehouse, Lakeflow pipelines, standard access mode**, or dedicated on DBR 15.4 and above.

> ON THE EXAM
>
> The identity column limitation matters for dimensional models, where surrogate keys are the normal design.

All four are Unity Catalog objects

- **All appear in the three-level namespace** and can be granted with GRANT SELECT.
- **Lineage works across all of them**, so you can trace a dashboard back to bronze.
- **A materialised view is a Unity Catalog managed table** that happens to be maintained for you.
- **So the governance answer is the same for all four.** The choice is about cost and freshness.

> ON THE EXAM
>
> That simplifies exam questions: if a scenario's constraint is governance, it usually does not discriminate between these four.

Choosing for a BI team

IF THE REQUIREMENT IS	CHOOSE
Light logic, must always be current	View
Expensive query, many readers, minutes of staleness acceptable	Materialised view
Complex multi-step build, or MERGE on write	Table, built by a job
Ingesting from files or a message bus	Streaming table
The source does not retain history	Streaming table
Surrogate keys in a dimensional model	Table, not a materialised view

ON THE EXAM

Six lines that answer the objective directly. Note that two different requirements both point at a streaming table.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
The dashboard is slow for every user	A materialised view, not a view
The figures must never be stale	A view, which computes at query time
Refreshes cost far more than expected	Row tracking may not be enabled on the sources
Records must be processed exactly once	A streaming table
The source is Kafka, or Auto Loader ingest	A streaming table, never a materialised view
Classic SQL warehouse, creating an MV	Not supported. Pro or serverless only.
Needs an identity column	A table. Materialised views do not support them.

ON THE EXAM

Seven lines. The third is the one that shows up as an unexplained cost rather than an error.

> A finance dashboard is backed by a view that joins four silver tables and aggregates 900 million rows. It takes about 80 seconds to load. Around 60 analysts open it through the morning, and each one waits.
>
> The underlying tables are updated once, by a pipeline that finishes at 05:30. Analysts arrive from 08:00.

What should the team do?

A. Increase the SQL warehouse size so the view query runs faster

B. Replace the view with a materialised view, refreshed after the 05:30 pipeline

C. Replace the view with a streaming table

D. Add result caching so the second and later analysts get a cached result

◆ PAUSE AND DECIDE

What should the team do?

A Increase the warehouse size

Makes 80 seconds into perhaps 40, for every analyst, at permanently higher cost. It does not stop the work being repeated 60 times.

B Materialised view refreshed after the pipeline

Correct. The data changes once a day, so compute once and let 60 analysts read a stored result. This is the documented use case.

C A streaming table

Streaming tables are for incremental ingestion, not for serving an aggregate. The sources are silver tables updated in batch.

D Result caching

Genuinely helps the second reader onwards, and it is fragile: it expires, and any variation in the query misses the cache.

THE SIGNAL

The data changes once a day and the query runs sixty times. That ratio is the whole argument for materialising.

A team creates a materialised view aggregating a large silver table. They expected refreshes to be cheap, since only about 0.3% of the source rows change each day.

The serverless pipeline bill shows each refresh consuming roughly the same compute as a complete rebuild. Refreshes are scheduled hourly.

What is the most likely cause?

A The materialised view is being refreshed with the FULL keyword

B Row tracking is not enabled on the source table, so incremental refresh is not possible

C Hourly refreshes are too frequent and should be reduced to daily

D Materialised views always fully recompute; incremental refresh applies only to streaming tables

◆ PAUSE AND DECIDE

What is the most likely cause?

A. Refreshing with FULL

Possible, and worth checking. But a scheduled refresh does not use FULL by default, so it would require someone to have written it deliberately.

B. Row tracking is not enabled on the source

Correct. Incremental refresh from Delta tables requires row tracking on the sources. Without it, Spark cannot identify changed rows and falls back to a full recompute.

C. Reduce refresh frequency

Reduces how often you pay, not what each refresh costs. It also makes the data staler without fixing anything.

D. MVs always fully recompute

False. Incremental refresh is a documented materialised view feature; it simply has requirements.

THE SIGNAL

The fallback is silent. An MV that cannot refresh incrementally does not fail — it recomputes fully and sends you the bill.

> A team ingests clickstream events from a Kafka topic with seven days of retention. They want a gold object holding a running daily count of events per product, going back two years.
>
> An engineer proposes a materialised view defined directly over the Kafka source, refreshed hourly.

What is wrong with the proposal?

A. Nothing; a materialised view refreshed hourly is appropriate here

B. A materialised view may recompute from source, and Kafka only retains seven days, so history would be lost

C. Materialised views cannot aggregate, only filter and join

D. Hourly refresh is too slow for clickstream data; it needs continuous processing

◆ PAUSE AND DECIDE

What is wrong with the proposal?

A **Nothing is wrong**

It would appear to work for a week, then silently lose everything older than the retention window on the first full recompute.

B **MV may recompute; Kafka retains only seven days**

Correct. Databricks documents this directly: do not define a materialised view against a source where records should only be processed once. Kafka is the named example.

C **MVs cannot aggregate**

False. Aggregation is one of the main reasons to use one.

D **Hourly is too slow**

Latency was never stated as a requirement. The problem is data loss, not freshness.

THE SIGNAL

Ingest with a streaming table, then build the materialised view on that table. The streaming table becomes the durable history.

> An operations team relies on a gold object showing open incidents. It was built as a materialised view six months ago. Overnight, a major incident is raised, and at 09:00 the dashboard still shows the previous day's figures.
>
> Investigation shows the materialised view was created without a SCHEDULE clause and has not been refreshed since the day it was created.

What is the correct conclusion?

A. Materialised views refresh automatically; something is broken and should be raised with support

B. A materialised view is as fresh as its last refresh; it needs a SCHEDULE, or a view should be used if it must always be current

C. The materialised view should be converted to a streaming table so it updates continuously

D. The dashboard is caching results and needs its cache cleared

◆ PAUSE AND DECIDE

What is the correct conclusion?

A They refresh automatically; raise it with support

They can be refreshed on a schedule or automatically, but that must be configured. Nothing is broken; nothing was ever scheduled.

B As fresh as its last refresh; add a SCHEDULE, or use a view

Correct. It names the property, gives the fix, and offers the alternative when 'always current' is the actual requirement.

C Convert to a streaming table

Streaming tables are append-only. An open-incidents figure needs incidents to leave the set when closed, which appending cannot express.

D Clear the dashboard cache

The stored result really is a day old. Clearing a cache would re-read the same stale materialised view.

THE SIGNAL

'As fresh as the last refresh' is the defining property. Six months without a refresh is the definition working, not failing.

Is the result stored, and when is it computed? **Then ask whether the source can be read again.** Those two questions choose all four objects.

MODULE 14 · DOMAIN 3 - DATA TRANSFORMATION AND MODELING

Data quality checks and validation

Table constraints and pipeline expectations, the three violation actions, and where each check belongs.

DOMAIN 3 of 7 EXAM WEIGHT 22% MODULE 14 of 36

admin@sthithapragna.com

A quality check is not complete until you have answered three questions: **what is valid, what happens to the invalid, and how will anyone find out.**

Two mechanisms

TABLE CONSTRAINTS

Enforced by Delta at write time

- NOT NULL and CHECK constraints on the table itself
- Invalid data is rejected; the write fails
- Works on any Delta table, no pipeline needed
- No choice of action — violation means failure

PIPELINE EXPECTATIONS

Applied during transformation

- Defined in Lakeflow declarative pipelines
- Choose to warn, drop the row, or fail the update
- Metrics recorded per expectation
- Only available inside a pipeline

THE TIEBREAKER

Constraints are a property of the table. Expectations are a property of the pipeline that fills it. Both can apply to the same data.

Table constraints

CONSTRAINT	MEANING
`NOT NULL`	This column may never contain a null
`CHECK (amount > 0)`	Every row must satisfy this boolean expression
`ALTER TABLE t ADD CONSTRAINT name CHECK (...)`	Add one to an existing table
`ALTER TABLE t DROP CONSTRAINT name`	Remove it

ON THE EXAM

Adding a CHECK constraint verifies the existing rows first. If any current row violates it, the ALTER fails.

What a constraint violation looks like

- **The write fails.** The whole transaction, not just the offending row.
- **Because Delta writes are atomic**, nothing from that write lands. There is no partial result.
- **That is strong protection and a blunt instrument.** One bad row stops a batch of ten million.
- **Use constraints for rules that must never be broken**, and expectations where you need a softer response.

> ON THE EXAM
>
> A constraint is a promise about the table that holds regardless of who writes to it. That is exactly its value and its cost.

Every expectation has three parts: a **name**, a **constraint**, and an **action**. The action is the part people forget.

The three violation actions

ACTION	SQL	WHAT HAPPENS TO THE INVALID RECORD
Warn	(no ON VIOLATION clause)	Written to the target anyway, and counted in metrics
Drop	ON VIOLATION DROP ROW	Discarded before the write. The count is logged.
Fail	ON VIOLATION FAIL UPDATE	The update stops. Partial updates are rolled back.

ON THE EXAM

Warn is the default. Retaining invalid records is what happens when you write an expectation with no ON VIOLATION clause.

When to choose each

- **Warn** when you want visibility without changing behaviour. Good for measuring a problem before acting on it.
- **Drop** when bad rows are expected and must not propagate downstream, and losing them is acceptable.
- **Fail** when any invalid record indicates a serious upstream problem that a human must look at.
- **A failure affects a single flow**, not the other flows in the pipeline.

> ON THE EXAM
>
> Fail is the right choice more often than people think, and the wrong one whenever the pipeline feeds something that must stay live.

The same expectation, two languages

SQL	PYTHON
CONSTRAINT valid_id EXPECT (id IS NOT NULL)	@dp.expect("valid_id", "id IS NOT NULL")
… EXPECT (amount >= 0) ON VIOLATION DROP ROW	@dp.expect_or_drop("amount_ok", "amount >= 0")
… EXPECT (id IS NOT NULL) ON VIOLATION FAIL UPDATE	@dp.expect_or_fail("id_present", "id IS NOT NULL")

ON THE EXAM

There are six Python decorators: the three above, plus expect_all, expect_all_or_drop and expect_all_or_fail for several rules at once.

A currency note on the Python syntax

- **Older material shows import dlt and @dlt.expect.** Your question bank almost certainly does.
- **The current form is from pyspark import pipelines as dp**, then @dp.expect.
- **The old names still work.** This is a rename, not a breaking change.
- **Recognise both.** The exam may use either, and the behaviour is identical.

> ON THE EXAM
>
> Same rename as module 32 collects: Delta Live Tables became Lakeflow Spark Declarative Pipelines, and the module prefix moved with it.

Naming expectations

- **The name is an identifier used to track and monitor** that expectation over time.
- **It must be unique within a dataset.**
- **It can be reused across datasets** in the same pipeline, which is how you track one rule in several places.
- **Choose a name that says what is being validated**, since it is what appears in the metrics.

> ON THE EXAM
>
> valid_customer_age tells you what failed. constraint_3 tells you nothing, and it is the name you will see at three in the morning.

Where the metrics go

- **The pipeline UI has a Data quality tab** on each dataset with expectations defined.
- **The event log holds the same metrics**, and can be queried like any table.
- **Fail records no metrics.** Because the update fails on detection, there is nothing to report.
- **Fail does give a detailed error**, naming the violated expectation and showing the offending input record.

> ON THE EXAM
>
> That third point is counter-intuitive. The strictest action is the one that leaves you with the least measurement.

The quarantine pattern

- **Neither drop nor fail keeps the bad rows for inspection.** Drop discards them; fail stops everything.
- **Quarantine writes them somewhere instead:** a flag column, or a separate table.
- **Build it with a boolean column** recording whether each row passed, then split downstream on that column.
- **Use it when bad records must be investigated** rather than merely counted or discarded.

> ON THE EXAM
>
> Quarantine is the answer whenever a scenario says invalid rows must be reviewed, corrected or reprocessed later.

Three dimensions worth checking

- **Nullability.** Required values are present. NOT NULL, or an IS NOT NULL expectation.
- **Cardinality.** Values are unique where they should be. Usually a count against a count distinct.
- **Range.** Values fall within acceptable bounds. A CHECK constraint or a BETWEEN expectation.
- **Referential integrity is not enforced.** Delta has no foreign keys that reject writes; use a left_anti join to detect orphans.

> ON THE EXAM
>
> Cardinality is the one that catches the fan-out from module 11. A silver table should have one row per business key.

Where checks belong

- **Not on the way into bronze.** Bronze takes everything, as module 3 established.
- **Bronze to silver is the main place.** This is where records become trustworthy, so this is where you decide.
- **Silver to gold checks aggregate sanity:** totals, expected row counts, no unexpected nulls in dimensions.
- **Check early enough to stop bad data spreading**, and late enough that you still have the original.

> ON THE EXAM
>
> A check placed in gold has already let bad data into silver, where several other gold tables may have read it.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Bad rows are still in the table despite expectations	Warn is the default. No ON VIOLATION clause was given.
The pipeline must not stop, bad rows must not propagate	ON VIOLATION DROP ROW
Any invalid record means something is seriously wrong	ON VIOLATION FAIL UPDATE
Invalid rows must be reviewed and corrected later	Quarantine, not drop
The rule must hold no matter who writes to the table	A table constraint, not an expectation
The team wants counts of how many rows failed	Warn or drop. Fail records no metrics.
Adding a CHECK constraint failed	Existing rows already violate it

ON THE EXAM

Seven lines. The first is the most commonly tested fact in the module.

A team adds an expectation to a streaming table: CONSTRAINT valid_amount EXPECT (amount >= 0). They deploy it and confirm the pipeline runs successfully.

A week later, an analyst reports that the silver table still contains rows with negative amounts. The pipeline's data quality tab shows several thousand failing records.

What is happening?

A The expectation is not being evaluated; the syntax is wrong

B With no ON VIOLATION clause the action is warn, so invalid records are written to the target and merely counted

C Expectations only apply to new records, and these predate the deployment

D The data quality tab is reporting records from the source, not the target

◆ PAUSE AND DECIDE

What is happening?

A The syntax is wrong

The data quality tab is showing failing counts, which means the expectation is being evaluated correctly. It is working exactly as written.

B No ON VIOLATION clause means warn, so records are written and counted

Correct. Retaining invalid records is the documented default. The expectation measures the problem; it was never told to act on it.

C Only applies to new records

Would explain old rows, not the several thousand currently failing. Those are being evaluated now and written anyway.

D The tab reports source records

It reports records evaluated against the expectation as they flow into the target. The counts and the bad rows are the same records.

THE SIGNAL

The failing count in the UI is the evidence that everything is working. Warn means observe, not enforce.

A pipeline ingests sensor readings. About 0.5% of readings arrive with an impossible temperature, caused by a known hardware fault. Those readings must not reach the gold aggregates.

The pipeline feeds an operational dashboard that the plant team relies on continuously. It must not stop. The engineering team also wants to track how often the fault occurs, to justify replacing the sensors.

Which action fits?

A EXPECT with no ON VIOLATION clause

B ON VIOLATION DROP ROW

C ON VIOLATION FAIL UPDATE

D A CHECK constraint on the target table

◆ PAUSE AND DECIDE

Which action fits?

A EXPECT with no ON VIOLATION

Tracks the fault, and writes the impossible readings into the target, so they reach the gold aggregates. Fails the first requirement.

B ON VIOLATION DROP ROW

Correct. Bad readings are discarded before the write, the pipeline keeps running, and the count of dropped records is logged in the metrics.

C ON VIOLATION FAIL UPDATE

Stops the update on the first bad reading, so the dashboard the plant team relies on goes down roughly every 200 readings.

D A CHECK constraint

Rejects the whole write, which stops the pipeline just as surely as fail does, and records no per-rule metrics.

THE SIGNAL

Three requirements: exclude the rows, keep running, and count the occurrences. Drop is the only action that does all three.

> A regulated silver table must never contain a row with a null account_id. It is currently written by one Lakeflow pipeline, which has an expectation with ON VIOLATION FAIL UPDATE.
>
> The platform team is concerned that another team may later write to this table from a notebook or a job, bypassing the pipeline entirely.

What should they add?

A Nothing; the FAIL UPDATE expectation already guarantees it

B A NOT NULL constraint on the table, which Delta enforces against every writer

C A second expectation with ON VIOLATION DROP ROW as a fallback

D A scheduled query that checks for nulls and alerts the team

◆ PAUSE AND DECIDE

What should they add?

A Nothing; FAIL UPDATE is enough

It guarantees the pipeline cannot write nulls. It says nothing about a notebook writing directly to the table, which is exactly the stated concern.

B A NOT NULL constraint on the table

Correct. A constraint is a property of the table, enforced by Delta at write time regardless of which tool performs the write.

C A second expectation as a fallback

Expectations only run inside the pipeline. Two of them protect against the same single path, and neither sees a notebook write.

D A scheduled query that alerts

Detection after the fact. The requirement is that the table must never contain such a row, not that someone is told when it does.

THE SIGNAL

The rule must hold regardless of the writer. That phrase points at the table, not at any pipeline filling it.

> A pipeline uses ON VIOLATION FAIL UPDATE on a critical expectation. It has begun failing most nights. The team wants to understand how widespread the problem is before deciding what to do, so they open the Data quality tab.
>
> The tab shows no metrics for that expectation at all.

Why, and what should they do?

A The metrics take 24 hours to populate; they should wait

B Fail records no metrics, because the update fails on detection; switch temporarily to warn or drop to measure the scale

C The expectation is misconfigured and is not running

D Metrics are only available in the event log, never in the UI

◆ PAUSE AND DECIDE

Why, and what should they do?

A Wait 24 hours

Invents a delay. The metrics are not late; for this action they are never produced.

B Fail records no metrics; switch temporarily to warn or drop

Correct. Because fail causes the update to fail on detection, there is nothing to aggregate. Warn measures the scale without stopping the pipeline.

C It is misconfigured

It is demonstrably running — it is failing the pipeline most nights, which is precisely what it was configured to do.

D Metrics are event log only

Both surfaces exist: the Data quality tab and the event log. Neither has data here, for the same underlying reason.

THE SIGNAL

The strictest action gives the least information. If you need to size a problem, warn first and tighten afterwards.

What is valid, what happens to the invalid, and how will anyone find out. **A check that answers only the first is not a check.**

MODULE 15 · DOMAIN 3 - DATA TRANSFORMATION AND MODELING

Tuning parameters and re-measuring

The four parameters the exam names, what each actually controls, and the discipline of measuring before and after.

DOMAIN 3 of 7 EXAM WEIGHT 22% MODULE 15 of 36

admin@sthithapragna.com

Tuning is what you do **after** you have measured, and after the cheaper fixes are exhausted. Reaching for a config first is the classic mistake.

The four named parameters

PARAMETER	CONTROLS	DEFAULT
spark.sql.shuffle.partitions	Partitions produced by a shuffle in Spark SQL	200
spark.default.parallelism	Default partitions for RDD operations only	Total cores
spark.executor.memory	Memory available to each executor	Node type
spark.driver.memory	Memory available to the driver	Node type
spark.sql.autoBroadcastJoinThreshold	Size below which a table is auto-broadcast	10 MB

ON THE EXAM

Second row is the trap: spark.default.parallelism is an RDD setting.

spark.sql.shuffle.partitions

- **How many partitions a shuffle produces.** Every join, groupBy, distinct and orderBy is affected.
- **Default 200**, which is a number chosen years ago and unrelated to your cluster.
- **Too few:** each partition is large, executors strain, and you risk spill or out-of-memory errors.
- **Too many:** thousands of tiny tasks, scheduling overhead dominates, and parallelism does not improve.

> ON THE EXAM
>
> It is also the default partition count used by repartition when you do not supply a number.

AQE **coalesces partitions downwards**. It will not increase beyond the configured value. Set it too low and adaptive execution cannot rescue you.

admin@sthithapragna.com

Choosing a starting value

- **A common heuristic is one to three times the total core count**, so parallelism roughly matches the cluster.
- **Or target a partition size:** total shuffle bytes divided by a sensible partition size, often a few hundred megabytes.
- **Err high and let AQE coalesce**, since it can reduce but not raise.
- **On Databricks, 'auto' is a valid value**, enabling auto-optimised shuffle.

> ON THE EXAM
>
> The default of 200 is wrong in both directions: far too many on a two-core cluster, far too few on a thousand-core one.

spark.default.parallelism

- **It applies to RDD operations**, not to DataFrames or Spark SQL.
- **So setting it does nothing to a DataFrame job.** That is the whole trap.
- **It defaults to the total number of worker cores.**
- **Know it because the exam names it**, and because the distinction from shuffle.partitions is the point.

> ON THE EXAM
>
> On Databricks, sc.defaultParallelism reflects worker cores and cannot be overridden. Almost every question mentioning it is testing the RDD distinction.

Executor memory

- **How much memory each executor has** for processing and caching.
- **Too little and Spark spills to disk**, which is slow but survivable, or fails with an out-of-memory error.
- **On Databricks it usually follows the node type**, so choosing a memory-optimised node is the practical lever.
- **More memory is not the first fix for spill.** Often the partitions are simply too large.

> ON THE EXAM
>
> Spill and OOM are usually a partitioning problem wearing a memory costume. Check partition sizes before adding memory.

Driver memory

- **The driver plans work, tracks tasks and receives results.** It does not process partitions.
- **collect and toPandas pull data to the driver**, and a large result will exhaust it.
- **Broadcast joins collect the small side on the driver first**, so an oversized broadcast is a driver failure.
- **A driver OOM has different causes from an executor OOM**, and reading the error correctly saves hours.

> ON THE EXAM
>
> If the error names the driver, stop looking at executors. The cause is almost always collect, toPandas, or a broadcast.

The broadcast threshold, revisited

- **spark.sql.autoBroadcastJoinThreshold** is the static, plan-time threshold. Default 10 MB.
- **Raising it lets larger dimension tables be broadcast**, avoiding a shuffle on the large side.
- **Raise it too far and you risk a driver out-of-memory error**, since the small side is collected there.
- **Set it to -1 to disable automatic broadcasting** entirely.

> ON THE EXAM
>
> Databricks has a separate AQE threshold, spark.databricks.adaptive.autoBroadcastJoinThreshold, defaulting to 30 MB.

Where to set a parameter

SCOPE	HOW	WHEN
Cluster	Spark config in the cluster or job definition	A setting that should apply to everything on it
Session	spark.conf.set(…) in the notebook or job	One workload that needs different settings
Query	A hint, such as BROADCAST	One join, without changing anything global

ON THE EXAM

Narrower is safer. A cluster-wide change to fix one job affects every other job on that cluster.

The re-measure discipline

01

Baseline

Record the current runtime and where the time goes.

02

One change

Change a single parameter. Not three.

03

Re-run

Same data, same cluster, ideally a fresh session.

04

Compare

Did it help, and by how much?

05

Keep or revert

Revert anything that did not help. Do not accumulate.

NAMED IN THE OBJECTIVE

The exam objective says 're-measure the performance'. This is what that phrase means.

Measuring honestly

- **The first run is not comparable.** Caching, warm clusters and file system caches all flatter a repeat.
- **Use a fresh session** and run more than once before recording a number.
- **Wall-clock time includes cluster start-up.** On a cold classic cluster that can dominate a short job.
- **Look at stage-level metrics**, not just total duration. The bottleneck is usually one stage.

> ON THE EXAM
>
> A change that appears to halve the runtime has often just benefited from a warm cache. Repeat before believing it.

Symptom to parameter

SYMPTOM	LIKELY LEVER
Most cores idle during a join or aggregation	Raise spark.sql.shuffle.partitions
Thousands of tiny tasks, high overhead	Lower it, or let AQE coalesce
Disk spill in the Spark UI	More partitions first, then more memory
Executor out of memory	Partition sizes, then node type
Driver out of memory	A collect, a toPandas, or an oversized broadcast
A shuffle where a broadcast would do	Raise autoBroadcastJoinThreshold, or hint

ON THE EXAM

Note that two symptoms point away from parameters entirely. Driver OOM is a code problem, not a config one.

What to try before tuning

- **Read less.** Filter earlier, select fewer columns, use partition pruning.
- **Check the compute type.** A job on all-purpose compute is paying a premium before any tuning.
- **Look for fan-out.** A join multiplying rows makes everything downstream slower.
- **Check for an accidental full recompute**, such as a materialised view without row tracking.

> THE ORDERING
>
> Every item here changes how much work exists. A parameter only changes how the same work is divided up.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Setting shuffle.partitions low and expecting AQE to raise it	AQE only coalesces downwards
A DataFrame job unchanged by spark.default.parallelism	That parameter applies to RDDs only
Cores idle during the shuffle stage	Too few shuffle partitions
The error names the driver	collect, toPandas, or a broadcast
One job on a shared cluster needs a change	Session scope, not cluster scope
Three parameters changed at once	You cannot attribute the result. Change one.
The job reads more data than it needs	Fix that before touching any parameter

ON THE EXAM

Seven lines. The first two are the ones most likely to appear as a question in their own right.

> A nightly aggregation runs on a cluster with 240 total cores. The Spark UI shows the input stage using around 600 partitions and running well. The stage after the groupBy shows exactly 200 tasks, and most executors are idle during it.
>
> That stage accounts for most of the job's runtime.

What should the team change?

A. Increase spark.sql.shuffle.partitions, which is at its default of 200

B. Increase spark.default.parallelism to match the core count

C. Increase executor memory so each of the 200 tasks completes faster

D. Add more workers to the cluster

◆ PAUSE AND DECIDE

What should the team change?

A Increase spark.sql.shuffle.partitions

Correct. Exactly 200 tasks after a groupBy is the default showing through. With 240 cores, 200 tasks leaves capacity unused on a stage that dominates runtime.

B Increase spark.default.parallelism

That parameter governs RDD operations. A DataFrame groupBy takes its partition count from spark.sql.shuffle.partitions, so this changes nothing.

C Increase executor memory

Executors are idle, not struggling. Memory is not the constraint when most of the cluster has no work to do.

D Add more workers

More cores sitting idle. The stage would still produce 200 tasks, so the extra capacity goes unused too.

THE SIGNAL

'Exactly 200' is the tell. A round, unchanging number in the Spark UI usually means a default is showing through.

> An engineer reads that adaptive query execution tunes shuffle partitions automatically. To let it work freely, she sets spark.sql.shuffle.partitions to 40 on a 500-core cluster, reasoning that AQE will adjust upwards as needed.
>
> The job slows considerably. The Spark UI shows 40 very large tasks and significant disk spill.

What went wrong?

A AQE was disabled on this cluster and needs enabling

B AQE coalesces partitions downwards only; it cannot increase beyond the configured value

C 40 is below the minimum permitted value, so Spark ignored it

D AQE only adjusts partitions for joins, not aggregations

◆ PAUSE AND DECIDE

What went wrong?

A AQE was disabled

AQE is on by default from DBR 7.3 onwards. And even fully enabled, it could not have raised 40 to a useful number.

B AQE coalesces downwards only

Correct. The configured value is an upper bound AQE works down from. Setting it low removes the headroom AQE needs.

C 40 is below a minimum

Invented limit. Spark accepted the value, which is exactly why there are 40 tasks.

D AQE only adjusts joins

It coalesces shuffle partitions generally, and also handles skew and join strategy. The limitation is directional, not by operation.

THE SIGNAL

The advice is to set it generously and let AQE reduce. Setting it low to 'let AQE decide' inverts how the mechanism works.

> A job that builds a gold table takes 50 minutes. Investigating, an engineer finds it reads a full year of the silver table on every run and then filters to the last 7 days after a join.
>
> He asks whether he should raise spark.sql.shuffle.partitions and increase executor memory to speed it up.

What is the right advice?

A. Yes; both changes will reduce the runtime on a job of this size

B. Filter to the 7 days before the join first, then re-measure; tuning parameters redistributes work rather than reducing it

C. Raise shuffle.partitions only, since memory is not the bottleneck

D. Convert the gold table to a materialised view instead

◆ PAUSE AND DECIDE

What is the right advice?

A Yes, make both changes

Two changes at once, so neither can be attributed. And both redistribute work that should not exist in the first place.

B Filter first, then re-measure

Correct. Reducing the data read is a change in how much work exists. Measure again afterwards, because the tuning question may vanish.

C Raise shuffle.partitions only

Better than A because it changes one thing. Still tunes the division of work that a filter would eliminate.

D Use a materialised view

Addresses repetition across readers, which is not the problem. One scheduled job reading too much data stays slow behind a materialised view.

THE SIGNAL

Spark can push some filters down, but a filter placed after a join still means joining a year of data to produce seven days.

> A job fails with an out-of-memory error. The stack trace names the driver. The job joins a large fact table to a dimension table that has recently grown, and ends by calling display on the result.
>
> A colleague suggests increasing spark.executor.memory and adding more workers.

What should be investigated first?

A Increase executor memory as suggested, since OOM means insufficient memory

B The error names the driver, so look at the broadcast of the grown dimension table, and at the display call

C Increase spark.sql.shuffle.partitions to make each partition smaller

D The dimension table needs OPTIMIZE to reduce its file count

◆ PAUSE AND DECIDE

What should be investigated first?

A Increase executor memory

The error names the driver. Adding executor memory addresses a component that is not failing, at permanent cost.

B Look at the broadcast and the display call

Correct. Both pull data to the driver. A grown dimension table may still be broadcast, and display collects results back.

C Raise shuffle.partitions

Shuffle partitions affect executor-side memory pressure. They do nothing for the driver, which does not process partitions.

D Run OPTIMIZE on the dimension table

File layout affects read efficiency. It does not change how much the driver has to hold.

THE SIGNAL

Reading which component failed is the whole diagnosis. Driver and executor OOM have entirely different causes and fixes.

Measure, change one thing, measure again. And before any of that, ask whether the work itself can be made smaller. **Parameters divide work; they do not reduce it.**

admin@sthithapragna.com

MODULE 16 · DOMAIN 4 - WORKING WITH LAKEFLOW JOBS

Lakeflow Jobs anatomy and task types

Jobs, tasks and the DAG; the task types the exam names; compute per task; and passing values between them.

DOMAIN 4 of 7 EXAM WEIGHT 16% MODULE 16 of 36

admin@sthithapragna.com

A job is a **graph of tasks**, not a script. Each task has its own type, its own compute, and its own place in the dependency graph.

The anatomy

01

Job

The unit you schedule, own and monitor.

02

Tasks

One or more. Each has a type and a compute resource.

03

Dependencies

Each task names the tasks it waits for.

04

The DAG

Databricks derives order and parallelism from those dependencies.

05

Runs

Each execution is a run, with per-task status and history.

DECLARE, DON'T SEQUENCE

You never write the order. You declare dependencies and the order follows. Anything undeclared may run in parallel.

Task types

TYPE	WHAT IT RUNS
Notebook	A notebook from the workspace or a Git folder
SQL	A query, a dashboard, an alert, or a SQL file
Pipeline	A Lakeflow declarative pipeline. Triggered only.
Python script / Python wheel	A .py file, or a packaged wheel
dbt	A dbt project against a SQL warehouse
JAR / Spark Submit	Compiled Scala or Java work
Run job	Another job, as a task inside this one
If/else and For each	Control flow rather than work. Module 17.

ON THE EXAM

The exam guide names four: notebook, SQL query, dashboard and pipeline. Know those precisely, and the others by sight.

The notebook task

- **The most common type.** Runs a notebook, with parameters passed in as widgets.
- **Source can be the workspace or a Git folder**, which is how notebooks get version-controlled deployment.
- **Recommended compute is serverless jobs**; classic jobs and classic all-purpose are also supported.
- **All-purpose compute is supported but discouraged**, for the reasons module 2 covered.

> ON THE EXAM
>
> Reading parameters with dbutils.widgets.get is how a notebook task receives values from the job.

The SQL task

- **Four things it can run:** a saved query, a dashboard, an alert, or a SQL file.
- **Runs on a SQL warehouse**, not on a cluster. Serverless is recommended; pro is supported.
- **Classic SQL warehouses are not supported** for SQL tasks.
- **A dashboard task refreshes the dashboard**, which is how a dashboard gets updated on a schedule after a pipeline.

> ON THE EXAM
>
> The dashboard task is why the exam guide names dashboards. It is a task type, not a Databricks SQL topic in its own right.

The pipeline task

- **The pipeline must be a triggered pipeline.** Continuous pipelines are not supported as a job task.
- **The reasoning:** a continuous pipeline is already running, so triggering it would be redundant.
- **You can optionally trigger a full refresh** from the task.
- **Parameter overrides can be set** on the task, without editing the pipeline.

> ON THE EXAM
>
> Jobs are procedural: they define relationships between tasks. Pipelines are declarative: they define relationships between datasets.

Two more worth knowing

- **Run job** runs another job as a task, which is how you compose a large workflow from smaller reusable ones.
- **It keeps ownership and scheduling separate**, so each sub-job can also be run on its own.
- **JAR and Spark Submit run on classic compute only.** Serverless is not supported for them.
- **Spark Submit is classic jobs only**, not even classic all-purpose.

> ON THE EXAM
>
> Run job is the answer when a scenario describes a workflow that must also be runnable independently by another team.

Tasks with no dependency between them **run in parallel**. That is a feature, and it is also how people accidentally create race conditions.

Shapes in the graph

- **Fan-out:** one task, several dependents. They all start when it finishes, in parallel.
- **Fan-in:** one task depending on several. It waits for all of them.
- **A diamond** is both: split, work in parallel, then converge.
- **No cycles.** It is a directed acyclic graph, so a task cannot depend on something downstream of itself.

> ON THE EXAM
>
> A fan-in task is the natural place for a validation step, because it is the first point where everything upstream is known to have succeeded.

Recommended compute by task type

TASK	RECOMMENDED	ALSO SUPPORTED
Notebook, Python script, Python wheel	Serverless jobs	Classic jobs, classic all-purpose
SQL, dbt	Serverless SQL warehouse	Pro SQL warehouse
Pipeline	Serverless pipeline	Classic pipeline
JAR	Classic jobs	Classic all-purpose
Spark Submit	Classic jobs	Classic jobs only

ON THE EXAM

Serverless is the recommendation nearly everywhere. The exceptions are JAR and Spark Submit, which need classic compute.

Sharing compute between tasks

- **Several tasks in one job can use the same compute resource.**
- **A shared job cluster starts once** and is reused, rather than each task paying start-up separately.
- **But it stays alive across the whole job**, including while a long unrelated task runs.
- **Serverless removes the trade-off**, since there is no start-up cost to amortise and no idle to pay for.

> ON THE EXAM
>
> With classic compute, sharing a cluster trades start-up cost for idle cost. Which wins depends on the shape of the DAG.

Parameters

- **Job parameters** are defined once on the job and available to every task in it.
- **Task parameters** are set on an individual task.
- **A notebook reads them as widgets**, using dbutils.widgets.get.
- **Dynamic value references** supply runtime values such as the job id, run id and start time.

> ON THE EXAM
>
> Parameters are what make one job definition serve dev, test and prod. Module 21 uses the same idea in bundles.

Passing values between tasks

- **Task values let a task return a result** into the job environment for later tasks to read.
- **Set one in a notebook** with dbutils.jobs.taskValues.set, and read it downstream with get.
- **They are what If/else conditions test**, alongside job parameters and dynamic values.
- **They are small values, not datasets.** Pass a row count or a status, not a DataFrame.

> ON THE EXAM
>
> Writing to a table is how you pass data between tasks. Task values are for passing a decision.

Notifications and monitoring

- **Notifications can fire on start, success, failure**, and on a duration threshold being exceeded.
- **They can be set per job or per task**, so a critical task can alert separately.
- **Destinations include email** and integrations such as Slack or PagerDuty.
- **The system.lakeflow schema** holds job and task run records, queryable across the account.

> ON THE EXAM
>
> Joining the jobs system tables to the billing tables is how you attribute cost to individual jobs. Module 23 uses this.

Who the job runs as

- **A job runs as a specific identity**, and that identity's permissions determine what it can read and write.
- **Use a service principal for production**, not a person.
- **A job owned by an individual breaks when they leave**, or when their permissions change.
- **Job permissions are separate:** who can view, manage run, or manage the job itself.

> ON THE EXAM
>
> 'Run as' answers a whole family of permission questions. If a job fails on access, check the identity before checking the grant.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Two tasks ran at the same time unexpectedly	No dependency was declared between them
A continuous pipeline in a job task	Not supported. Pipeline tasks need a triggered pipeline.
A dashboard must refresh after the data loads	A dashboard task, after the transformation tasks
A step must also be runnable on its own	A Run job task
Spark Submit on serverless	Not supported. Classic jobs only.
A value must reach a later task	Task values, via dbutils.jobs.taskValues
The job broke when an engineer left	It was running as a person, not a service principal

ON THE EXAM

Seven lines. The second is the cleanest elimination in the module.

> A team has a Lakeflow declarative pipeline running in continuous mode, ingesting from Kafka. They want to add a downstream aggregation notebook that runs after the pipeline has processed each batch.
>
> An engineer proposes a job with two tasks: a pipeline task pointing at the continuous pipeline, then a notebook task depending on it.

What is the problem?

A. Nothing; this is the standard way to sequence a pipeline and a notebook

B. Continuous pipelines cannot be used as a job task; the pipeline must be triggered, or the notebook must run separately

C. A notebook task cannot depend on a pipeline task

D. The pipeline task would need to be the last task in the job

◆ PAUSE AND DECIDE

What is the problem?

A. Nothing is wrong

It is standard for a triggered pipeline. The scenario specifies continuous, which is the one case it does not work for.

B. Continuous pipelines cannot be a job task

Correct. A pipeline task triggers a triggered pipeline. A continuous pipeline is already running, so there is nothing to trigger and it is not supported.

C. A notebook cannot depend on a pipeline task

It can. Any task can depend on any other; that is what the DAG is for.

D. The pipeline task must be last

Invented ordering rule. Task order comes entirely from declared dependencies.

THE SIGNAL

A continuous pipeline never 'finishes a batch', so there is no completion event for a downstream task to wait on.

A job has five tasks. Task A loads bronze. Tasks B, C and D each build a different silver table from bronze, and each takes about 20 minutes. Task E builds a gold table from all three.

Only B declares a dependency on A. C and D declare none. E depends on B, C and D. The job intermittently produces a gold table missing rows.

What is wrong?

A. E should depend only on B, since B already depends on A

B. C and D have no dependency on A, so they may start before bronze is loaded

C. The three silver tasks should run sequentially, not in parallel

D. Task E needs a retry policy to handle the missing rows

◆ PAUSE AND DECIDE

What is wrong?

A E should depend only on B

Backwards. Removing dependencies from E would let it run before C and D finish, making the problem worse and more frequent.

B C and D lack a dependency on A

Correct. Undeclared means parallel, so C and D start immediately alongside A and read bronze while it is still being written.

C Run the silver tasks sequentially

Would accidentally fix it by forcing an order, at the cost of 40 extra minutes. It treats the symptom, not the missing dependency.

D Add a retry policy to E

E is not failing. It succeeds and produces incomplete output, which retries cannot detect or correct.

THE SIGNAL

'Intermittently' is the tell. A race condition depends on timing, so it appears and disappears with no change to the code.

> A job runs eight short notebook tasks in sequence, each taking about 90 seconds. Each task is configured with its own classic job cluster. The job takes nearly 45 minutes end to end.
>
> The team is surprised, since the actual processing is around 12 minutes in total.

What is happening, and what is the best fix?

A. Each task is starting its own cluster; use a shared job cluster, or move to serverless

B. The tasks should run in parallel rather than in sequence

C. The clusters are undersized; increase the node type for each task

D. Combine all eight notebooks into a single notebook task

◆ PAUSE AND DECIDE

What is happening, and what is the best fix?

A Eight cluster starts; share a cluster or go serverless

Correct. Eight classic clusters at roughly four minutes each is about 32 minutes of pure start-up, which matches the gap exactly.

B Run them in parallel

The scenario says in sequence, which usually means they depend on each other. Parallelising would also still start eight clusters.

C The clusters are undersized

Processing is 12 minutes across eight tasks. The missing 33 minutes is not compute time, so faster nodes change almost nothing.

D Combine into one notebook

It would work by accident, since one task means one cluster. It also throws away per-task retries, monitoring and reuse.

THE SIGNAL

45 minutes wall clock, 12 minutes of work. Whenever that gap is large and the tasks are short, look at start-up before compute.

> A job's first task loads new records and needs to tell later tasks how many arrived. If zero rows arrived, an expensive downstream rebuild should be skipped entirely.
>
> An engineer proposes writing the count to a small Delta table, which the downstream task then reads to decide whether to proceed.

What is the more idiomatic approach?

A The Delta table approach is correct; it is the only way to share state between tasks

B Set a task value with dbutils.jobs.taskValues, and branch on it with an If/else condition

C Use a job parameter, set by the first task at runtime

D Combine both tasks into one notebook so the count stays in memory

◆ PAUSE AND DECIDE

What is the more idiomatic approach?

A The Delta table is the only way

It works, and 'only way' is wrong. It also creates a table whose sole purpose is passing one number between two tasks.

B A task value, branched on with If/else

Correct. Task values exist to return a result into the job environment, and If/else conditions can test them directly.

C A job parameter set at runtime

Job parameters are supplied when the run starts, not written by a task mid-run. Wrong direction.

D Combine into one notebook

Removes the ability to skip the expensive work as a separate task, which was the entire requirement.

THE SIGNAL

Task values pass a decision; tables pass data. A single row count is a decision, so it belongs in a task value.

Declare dependencies, not order. Choose compute per task. And remember that **anything you did not connect runs in parallel**.

MODULE 17 · DOMAIN 4 - WORKING WITH LAKEFLOW JOBS

Control flow: retries, branching, looping

Retries, the six Run if conditions, If/else branching, For each loops, disabled tasks and repair runs.

DOMAIN 4 of 7 EXAM WEIGHT 16% MODULE 17 of 36

admin@sthithapragna.com

Control flow is about the runs that **do not** go to plan. The happy path needs no configuration at all.

Retries

- **A retry policy re-runs a failed task** up to a set number of times before the task is considered failed.
- **You can set a minimum interval between retries**, so a struggling downstream system gets time to recover.
- **Errors are often transient**, and a restart resolves them. That is what retries are for.
- **Not all job configurations support task retries.**

> ON THE EXAM
>
> In continuous trigger mode, Databricks retries automatically with exponential backoff. You do not configure that.

A retry only helps if running the task twice is **safe**. Lakeflow Jobs does not make your tasks idempotent. Your task logic does.

The six Run if conditions

CONDITION	THE TASK RUNS WHEN
All succeeded	Every dependency ran and succeeded. The default.
At least one succeeded	At least one dependency succeeded
None failed	No dependency failed, and at least one ran
All done	Every dependency finished, whatever the outcome
At least one failed	At least one dependency failed
All failed	Every dependency failed

ON THE EXAM

All succeeded is the default. The last three exist so that failure handling and cleanup can be part of the same job.

Two different outcomes when the condition is unmet

- **All succeeded, At least one succeeded, None failed:** unmet means the task is marked **Upstream failed**.
- **At least one failed, All failed:** unmet means the task is marked **Excluded**.
- **Excluded tasks are skipped and treated as successful.**
- **If all of a task's dependencies are excluded, that task is excluded too**, whatever its own condition says.

> ON THE EXAM
>
> Upstream failed is a problem. Excluded is a cleanup task correctly deciding it has nothing to clean up.

A job's success or failure is determined by the outcome of its **leaf tasks** — the tasks with no downstream dependencies.

If/else tasks

- **Run if branches on the outcome of dependencies.** If/else branches on a value.
- **It can test task values, job parameters, or dynamic values.**
- **Two outputs, true and false**, and you attach different downstream tasks to each.
- **It is a task in its own right**, not a setting on another task.

> ON THE EXAM
>
> Row count is zero, so skip the rebuild. Environment is prod, so send the alert. Those are If/else, not Run if.

Run if or If/else?

RUN IF

Branch on outcome

- A property of a task, set on its dependencies
- Six fixed conditions about success and failure
- Answers: did the work before me succeed?
- Used for cleanup, fallback and failure handling

IF/ELSE

Branch on a value

- A task in its own right, with true and false outputs
- Tests task values, job parameters or dynamic values
- Answers: what does the data or configuration say?
- Used for skipping work, environment switches, thresholds

THE TIEBREAKER

If a scenario mentions succeeded or failed, that is Run if. If it mentions a number, a name or a flag, that is If/else.

For each tasks

- **Runs another task in a loop**, passing a different set of parameters to each iteration.
- **It requires a nested task.** The For each task alone does nothing.
- **The nested task is one of the standard task types**, such as a notebook.
- **Iterations can run concurrently**, which is the point: fourteen regions processed in parallel.

> ON THE EXAM
>
> For each exists because a DAG cannot contain a cycle. Looping had to become a task type rather than a graph shape.

Disabled tasks

- **Disabling skips a task at run time** without removing it from the job definition.
- **Disabled is neither success nor failure.** It is its own terminal state.
- **So a downstream task with All succeeded will not run**, because disabled does not satisfy the gate.
- **A downstream task with All done will run**, because disabled counts as done.

> ON THE EXAM
>
> An If/else or For each task whose provider task is disabled fails. Disabling upstream of control flow is not a safe way to pause work.

A repair re-runs only the **unsuccessful** tasks and their dependents. Successful tasks, and anything downstream of them, are not run again.

What repair does and does not guarantee

- **It re-runs each unsuccessful task from the beginning.** Not from where it stopped.
- **Lakeflow Jobs does not make tasks idempotent.**
- **So if a task wrote part of its output before failing, re-running it can duplicate that data.**
- **Whether a repair is safe depends on the task's own logic.** Merge is safe; append usually is not.

> ON THE EXAM
>
> This is the same idempotency question as module 10 and as retries, arriving for the third time. It is worth expecting.

Reading a failed run

- **The Runs tab shows a matrix view**, with each task's outcome across recent runs.
- **A task can be unsuccessful two ways:** it failed, or it was skipped because a dependency failed.
- **Hover a failed task for its metadata**, then open the task run for logs and the error.
- **Find the first failure, not the loudest one.** Downstream skips are consequences, not causes.

> ON THE EXAM
>
> The matrix view is designed for exactly this: seeing at a glance which task broke and whether it has broken before.

Cancellation

- **Cancelling a task run propagates to downstream tasks.**
- **Tasks whose Run if handles failure still run**, so a cleanup task fires on a cancellation.
- **That is deliberate**, and it is how you guarantee temporary resources get tidied up.
- **A cancelled run can also be repaired**, like a failed one.

> ON THE EXAM
>
> If a scenario needs something to happen 'whatever else occurs', All done covers success, failure and cancellation alike.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
A cleanup task must run whatever happened	Run if: All done
A task must run only when something failed	Run if: At least one failed
Skip expensive work when a count is zero	If/else on a task value
The same task for every region	For each with a nested task
Recover a failed run without redoing everything	Repair run
The repair duplicated data	The task is not idempotent. Repair re-runs from the start.
The job reported success but a task failed	Job status comes from leaf tasks; Excluded counts as success
A transient network error	A retry policy

ON THE EXAM

Eight lines. The sixth and seventh are the two facts most likely to surprise you on the day.

> A job creates a temporary staging table, runs three transformation tasks against it, then drops it. The drop task currently depends on all three transformations with the default Run if condition.
>
> When any transformation fails, the staging table is left behind. It has accumulated for months and is now large enough to matter.

What should the Run if condition on the drop task be?

A All succeeded, which is already correct

B All done

C At least one failed

D None failed

◆ PAUSE AND DECIDE

What should the Run if condition on the drop task be?

A All succeeded

This is the current setting and the cause of the problem. When a transformation fails the condition is unmet, so the drop never runs.

B All done

Correct. The drop runs once all three dependencies have finished, regardless of outcome, so the staging table is removed either way.

C At least one failed

Inverts the problem. The table would now be dropped only when something failed, and left behind on every successful run.

D None failed

Still requires that nothing failed, so it behaves like the current setting in exactly the case that matters.

THE SIGNAL

Cleanup should not depend on the outcome of the work it is cleaning up after. All done is the condition that expresses that.

> A job's third task appends new records to a fact table. One night it fails part way through, after writing roughly 60% of the batch. An engineer uses Repair run to recover, which succeeds.
>
> The next morning, the fact table contains about 160% of the expected rows for that date.

What happened?

A. Repair re-ran the task from the beginning, and because it appends, the partial output was duplicated

B. Repair re-ran all tasks in the job, including the earlier successful ones

C. The retry policy fired at the same time as the repair, running the task twice

D. Delta failed to roll back the partial write, which is a known defect

◆ PAUSE AND DECIDE

What happened?

A Repair re-ran from the beginning; the append duplicated

Correct. Repair re-runs each unsuccessful task from the start. Lakeflow Jobs does not make tasks idempotent, so a partial append is added to twice.

B Repair re-ran everything

The opposite of what repair does. Successful tasks and their dependents are deliberately not re-run.

C A retry fired alongside the repair

Invents a concurrent execution. The 160% figure matches one partial write plus one full write, not two full runs.

D Delta failed to roll back

Delta writes are atomic per write. A task that committed several batches leaves those committed; that is correct behaviour, not a defect.

THE SIGNAL

60% written, then 100% written again, gives 160%. The arithmetic identifies the cause before you read the options.

> A job ingests a daily file, then rebuilds a large gold aggregate that takes 40 minutes. On roughly a third of days the supplier sends no file, so there is nothing new, and the rebuild produces an identical result.
>
> The team wants to skip the rebuild on those days. The ingestion task can easily compute how many rows it loaded.

How should this be built?

A. Set the rebuild task's Run if condition to At least one succeeded

B. Have the ingestion task set a task value with the row count, then use an If/else task to branch on it

C. Give the ingestion task a retry policy so it waits for the file to arrive

D. Disable the rebuild task on days when no file arrives

◆ PAUSE AND DECIDE

How should this be built?

A Run if: At least one succeeded

Run if branches on outcome, and the ingestion task succeeds either way. Loading zero rows is not a failure.

B A task value plus an If/else task

Correct. The decision depends on a value rather than an outcome, which is exactly what If/else is for.

C A retry policy on ingestion

Retries handle failures. No file is not a failure, so there is nothing to retry, and retrying would only delay the run.

D Disable the rebuild task

Disabling is a manual configuration change. Somebody would have to do it before each run, having already checked whether a file arrived.

THE SIGNAL

The ingestion task succeeds whether or not a file arrived. Nothing about the outcome distinguishes the two days — only the value does.

A job has a validation task that runs after three loaders, with Run if set to At least one failed, so it can log details and alert an on-call engineer. It is the last task in the graph.

One night a loader fails. The next morning nobody has been alerted, and the job's run history shows the run as Succeeded.

Why did the run report success?

A The failed loader was automatically retried and eventually succeeded

B Job status is determined by leaf tasks, and the validation task ran and succeeded as the only leaf

C Run if conditions suppress the failure status of their dependencies

D The run history shows the status of the last task only, which is a display limitation

◆ PAUSE AND DECIDE

Why did the run report success?

A The loader was retried and succeeded

Then the validation task would not have run at all, since its condition requires a failure. The scenario implies it did run.

B Job status comes from leaf tasks

Correct. Databricks determines run status from the outcome of leaf tasks. The validation task is the only leaf, it ran, and it succeeded.

C Run if suppresses dependency failure status

It does not change any dependency's status. It only decides whether this task runs.

D A display limitation

The status is computed, not merely displayed. Calling it a display bug misses a designed behaviour.

THE SIGNAL

The design is working. Put notifications on the task, not only on the job, or a handled failure becomes an invisible one.

Run if branches on **outcome**. If/else branches on **value**. And no mechanism here makes a task safe to run twice — **only your task logic does that**.

MODULE 18 · DOMAIN 4 - WORKING WITH LAKEFLOW JOBS

Triggers and scheduling

The five trigger types, cooldown and debouncing, and the choice between time-based and data-driven.

DOMAIN 4 of 7 EXAM WEIGHT 16% MODULE 18 of 36

admin@sthithapragna.com

A time-based trigger runs when the clock says so. A data-driven trigger runs when the data says so. **Almost every scheduling problem is a symptom of using the first where you needed the second.**

The five trigger types

TYPE	FIRES WHEN
Scheduled	A time-based schedule is reached
File arrival	New files arrive in a monitored Unity Catalog storage location
Table update	One or more monitored Unity Catalog tables are updated
Continuous	The previous run completes or fails, keeping the job always running
None (manual)	Someone runs it, or an external orchestrator calls the API

ON THE EXAM

The exam guide names the first three. Note what continuous actually means: a new run starts when the last one ends, whether it succeeded or not.

Scheduled triggers

- **A time-based schedule**, expressed simply in the UI or as a cron expression for anything unusual.
- **Set the time zone explicitly.** Otherwise daylight saving will move your job twice a year.
- **The clock does not know whether the data arrived.** That is the whole limitation.
- **Still right for genuinely time-bound work:** a report due at 09:00, a month-end close, a regulatory deadline.

> ON THE EXAM
>
> A schedule is a promise about when you will start, never a promise about whether there is anything to do.

File arrival triggers

- **Monitors a Unity Catalog storage location** — an external location or a volume — and fires when new files land.
- **Enable file events on the external location** for best performance. With file events, there is no limit on the number of files.
- **The path cannot contain wildcards.** No asterisk, no question mark.
- **The path must not contain external tables**, or managed locations of catalogs and schemas.

> ON THE EXAM
>
> Best practice: create a volume mapped to exactly the subdirectory you want watched, and set the trigger on that volume's root.

Why the subpath advice matters

- **A trigger on a subpath of a large external location still sees changes elsewhere** in that location as metadata to process.
- **In a busy environment that can exceed the trigger's processing time limit**, putting it into an error state.
- **A dedicated volume scopes the watching precisely**, so unrelated activity does not affect it.
- **Configure notifications on job failure** to be told when a trigger fails to evaluate.

> ON THE EXAM
>
> A trigger that has quietly entered an error state stops firing. Without a notification, nobody finds out until someone asks where the data is.

Table update triggers

- **Fires when one or more Unity Catalog tables are updated.** Generally available since October 2025.
- **With several tables, choose the mode:** run when any one updates, or only once all of them have.
- **Limit of 10 tables per trigger.** A monitored view's source tables count towards that limit.
- **A separate limit of 10 dependent views** per monitored view.

> ON THE EXAM
>
> A view depending on 11 tables cannot be used in a table update trigger at all. Two views of 6 tables each count as 12.

The two timing controls

OPTION	PATTERN	WHAT IT DOES
Minimum time between triggers	Cooldown	At most one run per interval. Arrivals during the cooldown wait.
Wait after last change	Debouncing	Waits this long after the latest change. Each new change resets the timer.

ON THE EXAM

Both options are available on file arrival and table update triggers alike. They solve two different problems.

Using them together

- **Either can be set alone, or both together.**
- **With both, the cooldown applies first**, then the debounce timer.
- **Example:** minimum 120 seconds and wait 60 seconds means no run before 120 seconds have elapsed, even if a change happens in the first 60.
- **Debouncing is the answer to 'a batch of 200 files arrived and the job ran 200 times'.**

> ON THE EXAM
>
> Cooldown protects against running too often. Debouncing protects against running too early, on a partial batch.

Continuous mode

- **A new run starts whenever the previous run completes or fails.**
- **Failures are retried automatically with exponential backoff**, as module 17 covered.
- **It costs continuously**, because something is always running.
- **Serverless jobs support continuous scheduling only with bounded triggers** such as Trigger.AvailableNow.

> ON THE EXAM
>
> Continuous is for genuine streaming. For data arriving in bursts, a file arrival trigger gives similar latency at a fraction of the cost.

Time-based or data-driven?

TIME-BASED

The clock decides

- Predictable, and easy to reason about across teams
- Right when the deadline is the requirement
- Runs whether or not there is anything to do
- Needs a margin, which is waste on good days and too little on bad ones

DATA-DRIVEN

The data decides

- Runs as soon as the data is actually there
- No margin to guess at, and no wasted empty runs
- Lower latency and usually lower cost
- Needs Unity Catalog, and a path or tables to watch

A NAMED OBJECTIVE

The exam guide makes this choice an objective in its own right, phrased as 'based on data availability and pipeline dependencies'.

When time-based is still right

- **The deadline is the requirement:** a report at 09:00, a month-end close, a regulatory submission.
- **The source is genuinely periodic** and reliable, and there is nothing to gain from reacting sooner.
- **You must coordinate with an external system** that expects work in a fixed window.
- **The source is outside Unity Catalog**, so there is nothing for a data-driven trigger to watch.

> ON THE EXAM
>
> Do not treat data-driven as automatically better. Some work genuinely is time-bound, and a schedule expresses that honestly.

When data-driven wins

- **Arrival times are irregular.** The margin you would need is the problem you are solving.
- **A downstream job depends on an upstream one** that sometimes runs late.
- **Empty runs are costing money**, because the schedule fires whether or not anything changed.
- **Latency matters**, and you want processing to begin the moment data lands.

> ON THE EXAM
>
> 'Pipeline dependencies' in the objective points at table update triggers. A downstream job watching a table replaces guessing when the upstream finishes.

Concurrency and queueing

- **Maximum concurrent runs** controls how many runs of the same job can be in flight at once.
- **The default is one**, so a new trigger while a run is in progress does not start a second run.
- **Queueing holds the triggered run** rather than dropping it, so it starts when the current one finishes.
- **Concurrency above one needs idempotent tasks**, or two runs will write over each other.

> ON THE EXAM
>
> A job whose runtime exceeds its schedule interval will queue or skip. That is usually the first symptom of a job that has outgrown its window.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Files arrive at unpredictable times	File arrival trigger
A batch of files arrives and the job runs once per file	Wait after last change (debouncing)
A source updates constantly and runs are too frequent	Minimum time between triggers (cooldown)
A downstream job must wait for an upstream one	Table update trigger on the table it writes
More than 10 tables to monitor	Not possible in one trigger. Split, or watch fewer.
A wildcard in the monitored path	Not supported for file arrival triggers
A report due at a fixed time	Scheduled. The deadline is the requirement.

ON THE EXAM

Seven lines. The second and third are the pair most often confused with each other.

> A supplier uploads a daily file to a Unity Catalog volume. The delivery time varies between 02:00 and 09:00 depending on their batch. The team currently schedules the ingestion job at 10:00 to be safe.
>
> The business wants the data available sooner. On most days the file has arrived by 03:00, so seven hours are being wasted.

What should the team do?

A. Move the schedule to 03:00, since that is when the file usually arrives

B. Use a file arrival trigger on the volume, so the job starts when the file lands

C. Run the job every 30 minutes from 02:00 until it finds a file

D. Switch the job to continuous mode

◆ PAUSE AND DECIDE

What should the team do?

A. Move the schedule to 03:00

Trades one guess for a worse one. On the days the file arrives at 09:00, the job runs against nothing and the data is missed entirely.

B. A file arrival trigger on the volume

Correct. The trigger fires when the file lands, whether that is 02:00 or 09:00. No margin to guess and no wasted wait.

C. Poll every 30 minutes

Fourteen runs a day, thirteen of which find nothing, each paying start-up. It reinvents a file arrival trigger badly.

D. Continuous mode

Keeps a job permanently running to wait for one file a day. Enormous cost for a daily delivery.

THE SIGNAL

'To be safe' is the phrase to notice. A margin exists because nobody knows when the data arrives, which is exactly what a data-driven trigger removes.

> A partner delivers around 200 small files in a burst, over roughly four minutes, twice a day. A file arrival trigger is configured on the volume with no advanced options set.
>
> The job is running close to 200 times per delivery, mostly against partial data, and the compute cost has become the team's largest line item.

What should be configured?

A. Minimum time between triggers, set to a few minutes

B. Wait after last change, set to longer than the gap between files in a burst

C. Maximum concurrent runs, set to 1

D. Switch to a scheduled trigger twice a day

◆ PAUSE AND DECIDE

What should be configured?

A. Minimum time between triggers

A cooldown caps how often runs start, so it helps with cost. It does not wait for the batch to finish, so runs still fire on partial data.

B. Wait after last change

Correct. Debouncing waits for quiet, and each new file resets the timer, so one run fires after the whole burst has landed.

C. Maximum concurrent runs of 1

Already the default. It prevents overlapping runs, not repeated ones, so 200 runs would simply queue instead.

D. A scheduled trigger twice a day

Fixes it by abandoning the data-driven approach and reintroducing the margin problem. It also assumes the deliveries are punctual.

THE SIGNAL

Two symptoms: too many runs, and partial data. A cooldown addresses the first only. Debouncing addresses both.

> An ingestion job writes a silver table and usually finishes by 04:20. A downstream aggregation job is scheduled at 05:00 to give it margin. Once or twice a month the ingestion job overruns and the aggregation runs against stale data, producing wrong figures with no error.
>
> Both tables are in Unity Catalog.

What is the best fix?

A. Move the aggregation job's schedule to 06:00 for more margin

B. Use a table update trigger on the silver table, so the aggregation starts when the table is written

C. Combine both jobs into one job with a dependency between the tasks

D. Add a retry policy to the aggregation job

◆ PAUSE AND DECIDE

What is the best fix?

A More margin

A bigger guess is still a guess. It delays the data every day to protect against a failure that happens twice a month, and eventually overruns again.

B A table update trigger on the silver table

Correct. The dependency becomes explicit: the aggregation starts because the table was written, not because the clock reached a time.

C Combine into one job

Genuinely valid, and it couples the two. If the teams own them separately, or the aggregation has other triggers, this is a heavier change.

D A retry policy

The aggregation is not failing. It succeeds and produces wrong figures, which retries cannot detect.

THE SIGNAL

This is the objective's phrase 'pipeline dependencies' exactly. A dependency encoded as two cron times is a dependency waiting to break.

> A team wants a reconciliation job to run only after all fourteen of their regional silver tables have been refreshed. They plan a single table update trigger listing all fourteen, with the mode set to run once all tables have updated.
>
> The trigger cannot be saved.

Why, and what are the options?

A. Table update triggers support only one table; they must use fourteen separate jobs

B. There is a limit of 10 tables per trigger; they must reduce the count, or use a different approach

C. The 'once all tables have updated' mode is not supported; only 'any table' is available

D. Table update triggers require the tables to be in the same schema

◆ PAUSE AND DECIDE

Why, and what are the options?

A. Only one table supported

False. Multiple tables are supported, with a choice of any-one or all-of semantics. The constraint is the count, not the concept.

B. A limit of 10 tables per trigger

Correct. The documented limit is 10. A monitored view's source tables count towards it too, so views are not a way around it.

C. 'All tables' mode is unsupported

It is supported, and it is exactly what this team wants. The mode is not the problem.

D. Tables must share a schema

Invented restriction. The tables need to be in Unity Catalog, not in one schema.

THE SIGNAL

A view over the fourteen tables does not help: its source tables count towards the same limit of 10.

Ask what the job is actually waiting for. If the answer is **data**, a clock is the wrong instrument. If the answer is **a deadline**, it is the right one.

MODULE 19 · DOMAIN 4 - WORKING WITH LAKEFLOW JOBS

Jobs in production

Two sets of permissions, the run-as identity, parameters that actually reach your code, tags and cost.

DOMAIN 4 of 7 EXAM WEIGHT 16% MODULE 19 of 36

admin@sthithapragna.com

A job has **two separate permission systems**. One controls who can touch the job. The other controls what the job itself can touch.

Job privileges

LEVEL	WHAT IT ALLOWS
CAN VIEW	View the job configuration and monitor runs
CAN MANAGE RUN	Trigger, stop and restart runs
CAN MANAGE	Full administrative access: edit configuration, tasks and scheduling
IS OWNER	The creator by default. A job cannot have more than one owner.

ON THE EXAM

Each level includes the grants below it. Workspace admins are granted CAN MANAGE. Only grant CAN MANAGE or IS OWNER to people trusted to modify production code.

Anyone with **CAN MANAGE RUN or higher** can override job parameter values when running manually. A default parameter value is **not a security control**.

The run-as identity

- **Determines what the job can access** — the data, the secrets, the external locations.
- **Use a service principal for production jobs**, not a named person.
- **A job running as a person breaks when they leave**, or when their access changes for an unrelated reason.
- **It also lets you restrict write access to production data**, granting it to the service principal rather than to every engineer.

> ON THE EXAM
>
> If a job runs as a user, that user needs every permission the job needs — including write access to production tables.

Four things called parameters

CONCEPT	WHAT IT IS
Job parameter	A key-value pair defined at the job level and pushed down to tasks
Task parameter	A key-value pair or JSON array defined on one task
Dynamic value reference	Syntax for referencing job conditions, metadata and parameters
Task value	A value produced during a run, for later tasks to read

ON THE EXAM

A notebook widget is not a fifth store. It is the interface through which notebook code reads whichever value was delivered.

The task type changes how parameters arrive

- **A notebook task receives key-value parameters.** Job parameters are pushed down automatically.
- **A Python script or JAR task receives an ordered JSON array.** Job parameters are not pushed down the same way.
- **Those tasks reference job parameters explicitly**, using dynamic value references in their task parameters.
- **Job parameter defaults are resolved when the run starts**, and can be overridden for that run.

> ON THE EXAM
>
> 'I set a job parameter and the script cannot see it' is almost always this. Key-value tasks get it free; array tasks must ask.

Dynamic value references

- **Syntax for injecting values you cannot know when writing the job**, resolved at run time.
- **Job and run metadata:** the job id, the run id, the scheduled start time.
- **Other parameters and task values**, so a task parameter can be built from a job parameter.
- **The start time reference is how a task learns which date it is processing**, rather than reading the clock itself.

> ON THE EXAM
>
> A task that calls current_date() processes today. A task given the scheduled start time processes the right day, even on a re-run three days later.

Tags

- **Labels or key-value attributes attached to a job**, edited in the Job details pane.
- **They flow through to billing**, which is how compute cost gets attributed to a team, project or cost centre.
- **Without them, the bill is one undifferentiated number**, and nobody can act on it.
- **Enforce them with a cluster policy**, as module 2 covered, so an untagged job cannot be created.

> ON THE EXAM
>
> Tagging is the difference between knowing you spent a lot and knowing which team spent it. That is what makes it actionable.

Notifications that catch the quiet failures

- **On failure** is the obvious one, and it misses everything that does not fail.
- **Duration thresholds** alert when a run takes longer than expected, catching gradual degradation.
- **Per-task notifications** matter because of the leaf task rule from module 17.
- **A job that succeeds while doing nothing useful sends no notification at all.**

> ON THE EXAM
>
> Module 17's scenario is the case in point: a handled failure is not a job failure, so job-level alerting stays silent.

Compute governance

- **Serverless compute for jobs and SQL warehouses always use Unity Catalog.**
- **For classic compute, Databricks recommends standard access mode** for supported workloads.
- **Use dedicated access mode only when required**, as module 2 set out.
- **Unity Catalog governance requires a supported compute configuration.** No isolation shared is not one.

> ON THE EXAM
>
> If a scenario needs governed data access from a job, the compute configuration is as much a part of the answer as the grants are.

What lives in the Job details pane

SETTING	WHY IT MATTERS
Schedule or trigger	Module 18. Time-based or data-driven.
Job parameters	Pushed down to key-value tasks; overridable at run time
Compute configuration	Cost and governance both follow from this
Tags	Cost attribution
Notifications and duration thresholds	Whether anyone finds out
Maximum concurrent runs	What happens when a trigger fires mid-run
Git settings	Which branch or tag the job runs from
Permissions	Who can view, run and manage

ON THE EXAM

Eight settings, and only the first two are about what the job does. The other six are about operating it.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Someone ran the job with a different parameter	CAN MANAGE RUN allows it. Defaults are not security.
An on-call engineer must rerun but not edit	CAN MANAGE RUN
The job broke when an engineer left	It ran as a person. Use a service principal.
A Python script cannot see a job parameter	Array tasks need a dynamic value reference
A repaired run processed the wrong date	It read the clock instead of the scheduled start time
Nobody can tell which team the bill belongs to	Tags, enforced by a cluster policy
The job runs slower every month, but never fails	A duration threshold notification

ON THE EXAM

Seven lines. The first is the one most likely to be phrased as a security question rather than a jobs question.

> A job takes a parameter named target_catalog, with a default value of dev_catalog. The team relies on this default to ensure that only their deployment pipeline, which passes prod_catalog explicitly, can write to production.
>
> An analyst with CAN MANAGE RUN on the job uses Run now with different parameters and sets target_catalog to prod_catalog. Production data is overwritten.

What went wrong?

A. A defect; parameter overrides should require CAN MANAGE

B. A default parameter value is not a security control; CAN MANAGE RUN permits overrides, so job permissions must restrict this

C. The analyst must have been granted CAN MANAGE by mistake

D. Job parameters cannot be overridden; the deployment pipeline must have been misconfigured

◆ PAUSE AND DECIDE

What went wrong?

A **A defect requiring CAN MANAGE**

Documented behaviour, not a defect. CAN MANAGE RUN or higher can override parameters for a manual run, by design.

B **Defaults are not a security control**

Correct. The docs say so directly. The only protection is restricting who holds CAN MANAGE RUN on the job.

C **The analyst had CAN MANAGE by mistake**

CAN MANAGE RUN is sufficient. No misgrant is needed to explain this, and the scenario states the permission held.

D **Parameters cannot be overridden**

They can, both through Run now with different parameters and through the REST API.

THE SIGNAL

The team built a control out of a convenience. Defaults exist so the job runs sensibly, not to stop anyone changing them.

> A production job has run nightly for two years. It was created by an engineer who left the company last month. Her account was deprovisioned yesterday.
>
> The job now fails immediately with a permissions error on the source tables. The on-call engineer has full access to those tables and can query them without difficulty.

What is the cause and the fix?

A. The job runs as the departed engineer; change the run-as identity to a service principal

B. Grant the on-call engineer CAN MANAGE on the job

C. Re-grant SELECT on the source tables to the departed engineer's account

D. The tables were moved to a different catalog when the account was deprovisioned

◆ PAUSE AND DECIDE

What is the cause and the fix?

A It runs as the departed engineer; move to a service principal

Correct. The run-as identity determines data access. Deprovisioning that account removed the job's ability to read.

B Grant the on-call engineer CAN MANAGE

Job privileges are a separate system from run-as. His access changes who can edit the job, not what the job can read.

C Re-grant SELECT to the departed account

Reviving a deprovisioned account's access to keep production running is the opposite of the correct governance answer.

D The tables were moved

Deprovisioning an account does not move tables. It invents a cause when the timing already points at the account.

THE SIGNAL

The on-call engineer's own access is a red herring, and a deliberate one. His permissions have nothing to do with the job's.

A job's first task is a notebook that reads a job parameter called run_date and works correctly. Its second task is a Python script task that needs the same value.

The script receives nothing for run_date and fails on a missing argument. The job parameter is definitely set, and the notebook task proves it.

Why, and what is needed?

A. Job parameters only apply to the first task in a job

B. A Python script task receives an ordered JSON array, so it must reference the job parameter explicitly with a dynamic value reference

C. The script must call dbutils.widgets.get, as the notebook does

D. Job parameters must be redefined as task parameters on every task that needs them

◆ PAUSE AND DECIDE

Why, and what is needed?

A Only the first task gets them

Invented rule. Job parameters are pushed to every task that uses key-value parameters, regardless of position.

B Array tasks need an explicit dynamic value reference

Correct. Job parameters are pushed down to key-value tasks. A Python script task takes an ordered JSON array, so it must reference the parameter by name.

C Call dbutils.widgets.get in the script

Widgets are the notebook interface for reading a delivered value. They do not deliver anything to a script task.

D Redefine as task parameters everywhere

Would work, and it duplicates the value on every task and defeats the point of defining it once at job level.

THE SIGNAL

The difference is the task type, not the parameter. Key-value tasks get the push-down; array tasks must ask by name.

> A daily job processes the previous day's partition. The notebook computes the date itself with current_date() minus one day. This has worked for a year.
>
> A run fails on Friday. Nobody notices until Monday, when an engineer repairs the failed run. It succeeds. The following week, reconciliation shows Friday's data is still missing, and Sunday's has been processed twice.

What is the design fault?

A. The repair should have been a fresh run rather than a repair

B. The task derives its date from the clock, so a Monday repair processed Sunday; it should use the scheduled start time

C. The job needs a retry policy so failures are handled the same night

D. current_date() returns UTC, and the job runs in a different time zone

◆ PAUSE AND DECIDE

What is the design fault?

A A fresh run instead of a repair

A fresh run on Monday would have processed Sunday too. The problem is where the date comes from, not which button was pressed.

B It reads the clock, not the scheduled start time

Correct. A repair runs on Monday, so current_date() minus one gives Sunday. The run's own scheduled start time would have given Friday.

C A retry policy

Retries handle failures the same night, which helps. It does not fix a task that will always process the wrong day whenever it runs late.

D A time zone problem

A time zone offset shifts dates by hours, not by three days. It also would not explain a duplicate.

THE SIGNAL

Two symptoms confirm it: Friday missing and Sunday duplicated. That is one run processing the wrong day, not a run that failed to happen.

> A platform team is asked to reduce Databricks spend by 20%. The account bill is large but arrives as a single figure. There are around 400 jobs across eleven teams, and no job carries any tags.
>
> The finance director wants each team told what they are spending so they can act.

What should the platform team do first?

A Reduce all cluster sizes by 20% across the workspace

B Introduce tags on jobs and enforce them with a cluster policy, then attribute cost using the jobs and billing system tables

C Move every job to serverless compute

D Set aggressive auto-termination on every cluster

◆ PAUSE AND DECIDE

What should the platform team do first?

A Cut all cluster sizes by 20%

Applies a blunt change to 400 jobs blind. Some will fail, some will spill, and nobody can tell which mattered.

B Tag, enforce, then attribute with system tables

Correct. The stated requirement is that each team be told what they spend, which is impossible without attribution. Measure before cutting.

C Move everything to serverless

Often reduces cost, and it is a blind change across 400 jobs including JAR and Spark Submit tasks that cannot use it.

D Aggressive auto-termination everywhere

A real saving on idle clusters, and it does nothing for the stated requirement of telling each team their spend.

THE SIGNAL

The requirement is not only to spend less. It is that each team be told what they spend, which is a measurement problem before it is a cost one.

Two permission systems, one identity, and a default that protects nothing. **Most production job failures are about who, not about what.**

MODULE 20 · DOMAIN 5 - IMPLEMENTING CI/CD

Git folders and the development workflow

Cloning, branching, committing and pull requests — and the line between a development tool and a deployment mechanism.

DOMAIN 5 of 7 EXAM WEIGHT 10% MODULE 20 of 36

admin@sthithapragna.com

Databricks Git folders are a **development tool, not a deployment mechanism**. That single sentence resolves most of the questions in this domain.

Setting up

- **Add your Git provider username and a personal access token** to Databricks before you can clone a private repository.
- **Public repositories can be cloned without credentials.**
- **Credentials are per user**, which is what makes commits attributable to the right person.
- **No SSH support.** Authentication is by token, not by key.

> ON THE EXAM
>
> Because credentials are per user, a Git folder acts as that user's client. Commits carry their identity, not the workspace's.

What you can do from the workspace

OPERATION	WHERE
Clone a repository	Databricks UI, when creating the Git folder
Create and switch branches	Databricks UI
Commit and push	Databricks UI
Pull	Databricks UI
View diffs	Databricks UI
Merge branches	Databricks UI
Create and review a pull request	Your Git provider's own interface

ON THE EXAM

The last row is the one the exam guide phrases carefully. Databricks takes you there; the review happens at the provider.

The everyday workflow

01

Create a branch

From the Git folder, off the default branch.

02

Work

Edit notebooks and files. The UI highlights what changed.

03

Commit and push

With a commit message. Both happen together.

04

Open a pull request

At your Git provider, for review.

05

Merge, then pull

Merge at the provider, then pull into other folders.

BRANCH PROTECTION IS A FEATURE

If you lack permission to commit to the default branch, this is not an obstacle. It is the workflow working correctly.

Collaborating

- **Use a separate Git folder per user**, each mapped to the same remote repository.
- **Each person works in their own branch**, in their own folder.
- **A shared Git folder means shared working state**, so one person switching branch changes it for everyone.
- **One Git folder maps to one repository.**

> ON THE EXAM
>
> A Git folder is a working copy, not a repository. Two people sharing one working copy is the same problem as two people sharing one laptop.

Notebook format matters more than you would expect

- **Source formats (.py, .scala, .sql, .r)** give clean, readable diffs. Outputs are not committed by default.
- **Source-format notebooks do not store dashboard information.**
- **.ipynb preserves dashboards and visualisation definitions**, and can commit outputs.
- **But .ipynb is JSON**, so diffs are noisy and output cells clutter the history.

> ON THE EXAM
>
> Source format for code you review. IPYNB when the visualisations are part of what you are versioning.

A secret committed to Git lives in the history **forever**. Replacing it in the next commit does not remove it.

Limits worth knowing

- **Working branches are capped at 1 GB.**
- **Per-operation limits:** around 2 GB memory and 4 GB disk.
- **Databricks recommends against monorepo-backed Git folders.** Cloning one can exceed these limits.
- **Adding a large file to .gitignore does not reduce repository size**, because it is already in the history.

> ON THE EXAM
>
> Non-notebook files are stored on disk for up to 30 days. Local branches can linger for up to 30 days after the remote branch is deleted.

Sparse checkout

- **Clones only a subset of directories**, specified with cone patterns.
- **Set when you create the Git folder**, and editable in its settings afterwards.
- **Creating a file outside the cone pattern causes an error on commit and push.**
- **Fix that by editing the pattern** to include the new directory.

> ON THE EXAM
>
> Sparse checkout is the documented answer to a repository too large for Git folder limits. Splitting the repository is the other one.

What cannot live in a Git folder

- **Legacy dashboards and legacy alerts, MLflow workspace experiments and Genie Spaces** cannot be serialised.
- **Data files and Delta tables** do not belong there either.
- **Databricks will let you move them in.** The commit is what fails.
- **Keep Git folders lean:** notebooks, code files and configuration.

> ON THE EXAM
>
> The failure comes at commit time, not when you move the asset. That delay is what makes the cause hard to spot.

Git folders or bundles?

GIT FOLDERS

Development

- An embedded Git client in the workspace
- Per-user, per-branch working copies
- Great for writing, reviewing and iterating
- No environment-specific configuration

DECLARATIVE AUTOMATION BUNDLES

Deployment

- Packages code with jobs, pipelines and configuration
- Per-target variables for dev, test and prod
- Validated and deployed through the CLI
- The documented way to promote assets

THE TIEBREAKER

They are complements, not alternatives. You develop in a Git folder and deploy with a bundle.

Jobs can run from Git

- **A notebook task can take its source from a Git provider** rather than from the workspace.
- **You pin it to a branch, a tag or a commit.**
- **Pointing production at a branch means production changes when someone merges.**
- **Pinning to a tag or commit makes the deployed version explicit**, and rollback is changing the pin.

> ON THE EXAM
>
> Git settings appear in the Job details pane, as module 19 listed. This is where they matter.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Where are pull requests created and reviewed	At the Git provider, not in Databricks
Two engineers keep changing each other's branch	One Git folder each, not a shared folder
A repository too large for Git folder limits	Sparse checkout, or split the repository
Commit fails after moving an asset in	That asset type cannot be serialised
A credential was pushed to Git	Rotate it. Removing it from the next commit is not enough.
Promoting code across dev, test and prod	A bundle, not a Git folder
Notebook visualisations must be versioned	IPYNB format, not source format

ON THE EXAM

Seven lines. The sixth is the one that separates this module from the next, and it decides several exam questions.

> An engineer hard-codes a storage account access key into a notebook while debugging, then commits and pushes it to the team's GitHub repository. She notices an hour later.
>
> She removes the key from the notebook, commits again with the message 'remove key', and pushes. She reports the issue as resolved.

Is it resolved?

A Yes; the current version of the file no longer contains the key

B No; the key remains in the commit history and must be rotated. History rewriting is a separate question.

C No; she should delete the notebook from the repository instead

D Yes, provided the repository is private

◆ PAUSE AND DECIDE

Is it resolved?

A Yes, the current version is clean

Git stores history. The previous commit still contains the key and is readable by anyone with repository access.

B No; rotate the key

Correct. The credential is compromised the moment it is pushed. Rotation is the only action that actually invalidates it.

C Delete the notebook

Deleting a file removes it from the current version and not from history. Same mistake, more disruption.

D Yes if the repository is private

Private limits who can read it, and does not un-expose it. Every person with repo access, now or later, can read that commit.

THE SIGNAL

Rotate first, always. Rewriting history with git filter-repo is a legitimate second step, and it does not make the old key safe.

> A team is asked to bring their analytics assets under version control. An engineer drags a legacy Databricks dashboard and an MLflow workspace experiment into their Git folder alongside the notebooks. Databricks accepts the move without complaint.
>
> When he tries to commit, the operation fails.

What is happening?

A The Git folder has exceeded its 1 GB branch limit

B Those asset types cannot be serialised, so they cannot be committed, even though they can be moved in

C He needs to add them to the sparse checkout cone pattern

D His Git credentials have expired

◆ PAUSE AND DECIDE

What is happening?

A The 1 GB branch limit

A dashboard definition and an experiment are small. The limit exists, and nothing here approaches it.

B Those types cannot be serialised

Correct. Legacy dashboards, legacy alerts, MLflow workspace experiments and Genie Spaces can be moved into a Git folder but cannot be committed.

C Add them to the cone pattern

Sparse checkout governs which directories are cloned. It would matter for a new folder outside the pattern, not for an unsupported asset type.

D Expired credentials

Would fail every commit, including notebook-only ones. The scenario ties the failure to the moved assets.

THE SIGNAL

Databricks lets you move the asset and fails at commit. The gap between the action and the error is what makes it confusing.

> A platform team keeps all of its code in one large repository covering eleven projects and several thousand files. Engineers report that creating a Git folder takes a very long time, and some Git operations fail outright.
>
> Each engineer only ever works on one or two of the eleven projects.

What should they do?

A. Increase the Git folder size limit in workspace settings

B. Use sparse checkout with cone patterns so each engineer clones only the projects they work on

C. Add the unused projects to .gitignore

D. Have all engineers share one Git folder to avoid cloning it repeatedly

◆ PAUSE AND DECIDE

What should they do?

A Increase the size limit

The limits are platform limits, not a workspace setting you can raise.

B Sparse checkout with cone patterns

Correct. It is the documented answer for monorepos, cloning only the directories each engineer needs. Splitting the repository is the other option.

C Add unused projects to .gitignore

Misunderstands .gitignore. It stops new files being tracked; it does not remove committed content or reduce what is cloned.

D Share one Git folder

Solves the cloning cost by creating a worse problem: one working copy, so a colleague switching branch switches it for everybody.

THE SIGNAL

Databricks recommends against monorepo-backed Git folders. Sparse checkout is the mitigation; splitting the repository is the cure.

> A team promotes code to production by keeping a Git folder in the production workspace pointed at the main branch, and calling the Repos API to sync it after each merge. Jobs run notebooks from that folder.
>
> They now need the production jobs to write to a different catalog from development, and to have different cluster sizes and schedules per environment.

What should they move to?

A Keep the Git folder approach and add environment logic inside each notebook

B Declarative Automation Bundles, which package jobs and configuration with the code and support per-target variables

C Maintain a separate branch per environment, each with its own hard-coded settings

D Create a separate repository per environment

◆ PAUSE AND DECIDE

What should they move to?

A Environment logic inside each notebook

Scatters environment configuration across every notebook, where it cannot be reviewed as one thing and drifts silently.

B Declarative Automation Bundles

Correct. Bundles package jobs, pipelines and configuration alongside the code, with per-target variables and overrides for dev, test and prod.

C A branch per environment

Guarantees divergence. Branches that are never merged drift, and a fix to one must be cherry-picked into the others.

D A repository per environment

The same problem as C, with more ceremony. The code is identical; only the configuration differs.

THE SIGNAL

The Git folder approach has not failed. It has reached its limit, which is that it versions code and nothing around the code.

Develop in a Git folder. Deploy with a bundle. **The moment a question mentions dev, test and prod differing, you have left this module.**

MODULE 21 · DOMAIN 5 - IMPLEMENTING CI/CD

Declarative Automation Bundles

databricks.yml, targets and deployment modes, variables and overrides, and the CLI lifecycle.

DOMAIN 5 of 7 EXAM WEIGHT 10% MODULE 21 of 36

admin@sthithapragna.com

A bundle versions **the project**, not just the code. The jobs, the pipelines, the schedules and the configuration all live in the repository beside the notebooks.

What a bundle contains

PART	WHAT IT HOLDS
`databricks.yml`	The bundle definition: name, resources, targets, variables
`Source files`	Notebooks, Python files, SQL — the code itself
`Resource definitions`	Jobs, pipelines and other workspace assets, as YAML
`Targets`	dev, staging, prod — each with its own workspace and overrides
`Variables`	Values that differ per target

ON THE EXAM

All of it lives in one repository and goes through the same review as the code. That is the point.

The main top-level mappings

MAPPING	PURPOSE
bundle	The bundle's name, and its deployment metadata
include	Other YAML files to pull in, so one file does not hold everything
variables	Declared variables, with optional defaults
resources	Jobs, pipelines and other assets to deploy
targets	The environments, each able to override anything above
workspace	Paths in the target workspace, such as root_path
artifacts	Things to build during deployment, such as a Python wheel
run_as	The identity deployed resources run as

ON THE EXAM

Everything above targets can be overridden inside a target. That single rule is what makes one file serve three environments.

A target is an environment. Same code, same resource definitions, **different values**. That is the whole mechanism for promoting across dev, test and prod.

Development mode

- **Set mode: development** on the target, and a set of default behaviours applies.
- **Resource names are prefixed** with a dev marker including your username, so two developers do not collide.
- **Jobs and pipelines are tagged** with a dev tag.
- **Pipelines are marked development: true.**
- **--compute-id becomes available** on deploy, overriding every cluster definition in the bundle.

> ON THE EXAM
>
> The prefixing is why several engineers can deploy the same bundle to one workspace without overwriting each other's jobs.

Production mode

- **Set mode: production** and you get validations rather than conveniences.
- **Validates that all deployed pipelines are marked development: false.**
- **Validates that the current Git branch matches the branch specified in the target**, if one is specified.
- **That branch check can be overridden with --force**, which is occasionally necessary and usually a smell.

> ON THE EXAM
>
> Production mode's job is to stop you deploying something you did not mean to, from a branch you did not mean to deploy from.

Production without a service principal

- **Databricks recommends service principals for production deployments**, enforced by setting run_as.
- **If you do not use one, extra validations apply.**
- **It validates that artifact_path, file_path, root_path and state_path are not overridden to a specific user.**
- **The reasoning:** production assets deployed under one person's path belong to that person, and vanish with them.

> ON THE EXAM
>
> This is module 19's run-as argument again, enforced by the tooling rather than left to good intentions.

A variable is declared once, given a default, and **overridden per target**. That is how one catalog name serves three environments.

Declaring and overriding

WHERE	WHAT IT LOOKS LIKE
Declare	variables: catalog: {default: dev_catalog}
Reference	catalog: ${var.catalog}
Override in a target	targets: prod: variables: {catalog: prod_catalog}
Override at the command line	databricks bundle deploy -t prod --var="catalog=other"

ON THE EXAM

Four places the same value can come from. Command line beats target, target beats default.

Substitution syntax

- **${var.name}** references a declared variable.
- **${bundle.target}** gives the name of the target being deployed.
- **${workspace.current_user.userName}** is who is deploying, which is how dev mode builds its prefix.
- **${resources.jobs.my_job.id}** references another resource, so one asset can point at another.

> ON THE EXAM
>
> Referencing resources by name rather than by id is what lets the same definition deploy to three workspaces where the ids all differ.

One thing variables cannot do

- **Variables cannot dynamically set include directives.**
- **So you cannot include a different set of YAML files per target** using a variable.
- **The workaround is to substitute in CI**, editing databricks.yml before running the deploy.
- **In practice, prefer overrides**. Define everything and override what differs, rather than including different files.

> ON THE EXAM
>
> If a design needs entirely different resources per environment, that is usually a sign the environments have drifted apart.

The lifecycle

01
Create
Initialise a bundle, often from a template.

02
Develop
Write code and declare resources in YAML.

03
Validate
Check the configuration resolves before touching a workspace.

04
Deploy
Push to a named target.

05
Run
Trigger a job or pipeline from the bundle.

06
Destroy
Remove everything the bundle deployed.

DATABRICKS' OWN SEQUENCE

Six stages, and this is Databricks' own framing of the lifecycle. Validate sits deliberately before deploy.

The CLI commands

COMMAND	WHAT IT DOES
`databricks bundle init`	Create a new bundle, optionally from a template
`databricks bundle validate`	Check the configuration resolves. No workspace changes.
`databricks bundle deploy -t prod`	Deploy the named target
`databricks bundle run my_job`	Trigger a job or pipeline defined in the bundle
`databricks bundle summary`	Show what is deployed for a target
`databricks bundle destroy`	Remove everything the bundle deployed

ON THE EXAM

The -t flag names the target. Omit it and you get the default target, which is how accidents happen.

What bundles can deploy

- **Lakeflow Jobs**, with all their tasks, schedules and settings.
- **Lakeflow Spark Declarative Pipelines**, including ingestion pipelines from Lakeflow Connect.
- **Other workspace assets**, and the set continues to grow — alerts became definable in bundles recently.
- **Artifacts built during deployment**, such as a Python wheel compiled from your source.

> ON THE EXAM
>
> Module 06 noted this from the other side: the Lakeflow Connect docs name bundles as the way to deploy ingestion pipelines.

Bundles and drift

- **Editing a bundle-deployed job in the workspace UI creates drift** between the workspace and the repository.
- **The next deploy overwrites the manual change**, usually without warning anybody.
- **The repository is the source of truth.** Fix the YAML, review it, deploy it.
- **Restrict CAN MANAGE on production jobs**, so the tempting edit is not available in the first place.

> ON THE EXAM
>
> An urgent fix made in the UI at 2am is invisible to everyone and disappears at the next deploy. That is worse than the original outage.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Different catalog per environment	A variable, overridden in each target
Two developers overwriting each other's jobs	mode: development, which prefixes resource names
Deploying to prod from the wrong branch	Production mode's Git branch validation
A pipeline reached prod still in development mode	Production mode validates development: false
Check the configuration without deploying	databricks bundle validate
A manual UI fix disappeared	Drift. The next deploy overwrote it.
Production deployed under one person's path	Use run_as with a service principal

ON THE EXAM

Seven lines. The second and third are the two deployment-mode behaviours most likely to be tested directly.

> Four engineers share a development workspace and each deploys the same bundle from their own branch to test changes. Their target is configured with mode: production, which they chose because it sounded more rigorous.
>
> They keep overwriting each other's jobs. Whoever deployed most recently has their version of every job in the workspace.

What should they change?

A. Give each engineer their own workspace

B. Set mode: development on that target, which prefixes resource names per user

C. Add a variable for the job name and have each engineer override it

D. Have only one engineer deploy, and the others request changes from them

◆ PAUSE AND DECIDE

What should they change?

A A workspace each

Would work, at considerable cost and administration. The feature that solves this without any of that already exists.

B mode: development

Correct. Development mode prefixes resource names with a dev marker including the deploying user, so four engineers get four sets of jobs.

C A variable for the job name

Reimplements development mode by hand, and depends on every engineer remembering to set it correctly every time.

D One person deploys for everyone

Solves collisions by removing the ability to test independently, which is the thing a development environment is for.

THE SIGNAL

They chose production mode because it sounded rigorous. Modes are not a quality setting; they select behaviours suited to a purpose.

> A release engineer runs databricks bundle deploy -t prod. It fails, reporting that the current Git branch does not match the branch specified in the target. The target specifies main; she is on a feature branch.
>
> She is under time pressure and a colleague suggests adding --force to get the deployment through.

What is happening, and what should she do?

A. A bug in the CLI; --force is the documented workaround

B. Production mode validates the Git branch. --force would bypass it, but the correct action is to deploy from main.

C. The bundle is missing a git property, which must be added to the target

D. Her Git credentials have expired, and the CLI cannot read the branch

◆ PAUSE AND DECIDE

What is happening, and what should she do?

A A bug; --force is the workaround

Documented behaviour, not a bug. --force does override it, and calling a safety check a bug is how safety checks get removed.

B Production mode's branch validation; deploy from main

Correct. The check exists to stop exactly this: production deployed from a branch that has not been through review.

C A missing git property

Backwards. The git property is clearly present, which is why the target knows to expect main.

D Expired credentials

The error names a branch mismatch, not an authentication failure. The CLI read the branch successfully to compare it.

THE SIGNAL

--force exists and works. The question is whether the check was wrong, or whether you were. Here, the check was right.

> A bundle deploys one job to three targets. In dev it should write to dev_catalog on a 2-worker cluster with no schedule; in prod, to prod_catalog on an 8-worker cluster running nightly at 02:00.
>
> An engineer proposes maintaining three copies of the job YAML, one per environment, so each is explicit and easy to read.

What is the better approach?

A. Three copies is clearer and avoids indirection; keep the proposal

B. One job definition using variables, with each target overriding catalog, worker count and schedule

C. One definition, with the notebook reading an environment variable and branching internally

D. Three separate bundles, one per environment

◆ PAUSE AND DECIDE

What is the better approach?

A Three copies of the YAML

Readable on day one and divergent by month three. A change to the job must be made three times, and one will be missed.

B One definition, variables overridden per target

Correct. This is exactly what the objective describes: environment-specific configuration through variables and overrides, one codebase across targets.

C Branching inside the notebook

Moves configuration into code, where it cannot be reviewed as configuration. It also cannot set cluster size or schedule, which are outside the notebook.

D Three bundles

The same divergence problem as A, with more machinery. The code is identical; only the configuration differs.

THE SIGNAL

Option C fails on a hard limit as well as a soft one: a notebook cannot set its own cluster size or its job's schedule.

> During an overnight incident, an on-call engineer edits a bundle-deployed production job in the workspace UI, increasing a timeout and changing a cluster node type. The job recovers and the incident is closed.
>
> Three days later, a routine bundle deployment runs. The following night the same failure recurs.

What happened, and what is the durable fix?

A. The deployment reverted the manual change; make the fix in the bundle YAML, review it and deploy

B. The deployment failed and left the job in a broken state

C. The bundle should be redeployed with --force to preserve manual edits

D. The engineer's change was never saved

◆ PAUSE AND DECIDE

What happened, and what is the durable fix?

A The deploy reverted it; fix the YAML instead

Correct. The bundle is the source of truth. A deployment reconciles the workspace to the YAML, which still had the old timeout.

B The deployment failed

It succeeded. The job ran; it simply ran with the configuration the bundle specified, which was the pre-incident one.

C Redeploy with --force to preserve edits

Inverts what --force does. It overrides validations; it does not protect manual workspace changes from being overwritten.

D The change was never saved

It was saved and it worked, which is why the incident closed. It was overwritten later, which is a different thing.

THE SIGNAL

The fix worked and was invisible. Nothing in the repository recorded it, so nothing in the repository preserved it.

One definition, many targets. Variables carry what differs, modes carry what should be enforced, and the repository is the source of truth.

MODULE 22 · DOMAIN 5 - IMPLEMENTING CI/CD

The CLI and CI/CD automation

Authentication without a human, configuration profiles, and the shape of an automated deployment pipeline.

DOMAIN 5 of 7 EXAM WEIGHT 10% MODULE 22 of 36

admin@sthithapragna.com

In CI/CD there is **nobody there to log in.** Every design decision in this module follows from that one constraint.

What the CLI is

- **A command-line client for the Databricks REST APIs**, covering both workspace and account operations.
- **It is how bundles are deployed.** Every bundle command from module 21 is a CLI command.
- **It runs anywhere:** your laptop, a CI runner, a container.
- **Anything the CLI can do, a pipeline can do**, which is the point for this domain.

> ON THE EXAM
>
> The exam guide names the CLI for validating, deploying and managing bundles and other workspace assets. That is its role here.

Command groups you should recognise

GROUP	WHAT IT MANAGES
`databricks bundle`	Bundles: init, validate, deploy, run, summary, destroy
`databricks jobs`	Jobs: list, get, run-now, cancel
`databricks clusters`	Compute: create, start, list
`databricks secrets`	Secret scopes and secrets
`databricks fs`	Files in cloud storage and volumes
`databricks workspace`	Notebooks and workspace objects: import, export
`databricks auth`	Login and profile management

ON THE EXAM

The pattern is databricks <group> <command>. If you can name the group, you can usually guess the command.

Configuration profiles

- **Profiles live in a .databrickscfg file**, each with a name in square brackets and its own settings.
- **Select one with --profile or -p** on any command.
- **A workspace profile holds a host and credentials.** An account profile also holds an account_id.
- **Profiles are how one machine talks to several workspaces** — dev, staging and prod side by side.

> ON THE EXAM
>
> If a bundle target specifies a host and no token is set, the CLI looks for a profile in .databrickscfg with a matching host.

U2M or M2M?

OAUTH U2M

User to machine

- You log in interactively through a browser
- The CLI manages short-lived tokens for you
- Tokens expire in under an hour
- For a person at a keyboard

OAUTH M2M

Machine to machine

- A service principal and a client credential flow
- No interactive sign-in at any point
- Configured with client_id and client_secret
- For scripts, services and CI/CD

THE TIEBREAKER

The names say it. If a person is present, U2M. If nobody is, M2M. A CI runner is nobody.

Personal access tokens are the legacy option

- **Databricks documentation labels PAT authentication legacy.**
- **OAuth M2M is the current recommendation** for unattended access.
- **A PAT is long-lived**, so an exposed one stays valid until somebody notices and revokes it.
- **OAuth tokens are short-lived and requested on demand**, so exposure has a much smaller window.

> ON THE EXAM
>
> Older CI examples all use a PAT in a secret variable. It still works. It is no longer what Databricks points you at.

Credentials in a pipeline

- **Set them as environment variables** from your CI system's secret store.
- **Environment variables take precedence over .databrickscfg** for OAuth M2M, which is what makes this work.
- **Never commit a credential to the repository.** Module 20 covered why that is permanent.
- **Give the CI service principal only the permissions it needs** in each target workspace.

> ON THE EXAM
>
> A CI runner has no .databrickscfg. Environment variables are how the credential reaches a container that was created 30 seconds ago.

The shape of a deployment pipeline

01

On pull request

Validate the bundle. Fail the check if it does not resolve.

02

Run tests

Deploy to a dev target and run the job, or run unit tests.

03

Merge

Review and approve. The branch protection is the gate.

04

Deploy to staging

From the main branch, to the staging target.

05

Deploy to production

From main, as a service principal, in production mode.

WHERE THE JUDGEMENT LIVES

Notice where the human is. Reviewing a pull request, not running a deployment command.

Why validate belongs in CI

- **It checks the configuration resolves** without touching any workspace.
- **It catches a mistyped variable, a missing target, malformed YAML** — before anything is deployed.
- **It is fast and free**, which is why it belongs on every pull request.
- **It cannot catch a logic error in your notebook.** It validates configuration, not correctness.

> ON THE EXAM
>
> The exam objective lists validate before deploy and manage. That ordering is not accidental.

Deploying from the pipeline

- **Always name the target explicitly** with -t. Omitting it uses the default, which is how the wrong workspace gets deployed to.
- **Production mode validates the Git branch**, so a production deploy from a feature branch fails.
- **Set run_as to a service principal** so production assets do not belong to a person.
- **Avoid --force in a pipeline.** It exists to override validations, which is the opposite of what automation wants.

> ON THE EXAM
>
> A pipeline that routinely passes --force has turned its safety checks into decoration.

What to test, and where

- **Unit tests on your transformation logic** run without any workspace at all.
- **Integration tests deploy to a dev target and run the job**, then check the output.
- **bundle destroy tears the test environment down** afterwards, which keeps the cost bounded.
- **Data quality checks from module 14 run in the pipeline too**, on real data rather than on code.

> ON THE EXAM
>
> Deploy, run, assert, destroy is a complete integration test, and it is four CLI commands.

Managing other assets

- **The objective says bundles and other workspace assets.** The CLI covers both.
- **Secret scopes can be created and populated** from a pipeline, so no human ever sees the value.
- **Connections for Lakeflow Connect can be created programmatically**, except where browser OAuth is the only auth.
- **Workspace objects can be imported and exported**, which is how one-off migrations get scripted.

> ON THE EXAM
>
> Module 06's exception applies here: connectors whose only authentication is browser-based OAuth cannot be created programmatically.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
A pipeline must authenticate with no human present	OAuth M2M with a service principal
A developer logging in from their laptop	OAuth U2M, via databricks auth login
An older example using a token in a secret	Works, but PATs are the legacy option now
It works locally but fails in CI	The runner has no profile. Set environment variables.
A broken bundle reached production	No validate step on the pull request
Deployed to the wrong workspace	The target was not named with -t
The pipeline passes --force every time	The safety checks have become decoration

ON THE EXAM

Seven lines. The fourth is the single most common practical failure when a team first automates a deployment.

> A team is automating deployment with GitHub Actions. An engineer proposes generating a personal access token from his own account, then storing it as a repository secret. The workflow would use it to deploy to production.
>
> It works in a trial run. The team asks whether this is the right long-term arrangement.

What should they do instead?

A Nothing; a PAT in a repository secret is the standard approach

B Use OAuth M2M with a dedicated service principal, supplied to the workflow as environment variables

C Use OAuth U2M so the workflow logs in interactively at deploy time

D Commit the token to a config file in the repository so the workflow can read it

◆ PAUSE AND DECIDE

What should they do instead?

A — A PAT in a repository secret

It works, and it ties production deployment to one person's account and uses the option Databricks now labels legacy.

B — OAuth M2M with a service principal

Correct. No interactive sign-in, short-lived tokens, and production deployment belongs to a service principal rather than an individual.

C — OAuth U2M in the workflow

U2M requires a browser sign-in. There is nobody there to complete it, which is the defining constraint of CI.

D — Commit the token

Puts a live credential into Git history permanently, where it cannot be removed by a later commit.

THE SIGNAL

Two problems, not one: the authentication method is legacy, and the identity is a person. M2M with a service principal fixes both.

> A bundle deploys correctly from an engineer's laptop with databricks bundle deploy -t dev. The same command in a CI job fails immediately, reporting that the CLI is not configured.
>
> The CI secret store holds a valid service principal client id and secret.

What is missing?

A. The bundle needs a profile name added to the target definition

B. The credentials are in the secret store but not set as environment variables for the step

C. The CI runner needs the Databricks CLI reinstalled

D. The service principal has not been granted CAN MANAGE on the bundle

◆ PAUSE AND DECIDE

What is missing?

A Add a profile name to the target

A profile name refers to an entry in .databrickscfg, and the runner has no such file. Naming a profile that does not exist changes nothing.

B Credentials not exposed as environment variables

Correct. A fresh CI container has no .databrickscfg. Environment variables are how the credential reaches it, and they take precedence.

C Reinstall the CLI

The CLI ran and produced a configuration error, which means it is installed and executing.

D Grant CAN MANAGE on the bundle

A permissions problem produces an authorisation error, not 'not configured'. The CLI never got as far as authenticating.

THE SIGNAL

'It works on my laptop' is the whole clue. The laptop has a .databrickscfg written by databricks auth login. The runner has nothing.

> A team's pipeline deploys to production whenever main is updated. There is no validate step. A pull request renames a variable in databricks.yml but misses one reference elsewhere in the file.
>
> The change is reviewed, approved and merged. The production deployment fails part way through, leaving some resources updated and others not.

What would have prevented this?

A A more thorough code review

B A databricks bundle validate step on the pull request, failing the check before merge

C Deploying to production with --force

D Running databricks bundle destroy before each deployment

◆ PAUSE AND DECIDE

What would have prevented this?

A. A more thorough review

A dangling variable reference in a long YAML file is exactly what humans miss and machines catch instantly. Asking people to be more careful is not a control.

B. A validate step on the pull request

Correct. Validate resolves the configuration without touching a workspace, so an unresolved reference fails the check before anyone merges.

C. Deploy with --force

Overrides validations. It would make the pipeline more willing to deploy broken configuration, not less.

D. Destroy before each deploy

Tears down production before deploying to it, converting a partial failure into a total outage.

THE SIGNAL

The objective names validate before deploy and manage. This scenario is what that ordering is for.

> An urgent fix is needed in production. The pipeline's production deploy fails, reporting that the current Git branch does not match the branch specified in the target. The engineer is on hotfix/urgent-fix.
>
> A colleague suggests permanently adding --force to the pipeline's production deploy step so this never blocks a release again.

What should the team do?

A. Add --force permanently, since urgent fixes should never be blocked

B. Merge the hotfix to main and let the pipeline deploy from there; keep the branch validation

C. Remove the git property from the production target

D. Change the production target's branch to hotfix/urgent-fix

◆ PAUSE AND DECIDE

What should the team do?

A Add --force permanently

Turns a safety check into decoration. It still runs, still reports, and can no longer stop anything.

B Merge to main and deploy from there

Correct. The hotfix goes through review and lands in main, so production matches the repository and the validation keeps working.

C Remove the git property

Silently removes the check rather than overriding it. Same outcome as A, and harder to notice afterwards.

D Point production at the hotfix branch

Production now tracks a temporary branch. When it is deleted, production's declared source no longer exists.

THE SIGNAL

Urgency is the argument every safety check eventually faces. The question is whether the check was wrong, and here it was right.

No human is present. So the credential must be a service principal, the target must be explicit, and the gates must be in the pipeline.

MODULE 23 · DOMAIN 6 - TROUBLESHOOTING, MONITORING AND OPTIMISATION

Monitoring jobs and run history

Run outcomes, the matrix view, spotting trends against a baseline, and reading a DAG for the upstream blocker.

DOMAIN 6 of 7 EXAM WEIGHT 10% MODULE 23 of 36

admin@sthithapragna.com

Monitoring is **comparison**. A single run tells you almost nothing. A run against forty previous runs tells you everything.

The six job run outcomes

OUTCOME	MEANING
Succeeded	All tasks were successful
Succeeded with failures	Some tasks failed, but all leaf tasks were successful
Failed	One or more leaf tasks failed
Skipped	The run never executed — for example, maximum concurrent runs was exceeded
Timed Out	The run took too long and timed out
Canceled	Somebody, or the platform, cancelled the run

ON THE EXAM

Remember from module 17: job outcome is determined by the leaf tasks, the ones with no downstream dependencies.

Succeeded with failures

- **It is a distinct outcome**, not a variant of Succeeded and not a variant of Failed.
- **It means something inside the job failed**, and the tasks at the end of the graph handled it.
- **A job-level failure notification will not fire**, because the run did not fail.
- **This is exactly the state module 17's cleanup pattern produces**, and it is why per-task notifications matter.

> ON THE EXAM
>
> If you filter run history by Failed, you will not see these. Filter for it deliberately, or handled failures stay invisible.

The matrix view

- **One column per run, one row per task**, so recent history is visible at a glance.
- **The top row is the run's total duration** and overall state.
- **Bar height indicates duration.** That is how a trend becomes visible without any calculation.
- **Hover a cell** for that task's start time, duration, cluster and status.

> ON THE EXAM
>
> The matrix answers two questions at once: which task failed, and whether it has failed before. A list view answers only the first.

What the colours mean

COLOUR	STATE
Green	Successful
Red	Unsuccessful
Pink	Skipped
Yellow	Waiting for retry
Grey	Pending, cancelled, or timed out

ON THE EXAM

Pink and red mean different things. A skipped task did not fail; something it depended on did.

The runs list

- **Shows start time, name, type, the identity it ran as, and what launched it.**
- **Launched tells you the trigger:** a schedule, an API request, or a manual start.
- **Elapsed time for a running job, total time for a completed one.**
- **A warning is displayed if the duration exceeds a configured expected completion time.**

> ON THE EXAM
>
> The 'Launched' column answers a question you will ask during an incident: did somebody run this by hand?

Retention

- **Databricks retains run history for 60 days**, for both jobs and pipelines.
- **So a baseline older than 60 days does not exist in the UI.**
- **For longer history, query the system tables**, which is how you compare against last quarter.
- **By default the runs table shows failed, skipped and successful runs**, and the filters at the top apply to it.

> ON THE EXAM
>
> Sixty days is enough to see a trend and not enough to compare a year-end run against last year's. Plan accordingly.

Reading the DAG for an upstream blocker

- **The job run details page shows the task graph**, with each task's state on it.
- **Find the first failure in dependency order.** Everything downstream of it is a consequence.
- **A task can be unsuccessful two ways:** it failed, or it was skipped because a dependency failed.
- **Disabled tasks are marked on the DAG**, either explicitly disabled or disabled because an upstream task was.

> ON THE EXAM
>
> The loudest failure is rarely the cause. Follow the arrows backwards until you reach a task that failed with nothing failed before it.

Task-level detail

- **Click a task to open its run details**, then choose a previous run from the history dropdown.
- **That gives you a per-task baseline**, which is finer than the job-level one.
- **A For each task shows a table of its iterations** rather than a single result.
- **Filter to only failed iterations** when a loop of two hundred has three problems.

> ON THE EXAM
>
> A job that is 10 minutes slower is a job-level observation. Knowing which task absorbed those 10 minutes is a task-level one.

Termination reasons worth recognising

CODE	WHAT IT POINTS AT
DRIVER_ERROR	A problem communicating with the Spark driver
CLUSTER_ERROR	The cluster itself failed. Module 26 covers these.
CLOUD_FAILURE	A cloud provider issue, not your code
MAXIMUM_CONCURRENT_RUNS_REACHED	A run was skipped because one was already in flight
REPOSITORY_CHECKOUT_FAILED	The Git checkout failed — a CI/CD problem, not a data one
CLUSTER_TERMINATED_BY_USER	Someone terminated the cluster outside the jobs service

ON THE EXAM

These separate your problems from the platform's, and data problems from infrastructure ones. That is most of triage.

System tables

- **The system.lakeflow schema holds job and task run records** across the whole account.
- **Query it like any table**, so failure rates and duration trends become SQL rather than screenshots.
- **Join it to the billing tables** to attribute cost to individual jobs, which needs the tags from module 19.
- **This is how you exceed the 60-day UI window**, and how you monitor hundreds of jobs at once.

> ON THE EXAM
>
> The UI is for one job. System tables are for an estate. The objective's 'failure rates' is a system tables question at scale.

Pipeline monitoring

- **Every pipeline generates an event log** covering updates, lineage, data quality and resource usage.
- **The data quality metrics from module 14 live there**, alongside the Data quality tab.
- **Pipeline runs appear in the same Runs tab**, with the same 60-day retention.
- **For database connectors, gateway progress** can be monitored through event logs too.

> ON THE EXAM
>
> The event log is queryable, so pipeline health can be alerted on rather than watched.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
The job succeeded but a task failed	Succeeded with failures. Job status comes from leaf tasks.
Runtime has been creeping up for weeks	The matrix view. Bar height shows duration.
A run never executed at all	Skipped — often maximum concurrent runs exceeded
Many tasks red and pink at once	Find the first failure. Pink means skipped, not failed.
Compare against last quarter	System tables. The UI keeps 60 days.
Failure rates across hundreds of jobs	system.lakeflow, queried
A slow run that never failed	A configured expected completion time triggers a warning

ON THE EXAM

Seven lines. The first and fourth are the two most likely to be tested as 'what does this screen mean'.

> A platform team reviews the previous week's job runs by filtering the Runs tab to Failed. Nothing appears, and they report a clean week.
>
> Separately, an analyst reports that a reconciliation table has been missing three regions' data since Tuesday. The job that loads it shows a status of Succeeded with failures on every run since then.

What is happening?

A The job status is incorrect and should be reported to support

B Succeeded with failures means some tasks failed but all leaf tasks succeeded, and filtering by Failed does not show it

C The analyst is looking at a different job from the one the team reviewed

D The runs are older than 60 days and have been removed from history

◆ PAUSE AND DECIDE

What is happening?

A The status is incorrect

It is exactly correct. Job outcome comes from leaf tasks, and the leaf tasks succeeded, so the run is not Failed.

B A distinct outcome that a Failed filter excludes

Correct. Succeeded with failures is its own outcome. Filtering by Failed hides every handled failure in the estate.

C A different job

Invents a mix-up. The scenario states the loading job's status directly.

D Older than 60 days

The runs are from this week, and their status is visible, so they are plainly still in history.

THE SIGNAL

The filter created the blind spot, not the platform. A weekly review that only looks for Failed will miss this class of problem indefinitely.

> A nightly job has started missing its 06:00 deadline about twice a week. Nobody can say when it started. It has never failed, so no notification has ever fired.
>
> An engineer opens the job's Runs tab and looks at the matrix view for the last month.

What will the matrix view tell her most directly?

A Which task is failing, indicated in red

B Whether the runtime has been increasing, because bar height indicates duration

C The cost of each run, so she can find the expensive one

D Nothing useful; she needs to query the system tables instead

◆ PAUSE AND DECIDE

What will the matrix view tell her most directly?

A. Which task is failing

Nothing is failing. The scenario says so, which is why no notification fired. There will be no red to find.

B. Whether runtime is increasing, via bar height

Correct. Bar height indicates duration, so a rising staircase across the columns is a visible trend, per task and for the run overall.

C. The cost of each run

The matrix shows duration and status, not cost. Cost attribution comes from the billing system tables with tags.

D. Nothing useful; use system tables

System tables are right for a longer window or many jobs. For one job over one month, the matrix answers it immediately.

THE SIGNAL

'It has never failed' rules out looking for red. The information she needs is encoded in the height of the bars, not their colour.

> A job is scheduled every 15 minutes. Reviewing the Runs tab, an engineer sees that roughly a third of the runs show a status of Skipped, coloured pink. The runs that do execute succeed normally.
>
> Recent successful runs have been taking between 18 and 24 minutes.

What is happening?

A The job is failing intermittently and the failures are being reported as Skipped

B Runs are being skipped because a previous run is still in flight and maximum concurrent runs is 1

C The cluster fails to start on those runs

D The schedule's time zone is misconfigured

◆ PAUSE AND DECIDE

What is happening?

A Failures reported as Skipped

Skipped means the run never executed. A failure would be red and would carry an error; these have neither.

B A previous run is still running; max concurrent runs is 1

Correct. Runs take 18–24 minutes on a 15-minute schedule, so they overlap, and the default maximum of 1 causes the later trigger to be skipped.

C The cluster fails to start

That would produce a cluster error and a failed run, not a skipped one. The run would have started.

D Time zone misconfiguration

A time zone offset shifts when runs happen. It does not cause a third of them not to happen at all.

THE SIGNAL

The arithmetic gives it away: a job taking 18–24 minutes cannot run every 15. It has outgrown its schedule.

> A job with fourteen tasks fails. The matrix view shows one red cell early in the graph and nine pink cells after it. An on-call engineer opens the last pink task, which reports that its dependency did not succeed.
>
> He escalates, saying nine tasks have failed and the job is badly broken.

What should he have done?

A Repair the run, which will re-run all ten unsuccessful tasks

B Traced back to the single red task — the first failure in dependency order — since the pink ones were skipped, not failed

C Opened each of the nine pink tasks in turn to find the common cause

D Checked the system tables for the failure rate across all fourteen tasks

◆ PAUSE AND DECIDE

What should he have done?

A Repair the run

The right eventual action, and not the right first one. Repairing before knowing why the red task failed just reproduces the failure.

B Trace back to the single red task

Correct. One task failed. Nine were skipped because a dependency failed, which is a consequence, not nine separate problems.

C Open each pink task

Nine tasks will each report the same thing: their dependency did not succeed. Nine identical answers, none of them the cause.

D Check failure rates in system tables

Useful for estate-level trends. It does not tell you why this run failed tonight.

THE SIGNAL

Pink is not red. Nine skipped tasks are one failure with consequences, and the escalation described a crisis that does not exist.

One run is a fact. Forty runs are information. **Find the first failure, and read the height of the bars, not just their colour.**

MODULE 24 · DOMAIN 6 - TROUBLESHOOTING, MONITORING AND OPTIMISATION

The Spark UI: skew, shuffle and spill

Finding the longest stage, reading Summary Metrics, and telling the three bottlenecks apart.

DOMAIN 6 of 7 EXAM WEIGHT 10% MODULE 24 of 36

admin@sthithapragna.com

Look at the distribution, not the total. A stage taking twenty minutes tells you nothing. Twenty minutes where one task took eighteen tells you everything.

Where to look

01

Stages tab

Sort by duration, descending.

02

The longest stage

Open it. Everything else is noise until you understand this one.

03

Check for spill

Spill stats appear at the top of the stage page.

04

Summary Metrics

Scroll down. Compare Max duration to the 75th percentile.

05

Conclude

Skew, spill, or neither — and neither points elsewhere.

FOLLOW THE PUBLISHED ORDER

This is Databricks' own sequence: spill first, then skew, and if neither, go back to the job page.

The stage columns

COLUMN	WHAT IT MEASURES
Duration	How long the stage took
Input	Bytes read from storage
Output	Bytes written to storage
Shuffle read	Bytes and records read, locally and from remote executors
Shuffle write	Bytes and records written to disk for a future shuffle

ON THE EXAM

Shuffle read and write are how you see a shuffle at all. A stage boundary exists because of one.

Spill happens when Spark runs low on **execution memory** — the memory used for shuffles, joins, sorts and aggregations — and starts moving data to disk.

Two spill numbers

- **Shuffle spill (memory)** is the size of the deserialised data in memory.
- **Shuffle spill (disk)** is the size of the serialised form written to disk.
- **The memory figure is usually much larger**, because deserialised data is bigger than serialised data.
- **Do not read the gap as an error.** They measure the same spill in two representations.

> ON THE EXAM
>
> Any non-zero spill is worth attention. A large memory figure beside a small disk figure is normal, not alarming.

Fixing spill

- **More partitions first.** Smaller partitions fit in the memory available to process them.
- **Then more memory**, by choosing a memory-optimised node type.
- **Reduce the data being shuffled:** filter earlier, select fewer columns.
- **Spill is a symptom of partitions that are too large** at least as often as of a cluster that is too small.

> ON THE EXAM
>
> This is module 15's ordering again. Raising spark.sql.shuffle.partitions is usually cheaper than a bigger node type.

In Summary Metrics, compare **Max duration** to the **75th percentile**. If Max is 50% higher, you may be suffering from skew.

What skew actually is

- **One or a few tasks take much longer than the rest**, because their partitions hold much more data.
- **The cluster is idle while they finish.** A stage is only done when its slowest task is done.
- **Usually caused by an uneven key:** one customer with a million orders, or a null that everything joins to.
- **Look at shuffle read per task too.** Large differences there point at skew as clearly as duration does.

> ON THE EXAM
>
> Skew wastes the cluster you are paying for. Ninety-nine executors idle while one finishes is the definition of poor utilisation.

Telling the three apart

WHAT YOU SEE	WHAT IT MEANS
A few tasks much slower than the rest	Data skew
Most tasks slow, fairly evenly	Cluster resources or under-parallelism, not skew
Non-zero spill, high GC time, or OOM	Memory pressure
Large shuffle read and write, many stages	Shuffle-bound work
Large differences in shuffle read per task	Skew again, seen from the data side

ON THE EXAM

The first two rows are the discriminator that matters most. A few slow tasks and all tasks slow have opposite fixes.

Fixing skew

- **Enable AQE first.** It detects and splits skewed partitions at runtime, and it is on by default from DBR 7.3.
- **Filter out the pathological key** if it is a null or a placeholder that should not be joined on at all.
- **Salting:** add a random suffix to the hot key so it spreads across partitions, then aggregate twice.
- **Repartition on a better key**, or broadcast the small side so no shuffle happens.

> ON THE EXAM
>
> Enable AQE before any manual tuning. It resolves a large share of shuffle-partition and join-strategy problems on its own.

Shuffle

- **Every wide transformation causes one:** join, groupBy, distinct, orderBy.
- **It moves data across the network** between executors, which is the slowest thing Spark does.
- **Shuffle read and write on the stage page** tell you how much moved.
- **Shuffle is not a fault.** The question is whether this one was necessary and whether it moved more than it needed to.

> ON THE EXAM
>
> A stage boundary is a shuffle. Counting stages in the UI counts the shuffles you caused.

Reducing shuffle

- **Broadcast the small side of a join**, which removes the shuffle on the large side entirely.
- **Filter and select before the wide transformation**, so less data crosses the network.
- **Avoid unnecessary ordering.** orderBy is a full shuffle and is often decoration on an intermediate result.
- **Right-size shuffle partitions**, remembering AQE coalesces downwards only.

> ON THE EXAM
>
> Module 11 and module 15 both apply here. Broadcasting and partition sizing are the same levers, seen from the diagnosis side.

When it is none of the three

- **No spill and no skew is a real finding**, not a failed investigation.
- **It points at high input or output** — the stage is simply reading or writing a lot.
- **Or at under-parallelism:** too few tasks for the cluster, so cores sit idle.
- **Go back to the job page** and look at the overall shape before drilling in again.

> ON THE EXAM
>
> Databricks' guide says this explicitly: if the stage has no spill or skew, look at the stage's I/O next.

Validating a fix

- **A single faster run is not validation.** Caching and a warm cluster both flatter a repeat.
- **The underlying metric must move**, not just the wall-clock time.
- **Check the specific number:** spill to zero, or the Max-to-median ratio down below about two.
- **Test at full production volume**, because skew often only appears at scale.

> ON THE EXAM
>
> Module 15's re-measure discipline, made specific. Confirm the metric you were targeting actually changed.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Max duration far above the 75th percentile	Data skew
Most tasks slow, fairly evenly	Not skew — resources or under-parallelism
Non-zero spill statistics on the stage	Memory pressure. More partitions before more memory.
Spill in memory much larger than on disk	Normal. Deserialised is bigger than serialised.
One customer has millions of rows	A skewed key. AQE, filter, salt, or broadcast.
Large shuffle read and write, many stages	Shuffle-bound. Broadcast, or move less data.
It ran faster once, so the fix worked	Not validation. Check the metric moved.

ON THE EXAM

Seven lines. The first two are the discriminator; the rest follow from it.

> A job's longest stage takes 41 minutes. In Summary Metrics, the median task duration is 22 seconds, the 75th percentile is 28 seconds, and the maximum is 37 minutes.
>
> There are 200 tasks. No spill statistics appear on the stage page.

What is the bottleneck?

A The cluster is undersized; most tasks are taking too long

B Data skew — the maximum is far above the 75th percentile, so one partition holds most of the data

C Disk spill, which is why the stage is slow

D Too few shuffle partitions for the cluster

◆ PAUSE AND DECIDE

What is the bottleneck?

A The cluster is undersized

Most tasks finish in about 22 seconds, which is healthy. If resources were the constraint, the whole distribution would be slow, not one task.

B Data skew

Correct. 37 minutes against a 28-second 75th percentile is vastly beyond the 50% threshold. One task is doing almost all the work.

C Disk spill

The scenario states no spill statistics appear, and Databricks' guide says that means the stage has no spill.

D Too few shuffle partitions

200 partitions with 199 finishing in seconds is not an under-parallelism problem. Adding partitions would not split the one hot key.

THE SIGNAL

41-minute stage, 37-minute maximum task. One task accounts for nearly the whole stage while 199 others idle.

> A different stage takes 30 minutes across 200 tasks. The median task duration is 8.4 minutes, the 75th percentile is 9.1 minutes, and the maximum is 9.8 minutes.
>
> The cluster has 16 cores. No spill is reported.

What is the bottleneck?

A Data skew, since the maximum is above the median

B Not skew — the distribution is even. With 200 tasks on 16 cores, the cluster is under-resourced for the work

C Memory pressure, indicated by the long task durations

D An excessive number of shuffle partitions creating scheduling overhead

◆ PAUSE AND DECIDE

What is the bottleneck?

A **Data skew**

9.8 against 9.1 minutes is about 8% above the 75th percentile, far below the 50% threshold. This is one of the most even distributions you will see.

B **Not skew; the cluster is under-resourced**

Correct. Every task is slow, evenly. 200 tasks averaging over 8 minutes on 16 cores means roughly 13 waves of work.

C **Memory pressure**

Memory pressure shows as spill or GC time, and the scenario states there is no spill. Long tasks alone are not evidence of it.

D **Too many shuffle partitions**

Excessive partitions produce many tiny tasks. These tasks take over 8 minutes each, which is the opposite symptom.

THE SIGNAL

This scenario is the mirror of the last one. Same tools, same tab, opposite conclusion — because the distribution is even.

> A stage reports shuffle spill of 46 GB in memory and 7 GB on disk. Task durations are fairly even, with the maximum about 20% above the 75th percentile. The stage takes far longer than the team expects.
>
> An engineer proposes moving to a node type with four times the memory.

What should be tried first?

A. The four-times-memory node type, since spill means insufficient memory

B. Increase spark.sql.shuffle.partitions so each partition is smaller and fits in the memory available

C. Nothing — 46 GB against 7 GB shows the metrics are inconsistent and cannot be trusted

D. Enable AQE to split the skewed partitions

◆ PAUSE AND DECIDE

What should be tried first?

A Four times the memory

It would help, at four times the cost, permanently. Spill is as often partitions being too large as memory being too small.

B More shuffle partitions

Correct. Smaller partitions fit in the memory each task has, which removes the spill without changing the cluster.

C The metrics are inconsistent

They are consistent. Memory spill measures deserialised data; disk spill measures the serialised form, which is much smaller.

D Enable AQE for skew

There is no skew — the maximum is 20% above the 75th percentile, under the 50% threshold. AQE would not target this.

THE SIGNAL

46 GB against 7 GB is exactly what you should expect. Deserialised in memory, serialised on disk — the same spill, twice.

> An engineer identifies skew in a stage: maximum task duration is 12 times the median. She adds a salting step, redeploys, and re-runs the job immediately. The job completes in 19 minutes rather than 31.
>
> She reports the problem solved. A colleague asks whether she checked Summary Metrics on the re-run.

Is the colleague's question reasonable?

A No; a 12-minute improvement is conclusive evidence the fix worked

B Yes; an immediate re-run benefits from warm caches, and the Max-to-median ratio must actually have come down

C Yes, but only because salting always needs two aggregation passes

D No; Summary Metrics do not show skew, only spill

◆ PAUSE AND DECIDE

Is the colleague's question reasonable?

A The improvement is conclusive

Wall-clock time on an immediate re-run is the least reliable measurement available. Warm caches alone can produce a gain this size.

B Yes; check the ratio actually moved

Correct. The metric you targeted must move. If Max-to-median is still 12, the improvement came from something else and will not persist.

C Yes, but for the salting reason

Salting does involve two passes, and that is not why the colleague's question is reasonable. He is asking about validation, not implementation.

D Summary Metrics show only spill

Summary Metrics is precisely where the skew comparison lives. Spill statistics appear at the top of the stage page instead.

THE SIGNAL

If the ratio is still 12 and the job got faster, something else changed. That is worth knowing before you tell everyone it is fixed.

Longest stage. Spill first, then Max against the 75th percentile. **A few slow tasks is skew; all tasks slow is not.**

MODULE 25 · DOMAIN 6 - TROUBLESHOOTING, MONITORING AND OPTIMISATION

Liquid clustering and predictive optimisation

From partitioning to CLUSTER BY AUTO, and the maintenance Databricks now runs for you.

DOMAIN 6 of 7 EXAM WEIGHT 10% MODULE 25 of 36

admin@sthithapragna.com

Query speed depends on **how little data you can avoid reading**. Both features are about the same thing: who decides the layout, and who maintains it.

How we got here

01

Partitioning

Physical directories per value. Decided at creation, permanent.

›

02

Z-ORDER

Co-locates related data within files. Needs OPTIMIZE, and fixed columns.

›

03

Liquid clustering

CLUSTER BY. Keys can change without rewriting the table.

›

04

CLUSTER BY AUTO

Databricks picks and updates the keys from your query patterns.

THE DIRECTION OF TRAVEL

Each step removes a decision you had to get right in advance and could not easily change afterwards.

Why partitioning became a problem

- **Chosen at creation and effectively permanent.** Changing partition columns means rewriting the table.
- **High-cardinality columns produce thousands of tiny directories**, and the small files problem from module 3.
- **Low-cardinality columns barely prune anything.** Two partitions on a billion rows helps nobody.
- **A query that does not filter on the partition column gets no benefit at all.**

> ON THE EXAM
>
> Partitioning asks you to predict, at creation time, how a table will be queried for the rest of its life.

Liquid clustering

- **CLUSTER BY replaces both partitioning and Z-ordering.** You cannot combine it with either.
- **Clustering keys can be changed** without rewriting the table.
- **It handles skewed and high-cardinality columns** far better than partitioning does.
- **After altering clustering columns you must run OPTIMIZE** to feel the benefit.

> ON THE EXAM
>
> The headline feature is that the decision is reversible. A wrong clustering key is a change; a wrong partition column is a migration.

Automatic liquid clustering

- **Set CLUSTER BY AUTO** on a Unity Catalog managed table, and Databricks chooses the keys.
- **Predictive optimisation analyses the historical query workload** and selects the most effective keys.
- **It only changes keys when the benefit outweighs the overhead**, since clustering itself costs something.
- **Available for Unity Catalog managed tables only.**

> ON THE EXAM
>
> AUTO uses the current partition columns as its starting point, then adapts from there as query patterns emerge.

Two details about AUTO

- **Reading and writing works on any runtime that supports liquid clustering.**
- **But intelligent key selection relies on metadata introduced in Databricks Runtime 15.4 LTS.**
- **You can enable it on any UC managed table**; the heuristics decide whether clustering is worth it.
- **Skip reasons are visible** in Catalog Explorer's History tab, on the AUTO LIQUID row marked Not applied.

> ON THE EXAM
>
> If AUTO appears to do nothing, that may be the heuristics correctly deciding this table would not benefit. The History tab tells you why.

Databricks runs **OPTIMIZE, VACUUM and ANALYZE** for you, on Unity Catalog managed tables, deciding which tables need them and when.

The three operations

OPERATION	WHAT IT DOES
OPTIMIZE	Optimises file sizes, and triggers incremental clustering on clustered tables
VACUUM	Deletes data files no longer referenced by the table, reducing storage cost
ANALYZE	Collects and incrementally updates statistics, improving query planning

ON THE EXAM

These are the same three commands you would have scheduled by hand. The difference is that something decides when they are needed.

Where it does not run

- **Only on Unity Catalog managed tables.** External tables are excluded.
- **Not on streaming tables or materialized views** created in Databricks SQL or in pipelines.
- **Not available in all regions.**
- **It covers Delta Lake and Apache Iceberg** managed tables.

> ON THE EXAM
>
> That first exclusion is the strongest argument in the course for managed over external tables.

The Z-order interaction

- **OPTIMIZE does not run ZORDER when executed by predictive optimisation.**
- **On tables that use Z-order, predictive optimisation ignores the Z-ordered files.**
- **So a Z-ordered table gets less benefit** than you might expect from enabling it.
- **This is a real reason to migrate to liquid clustering**, rather than merely a stylistic preference.

> ON THE EXAM
>
> Enabling predictive optimisation on a Z-ordered table and expecting full maintenance is a specific, documented misunderstanding.

Whether it is already on

- **Enabled by default for accounts created on or after 11 November 2024.**
- **Existing accounts have been enabled by a gradual rollout**, which Databricks expected to complete by August 2026.
- **There is a documented way to verify** whether your account is enabled.
- **Databricks recommends disabling scheduled OPTIMIZE jobs** once predictive optimisation is running.

> ON THE EXAM
>
> That last point matters operationally. Leaving a nightly OPTIMIZE job in place means paying for maintenance twice.

The VACUUM warning

- **VACUUM retention comes from delta.deletedFileRetentionDuration, which defaults to 7 days.**
- **So predictive optimisation will vacuum files older than that**, and time travel beyond 7 days stops working.
- **Set the property before enabling predictive optimisation** if you need longer.
- **ALTER TABLE … SET TBLPROPERTIES ('delta.deletedFileRetentionDuration' = '30 days')**

> ON THE EXAM
>
> Databricks flags this as a warning in its own documentation. Time travel you were relying on can quietly disappear.

Cost and visibility

- **It runs on serverless compute for jobs**, billed with a serverless jobs SKU.
- **So it is not free**, and it appears on your bill as compute you did not schedule.
- **Track it with system tables**, so the spend is attributable rather than mysterious.
- **The trade is usually favourable**, since it replaces jobs you were running anyway, and skips runs that were unnecessary.

> ON THE EXAM
>
> An unexplained serverless jobs line on the bill, shortly after enablement, is often this. It is worth recognising rather than investigating.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Partition columns turned out to be wrong	Liquid clustering — keys can change without a rewrite
Thousands of tiny partition directories	High-cardinality partitioning. Cluster instead.
Databricks should choose the clustering keys	CLUSTER BY AUTO, on a UC managed table
An external table is not being maintained	Predictive optimisation only runs on managed tables
A Z-ordered table gets little benefit	PO ignores Z-ordered files and does not run ZORDER
Time travel stopped working after enablement	VACUUM at the 7-day default retention
Unexplained serverless jobs cost appeared	Predictive optimisation, billed on a serverless jobs SKU

ON THE EXAM

Seven lines. The fourth, fifth and sixth are the three most likely to be tested, because each is an exclusion or a side effect.

> A team partitioned a large events table by event_id when it was created three years ago. The table now has several million partition directories, most containing a single small file. Queries are slow and OPTIMIZE takes hours.
>
> Almost all queries filter on event_date and customer_id, neither of which is the partition column.

What should they do?

A. Repartition the table by event_date, which most queries filter on

B. Migrate to liquid clustering with CLUSTER BY on event_date and customer_id

C. Add a Z-ORDER on event_date and customer_id alongside the existing partitioning

D. Enable predictive optimisation and let it resolve the layout

◆ PAUSE AND DECIDE

What should they do?

A Repartition by event_date

Better than event_id, and it repeats the original mistake: a permanent choice made now, unchangeable when query patterns shift again.

B Liquid clustering on the queried columns

Correct. It removes the partition explosion, targets the columns actually filtered on, and the keys can change later without a rewrite.

C Z-ORDER alongside partitioning

It leaves several million directories in place, and predictive optimisation ignores Z-ordered files, so future maintenance is worse too.

D Let predictive optimisation fix it

PO maintains a layout; it does not redesign one. It will not remove partitioning that was chosen at creation.

THE SIGNAL

Partitioning on a near-unique column is the textbook failure. The fix is not a better partition column — it is not partitioning.

> A platform team enables predictive optimisation across the account. After a month they audit the results. Most tables are being optimised and vacuumed automatically as expected.
>
> Three sets of tables are receiving nothing: the tables in their partner-shared external location, their Databricks SQL materialized views, and the streaming tables in their ingestion pipelines.

Why?

A Predictive optimisation needs to be enabled per table as well as per account

B It only runs on Unity Catalog managed tables, and does not run on streaming tables or materialized views

C Those tables are too small to benefit, so the heuristics skipped them

D The rollout has not reached those tables yet

◆ PAUSE AND DECIDE

Why?

A Enable it per table too

Does not explain why exactly these three categories were affected while everything else worked.

B Managed tables only; not streaming tables or MVs

Correct. Two documented exclusions cover all three groups: external tables are not managed, and streaming tables and MVs are excluded explicitly.

C Too small to benefit

The heuristics do skip tables, and that would not correlate so neatly with three distinct object types.

D The rollout has not reached them

The rollout is per account, not per table. Everything else in the account is working.

THE SIGNAL

Three symptoms, two documented exclusions. When affected objects share a type rather than a size, look for a categorical rule.

> A team has predictive optimisation enabled. Their largest fact table uses Z-ORDER on three columns, maintained by a nightly OPTIMIZE job they wrote two years ago.
>
> They remove the nightly job, expecting predictive optimisation to take over. Query performance degrades over the following weeks.

What happened?

A. Predictive optimisation does not run ZORDER, and ignores Z-ordered files, so the table is no longer being maintained

B. Predictive optimisation needs several months of query history before it acts

C. Removing the job also removed the table's statistics

D. The table should have been converted to an external table first

◆ PAUSE AND DECIDE

What happened?

A PO does not run ZORDER and ignores Z-ordered files

Correct. Both statements are documented. Removing the nightly job left the Z-ordered table with no maintenance at all.

B It needs months of history

Invented delay. PO queues maintenance based on writes and table state, not on a long warm-up period.

C Removing the job removed statistics

Deleting a job does not delete table statistics. And ANALYZE is one of the operations PO would run anyway.

D Convert to an external table

Backwards. PO runs only on managed tables, so converting to external would guarantee no maintenance at all.

THE SIGNAL

Databricks does recommend disabling scheduled OPTIMIZE jobs — but that advice assumes PO will actually maintain the table.

> A finance team relies on being able to query a regulatory table as it stood on any day in the previous quarter, using Delta time travel. This has worked for two years.
>
> Predictive optimisation is enabled on the account. Six weeks later, an auditor requests the table as at 40 days ago, and the query fails.

What happened, and what should have been done?

A Time travel is not supported on managed tables once predictive optimisation is enabled

B VACUUM ran at the default 7-day retention; delta.deletedFileRetentionDuration should have been set before enabling

C OPTIMIZE rewrote the files, which invalidates all previous versions

D The table needed to be converted to Iceberg to retain long-range time travel

◆ PAUSE AND DECIDE

What happened, and what should have been done?

A Time travel is unsupported with PO

Time travel still works. What changed is how far back the files exist, which is a retention question rather than a support question.

B VACUUM at the 7-day default; set retention first

Correct. Databricks flags this as a warning: set delta.deletedFileRetentionDuration before enabling, because vacuumed files are gone.

C OPTIMIZE invalidates old versions

OPTIMIZE rewrites files and the old ones remain referenced by earlier versions until VACUUM removes them. VACUUM is the operation that ends time travel.

D Convert to Iceberg

PO covers managed Delta and Iceberg alike, with the same VACUUM retention behaviour. Converting changes nothing here.

THE SIGNAL

For two years nobody ran VACUUM, so files accumulated and long time travel worked by accident. Enabling PO removed the accident.

Liquid clustering makes the layout decision **reversible**. Predictive optimisation makes the maintenance **automatic**. Both only fully apply to Unity Catalog managed tables.

MODULE 26 · DOMAIN 6 - TROUBLESHOOTING, MONITORING AND OPTIMISATION

Cluster failures, libraries and memory

Why clusters do not start, why libraries fight each other, and which component actually ran out of memory.

DOMAIN 6 of 7 EXAM WEIGHT 10% MODULE 26 of 36

admin@sthithapragna.com

Ask **when** it failed. Before the VMs launched, during startup, or while running. Those three points have almost no causes in common.

Three moments, three families

WHEN IT FAILED	WHAT TO LOOK AT
The VMs never launched	Cloud quota, capacity, subnet, IP addresses
The cluster never became ready	Init scripts, libraries, bootstrap timeouts
The job failed while running	Memory, skew, spill — and which component

ON THE EXAM

The termination reason on the cluster event log tells you which row you are in. Read it before doing anything else.

When the cloud provider refuses

- **Quota exceeded.** The launch would take the account past a limit on instances, cores, volumes or IP addresses.
- **Stockout.** The availability zone has no capacity for that instance type right now.
- **A full subnet, or no free public IPs.** Both are common and both look like a launch failure.
- **The reason is CLOUD_PROVIDER_LAUNCH_FAILURE**, and the parameters carry the provider's own error code.

> ON THE EXAM
>
> None of these are fixed by changing your Databricks configuration. They are fixed in the cloud account, or by asking for different machines.

Fixing a launch failure

- **For quota:** request an increase, terminate unused clusters, or use different instance types.
- **For a stockout:** enable flexible node types, try another availability zone, or choose alternative instances.
- **Check compute policies allow the fallback**, or your alternative instance type is blocked before it is tried.
- **Intermittent launch failures are often transient**, and re-running succeeds. Persistent ones are not.

> ON THE EXAM
>
> Terminating idle clusters is the fastest quota fix, and module 2's auto-termination is what stops the problem recurring.

Init script failures

- Init scripts run on the driver and on every worker node.
- A failure on any single node can terminate the whole cluster.
- Cluster-scoped failures surface as INIT_SCRIPT_FAILURE.
- **Admin-configured global init scripts have their own reason**, and usually need an admin to fix.

> ON THE EXAM
>
> A script that works on your laptop and on the driver can still fail on a worker. Every node runs it, so every node must succeed.

Reading an init script failure

- **Read the init script logs** to find which script failed and why.
- **If it could not be fetched**, check the path still exists and the identity can read it.
- **If it ran and exited non-zero**, find the failing command in its output.
- **If it installs packages, check the repositories are reachable** from the cluster's network.

> ON THE EXAM
>
> Enable init script logging before you need it. Diagnosing a script with no logs means reproducing the failure to see it.

Startup timeouts

- **VM bootstrap has a 700-second timeout**, and can be exceeded by network problems or slow artifact downloads.
- **The Spark driver must become ready within a few minutes**, or the launch fails.
- **Too many libraries configured in the cluster UI** can push it past that window.
- **Library installation has its own timeout**, and JAR fetching over a slow network is a common cause.

> ON THE EXAM
>
> A cluster connected to an external Hive metastore can download around 200 JARs at startup, which alone can exceed the driver-ready window.

The Databricks Runtime already contains hundreds of libraries. A conflict is usually **your version fighting the one that was already there**.

Cluster libraries or notebook-scoped?

CLUSTER-SCOPED

Installed on the cluster

- Available to every notebook and job on that cluster
- Installed at cluster startup, before it is usable
- A bad one can prevent the cluster starting
- Right for dependencies the whole team needs

NOTEBOOK-SCOPED

%pip install in the notebook

- Scoped to that notebook's session
- Installed when the cell runs, not at startup
- A bad one fails the cell, not the cluster
- Right for experimenting, and for per-job dependencies

THE TIEBREAKER

Notebook-scoped isolates the blast radius. A broken cluster library affects everyone; a broken %pip affects one notebook.

The classic conflict

- **Installing your own NumPy or pandas over the runtime's version** can break the runtime's compiled extensions.
- **The symptom is often not a package error.** The driver crashes on startup, and you see a 'could not reach driver' message.
- **The fix is to remove the custom versions** and use the ones that ship with the runtime.
- **Or move to a runtime version that ships the version you need**, rather than overriding it.

> ON THE EXAM
>
> 'Could not reach driver' sounds like a network problem and is frequently a library problem. That mismatch is what makes it hard.

Libraries and access mode

- **On standard access mode, libraries and init scripts must be on the Unity Catalog allowlist.**
- **The allowlist is empty by default**, and modifying it requires the MANAGE ALLOWLIST privilege.
- **Libraries used as JDBC drivers or custom data sources need ANY FILE permission** on standard access mode.
- **Identity differs by mode:** on standard access mode, libraries use the installer's identity and init scripts the cluster owner's.

> ON THE EXAM
>
> A library that installs on a dedicated cluster and fails on a standard one is usually the allowlist, not the library.

The first question is never how much memory. It is **which component ran out** — because driver and executor OOM have almost nothing in common.

Driver out of memory

- **collect and toPandas** pull the entire result back to the driver.
- **An oversized broadcast**, because the small side is collected on the driver before distribution.
- **Very large query plans**, or a loop building thousands of DataFrames.
- **Adding executors does nothing.** The driver is one machine and more workers do not help it.

> ON THE EXAM
>
> The driver does not process partitions. So dataset size alone does not pressure it — something has to pull data to it.

Executor out of memory

- **Partitions too large to process in the memory each task has.**
- **Skew**, where one partition is enormous while the rest are fine.
- **A wide row or a large object held per task**, such as a model loaded inside a UDF.
- **Spill usually comes first.** An executor that is spilling heavily is on its way to an OOM.

> ON THE EXAM
>
> Module 24's ordering applies: more partitions before more memory, because the partition is what did not fit.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
The VMs never launched	Cloud quota, capacity, subnet or IP addresses
The cluster started then terminated during startup	An init script — it runs on every node
A global init script failed	An admin must fix it, not the cluster's user
'Could not reach driver' after a library change	A library conflicting with the runtime's version
A library works on dedicated but not standard	The Unity Catalog allowlist
The error names the driver	collect, toPandas, or an oversized broadcast
The error names an executor	Partition size or skew, before memory

ON THE EXAM

Seven lines. The first two are the split that saves the most time, and the sixth and seventh are the split that saves the most money.

> A team's scheduled jobs begin failing intermittently each morning between 08:00 and 09:30, when several teams start work. The cluster event log shows CLOUD_PROVIDER_LAUNCH_FAILURE with a provider error code about an address limit.
>
> Re-running the same job at 11:00 works every time.

What is happening?

A. A code defect that only manifests under morning data volumes

B. A cloud account limit is being reached at peak, most likely public IP addresses consumed by concurrently running clusters

C. An init script is failing during the busy period

D. The jobs need a longer timeout configured

◆ PAUSE AND DECIDE

What is happening?

A A code defect

The job never started. The cluster failed to launch, so no code of yours has run at all.

B A cloud account limit reached at peak

Correct. The reason names the cloud provider and an address limit, and the timing tracks how many clusters are running concurrently.

C An init script failing

That produces INIT_SCRIPT_FAILURE, and it would mean the VMs launched. The reason here says they did not.

D A longer timeout

Nothing timed out. The provider refused the request outright, and waiting longer does not create IP addresses.

THE SIGNAL

The termination reason tells you the row before you reason at all. CLOUD_PROVIDER_LAUNCH_FAILURE means nothing of yours ever ran.

> An engineer adds a cluster-scoped init script that installs a package from an internal repository. On his own 2-node development cluster it works every time.
>
> On the 40-node production cluster the launch fails with INIT_SCRIPT_FAILURE, but not on every attempt — roughly one launch in three succeeds.

What is the most likely explanation?

A. The production cluster has an incompatible runtime version

B. Init scripts run on every node, so a transient failure reaching the repository on any one of 40 nodes terminates the cluster

C. Init scripts are not supported on clusters above a certain size

D. The script needs to be converted to a global init script

◆ PAUSE AND DECIDE

What is the most likely explanation?

A. An incompatible runtime version

A version incompatibility would fail consistently, not one launch in three. Intermittency points at something probabilistic.

B. Every node runs it; one failure of 40 kills the cluster

Correct. A per-node success rate that is fine on 2 nodes becomes unreliable on 40, because all 40 must succeed.

C. Unsupported above a size

Invented limit. Init scripts run on clusters of any size.

D. Make it a global init script

Changes who manages it and applies it to every cluster in the workspace. It does not change the per-node execution that is failing.

THE SIGNAL

Two nodes and forty nodes are not the same test. Twenty times the nodes is twenty times the chances for one of them to fail.

> A data scientist adds pandas and NumPy at specific versions as cluster-scoped libraries, because a modelling package she needs requires them. The cluster now fails to start, reporting that the driver could not be reached.
>
> The team spends a day checking subnets, security groups and quotas. All are fine, and the cluster started normally the previous week.

What should they check?

A Whether the newly added libraries conflict with the versions the Databricks Runtime ships

B Whether the workspace has run out of public IP addresses

C Whether the cluster needs a larger driver node

D Whether an init script is failing on the worker nodes

◆ PAUSE AND DECIDE

What should they check?

A A conflict with the runtime's own versions

Correct. Overriding fundamental packages can break the runtime's compiled extensions, crashing the driver at startup. The message misleads.

B Public IP addresses

That produces CLOUD_PROVIDER_LAUNCH_FAILURE before any driver exists. Here the machines launched and the driver crashed.

C A larger driver node

Nothing suggests memory pressure. The driver did not run out of memory; it failed to come up at all.

D An init script on the workers

That surfaces as INIT_SCRIPT_FAILURE, and no init script was added. The change made was to cluster libraries.

THE SIGNAL

'Could not reach driver' reads as a network symptom, which is why a day went into networking. The change log said libraries.

> A nightly job fails with an out-of-memory error. The team doubles the worker node memory. It fails again. They double it once more, and it still fails, now at considerable cost.
>
> Reading the stack trace properly, they find it names the driver. The final step of the job calls toPandas() on the result to write a formatted Excel report.

What should they do?

A. Continue increasing worker memory until it succeeds

B. The failure is on the driver; toPandas pulls the whole result to it. Aggregate first, or write the file differently.

C. Increase spark.sql.shuffle.partitions to reduce partition size

D. Enable AQE to handle the skew

◆ PAUSE AND DECIDE

What should they do?

A Keep increasing worker memory

Two doublings have already failed, which is the evidence that workers are not the constraint. A third will fail identically, more expensively.

B It is the driver; toPandas pulls everything to it

Correct. The trace names the driver, and toPandas is the operation bringing the full result to a single machine.

C More shuffle partitions

Shuffle partitions govern executor-side partition size. The driver does not process partitions, so this changes nothing for it.

D Enable AQE for skew

AQE optimises distributed execution. It has no effect on a single-machine collection to the driver.

THE SIGNAL

Two failed doublings were already the answer. If more executor memory does not help, the executors were never the problem.

Read the failure before you fix it. When did it fail, which component named the error, and what changed since it last worked.

MODULE 27 · DOMAIN 7 - GOVERNANCE AND SECURITY

Managed and external tables

Who owns the storage lifecycle, what DROP really does, and converting between the two with SET MANAGED.

DOMAIN 7 of 7 EXAM WEIGHT 15% MODULE 27 of 36

admin@sthithapragna.com

The difference is not where the data sits. It is **who owns the storage lifecycle** — who decides when files are created, optimised and deleted.

The differences that matter

	MANAGED	EXTERNAL
Storage location	Unity Catalog decides	You specify with LOCATION
On DROP	Data files are deleted	Data files are left alone
Maintenance	Predictive optimisation handles it	You schedule it yourself
Access from other tools	Through Databricks	Directly at the path, if permitted
Status	The default and recommended type	For specific reasons

ON THE EXAM

Read the second row twice. It is the single most consequential difference and the one that appears in incident reports.

Creating each

- **CREATE TABLE catalog.schema.name (...)** with no LOCATION creates a managed table.
- **Adding LOCATION 'path' makes it external**, pointing at storage you control.
- **The LOCATION must be covered by an external location** in Unity Catalog, with a storage credential behind it.
- **CTAS works for both:** CREATE TABLE … AS SELECT, with or without a LOCATION.

> ON THE EXAM
>
> The presence or absence of LOCATION is the whole distinction at creation time. There is no MANAGED keyword to remember.

What DROP does

- **Dropping a managed table deletes the underlying data files.**
- **Dropping an external table removes the metadata only.** The files remain at the path.
- **UNDROP TABLE can recover a dropped managed table** within the retention window.
- **But do not rely on it.** It is a safety net with a limit, not a substitute for care.

> ON THE EXAM
>
> An external table can be recreated over its files in seconds. A managed table's files are gone once the window closes.

Unity Catalog managed tables are now **the default and recommended table type**. External tables are for specific reasons, not a neutral alternative.

When external is still the right choice

- **Another system writes the data**, and Databricks is a reader rather than the owner.
- **Non-Databricks tools must read the files directly** at their path.
- **The data already exists** and copying it is impractical or prohibited.
- **A contractual or regulatory requirement** fixes where the data must live.

> ON THE EXAM
>
> Notice the shape: external is right when something outside Databricks has a legitimate claim on the files.

ALTER TABLE … SET MANAGED converts an external table to a managed one, keeping its name, history, permissions and views.

How SET MANAGED works

- **Requires Databricks Runtime 17.0 or above, or serverless compute.**
- **All readers and writers must use name-based access**, not path-based, as a prerequisite.
- **Step one copies the data and the Delta log** to the managed location, with no downtime.
- **Step two switches over**, moving commits made during step one and updating the metadata. Brief writer downtime here.

> ON THE EXAM
>
> If interrupted while copying, restart it and it continues from where it stopped. Do not run two conversions on one table at once.

After conversion

- **Read and write streams fail** and must be restarted with the same configurations.
- **Predictive optimisation is automatically enabled**, unless you had turned it off.
- **With predictive optimisation on, the data in the old external location is deleted after 14 days.**
- **From Databricks Runtime 18.1, path-based reads redirect** to the new managed location, so legacy code keeps working.

> ON THE EXAM
>
> That 14-day timer is the detail to carry. The old files are not kept indefinitely as a backup.

Rolling back

- **ALTER TABLE … UNSET MANAGED** converts a managed table back to external.
- **Seven days after rollback, the data in the managed location is deleted automatically.**
- **Commits made between conversion and rollback allow time travel by version, but not by timestamp.**
- **Streaming jobs must be restarted** after a rollback, as after a conversion.

> ON THE EXAM
>
> Two timers to keep straight: 14 days deletes the old external data after conversion; 7 days deletes the managed data after rollback.

One order that matters

- **You must run UNSET MANAGED before dropping a converted table.**
- **Dropping it without doing so may cause data loss or inconsistency.**
- **The reason is the drop rule.** Dropping a managed table deletes its files, including the converted ones.
- **Check the table type with DESCRIBE EXTENDED** or Catalog Explorer before you act.

> ON THE EXAM
>
> Two rules from this module combine into a trap: converted tables are managed, and dropping a managed table deletes data.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Dropping the table deleted the data	It was managed. Managed drops delete files.
Dropping the table left the files	It was external. Metadata only.
Convert external to managed	ALTER TABLE … SET MANAGED, on DBR 17.0+ or serverless
Roll a converted table back	ALTER TABLE … UNSET MANAGED
The old external files disappeared later	14 days after conversion, with predictive optimisation on
Another system writes these files	External is correct. Databricks is not the owner.
Which type is this table?	DESCRIBE EXTENDED, or Catalog Explorer

ON THE EXAM

Seven lines. The first two are the classic pair, and the fifth is the newest.

> A team tidies up a schema, dropping eleven tables they believe are obsolete. Two turn out to be still needed. One is recreated in minutes by pointing a CREATE TABLE at its original path. The other cannot be recovered the same way.
>
> Both tables were created by the same engineer, in the same schema, on the same day.

What is the difference between them?

A. The recoverable one was external, so DROP removed only its metadata; the other was managed, so its files were deleted

B. The recoverable one had been backed up and the other had not

C. The unrecoverable one was in a different catalog with stricter retention

D. DROP behaves differently depending on whether the table has been queried recently

◆ PAUSE AND DECIDE

What is the difference between them?

A One was external, one was managed

Correct. The recovery method described — pointing CREATE TABLE at the original path — only works when the files were left in place, which is external behaviour.

B One was backed up

Nothing suggests a backup, and the described recovery was recreating a table over an existing path, not restoring one.

C A different catalog

The scenario states both were in the same schema, so they share a catalog.

D Recency of querying

Invented behaviour. DROP does not consider query history.

THE SIGNAL

The recovery method is the evidence. Pointing CREATE TABLE at a path only works if the files are still there.

> A team has forty external Delta tables they want to convert to managed tables, to get predictive optimisation. They need to keep table history, keep existing grants and views working, and minimise downtime for the streaming jobs that write to them.
>
> An engineer proposes CREATE TABLE AS SELECT into new managed tables, then renaming.

What should they use instead?

A. The CTAS approach is fine and is the documented method

B. ALTER TABLE … SET MANAGED, which retains history, permissions and views, and minimises downtime

C. DEEP CLONE into a managed table, then drop the originals

D. Move the files into the managed storage location manually

◆ PAUSE AND DECIDE

What should they use instead?

A CTAS is fine

It works, and it loses table history, creates a new object needing grants reapplied, and requires downtime while it copies.

B ALTER TABLE … SET MANAGED

Correct. Databricks recommends it over CTAS precisely for these reasons: history, name, settings, permissions and views retained, with minimal downtime.

C DEEP CLONE then drop

Also a real option Databricks names, and SET MANAGED minimises or eliminates downtime compared to it.

D Move files manually

Unity Catalog manages the storage layout of managed tables. Hand-moving files leaves the metastore and the storage disagreeing.

THE SIGNAL

The team's three requirements — history, grants and views, low downtime — are the three benefits SET MANAGED is documented as providing.

> Three weeks after converting a large external table with SET MANAGED, a data science team reports that a Python script has broken. The script reads the table by its old storage path rather than by name, and the path now returns nothing.
>
> The cluster runs Databricks Runtime 16.4 LTS. Predictive optimisation is enabled.

What happened?

A The conversion failed silently and should be re-run

B With predictive optimisation on, the data in the old external location is deleted 14 days after conversion; the script needs to use name-based access

C The table was rolled back with UNSET MANAGED by someone else

D Path-based reads always fail against managed tables, from the moment of conversion

◆ PAUSE AND DECIDE

What happened?

A — The conversion failed silently

It plainly succeeded — the table worked by name for three weeks. Only the old path stopped working.

B — The 14-day deletion of the old external data

Correct. Three weeks is past 14 days, and path-based redirect requires DBR 18.1 or above, which this cluster is not on.

C — Someone rolled it back

A rollback would make the table external again and the managed data would be deleted after 7 days — a different symptom on a different timer.

D — Path-based reads always fail immediately

They worked for roughly two weeks after conversion, which is exactly why the failure was delayed and confusing.

THE SIGNAL

The delay is the clue. Nothing broke at conversion, and something broke three weeks later — which points at a timer, not an error.

> A team converted an external table to managed six months ago. They now want to decommission it entirely and free the storage. An engineer plans to run DROP TABLE, reasoning that the original files are in their own storage account and will therefore survive.
>
> A colleague stops him.

Why?

A The table is now managed, so DROP deletes its data; and the original external files were deleted 14 days after conversion anyway

B DROP TABLE is not permitted on converted tables

C He must run OPTIMIZE before dropping to release the storage

D Nothing is wrong; his reasoning is correct

◆ PAUSE AND DECIDE

Why?

A It is managed now, and the old files are long gone

Correct. Both halves of his reasoning are out of date: the table type changed, and the original external files were removed after 14 days.

B DROP is not permitted

It is permitted, which is the danger. Nothing stops the command; it simply does more than he expects.

C Run OPTIMIZE first

OPTIMIZE compacts files. It has nothing to do with releasing storage on a drop.

D His reasoning is correct

It was correct before the conversion. Conversion changed the table type, and with it what DROP does.

THE SIGNAL

He is applying a true fact about the table's past. Databricks says to run UNSET MANAGED before dropping a converted table.

Managed means Unity Catalog owns the lifecycle — including deleting the files when you drop the table. **Conversion changes which rules apply.**

MODULE 28 · DOMAIN 7 - GOVERNANCE AND SECURITY

GRANT, REVOKE, DENY and the hierarchy

Securables, inheritance, the traversal privileges — and an honest look at where DENY actually exists.

DOMAIN 7 of 7 EXAM WEIGHT 15% MODULE 28 of 36

admin@sthithapragna.com

In Unity Catalog, **nobody has access to anything by default**. Every permission is something somebody granted, deliberately.

The securable hierarchy

LEVEL	CONTAINS
Metastore	The top-level container. Catalogs, and the objects below.
Catalog	Schemas. The first level of the three-part name.
Schema	Tables, views, volumes, functions, models.
Table / view / volume / function	The objects people actually query

ON THE EXAM

Also securable, and outside the three-level namespace: external locations, storage credentials, connections and shares.

Inheritance

- **Privileges are inherited downward.** A grant on a catalog applies to every schema and object in it.
- **Including future objects.** A table created tomorrow inherits today's catalog grant.
- **The highest level inherited from is the catalog.**
- **Privileges granted on a metastore are not inherited.**

> ON THE EXAM
>
> Granting at catalog level is powerful and blunt. It covers everything, including tables nobody has created yet.

SELECT on a table is not enough. You also need **USE CATALOG** on its catalog and **USE SCHEMA** on its schema, or you cannot reach it.

Privileges worth knowing

PRIVILEGE	ALLOWS
USE CATALOG	Traverse into a catalog. Required to reach anything inside it.
USE SCHEMA	Traverse into a schema. Required to reach anything inside it.
SELECT	Read a table or view
MODIFY	Insert, update and delete data in a table
CREATE TABLE	Create a table in a schema
EXECUTE	Run a function — including a registered model
READ VOLUME / WRITE VOLUME	Read from or write to a volume
BROWSE	See an object's metadata without access to its data

ON THE EXAM

BROWSE is how a data catalog stays discoverable. People can find that a table exists and request access, without reading it.

ALL PRIVILEGES

- **Implies all applicable privileges** for that object type, without granting each individually.
- **It deliberately excludes some.** MANAGE, EXTERNAL USE SCHEMA and EXTERNAL USE LOCATION are not included.
- **The reason is to prevent accidental data exfiltration or privilege escalation.**
- **So ALL PRIVILEGES is not quite all privileges**, which is exactly the kind of thing an exam asks about.

> ON THE EXAM
>
> Schema owners do not get EXTERNAL USE SCHEMA by default either, and external location owners do not get EXTERNAL USE LOCATION.

GRANT and REVOKE

- **GRANT privilege ON securable-type name TO principal**
- **REVOKE privilege ON securable-type name FROM principal**
- **Both are available in SQL and in Catalog Explorer's Permissions tab.**
- **REVOKE succeeds even if the privilege was never granted.** It ensures absence rather than reversing a grant.

> ON THE EXAM
>
> On a metastore grant you omit the metastore name, because the metastore attached to your workspace is assumed.

The exam guide names DENY. Databricks documentation says: **no support for DENY** in Unity Catalog. Both of those are true, and you need both.

Where DENY does exist

- **In the legacy Hive metastore table access control model**, alongside GRANT, REVOKE, MSCK and SHOW GRANTS.
- **There, denying a privilege on a catalog denies it on every schema within**, the same way grants inherit.
- **A denied user cannot even see the object** when listing.
- **That model is legacy**, and Databricks recommends upgrading to Unity Catalog.

> ON THE EXAM
>
> DENY belongs to hive_metastore. If a scenario is set in Unity Catalog, the answer involves not granting, rather than denying.

How to handle this on the day

- **Know what DENY does**, because the guide names it and a question may use it.
- **Know that Unity Catalog does not support it**, because that may itself be the answer.
- **Do not assume a DENY option is automatically wrong.** Read whether the scenario is UC or hive_metastore.
- **In a UC scenario, achieving exclusion means revoking or not granting**, or narrowing the grant to a smaller scope.

> ON THE EXAM
>
> This is the clearest case in the course of the exam guide and the product documentation disagreeing. Carry both.

Every securable has exactly one owner. Owners can do everything on their object — but Databricks **does not explicitly grant them ALL PRIVILEGES**.

MANAGE is not ownership

- **MANAGE grants the ability to grant and revoke privileges** on an object.
- **It does not grant all privileges on the object.** A user with MANAGE cannot necessarily read the data.
- **But they can grant themselves SELECT**, which makes the distinction largely theoretical in practice.
- **Owners can give MANAGE to others**, delegating permission administration without transferring ownership.

> ON THE EXAM
>
> MANAGE is how you let a team lead administer access to their own data without handing over the object itself.

Who can grant

- **The owner of the object.**
- **The owner of the catalog or schema that contains it.**
- **A principal with MANAGE on the object.**
- **A metastore admin**, and account admins have default privileges for managing Unity Catalog.

> ON THE EXAM
>
> Note the second line. Owning a schema lets you grant on tables inside it, even ones you did not create.

Principals

- **Users, service principals and groups** — the three the objective names.
- **Grant to groups, not to individuals.** Joining and leaving a team then changes access automatically.
- **Unity Catalog uses account-level groups**, whereas the legacy Hive metastore used workspace-local groups.
- **Service principals are principals too**, which is how the production jobs from module 19 get their access.

> ON THE EXAM
>
> Granting to individuals produces an access model nobody can audit, and one that breaks every time somebody changes team.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
SELECT was granted but the query fails	Missing USE CATALOG or USE SCHEMA
A grant should cover future tables too	Grant at catalog or schema level; inheritance is downward
A metastore grant did not reach a catalog	Metastore grants are not inherited
Exclude a group in Unity Catalog	Revoke or do not grant. UC has no DENY.
The scenario is in hive_metastore	DENY exists there, and inherits downward
Someone must administer access but not read data	MANAGE, not ownership
ALL PRIVILEGES did not include something	MANAGE and the EXTERNAL USE privileges are excluded

ON THE EXAM

Seven lines. The first is the most common real-world problem; the fourth and fifth are the pair to keep straight.

> An analyst is granted SELECT on a table in finance.reporting. SHOW GRANTS confirms the grant exists and names her group correctly.
>
> Every query she runs against the table fails with a permissions error. Other members of the same group can query it without difficulty.

What is the most likely explanation?

A. The grant needs time to propagate across the metastore

B. She is missing USE CATALOG on finance or USE SCHEMA on reporting, which the others hold from another group

C. A DENY on the table is overriding her SELECT

D. The table is external, so SELECT does not apply

◆ PAUSE AND DECIDE

What is the most likely explanation?

A Propagation delay

Invents a delay, and would not explain why other members of the same group succeed right now.

B Missing USE CATALOG or USE SCHEMA

Correct. Reaching a table requires traversal privileges on its catalog and schema as well as SELECT. Her colleagues hold those from elsewhere.

C A DENY is overriding it

Unity Catalog has no DENY. Its model is least privilege — unGRANTed is implicitly denied, and there is nothing to override with.

D SELECT does not apply to external tables

SELECT applies to both managed and external tables. The table type does not change how privileges work.

THE SIGNAL

Same group, same grant, different outcome. The difference must come from a privilege held outside that group.

A catalog contains forty schemas. A group has been granted SELECT at catalog level, which they need for thirty-nine of them. One schema holds salary data that this group must not see.

An engineer proposes DENY SELECT on that one schema, so the catalog-level grant stays in place.

What should they do instead?

A Use DENY as proposed; it is the standard way to carve out an exception

B Unity Catalog has no DENY. Revoke the catalog-level grant and grant at the level of the 39 schemas, or on the objects within them.

C Move the salary schema into the hive_metastore catalog so DENY can be used

D Grant ALL PRIVILEGES on the catalog and rely on ownership to exclude the schema

◆ PAUSE AND DECIDE

What should they do instead?

A Use DENY

It is the standard way in many database systems, and Unity Catalog does not support it. The proposal would simply fail.

B Narrow the grant instead

Correct. With no DENY, exclusion is achieved by granting at a narrower scope — the schemas they should see, rather than the catalog.

C Move it to hive_metastore

Solves it by moving sensitive data into a legacy model Databricks recommends migrating away from. Precisely the wrong direction for salary data.

D ALL PRIVILEGES plus ownership

ALL PRIVILEGES at catalog level makes the exposure broader, not narrower. Ownership does not exclude anyone from an inherited grant.

THE SIGNAL

Inheritance is downward and there is no mechanism to interrupt it. So the grant must start below the thing you are excluding.

> A data platform team owns every catalog. Team leads keep raising tickets asking for colleagues to be granted access to their own team's tables, and the platform team has become a bottleneck.
>
> They want team leads to administer access to their own data, without giving them the ability to drop objects or transfer ownership.

What should they grant?

A. Transfer ownership of each schema to the relevant team lead

B. Grant MANAGE on the relevant schemas to the team leads

C. Grant ALL PRIVILEGES on the relevant schemas to the team leads

D. Make each team lead a metastore admin

◆ PAUSE AND DECIDE

What should they grant?

A Transfer ownership

Owners can also drop the object and transfer ownership onward, which the requirement explicitly excludes.

B Grant MANAGE on the schemas

Correct. MANAGE confers exactly the ability to grant and revoke privileges on the object, without ownership's other capabilities.

C ALL PRIVILEGES

Deliberately excludes MANAGE, so it would not let them grant to others — the one thing they actually need.

D Metastore admin

Grants administrative reach over the entire metastore to solve a per-schema problem. Wildly broader than the requirement.

THE SIGNAL

Option C is the trap: ALL PRIVILEGES sounds like more than MANAGE, and specifically excludes it.

> An audit finds that access to a sensitive catalog has been granted directly to 63 individual users over three years. Nobody can say who should still have access, and four of the users have left the company.
>
> The team is asked to make the access model auditable.

What is the right change?

A Grant to account-level groups instead, and revoke the individual grants

B Document the 63 grants in a spreadsheet reviewed quarterly

C Move the grants to the metastore level so they are managed in one place

D Grant ALL PRIVILEGES to a single service principal and route all queries through it

◆ PAUSE AND DECIDE

What is the right change?

A Grant to groups, revoke the individuals

Correct. Group membership becomes the single place access is decided, and leavers lose access through the same process that removes them from the group.

B A spreadsheet reviewed quarterly

Documents the problem rather than fixing it, and adds a second source of truth that will diverge from the real grants.

C Move grants to the metastore level

Metastore privileges are not inherited downward, so this would not grant catalog access at all.

D One service principal for everyone

Destroys attribution entirely. Every query would appear to come from one identity, which is worse for audit, not better.

THE SIGNAL

Sixty-three individual grants is not sixty-three decisions. It is one decision — 'who is on this team' — recorded sixty-three times.

Nothing is permitted until it is granted. **Traversal is a privilege too**, inheritance only flows down, and in Unity Catalog exclusion means narrowing the grant.

MODULE 29 · DOMAIN 7 - GOVERNANCE AND SECURITY

Row filters, column masks and dynamic views

Three ways to show different users different data from the same table, and when each is right.

DOMAIN 7 of 7 EXAM WEIGHT 15% MODULE 29 of 36

admin@sthithapragna.com

All three work the same way: the query runs, and **the data is filtered or transformed at query time** based on who is asking.

The three mechanisms

MECHANISM	ATTACHED TO	BEST FOR
Dynamic view	A view over one or more tables	Curated or joined data for users with no access to the base tables
Column mask	One column on one table	Transforming a sensitive value in place
Row filter	One table	Restricting which rows a user sees
ABAC policy	A catalog or schema	Consistent rules across many tables. Module 30.

ON THE EXAM

Read the middle column. The scope each one attaches to is what really separates them, and what a scenario is usually pointing at.

The identity functions

- **is_account_group_member('group')** returns true if the querying user belongs to that account-level group.
- **current_user() and session_user()** return the identity running the query.
- **These are what make any of this dynamic.** Without them you have an ordinary view or a constant expression.
- **Account groups, not workspace-local groups**, which is module 28's Unity Catalog distinction again.

> ON THE EXAM
>
> The objective says 'based on user groups'. is_account_group_member is the function that phrase is pointing at.

Dynamic views

- **An ordinary view with an identity function inside it**, typically in a CASE expression or a WHERE clause.
- **Users are granted SELECT on the view**, and need no access to the base tables at all.
- **It can span several tables**, joining and reshaping as well as restricting.
- **They prevent users from modifying the underlying tables**, because the user never has access to them.

> ON THE EXAM
>
> A CASE returning either the real email or just its domain, depending on group membership, is the canonical example.

Dynamic views or filters and masks?

DYNAMIC VIEWS

A curated layer

- Can span and join multiple tables
- Fully support query optimisation and predicate pushdown
- Often better query performance
- Users need no access to the base tables

ROW FILTERS AND COLUMN MASKS

Applied to the table itself

- Protect the table wherever it is queried from
- Users cannot see base values before filtering
- Visible as policy metadata for auditing
- Scoped to one table, or one column

THE TIEBREAKER

The deciding question: should the protection follow the table, or should you present a different object entirely?

Two drawbacks of dynamic views

- **Limited auditing.** They lack semantic metadata such as tags or policy definitions in system tables.
- **So they are harder to audit at scale.** The rule lives in view SQL rather than in a policy you can query.
- **Vulnerability to probing.** They lack a SecureView barrier.
- **A user can craft a predicate with side effects** to infer information about the rows they cannot see.

> ON THE EXAM
>
> Both drawbacks are about governance rather than function. The view works; it is harder to prove what it does, and harder to defend.

A function that takes the column's value and returns **something of the same type** — the real value for some users, a redacted one for others.

Column mask syntax

STEP	STATEMENT
Create the function	CREATE FUNCTION ssn_mask(ssn STRING) RETURN CASE WHEN is_account_group_member('HR') THEN ssn ELSE '***-**-****' END
Apply to a column	ALTER TABLE users ALTER COLUMN ssn SET MASK ssn_mask
Or apply at creation	CREATE TABLE users (name STRING, ssn STRING MASK ssn_mask)
Remove	ALTER TABLE users ALTER COLUMN ssn DROP MASK

ON THE EXAM

A mask can take extra columns as parameters using USING COLUMNS, so the decision can depend on other values in the row.

A function that returns a **boolean**. Rows where it evaluates to true are visible; the rest are not returned at all.

Row filter syntax

STEP	STATEMENT
Create the function	CREATE FUNCTION region_filter(region STRING) RETURN is_account_group_member('global') OR is_account_group_member(region)
Apply to a table	ALTER TABLE sales SET ROW FILTER region_filter ON (region)
Remove	ALTER TABLE sales DROP ROW FILTER

ON THE EXAM

Note ON (region): you name which columns are passed to the function. That is how the filter sees the row's own values.

Limitations worth knowing

- **Path-based access to files in tables with policies is not supported.**
- **MERGE is not supported** where the policy contains nesting, aggregations, windows, limits, or non-deterministic functions.
- **Below Databricks Runtime 17.2**, DELETE, UPDATE and MERGE are unsupported on partitioned tables with a policy on the partition column.
- **Circular dependencies are not supported**, where a policy refers back to itself through another object.

> ON THE EXAM
>
> The path-based restriction is the most sweeping. A policy on a table cannot protect a file somebody reads directly.

The referencing restriction

- **Row filters and column masks cannot reference tables that themselves have active row filters or column masks.**
- **So a lookup table used by a policy must be unprotected**, or the policy will not work.
- **This bites when you build a mapping table** of users to regions and then protect that table too.
- **In ABAC configurations there is a workaround**: exclude the policy function owner from the referenced table's policy.

> ON THE EXAM
>
> A natural design — a policy that looks up entitlements in a table, and that table protected as well — is exactly what this forbids.

Where ABAC fits

- **Row filters and column masks are per table and per column**, managed by the table owner.
- **That does not scale to consistent rules across hundreds of tables.**
- **ABAC policies attach at catalog or schema level** and apply automatically based on governed tags.
- **If both apply the same function, it works. Different functions, and access is blocked with an error** — even if they would produce identical output.

> ON THE EXAM
>
> Module 30 covers ABAC properly. Know now that it exists, what it solves, and that mixing it with table-level policies has rules.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Hide a value but keep the row	A column mask
Hide the row entirely	A row filter
Users must not have access to the base tables	A dynamic view
Data must be joined from several tables first	A dynamic view
The rule must apply wherever the table is queried	A row filter or column mask, not a view
Consistent rules across many tables	ABAC policies. Module 30.
A policy stopped working after protecting a lookup table	Policies cannot reference protected tables

ON THE EXAM

Seven lines. The first two are the fundamental split, and the third and fifth are the pair that separates views from table-level policies.

> A customers table holds a national_id column. Everyone in the company needs to query the table, and the row count must be correct for everyone. Only the compliance team may see the actual identifier values.
>
> An engineer proposes creating a second table with the column removed, for everyone else to use.

What is the better approach?

A. The second table is correct; it is the only way to hide a column

B. A column mask on national_id, returning the real value for the compliance group and a redacted value for everyone else

C. A row filter on the customers table

D. Revoke SELECT on the table from everyone except compliance

◆ PAUSE AND DECIDE

What is the better approach?

A A second table without the column

Works on day one and creates a copy that must be kept in sync forever. Two tables mean two sets of grants and eventual drift.

B A column mask on national_id

Correct. One table, correct row counts for everyone, and the value transformed at query time based on group membership.

C A row filter

Row filters remove rows. The requirement is that row counts stay correct, so removing rows is the wrong axis entirely.

D Revoke SELECT from everyone else

Denies the whole table to people who legitimately need the other columns. Too blunt by an entire table.

THE SIGNAL

Two requirements pin it: everyone queries the table, and row counts must be right. That rules out filtering and rules out a separate copy.

> A team must share a curated slice of data with an external-facing analyst group. The slice joins three tables, renames columns, and shows only aggregated figures. The analysts must not be able to reach any of the three base tables.
>
> A colleague suggests applying row filters and column masks to all three tables instead.

Which approach fits?

A Row filters and column masks on all three base tables, then grant SELECT on them

B A dynamic view over the three tables, with SELECT granted on the view only

C Copy the joined result into a new table on a schedule

D Grant the analysts BROWSE on the three tables

◆ PAUSE AND DECIDE

Which approach fits?

A Filters and masks on the base tables

Requires granting SELECT on all three, which the requirement forbids. It also cannot join or aggregate.

B A dynamic view, SELECT on the view only

Correct. Views span multiple tables, reshape and aggregate, and users need no access to the base tables at all.

C A scheduled copy

Works, and creates a second copy to maintain and secure, plus staleness between refreshes. A view is always current.

D Grant BROWSE

BROWSE shows metadata without data. The analysts need the data, in a specific shape.

THE SIGNAL

'Must not reach the base tables' plus 'joins three tables' is the dynamic view signature. Neither is a filter-and-mask job.

> A row filter on a sales table calls a function that looks up the querying user's permitted regions in a small user_regions mapping table. It has worked correctly for months.
>
> A security review notes that user_regions itself lists who can see what, so a column mask is applied to it. The sales table filter immediately stops working.

Why?

A The mask on user_regions changed the data type the filter expects

B Row filters and column masks cannot reference tables that themselves have active row filters or column masks

C Applying a mask invalidates all functions in the same schema

D The filter needs to be dropped and re-applied after any change to a referenced table

◆ PAUSE AND DECIDE

Why?

A The mask changed the data type

A mask must return the same type as the column it masks, so the type is unchanged by construction.

B Policies cannot reference protected tables

Correct. This is a documented restriction, and the security review's change created exactly the situation it forbids.

C Masks invalidate functions in the schema

Invented behaviour. A mask affects the column it is applied to, not unrelated functions.

D Drop and re-apply the filter

Would not help. The restriction is structural, so re-applying the filter recreates the same forbidden reference.

THE SIGNAL

The security review was reasonable and produced a broken configuration. Protecting the lookup table is exactly what the restriction forbids.

> A bank protects personal data across 340 tables using dynamic views, one per table, each embedding is_account_group_member checks. An auditor asks for a list of every column containing personal data and the rule applied to each.
>
> The team cannot produce it without reading 340 view definitions by hand.

What does this illustrate?

A. Dynamic views lack semantic metadata such as tags or policy definitions in system tables, making them hard to audit at scale

B. Dynamic views do not actually restrict access, only appearance

C. The views should have been created in a single schema

D. Auditors cannot query system tables in Unity Catalog

◆ PAUSE AND DECIDE

What does this illustrate?

A Limited auditing — no policy metadata in system tables

Correct. This is one of the two documented drawbacks of dynamic views. The rule lives in view SQL, which is not queryable as policy.

B They do not really restrict access

They do restrict access properly. The problem is proving what they restrict, not whether they work.

C Put them in one schema

Organises 340 views without making their logic queryable. The auditor still has to read every definition.

D Auditors cannot query system tables

They can. The issue is that dynamic view logic does not appear there as policy metadata to be queried.

THE SIGNAL

The views work. What fails is governance at scale — which is precisely the gap ABAC policies and governed tags exist to close.

A mask returns the column's type and changes **what you see**. A filter returns a boolean and changes **which rows you see**. A view changes **which object you query**.

MODULE 30 · DOMAIN 7 - GOVERNANCE AND SECURITY

ABAC, governed tags and classification

Describing the data instead of configuring each table — and the newest material on this exam.

DOMAIN 7 of 7 EXAM WEIGHT 15% MODULE 30 of 36

admin@sthithapragna.com

Stop configuring each table. **Describe the data** with tags, write one policy about the description, and every matching object is covered — including ones created next year.

Three capabilities that work together

01

Governed tags

An account-level vocabulary describing what data is.

02

Classification

Finds sensitive data and applies those tags automatically.

03

ABAC policies

Act on the tags, filtering rows and masking columns.

04

Separation of duties

Different people own the vocabulary, the tagging and the policies.

WHY THEY ARRIVED TOGETHER

Databricks released all three together, deliberately. Each is much less useful without the others.

Governed tags

- **Key-value pairs defined at the account level**, forming a controlled vocabulary.
- **Applied to catalogs, schemas, tables, columns, models and volumes**, and to workspace objects.
- **They describe characteristics:** sensitivity, classification, business domain. For example pii:ssn or sensitivity:confidential.
- **Permissions control who can apply which tags to which objects**, which is what makes them governed rather than free text.

> ON THE EXAM
>
> The word governed is the important one. Anyone can add a comment; only authorised people can apply a governed tag.

Tag inheritance, and its one exception

- **Securables inherit tags from their parent catalog or schema by default.**
- **You can override inherited tags at every level except one.**
- **Column tags do not inherit from the parent table.** They must be applied directly.
- **Data creators automatically have APPLY TAG on objects they create**, so they can add tags to their own data.

> ON THE EXAM
>
> That exception matters constantly. Tagging a table as containing PII does not tag its columns, and a column-matching policy will not fire.

A warning about tag content

- **Tag data can be replicated globally.**
- **So do not use tag names or values that could compromise security**, and never put personal or sensitive information in a tag.
- **Tags describe the data. They are not the data.**
- **pii:ssn is a correct tag.** A tag containing an actual identifier is not.

> ON THE EXAM
>
> Databricks flags this explicitly in its documentation. A tag is metadata, and metadata travels further than you might expect.

A policy attaches to a **catalog, schema or table**, and applies automatically to every object whose tags match its condition.

CREATE POLICY

CLAUSE	WHAT IT DOES
ON CATALOG | SCHEMA | TABLE	Where the policy attaches, and therefore its scope
ROW FILTER fn / COLUMN MASK fn	Which kind of policy, and the UDF that implements it
TO principal [EXCEPT principal]	Who it applies to, and who is exempted
FOR TABLES	Required
WHEN condition	Which securables match — usually a table-level tag test
MATCH COLUMNS condition AS alias	Which columns match, and a name to refer to them by
USING COLUMNS (args)	Arguments passed to the function, in order

ON THE EXAM

The policy name is scoped to the securable it is defined on. You need MANAGE on that securable, or ownership, to create it.

Only two functions in conditions

- **has_tag('key')** matches objects carrying that tag key.
- **has_tag_value('key', 'value')** matches objects carrying that key with that specific value.
- **Those are the only user-facing functions supported in policy conditions.**
- **The camelCase forms hasTag and hasTagValue still work**, for backward compatibility.

> ON THE EXAM
>
> Conditions are evaluated by the control plane on securable metadata, which is why the vocabulary is deliberately tiny.

WHEN and MATCH COLUMNS do different jobs

- **WHEN matches securables.** Which tables does this policy apply to at all?
- **MATCH COLUMNS matches columns.** Within those tables, which columns does it act on?
- **A column mask needs both**, plus ON COLUMN naming the alias to mask.
- **Confusing the two is the easiest mistake**, because both take a tag condition and they read alike.

> ON THE EXAM
>
> WHEN filters at table level, MATCH COLUMNS at column level. A policy can use either, or both together.

Two worked policies

POLICY	STATEMENT
Row filter	CREATE POLICY hide_eu_customers ON SCHEMA prod.customers ROW FILTER non_eu_region TO us_analysts FOR TABLES MATCH COLUMNS has_tag('geo_region') AS region USING COLUMNS (region)
Column mask	CREATE POLICY mask_ssn ON SCHEMA prod.customers COLUMN MASK mask_ssn TO us_analysts EXCEPT admins FOR TABLES MATCH COLUMNS has_tag_value('pii','ssn') AS ssn ON COLUMN ssn

ON THE EXAM

Read the row filter one carefully: it never names a table. It names a schema, and a tag. Every matching table is covered.

How a policy is evaluated

- Unity Catalog finds every policy whose scope covers the object being accessed.
- It checks whether the user is in the TO list and not in the EXCEPT list.
- It evaluates the WHEN condition against the object's tags, including inherited ones.
- **If the policy applies, the filter or mask takes effect** for that query.

> ON THE EXAM
>
> Every access, every time. Nothing is precomputed, so tagging a table this morning protects it this afternoon.

Requirements

- **MANAGE on the target securable, or ownership**, for every policy operation.
- **Databricks Runtime 16.4 or above, or serverless compute.**
- **Dedicated compute additionally needs fine-grained access control filtering enabled.**
- **Older runtimes cannot access tables secured by ABAC at all.**

> ON THE EXAM
>
> That last point is a hard failure, not a degradation. A cluster on an older runtime does not see unprotected data — it cannot read the table.

ABAC or table-level?

ABAC POLICIES

Describe once, apply everywhere

- Attach at catalog or schema level
- Cover new tables automatically as they are tagged
- Separation of duties between policy authors and stewards
- Queryable as policy metadata, so auditable at scale

TABLE-LEVEL FILTERS AND MASKS

Configure per object

- Attached with ALTER TABLE, managed by the table owner
- Apply to that one table or column only
- Right for genuinely per-table logic
- Simpler when there are only a few sensitive tables

THE TIEBREAKER

Databricks recommends ABAC when you need consistent rules across many tables. Table-level remains right for one-off logic.

When both apply

- **Same function on both sides:** execution is allowed.
- **Different functions:** access is blocked and an error is returned.
- **Even if the two functions would produce identical output.**
- **The design is fail-closed.** An ambiguous protection is refused rather than guessed at.

> ON THE EXAM
>
> So migrating table-by-table means removing the old policy as you add the new one, not leaving both in place.

Data classification

- **Automatically identifies sensitive data** such as personal and health information, and applies governed tags.
- **Built-in classifiers cover standards including GDPR and HIPAA.**
- **Custom classifiers learn from already-tagged columns**, extending detection to business-specific patterns.
- **New data is scanned automatically**, so sensitive data introduced later is caught.

> ON THE EXAM
>
> This is what makes the whole model work at scale. A policy is only as good as the tagging it acts on.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Consistent masking across hundreds of tables	An ABAC policy at catalog or schema level
New tables must be covered automatically	ABAC, as soon as they are tagged
A column-matching policy is not firing	Column tags do not inherit from the table
Only two functions work in a condition	has_tag and has_tag_value
Access blocked where both mechanisms apply	Different functions. Same function would be allowed.
An older cluster cannot read the table	ABAC needs DBR 16.4+, or serverless
Sensitive data must be found, not just protected	Data classification, applying governed tags

ON THE EXAM

Seven lines. The third is the one most likely to appear as a diagnosis question.

> A bank must mask every column containing a national identifier across 400 tables in six catalogs, for everyone except the compliance group. New tables are created weekly by twelve different teams.
>
> An engineer proposes writing a masking function and applying it to each column with ALTER TABLE.

What should they do instead?

A. The ALTER TABLE approach, scripted so it can be re-run weekly

B. Tag the columns with a governed tag and write one ABAC column mask policy per catalog

C. A dynamic view over each of the 400 tables

D. Revoke SELECT on all 400 tables from everyone except compliance

◆ PAUSE AND DECIDE

What should they do instead?

A Scripted ALTER TABLE, re-run weekly

Works, and the protection is only ever as current as the last run. A table created on Tuesday is unmasked until the script runs.

B Governed tags plus one ABAC policy per catalog

Correct. Six policies instead of hundreds of statements, and new tables are covered the moment they are tagged.

C 400 dynamic views

Recreates module 29's auditing problem at scale, and requires every consumer to be redirected to a view.

D Revoke SELECT everywhere

Denies whole tables to people who need the other columns. Far broader than masking one column.

THE SIGNAL

'New tables are created weekly by twelve teams' is the deciding detail. Any approach that must be re-run has a window where data is exposed.

A team tags a table with pii:true, then writes a column mask policy on the schema using MATCH COLUMNS has_tag_value('pii', 'true'). The policy is created without error.

Querying the table returns unmasked values for everyone. SHOW POLICIES confirms the policy exists on the schema.

Why is nothing being masked?

A The policy needs to be attached to the table rather than the schema

B The tag was applied to the table, and column tags do not inherit from the parent table; the columns must be tagged directly

C has_tag_value is not supported in MATCH COLUMNS

D The policy requires a WHEN clause to function

◆ PAUSE AND DECIDE

Why is nothing being masked?

A **Attach it to the table**

Schema-level attachment is normal and correct. The scope is fine; the tags are in the wrong place.

B **Column tags do not inherit from the table**

Correct. Inheritance can be overridden at every level except columns — column tags must be applied directly, so MATCH COLUMNS finds nothing.

C **has_tag_value is unsupported there**

Both has_tag and has_tag_value are supported in conditions. They are the only two, and this is one of them.

D **A WHEN clause is required**

WHEN is optional. It narrows which securables match; its absence means the policy considers all of them.

THE SIGNAL

The policy is correct and matches nothing. MATCH COLUMNS looks at column tags, and the only tag applied was on the table.

> A team is migrating from table-level column masks to ABAC. On one table they add an ABAC column mask policy while the existing ALTER TABLE mask is still in place. Both use different functions, though both redact to the same fixed string.
>
> Queries against that table now fail with an error rather than returning masked data.

What is happening?

A. When both mechanisms apply different functions, access is blocked — even if the output would be identical

B. ABAC policies cannot be applied to tables that already have masks; the ABAC policy was never created

C. The two masks are being applied in sequence, double-masking the value

D. The table needs OPTIMIZE before the new policy takes effect

◆ PAUSE AND DECIDE

What is happening?

A Different functions block access

Correct. Same function on both sides is allowed; different functions are blocked with an error, regardless of whether the output matches.

B The policy was never created

Both can coexist — the same function on both sides is explicitly allowed. Creation is not what failed.

C Double-masking

The system does not chain them. It refuses rather than composing two protections it cannot reconcile.

D Run OPTIMIZE first

OPTIMIZE concerns file layout. Policies are evaluated at query time from metadata and need no maintenance operation.

THE SIGNAL

'Though both redact to the same fixed string' is placed to tempt you. The system does not compare outputs — only the functions.

> A healthcare company must find and protect health information across an estate nobody has fully catalogued. They have a compliance team who understand HIPAA, a data platform team who write policies, and hundreds of data producers.
>
> A manager proposes the compliance team manually review every column and hand a list to the platform team.

What is the better design?

A The manual review, since compliance expertise cannot be automated

B Automated classification applies governed tags; compliance owns the tag vocabulary; the platform team writes ABAC policies against those tags

C Give the compliance team ownership of every table so they can apply masks directly

D Have each data producer write their own column masks

◆ PAUSE AND DECIDE

What is the better design?

A. Manual review by compliance

Does not scale to an uncatalogued estate, and is a snapshot — new columns appear continuously and the review is immediately stale.

B. Classification tags, compliance owns the vocabulary, platform writes policies

Correct. This is exactly the separation of duties the three capabilities were designed to enable, with built-in HIPAA classifiers doing the discovery.

C. Compliance owns every table

Makes a compliance team responsible for operating data they did not create. It also concentrates rather than separates duties.

D. Each producer writes their own masks

Hundreds of independent interpretations of HIPAA, with no consistency and nothing centrally auditable.

THE SIGNAL

Three roles, three capabilities. Compliance defines what sensitive means, classification finds it, and policies act on it.

Tag the data, write one policy about the tag. **Coverage then follows the description, not the configuration** — including for tables that do not exist yet.

MODULE 31 · EXAM STRATEGY - ALL DOMAINS

Confusion pairs

Forty things that look alike, and the one detail that separates each pair.

COVERS All 7 domains TYPE Revision MODULE 31 of 36

admin@sthithapragna.com

For each pair, do not memorise both. Memorise **the one detail that separates them**, and you can derive the rest.

Compute and platform

THE PAIR	THE DISCRIMINATOR
Jobs compute / all-purpose	Terminates with the job / bills 2–3x more
Standard / dedicated access mode	Dedicated for Runtime ML, GPU, RDD API and R
Classic / serverless SQL warehouse	Cold start: about 4 minutes / seconds
Control plane / compute plane	Notebooks and job config / processes data
Managed / external table	DROP deletes the files / leaves them

ON THE EXAM

The last row is the most consequential distinction in the course.

Ingestion

THE PAIR	THE DISCRIMINATOR
COPY INTO / Auto Loader	Thousands of files is COPY INTO. Millions is Auto Loader.
mergeSchema: FORMAT_OPTIONS / COPY_OPTIONS	Reconciles source files / evolves the target table
FILES / PATTERN	Mutually exclusive. FILES caps at 1000.
addNewColumns / rescue	Fails then resumes / never fails, never evolves
failOnNewColumns / none	Fails and will not restart / ignores the column
Directory listing / file events	File events is recommended, and must run at least every 7 days.

ON THE EXAM

Auto Loader's four evolution modes are best learnt as two pairs.

Writing data

THE PAIR	THE DISCRIMINATOR
saveAsTable / insertInto	By name / by position. insertInto swaps silently.
CREATE OR REPLACE TABLE / INSERT OVERWRITE	DDL replaces the schema / DML keeps it
append / overwrite / error / ignore	error is the default, not append
cast / try_cast	Under ANSI cast throws. try_cast returns null.
MERGE / append	MERGE is idempotent on a key. append is not.

ON THE EXAM

Row one is a silent failure: same-type columns raise no error.

Combining and deduplicating

THE PAIR	THE DISCRIMINATOR
SQL UNION / DataFrame union()	Deduplicates / does not. Nor does unionAll.
union() / unionByName()	By position / by name. Same hazard as insertInto.
distinct() / dropDuplicates(subset)	Whole rows / named columns
dropDuplicates(subset) / row_number window	Arbitrary row / you state which one wins
EXCEPT / left_anti	Whole rows / join keys only
explode / explode_outer	Drops empty arrays / keeps them

ON THE EXAM

Rows one and two are where SQL habits mislead you most.

Aggregation and joins

THE PAIR	THE DISCRIMINATOR
count(*) / count(col)	Rows / non-null values. They disagree exactly when nulls exist.
count_distinct / approx_count_distinct	Exact / estimated. rsd defaults to 0.05; below 0.01 use the exact one.
describe() / summary()	summary adds quartiles, which is what reveals skew.
Filter before groupBy / after	Before changes the values. After changes which groups appear.
groupBy / window function	groupBy collapses rows. A window keeps them and adds the aggregate.
Broadcast / sort-merge join	Move the join to the data / move the data to the join.

ON THE EXAM

Filter placement is the most commonly tested of these, because it changes both the values and the row count.

Gold objects and quality

THE PAIR	THE DISCRIMINATOR
View / materialised view	A view is always current and always costs. An MV is fast and can be stale.
Materialised view / streaming table	If the source must be read only once, it must be a streaming table.
Table constraint / pipeline expectation	A constraint binds every writer. An expectation binds the pipeline.
warn / drop / fail	warn is the default, and it writes the invalid record to the target.
drop / quarantine	drop keeps only a count. Quarantine keeps the rows for review.
fail / the others	fail records no metrics, because the update stops on detection.

ON THE EXAM

warn being the default is the single most testable fact in Domain 3's quality objective.

Jobs

THE PAIR	THE DISCRIMINATOR
Run if / If/else	Run if branches on outcome. If/else branches on a value.
Upstream failed / Excluded	Excluded counts as successful. Upstream failed does not.
Succeeded / Succeeded with failures	Job status comes from leaf tasks. A Failed filter hides the second.
Retry / repair	A retry re-runs a task. A repair re-runs a run's unsuccessful tasks.
Cooldown / debouncing	Cooldown limits frequency. Debouncing waits for the batch to finish arriving.
Triggered / continuous pipeline	Only a triggered pipeline can be a job task.

ON THE EXAM

Rows two and three combine into Domain 4's most surprising behaviour: a job can report success while a task inside it failed.

CI/CD

THE PAIR	THE DISCRIMINATOR
Git folders / bundles	A development tool / a deployment mechanism.
Development mode / production mode	Development prefixes resource names. Production validates the Git branch.
OAuth U2M / M2M	A person at a keyboard / a service principal. A CI runner is nobody.
PAT / OAuth	PATs are long-lived and now labelled legacy.
validate / deploy	validate touches no workspace. It belongs on every pull request.
Source format / .ipynb	Source gives clean diffs. IPYNB preserves dashboards and visualisations.

ON THE EXAM

The first row decides most of Domain 5. If a question mentions dev, test and prod differing, you have left Git folders behind.

Optimisation

THE PAIR	THE DISCRIMINATOR
Skew / under-resourced	A few slow tasks / all tasks slow. Opposite fixes.
Spill (memory) / spill (disk)	Deserialised / serialised. Memory is normally much larger.
Driver OOM / executor OOM	collect, toPandas, broadcast / partition size and skew.
shuffle.partitions / default.parallelism	Spark SQL / RDDs only.
Partitioning / liquid clustering	Partition columns are permanent. Clustering keys can change.
Z-ORDER / liquid clustering	Predictive optimisation ignores Z-ordered files.

ON THE EXAM

The first row is the discriminator the exam's own sample question is built on. Max versus the 75th percentile.

Governance

THE PAIR	THE DISCRIMINATOR
SELECT / USE CATALOG and USE SCHEMA	SELECT is not enough. Traversal is a separate privilege.
Ownership / MANAGE	MANAGE grants the ability to grant. It is not all privileges.
ALL PRIVILEGES / actually all	It excludes MANAGE and the EXTERNAL USE privileges.
Row filter / column mask	Returns boolean / returns the column's own type.
Dynamic view / table-level policy	The view protects one object. The policy follows the table.
ABAC / table-level	Catalog or schema level, by tag / one table, by ALTER TABLE.
WHEN / MATCH COLUMNS	WHEN matches securables. MATCH COLUMNS matches columns.

ON THE EXAM

Row one is the most common real access failure in Unity Catalog, and row three is the most quietly misleading name.

warn is the default. union does not deduplicate. dropDuplicates picks arbitrarily. **A few slow tasks is skew; all slow is not.** Column tags do not inherit.

Three shapes of wrong answer

- **The other member of the pair.** Plausible, related, and wrong on one specific detail.
- **An invented limit or rule.** 'Not supported above 10 nodes', 'requires a minimum value'.
- **A true statement that does not answer the question.** Correct, and irrelevant to what was asked.
- **The correct answer often names a mechanism**, rather than describing a symptom or restating the problem.

> ON THE EXAM
>
> You have met all three shapes repeatedly in this course's scenarios. Recognising the shape narrows four options to two.

> A pipeline writes a DataFrame with columns (order_id, customer_ref, region_code) into an existing table whose columns are (order_id, region_code, customer_ref). All three are strings.
>
> The write succeeds. No error is raised. Weeks later, reporting shows region codes appearing as customer references.

Which write method was used, and what should have been used?

A saveAsTable was used; insertInto would have matched by name

B insertInto was used; saveAsTable matches by name and would have aligned the columns

C Either would produce this; the schemas must be reordered at source

D append mode was used; overwrite would have corrected the ordering

◆ PAUSE AND DECIDE

Which write method was used, and what should have been used?

A saveAsTable used; insertInto matches by name

Exactly backwards. saveAsTable matches by name; insertInto matches by position.

B insertInto used; saveAsTable matches by name

Correct. Positional matching put the second column into the second slot, and all three being strings meant no type error surfaced.

C Either would do this

saveAsTable would have aligned the columns by name and written correctly. The methods genuinely differ here.

D append versus overwrite

Write mode governs what happens to existing rows, not how columns are matched.

THE SIGNAL

'All three are strings' is doing the work. A type mismatch would have raised an error and the bug would have been caught immediately.

> An engineer must produce a list of every customer who has never placed an order. She writes a query using EXCEPT between the customers table and the orders table, and it returns every customer.
>
> Both tables are in Unity Catalog and are correctly populated.

Why did EXCEPT return everything?

A. EXCEPT compares whole rows, and the two tables have different schemas, so no row matches

B. EXCEPT requires both tables to be sorted first

C. EXCEPT deduplicates, which removed the matching rows

D. She should have used INTERSECT

◆ PAUSE AND DECIDE

Why did EXCEPT return everything?

A EXCEPT compares whole rows

Correct. A customer row and an order row are never identical, so nothing is excluded. She needs left_anti on customer_id.

B It requires sorting

Invented rule. Set operations in Spark do not depend on ordering.

C Deduplication removed them

EXCEPT does deduplicate, and that affects duplicates within the result, not which rows are excluded.

D Use INTERSECT

INTERSECT returns rows in both, which is the opposite question, and it would return nothing for the same reason.

THE SIGNAL

EXCEPT compares whole rows; left_anti compares join keys. This question needs a key comparison, so it needs the anti join.

> Two jobs are investigated on the same morning. Job A's longest stage: median task 14 seconds, 75th percentile 19 seconds, max 26 minutes. Job B's longest stage: median 11 minutes, 75th percentile 12 minutes, max 12.5 minutes.
>
> Neither reports any spill.

What do they need?

A Both need more shuffle partitions

B A needs skew handling; B needs more capacity or parallelism

C A needs more memory; B needs skew handling

D Both need more executor memory

◆ PAUSE AND DECIDE

What do they need?

A Both need more shuffle partitions

Would help B. It does not split A's single hot key, which is one partition doing nearly all the work.

B A is skew; B is capacity

Correct. A's max is vastly above its 75th percentile. B's is 4% above, which is about as even as a distribution gets.

C A needs memory; B needs skew

Inverts both diagnoses. Neither reports spill, and B's distribution shows no skew at all.

D Both need more memory

Neither reports spill, which is the symptom memory pressure produces. Adding memory addresses a problem neither has.

THE SIGNAL

Same tab, same three numbers, opposite conclusions. Compare Max to the 75th percentile: 26 minutes against 19 seconds, or 12.5 against 12.

A team needs a gold object holding daily totals, computed from a silver table that is rebuilt nightly with a full overwrite. Queries against it must always reflect the most recent nightly build, and roughly 200 analysts query it each morning. The computation takes six minutes.

A colleague suggests a streaming table so it stays continuously up to date.

What should they build?

A. A streaming table, as suggested

B. A materialised view, refreshed after the nightly build

C. A plain view, so it is always current

D. A dynamic view

◆ PAUSE AND DECIDE

What should they build?

A A streaming table

Streaming tables are append-only and process each record once. The source is fully overwritten nightly, which they cannot consume.

B A materialised view refreshed after the build

Correct. Compute once at six minutes; 200 analysts read a stored result. Freshness matches the nightly build exactly.

C A plain view

Always current, and 200 analysts each pay six minutes. The data only changes once a day, so 199 of those are waste.

D A dynamic view

Dynamic views vary output by user identity. Nothing here differs per user.

THE SIGNAL

A full overwrite each night is the detail that eliminates the streaming table. Append-only cannot consume a source that is replaced.

> A job must skip an expensive rebuild on days when its upstream ingestion task loaded no new rows. The ingestion task succeeds on those days, having simply found nothing to load.
>
> An engineer configures the rebuild task with Run if set to 'At least one succeeded'.

Why will this not work?

A. Run if branches on outcome, and the ingestion task succeeds either way; this needs If/else on a task value

B. 'At least one succeeded' should be 'All succeeded'

C. The rebuild task should be disabled on those days

D. Run if conditions cannot be applied to tasks with a single dependency

◆ PAUSE AND DECIDE

Why will this not work?

A Run if sees outcome; this needs If/else on a value

Correct. Loading zero rows is a success, so no Run if condition can distinguish the two days. Only the row count can.

B Use 'All succeeded' instead

Both conditions are satisfied on both kinds of day, because the ingestion task succeeds either way. Swapping them changes nothing.

C Disable the task

Disabling is a manual configuration change, so somebody must check for data and edit the job each morning.

D Run if needs multiple dependencies

Invented restriction. Run if applies regardless of how many dependencies a task has.

THE SIGNAL

Nothing about the outcome differs between the two days. When outcome cannot distinguish them, Run if cannot help.

> A governance team tags a catalog with sensitivity:high, then writes an ABAC column mask policy on that catalog using MATCH COLUMNS has_tag_value('pii','ssn').
>
> Nothing is masked. Separately, a colleague reports that a user with SELECT on a table in that catalog cannot query it at all.

What are the two causes?

A. Both are the same cause: the catalog tag has not propagated

B. The columns were never tagged pii:ssn — column tags do not inherit; and the user is missing USE CATALOG or USE SCHEMA

C. has_tag_value is unsupported; and the user needs ALL PRIVILEGES

D. The policy must be attached to each table; and the user needs MANAGE

◆ PAUSE AND DECIDE

What are the two causes?

A One cause: propagation

Two unrelated problems. And the catalog tag did propagate — it is simply not the tag the policy matches on.

B Column tags do not inherit; and traversal is missing

Correct. MATCH COLUMNS needs column-level tags applied directly, and reaching a table needs USE CATALOG and USE SCHEMA as well as SELECT.

C has_tag_value unsupported; needs ALL PRIVILEGES

has_tag_value is one of only two supported functions. And ALL PRIVILEGES is far more than traversal requires.

D Attach per table; user needs MANAGE

Catalog-level attachment is correct and intended. MANAGE grants the ability to grant, which is not what reading a table needs.

THE SIGNAL

Two Domain 7 pairs in one scenario: tag inheritance stops at columns, and SELECT is not traversal.

When two options both look right, you have found a pair. **Ask which single detail separates them**, and the question usually answers itself.

MODULE 32 · EXAM STRATEGY - ALL DOMAINS

The naming migration

What got renamed, what got replaced, and which old material is merely old-fashioned rather than wrong.

COVERS All 7 domains TYPE Revision MODULE 32 of 36

admin@sthithapragna.com

Three categories, and only one is safe. **Renamed but unchanged. Renamed and changed. Replaced entirely.**

The four platform renames

OLD NAME	CURRENT NAME
Databricks Asset Bundles	Declarative Automation Bundles
Databricks Repos	Databricks Git Folders
Delta Live Tables (DLT)	Lakeflow Spark Declarative Pipelines
Databricks Workflows / Jobs	Lakeflow Jobs

ON THE EXAM

All four are in the safe category. Existing code, YAML and CLI commands run unchanged, and the DAB acronym was deliberately kept.

Why it happened

- **Lakeflow is an umbrella** covering ingestion, transformation and orchestration as one product family.
- **Lakeflow Connect for ingestion, Lakeflow Spark Declarative Pipelines for transformation, Lakeflow Jobs for orchestration.**
- **The old names grew separately**, so Workflows and Delta Live Tables sounded unrelated despite doing adjacent jobs.
- **Knowing the family helps you guess** when you meet a Lakeflow name you have not seen.

> ON THE EXAM
>
> Traces of the old names survive in places: classic SKUs still begin with DLT, and pipeline event log schemas still use dlt.

Code-level renames

OLD	CURRENT
import dlt	from pyspark import pipelines as dp
@dlt.expect	@dp.expect
@view	@temporary_view
APPLY CHANGES INTO	AUTO CDC INTO
hasTag / hasTagValue	has_tag / has_tag_value
Single user access mode	Dedicated access mode
Shared access mode	Standard access mode

ON THE EXAM

Every old form here still works. Recognise both, because the exam may show you either.

One code change that is not just a rename

- **@table now creates streaming tables.**
- **A new @materialized_view decorator** creates materialised views.
- **So the decorator's meaning shifted**, not merely its prefix.
- **Old material showing @dlt.table for a materialised view** is describing different behaviour, not just an old name.

> ON THE EXAM
>
> This is the clearest example of the second category. The name changed and so did what it does.

The exam guide itself uses old and new names, sometimes in the same sentence. **You must recognise both, because the guide does not choose.**

Where the guide uses which

- **Databricks Repos** — the old name, in the Domain 5 development workflow objective.
- **Automation Bundle and Declarative Automation Bundles** — both, each with 'formerly Databricks Asset Bundles'.
- **Lakeflow Jobs** — the new name, used consistently across Domain 4.
- **Lakeflow Spark Declarative Pipelines** — the new name.

> ON THE EXAM
>
> Note the pattern: where the guide expects confusion it supplies 'formerly X' itself. That is a hint about what it thinks is unfamiliar.

Renamed, and nothing else changed

TOPIC	OLD MATERIAL IS
Bundles / Asset Bundles	Still correct. YAML, CLI and the DAB acronym are unchanged.
Git Folders / Repos	Still correct on workflow, branching and pull requests.
Lakeflow Jobs / Workflows	Still correct on tasks, dependencies and the DAG.
Dedicated / single user	Still correct on when you need it: Runtime ML, GPU, RDD, R.
Standard / shared	Still correct on Unity Catalog support and isolation.

ON THE EXAM

For everything in this table, translate the vocabulary and trust the content.

Actually changed — old material is wrong

TOPIC	WHAT CHANGED
Failed casts	ANSI mode is on by default from DBR 17. cast throws instead of returning null.
@table in pipelines	Now creates streaming tables. Materialised views need @materialized_view.
DBFS root and mounts	Deprecated. New accounts are provisioned without access.
Personal access tokens	Labelled legacy. OAuth M2M is the recommendation for CI/CD.
Table maintenance	Predictive optimisation runs OPTIMIZE, VACUUM and ANALYZE automatically.
Partitioning and Z-ORDER	Superseded by liquid clustering, which PO is built around.

ON THE EXAM

This is the short, dangerous list. Old material on these topics is not dated vocabulary — it is incorrect behaviour.

Replaced rather than renamed

- **Hive metastore table access control** still exists, and Unity Catalog is a different model with no DENY.
- **Z-ORDER** still works, and liquid clustering is what predictive optimisation is designed around.
- **Directory listing** still works in Auto Loader, and file events is the recommendation.
- **cloudFiles.useIncrementalListing is deprecated** from Databricks Runtime 17.3 LTS.

> ON THE EXAM
>
> The pattern here is that the old thing keeps working. A question offering it is not automatically wrong — it may just be the worse answer.

What did not change

- **Delta Lake, the transaction log, and time travel.**
- **The medallion architecture**, and everything about bronze, silver and gold.
- **Spark itself:** lazy evaluation, wide and narrow transformations, shuffles, skew and spill.
- **The three-level namespace** and Unity Catalog's core object model.

> ON THE EXAM
>
> Guard against over-correcting. Most of what you learnt from older material is still exactly right.

Quick translation

IF YOU READ	READ IT AS
DLT, Delta Live Tables	Lakeflow Spark Declarative Pipelines
Workflows	Lakeflow Jobs
Repos	Git Folders
Asset Bundles, DABs	Declarative Automation Bundles
Single user / shared access mode	Dedicated / standard
APPLY CHANGES INTO	AUTO CDC INTO
import dlt, @dlt.expect	pyspark.pipelines as dp, @dp.expect

ON THE EXAM

Seven translations. Read them in both directions, because the exam may use either side.

> A candidate is working through a 2024 question bank. One question describes configuring a Delta Live Tables pipeline with a target schema, a pipeline mode, and expectations, and asks which settings are required.
>
> None of the terminology matches the current exam guide.

How should they treat this question?

A Discard it; the terminology is out of date so the content cannot be trusted

B Translate DLT to Lakeflow Spark Declarative Pipelines and treat the content as still valid

C Discard it; pipelines were removed from the current exam

D Treat it as valid without translation, since the exam still uses DLT terminology

◆ PAUSE AND DECIDE

How should they treat this question?

A **Discard it entirely**

Over-correction. The rename was non-breaking, so the pipeline settings it describes are still accurate.

B **Translate the name, trust the content**

Correct. This is the safe category: renamed, nothing else changed. Existing pipeline code runs unchanged.

C **Pipelines were removed from the exam**

They were de-emphasised — the old exam had a whole pipelines section, and the current guide folds expectations into one Domain 3 objective. Not removed.

D **No translation needed**

The current guide uses the new name. Recognising both is required, but the exam will not present it as DLT.

THE SIGNAL

The rename was non-breaking and existing code runs unchanged. That is the definition of the safe category.

> A practice question shows a code sample using @dlt.table to define what it calls a materialized view, and asks what the decorator produces.
>
> The candidate knows about the dlt to dp rename and translates it mentally to @dp.table.

Is the translation sufficient?

A. Yes; @dp.table is simply the new spelling of @dlt.table

B. No; @table now creates streaming tables, and materialized views need @materialized_view

C. No; the decorator was removed entirely

D. Yes, but only on Databricks Runtime 17 and above

◆ PAUSE AND DECIDE

Is the translation sufficient?

A Just a new spelling

The prefix change is a rename; the decorator's meaning also changed, which a spelling translation does not capture.

B @table now creates streaming tables

Correct. This is the second category — renamed and changed. A new @materialized_view decorator exists for materialised views.

C Removed entirely

It exists and is widely used. What changed is what it produces.

D Sufficient on DBR 17+

Invents a version condition. The decorator's behaviour is not gated on runtime version in that way.

THE SIGNAL

He did the translation correctly and it was not enough. A correct rename can still leave you with wrong behaviour.

> A study guide from 2024 states: 'If a value cannot be converted, CAST returns NULL rather than failing, so casting is safe to use on untrusted input.' The candidate has no reason to doubt it.
>
> Nothing in the sentence uses an outdated product name.

What is wrong here?

A Nothing; this is still correct behaviour

B ANSI mode is on by default from DBR 17, so cast throws an error; try_cast returns null in both modes

C CAST was renamed to TRY_CAST

D The statement is only wrong on serverless compute

◆ PAUSE AND DECIDE

What is wrong here?

A **Still correct**

It was correct for years, which is why it is dangerous. From DBR 17 the default changed and cast raises CAST_INVALID_INPUT.

B **ANSI mode changed the default**

Correct. try_cast is the function that returns null in both modes, and is the safe choice on untrusted input.

C **CAST was renamed**

Both exist and always did. try_cast is a separate function, not a new name for cast.

D **Only wrong on serverless**

The default follows the runtime, not the compute type. DBR 17 and above behaves this way generally.

THE SIGNAL

No outdated name appears anywhere in the sentence. That is precisely why this category is the dangerous one.

Translate the vocabulary, then ask the harder question: **has the behaviour changed too?** An old name is a warning. A familiar name is not a guarantee.

admin@sthithapragna.com

MODULE 33 · EXAM STRATEGY - ALL DOMAINS

How the exam is written

The format, the time budget, how scenarios encode their answers, and the guide's own sample question worked through.

COVERS Exam technique TYPE Revision MODULE 33 of 36

admin@sthithapragna.com

The format

	VALUE
Questions	45 scored
Time	90 minutes
Options	Four, A to D
Cost	USD 200
Pass mark	Not published
Validity	Two years
Domains	Seven, weighted

ON THE EXAM

Four options is the current format. A five-option question is from the previous version, which is a useful way to date any practice material.

Where the marks are

DOMAIN	WEIGHT	ROUGHLY
Transform (Domain 3)	22%	10 questions
Ingest (Domain 2)	21%	9 questions
Jobs (Domain 4)	16%	7 questions
Governance (Domain 7)	15%	7 questions
CI/CD (Domain 5)	10%	4–5 questions
Troubleshoot (Domain 6)	10%	4–5 questions
Platform (Domain 1)	6%	3 questions

ON THE EXAM

Domains 2 and 3 together are 43% — nearly nineteen questions. That is where preparation time earns the most.

The time budget

- **Ninety minutes for forty-five questions is two minutes each.**
- **Most questions take well under that.** A definition or a single-fact question takes twenty seconds.
- **Scenario questions are where the time goes**, and they are the ones worth spending it on.
- **Two minutes is an average, not an allowance.** Bank time on the short ones.

> ON THE EXAM
>
> If you are still on question 20 at the halfway mark, that is a signal to speed up rather than a crisis.

A well-written scenario contains **one detail that decides the answer**. Your job is to find it, not to evaluate all four options equally.

Where the deciding detail hides

- **In a number.** 18–24 minute runtime on a 15-minute schedule. 37 minutes against a 28-second percentile.
- **In a negation.** 'has never failed', 'nothing upstream changed', 'no spill is reported'.
- **In a timeframe.** 'three weeks later', 'intermittently', 'has worked for two years'.
- **In a constraint you might skim.** 'must not stop', 'the row count must be correct', 'they must not reach the base tables'.

> ON THE EXAM
>
> Numbers and negations are the two richest sources. A stated negative is almost always there to eliminate an option.

Words that carry weight

PHRASE	WHAT IT USUALLY MEANS
'intermittently'	A race condition or a timing dependency, not a logic error
'has worked for years'	Something outside the code changed, or it was always quietly wrong
'to be safe' / 'for margin'	A guess is being made where a mechanism exists
'must not' / 'must never'	A hard requirement. Any option that risks it is out.
'the count must be correct'	Rules out anything that removes rows
'nobody was alerted'	Notification design, not necessarily a failure
'no error was raised'	A silent failure — look for a same-type or positional mistake

ON THE EXAM

These are not exam tricks. They are how a competent question-writer signals which mechanism is being tested.

Count the requirements

- **Many scenarios state two or three requirements**, spread across the paragraphs.
- **Test each option against every requirement**, not against the one you noticed first.
- **A distractor usually satisfies all but one.** That is what makes it tempting.
- **If an option fails any stated requirement, it is out**, however good it looks otherwise.

> ON THE EXAM
>
> Sensor readings that must not reach gold, a dashboard that must not stop, and a fault that must be counted. One action does all three.

The four shapes of wrong answer

SHAPE	HOW TO SPOT IT
The other member of a pair	Plausible, related, wrong on one specific detail
The pair stated backwards	Names both halves and swaps them
An invented limit or rule	'Not supported above', 'requires a minimum', 'must be re-applied'
True but irrelevant	A correct statement that does not answer what was asked

ON THE EXAM

Module 31 covered the first three. The fourth is the hardest, because checking whether a statement is true is not the same as checking whether it answers the question.

Sometimes more than one option is correct. The question is then asking which is **best**, and best usually means **simplest, cheapest, or most automatic**.

The guide's sample questions

- **The exam guide contains five sample questions**, each labelled with the objective it aligns to.
- **They are retired from a previous version of the exam.** The guide says so explicitly.
- **But the objectives they are labelled with are the current ones**, so Databricks has matched them to the new outline.
- **They are not one per domain.** Two of the five are Domain 6.

> ON THE EXAM
>
> Retired questions matched to current objectives is still the best available evidence of style, difficulty and distractor design.

> A data engineer notices a batch job's duration has doubled after a new data source was onboarded. In the Spark UI, the longest stage shows that most tasks finish in under 30 seconds, but one task takes over 10 minutes.
>
> The stage's task summary shows Min/Median shuffle read near 400 MB while Max shuffle read exceeds 5 GB.

Which solution reduces the job runtime?

A. Increase cluster size to add more executors so the slow task finishes faster

B. Confirm adaptive query execution with skew join handling is active to automatically split the oversized partition at runtime

C. Reduce spark.sql.shuffle.partitions to coalesce more work into fewer tasks

D. Manually repartition the dataset using a salt key before the join to distribute skewed keys evenly

◆ PAUSE AND DECIDE

Which solution reduces the job runtime?

A Add more executors

More idle capacity. One task holds the stage open, so the other executors already have nothing to do.

B Confirm AQE skew join handling is active

Correct. AQE splits the oversized partition at runtime, it is on by default, and confirming it costs nothing.

C Reduce shuffle.partitions

Fewer, larger tasks. It makes an over-large partition worse, and AQE coalesces downwards anyway.

D Salt the key manually

Genuinely fixes skew, and it is code to write, test and maintain. Ask this only after AQE has failed to help.

THE SIGNAL

Two signals, both from module 24: max duration far above the median, and max shuffle read far above the median. Skew, twice over.

What that question tells you

- **It gives the same signal twice**, through duration and through shuffle read per task.
- **Two of the four options are wrong for reasons you can state**, not merely wrong.
- **The fourth option actually works.** The question is asking which is best.
- **It names a specific mechanism**, AQE skew join handling, rather than describing a symptom.

> ON THE EXAM
>
> Four scenarios in this course were built on the same discriminator. That was not a coincidence; it is the shape the exam favours.

When you genuinely do not know

- **Eliminate first.** Invented limits and true-but-irrelevant options can be removed without knowing the answer.
- **Prefer the option naming a mechanism** over the one restating the problem.
- **Be wary of absolutes** in options: 'always', 'never', 'the only way'.
- **Answer everything.** There is no penalty for a wrong answer, and a blank is certainly wrong.

> ON THE EXAM
>
> Eliminating two options and guessing gives you a coin flip on a question you would otherwise have lost entirely.

On the day

- **Read the whole question before the options.** Options are designed to steer you.
- **Flag and move on** rather than stalling. A question you return to often reads differently.
- **Do not change an answer without a reason you can state.** A vague second-guess is usually worse.
- **Check the halfway mark.** Around question 22 at 45 minutes means you are on pace.

> ON THE EXAM
>
> The most common time mistake is not going too slowly. It is stalling on one hard question early and losing composure.

> A daily job must load a supplier file that arrives between 02:00 and 09:00. The data must be available as early as possible. The job must not run when no file has arrived, because empty runs are being billed. And the operations team must be alerted if no file arrives by 10:00.
>
> Four designs are proposed.

Which design meets every stated requirement?

A. A file arrival trigger on the volume

B. A file arrival trigger, plus a separate scheduled job at 10:00 that alerts if the table was not updated

C. A schedule every 30 minutes from 02:00, exiting early when no file is present

D. A schedule at 09:15, with a failure notification configured

◆ PAUSE AND DECIDE

Which design meets every stated requirement?

A A file arrival trigger alone

Meets the first two requirements well and provides no alert. A trigger that never fires produces no run and therefore no notification.

B File arrival trigger plus a 10:00 check job

Correct. Early availability, no empty runs, and an alert on absence — which needs a separate scheduled check, because absence is not an event.

C Poll every 30 minutes

Fourteen runs a day, most of them empty, which fails the billing requirement explicitly.

D Schedule at 09:15

Fails 'as early as possible' on every day the file arrives at 02:00.

THE TECHNIQUE

Option A is the trap, and it is the right answer to the first two requirements. The third one is what it cannot do.

> A nightly job's gold aggregation has slowed from 12 minutes to 40. Investigation shows the silver table it reads has grown to several thousand small files, written by an hourly append job.
>
> The table is a Unity Catalog managed table.

What is the best first action?

A. Schedule a nightly OPTIMIZE job on the silver table

B. Verify predictive optimisation is enabled on the account, since it runs OPTIMIZE on managed tables automatically

C. Rewrite the hourly job to append less frequently

D. Increase the gold job's cluster size

◆ PAUSE AND DECIDE

What is the best first action?

A Schedule a nightly OPTIMIZE

Genuinely fixes it, and Databricks recommends disabling scheduled OPTIMIZE jobs where predictive optimisation runs. You would be building what may already exist.

B Verify predictive optimisation is enabled

Correct. The table is UC managed, PO runs OPTIMIZE automatically, and checking costs nothing before you build anything.

C Append less frequently

Reduces file creation and changes the data's freshness to solve a maintenance problem. A large change for a side effect.

D A bigger cluster

Pays permanently to read a badly laid-out table faster, rather than fixing the layout.

THE TECHNIQUE

'Unity Catalog managed table' is placed deliberately. It is the fact that makes B available and A redundant.

Find the deciding detail. Count the requirements. **When two options both work, choose the one that needs no new machinery.**

MODULE 34 · EXAM STRATEGY - ALL DOMAINS

Practice exam 1

25 questions across all seven domains. Four options each, as on the real paper.

QUESTIONS 25 FORMAT 4 options, A–D MODULE 34 of 36

admin@sthithapragna.com

How to take this paper

- **Each question is read aloud, then the slide holds** while you decide.
- **Pause if you need longer.** The hold is a prompt, not a limit.
- **Answer every question**, even when unsure. There is no penalty for a wrong answer.
- **The worked answer follows immediately**, with a reason for each wrong option.

> ON THE EXAM
>
> The real exam is 45 questions in 90 minutes — two minutes each. Try to keep near that pace.

> A security review asks where notebook source and job configuration are physically held, and where data is actually processed.
>
> The workspace uses serverless compute for all its jobs.

Which statement is correct?

A. Everything including data processing runs in the control plane

B. The control plane holds notebook source and job configuration; serverless compute runs in a Databricks-managed layer in the same region

C. Serverless compute may run in any region, which is what makes it cheaper

D. Notebook source and data processing both sit in the customer's cloud account

◆ PAUSE AND DECIDE

Which statement is correct?

A Everything including data processing runs in the control plane

The control plane does not process data. That is the compute plane, classic or serverless.

B The control plane holds notebook source and job configuration; serverless compute runs in a Databricks-managed layer in the same region

Correct. The control plane holds notebooks, query definitions and job configuration. Serverless compute is Databricks-managed and regionally co-located.

C Serverless compute may run in any region, which is what makes it cheaper

Serverless runs in the same cloud region as the workspace's classic compute plane.

D Notebook source and data processing both sit in the customer's cloud account

Serverless compute runs in a Databricks-managed layer, not the customer's account.

THE SIGNAL

Two planes, two questions. Where the definitions live, and where the processing happens.

A team runs fourteen scheduled production jobs, all of them on an existing all-purpose cluster that they also use interactively during the working day.

Finance queries why compute spend is high relative to the amount of processing being done.

What is the principal cause?

A Scheduled jobs cannot autoscale on all-purpose compute

B Fourteen jobs on one cluster causes contention that raises DBU consumption

C All-purpose clusters cannot use spot instances for workers

D All-purpose compute bills at a higher DBU rate and does not terminate when a job ends

◆ PAUSE AND DECIDE

What is the principal cause?

A Scheduled jobs cannot autoscale on all-purpose compute

Autoscaling is available. The cost difference is the billing rate and the termination behaviour.

B Fourteen jobs on one cluster causes contention that raises DBU consumption

Contention affects runtime, not the DBU rate. Each job would still be billed at the higher rate.

C All-purpose clusters cannot use spot instances for workers

Spot is available for workers on all-purpose compute, so this is not the cause.

D All-purpose compute bills at a higher DBU rate and does not terminate when a job ends

Correct. The rate difference is roughly two to three times, and the cluster terminates on inactivity rather than on job completion.

THE SIGNAL

Databricks documents four reasons against running jobs on all-purpose compute. Rate and termination are the two that show up on a bill.

A supplier re-sends a corrected file under exactly the same name as one delivered yesterday, overwriting it in the landing location. The ingestion job uses COPY INTO.

The following morning the corrections have not appeared in the bronze table, and the job reported success.

Why?

A The file arrived after the job started, so it will be picked up on the next run

B COPY INTO tracks files by identity, not by content or timestamp, so the file was skipped as already loaded

C COPY INTO only loads files matching the PATTERN clause, and the name changed

D COPY INTO requires FORMAT_OPTIONS('mergeSchema'='true') to reload a changed file

◆ PAUSE AND DECIDE

Why?

A The file arrived after the job started, so it will be picked up on the next run

It would then be loaded the following night. The scenario states the corrections never appeared.

B COPY INTO tracks files by identity, not by content or timestamp, so the file was skipped as already loaded

Correct. Idempotency is by file identity. A modified file with the same name is still considered loaded.

C COPY INTO only loads files matching the PATTERN clause, and the name changed

The name is stated as identical, and no pattern change is mentioned.

D COPY INTO requires FORMAT_OPTIONS('mergeSchema'='true') to reload a changed file

mergeSchema reconciles differing schemas. It has nothing to do with whether a file is reloaded.

THE SIGNAL

'Exactly the same name' is the deciding phrase. COPY_OPTIONS('force'='true') is what reloads it.

> An Auto Loader stream ingests JSON from a partner who adds new optional fields without notice, several times a month. The stream feeds an operational dashboard that must not go down.
>
> The team wants any unexpected fields preserved for later inspection rather than lost, and they do not want to be woken up.

Which schema evolution mode fits?

A failOnNewColumns

B none

C rescue

D addNewColumns

◆ PAUSE AND DECIDE

Which schema evolution mode fits?

A failOnNewColumns

Fails and will not restart until the schema is updated or the file removed. The worst option for uptime.

B none

Ignores new columns entirely, so the data is neither surfaced nor preserved.

C rescue

Correct. The schema never evolves and the stream never fails; unexpected fields land in _rescued_data for inspection.

D addNewColumns

The default when no schema is provided. It fails the stream on a new column and resumes on restart, so the dashboard goes down each time.

THE SIGNAL

Two requirements: never fail, and never lose the data. Only rescue satisfies both.

A team must ingest from Salesforce into bronze. They have no unusual transformation requirements, and they want the least operational burden.

An engineer proposes writing a Structured Streaming job against the Salesforce REST API, because it gives full control.

What should they do instead?

A. Write the Structured Streaming job as proposed, since it offers the most control

B. Use Lakehouse Federation to query Salesforce in place

C. Use Auto Loader against the Salesforce API

D. Use the managed Salesforce connector in Lakeflow Connect

◆ PAUSE AND DECIDE

What should they do instead?

A Write the Structured Streaming job as proposed, since it offers the most control

Most control and most maintenance: auth, CDC, edge cases, API changes and retries all become yours.

B Use Lakehouse Federation to query Salesforce in place

Federation queries without copying. The requirement is to ingest into bronze.

C Use Auto Loader against the Salesforce API

Auto Loader reads files in cloud object storage. An API is the wrong source type for it.

D Use the managed Salesforce connector in Lakeflow Connect

Correct. Start at the most managed layer and drop down only when a requirement forces it. Salesforce has a managed connector.

THE SIGNAL

Three layers, most managed to least. The rule is to start at the top and drop down only when forced.

A JDBC read from an Oracle database is configured with partitionColumn set to order_id, lowerBound 1000, upperBound 5000 and numPartitions 8. The source table holds order_ids from 1 to 900,000.

An engineer expects roughly 4,000 rows to be returned and is surprised when all 900,000 arrive.

Why?

A lowerBound and upperBound set the partitioning stride only; they do not filter rows

B The bounds are ignored unless fetchSize is also set

C Oracle does not support partitioned JDBC reads, so the bounds were discarded

D numPartitions must equal 1 for bounds to be applied

◆ PAUSE AND DECIDE

Why?

A lowerBound and upperBound set the partitioning stride only; they do not filter rows

Correct. They determine how Spark splits the range across partitions. Every row is still returned.

B The bounds are ignored unless fetchSize is also set

fetchSize controls how many rows are retrieved per round trip. It does not filter.

C Oracle does not support partitioned JDBC reads, so the bounds were discarded

Partitioned reads work against Oracle. The behaviour described is the documented one.

D numPartitions must equal 1 for bounds to be applied

Invented rule. numPartitions controls parallelism, not filtering.

THE SIGNAL

A very common misreading. To filter, use a WHERE clause in the query or dbtable option.

A one-off migration must load 1,400 Parquet files from an archive location into a new bronze table. The schema is fixed and well known, the files will never change, and the team works mostly in SQL.

An engineer proposes configuring an Auto Loader stream with a checkpoint and a schema location.

What is the better choice?

A Auto Loader as proposed, because it is always the recommended ingestion method

B A Lakeflow Connect managed connector

C Lakehouse Federation over the archive location

D COPY INTO, which suits thousands of files with a known schema and needs no checkpoint or schema location

◆ PAUSE AND DECIDE

What is the better choice?

A Auto Loader as proposed, because it is always the recommended ingestion method

Auto Loader suits continuous arrival at scale. Here it adds a checkpoint and schema location to maintain for a load that runs once.

B A Lakeflow Connect managed connector

Managed connectors cover specific SaaS and database sources. This is a file load from object storage.

C Lakehouse Federation over the archive location

Federation queries external databases in place. It is not a file ingestion mechanism.

D COPY INTO, which suits thousands of files with a known schema and needs no checkpoint or schema location

Correct. This is a bounded, one-off load. COPY INTO is idempotent by default and adds no ongoing state.

THE SIGNAL

Thousands of files, fixed schema, runs once. Auto Loader's machinery is for data that keeps arriving.

> A silver build has run nightly for three years. It reads a text column called quantity and casts it to INT. Last week the platform team upgraded the job cluster from Databricks Runtime 15.4 LTS to 17.3 LTS.
>
> The job now fails with CAST_INVALID_INPUT. The upstream data has not changed.

What is the correct response?

A. Recognise that those rows were previously becoming nulls silently, and use try_cast where null is the correct meaning

B. Pin the job to Databricks Runtime 15.4 LTS permanently

C. Set spark.sql.ansi.enabled to false permanently, restoring the previous behaviour

D. Change the target column to DOUBLE, which accepts a wider range of values

◆ PAUSE AND DECIDE

What is the correct response?

A Recognise that those rows were previously becoming nulls silently, and use try_cast where null is the correct meaning

Correct. ANSI mode is on by default from DBR 17. The errors are new information about data that was always bad.

B Pin the job to Databricks Runtime 15.4 LTS permanently

Freezes the platform to preserve a bug, forfeiting later improvements and security fixes.

C Set spark.sql.ansi.enabled to false permanently, restoring the previous behaviour

Restores the silent corruption that was happening for three years. Defensible as a temporary unblock, wrong as the fix.

D Change the target column to DOUBLE, which accepts a wider range of values

CAST_INVALID_INPUT means the text is not numeric at all. A wider numeric type parses it no better.

THE SIGNAL

'The upstream data has not changed' is the tell. The data was always like this; the default changed.

A silver orders table with 4.1 million rows is joined to a product_variants table on product_id using an inner join, to attach a variant description.

The result has 6.8 million rows, and total order value calculated from it is around 66% higher than from orders alone.

What is the cause?

A The join should have been a left join to preserve the correct count

B Some products have several variants, so those orders are duplicated by the join

C Order value is being summed from both tables

D Null product_ids matched multiple variant rows

◆ PAUSE AND DECIDE

What is the cause?

A The join should have been a left join to preserve the correct count

A left join keeps unmatched rows, which would produce even more rows. This is growth, not shrinkage.

B Some products have several variants, so those orders are duplicated by the join

Correct. Fan-out. An inner join discards unmatched rows, so growth can only come from duplication on one side.

C Order value is being summed from both tables

A variants table would not carry order value, and a duplicated column raises an ambiguous reference rather than inflating a sum.

D Null product_ids matched multiple variant rows

Backwards. Null keys never match anything, so they would reduce the row count.

THE SIGNAL

An inner join that grows the row count is the signature of fan-out. Inner joins normally shrink data.

A silver customer table is built by deduplicating bronze on customer_id using dropDuplicates. Bronze holds several versions of some customers, because corrections arrive as new rows.

Analysts report that a customer's marketing consent flag sometimes flips between runs, with no upstream change.

What is happening?

A distinct should be used instead, as it is deterministic

B dropDuplicates keeps the first row encountered, so bronze must be sorted first

C dropDuplicates on a subset keeps an arbitrary row per key, so a different version wins on different runs

D The consent flag is being rescued into _rescued_data on some runs

◆ PAUSE AND DECIDE

What is happening?

A distinct should be used instead, as it is deterministic

distinct compares whole rows, so differing versions of a customer are all kept. It does not deduplicate on the key.

B dropDuplicates keeps the first row encountered, so bronze must be sorted first

There is no first-row guarantee, and sorting beforehand does not create one.

C dropDuplicates on a subset keeps an arbitrary row per key, so a different version wins on different runs

Correct. There is no guarantee of first, newest or most complete. Use row_number over a window ordered by the update timestamp.

D The consent flag is being rescued into _rescued_data on some runs

Rescued data applies to schema evolution in Auto Loader, not to deduplication.

THE SIGNAL

Same input, different output, is non-determinism. Something is choosing arbitrarily.

A materialised view aggregates a large managed silver table and refreshes hourly. Only about 0.2% of source rows change each hour, but each refresh consumes roughly the compute of a full rebuild.

Task durations across the refresh are even, and no errors are reported.

What is the most likely cause?

A Materialised views always fully recompute

B Hourly is too frequent; the refresh should be daily

C The refresh is being run with the FULL keyword by the scheduler

D Row tracking is not enabled on the source table, so incremental refresh is not possible

◆ PAUSE AND DECIDE

What is the most likely cause?

A Materialised views always fully recompute

Incremental refresh is a documented materialised view feature. It simply has requirements.

B Hourly is too frequent; the refresh should be daily

Changes how often you pay, not what each refresh costs, and it makes the data staler.

C The refresh is being run with the FULL keyword by the scheduler

Possible, and a SCHEDULE clause does not add FULL by default. Someone would have had to write it deliberately.

D Row tracking is not enabled on the source table, so incremental refresh is not possible

Correct. Incremental refresh from Delta tables requires row tracking. Without it Spark falls back to a full recompute, silently.

THE SIGNAL

The fallback is silent. An MV that cannot refresh incrementally does not fail — it recomputes and sends the bill.

> A pipeline defines an expectation on a streaming table: CONSTRAINT positive_qty EXPECT (quantity > 0). It was deployed a month ago and the pipeline has run cleanly since.
>
> A downstream analyst reports negative quantities in the silver table. The pipeline's Data quality tab shows several hundred failing records per day.

What is happening?

A With no ON VIOLATION clause the action is warn, so invalid records are written to the target and only counted

B Expectations only apply to records that arrive after deployment

C The expectation syntax is invalid and is being ignored

D ON VIOLATION DROP ROW is the default and is failing to drop

◆ PAUSE AND DECIDE

What is happening?

A With no ON VIOLATION clause the action is warn, so invalid records are written to the target and only counted

Correct. Retaining invalid records is the documented default. The expectation is measuring, not enforcing.

B Expectations only apply to records that arrive after deployment

Would not explain several hundred currently failing records being written each day.

C The expectation syntax is invalid and is being ignored

The Data quality tab is reporting failing counts, which means it is being evaluated correctly.

D ON VIOLATION DROP ROW is the default and is failing to drop

Drop is not the default. Warn is, and warn writes the record.

THE SIGNAL

The failing count in the UI is evidence the expectation works. Warn means observe, not enforce.

> A job runs on a cluster with 320 total cores. Its longest stage shows exactly 200 tasks, and most executors sit idle throughout it. Task durations within that stage are even.
>
> An engineer sets spark.default.parallelism to 640 and re-runs. Nothing changes.

Why, and what should be changed?

A AQE overrode the setting and coalesced back to 200

B The value must be a power of two, so 640 was rejected

C spark.default.parallelism applies to RDD operations only; set spark.sql.shuffle.partitions instead

D The cluster needs more workers before extra parallelism can be used

◆ PAUSE AND DECIDE

Why, and what should be changed?

A AQE overrode the setting and coalesced back to 200

AQE coalesces downwards from the configured value. It did not raise anything here, and 200 is the untouched default.

B The value must be a power of two, so 640 was rejected

Invented rule. Spark accepts arbitrary values.

C spark.default.parallelism applies to RDD operations only; set spark.sql.shuffle.partitions instead

Correct. A DataFrame or SQL shuffle takes its partition count from spark.sql.shuffle.partitions. Exactly 200 tasks is that default showing through.

D The cluster needs more workers before extra parallelism can be used

320 cores are already available and idle. Capacity is not the constraint.

THE SIGNAL

Exactly 200 is a default showing through. A round, unchanging number in the Spark UI rarely means coincidence.

A job has four tasks. Task A truncates and reloads a staging table. Tasks B and C read that staging table and write two different silver tables. Task D reads both silver tables.

B declares a dependency on A. C declares none. D depends on B and C. Roughly once a fortnight, one silver table is short of rows.

What is wrong?

A C has no dependency on A, so it may read the staging table while A is still loading it

B D should depend only on B, since B already depends on A

C Task C needs a retry policy to handle the intermittent shortfall

D The staging table should be a view rather than a table

◆ PAUSE AND DECIDE

What is wrong?

A C has no dependency on A, so it may read the staging table while A is still loading it

Correct. Undeclared means parallel. C starts alongside A and sometimes wins the race.

B D should depend only on B, since B already depends on A

Would let D run before C finishes, making things worse and more often.

C Task C needs a retry policy to handle the intermittent shortfall

C is not failing. It succeeds with incomplete input, which retries cannot detect.

D The staging table should be a view rather than a table

Does not address the ordering. A view over data still being written has the same problem.

THE SIGNAL

'Roughly once a fortnight' is the tell. A missing dependency fails on timing, not consistently.

A job creates a temporary compute-heavy staging dataset, runs four analyses against it, then deletes it. The delete task depends on all four analyses.

When any analysis fails, the staging dataset is left behind. When somebody cancels the run, it is also left behind.

Which Run if condition should the delete task use?

A All succeeded

B None failed

C All done

D At least one failed

◆ PAUSE AND DECIDE

Which Run if condition should the delete task use?

A All succeeded

The current behaviour and the cause of the problem. When an analysis fails, the condition is unmet and the delete never runs.

B None failed

Still requires that nothing failed, so it behaves like the current setting in exactly the case that matters.

C All done

Correct. It runs once every dependency has finished, whatever the outcome, and cancellation is covered too.

D At least one failed

Inverts the problem: the dataset would be deleted only when something failed, and left behind on every successful run.

THE SIGNAL

Cleanup should not depend on the outcome of the work it is cleaning up after. All done also covers cancellation.

> An ingestion job writes a silver table and normally finishes by 03:40. A downstream aggregation job is scheduled at 04:15. Once or twice a month the ingestion overruns and the aggregation reads yesterday's data, producing wrong figures with no error.
>
> Both tables are Unity Catalog managed tables in the same catalog.

What is the best fix?

A Move the aggregation schedule to 05:30 for more margin

B Use a table update trigger on the silver table so the aggregation starts when it is written

C Add a retry policy to the aggregation job

D Set maximum concurrent runs on the aggregation job to 1

◆ PAUSE AND DECIDE

What is the best fix?

A Move the aggregation schedule to 05:30 for more margin

A bigger guess is still a guess. It delays the data every day to protect against a monthly event, and will eventually overrun again.

B Use a table update trigger on the silver table so the aggregation starts when it is written

Correct. The dependency becomes explicit rather than being encoded as two cron times 35 minutes apart.

C Add a retry policy to the aggregation job

The aggregation is not failing. It succeeds and produces wrong figures, which retries cannot detect.

D Set maximum concurrent runs on the aggregation job to 1

Governs overlapping runs of the same job. It says nothing about whether upstream data is ready.

THE SIGNAL

A dependency encoded as two cron expressions is a dependency expressed as a hope.

A job takes a parameter called environment, defaulting to dev. The team relies on that default so that only their deployment pipeline, which passes prod explicitly, can write to production.

An analyst who needs to re-run failed jobs has been granted CAN MANAGE RUN.

What risk does this create?

A None; the analyst cannot change parameters with CAN MANAGE RUN

B The analyst becomes the run-as identity, gaining the job's data access

C The analyst could change the job's schedule and cause unexpected runs

D The analyst can override the parameter on a manual run; a default parameter value is not a security control

◆ PAUSE AND DECIDE

What risk does this create?

A None; the analyst cannot change parameters with CAN MANAGE RUN

CAN MANAGE RUN or higher can override parameter values for a manual run, which is documented behaviour.

B The analyst becomes the run-as identity, gaining the job's data access

The run-as identity is a separate setting and is not changed by who triggers a run.

C The analyst could change the job's schedule and cause unexpected runs

Changing configuration requires CAN MANAGE. CAN MANAGE RUN allows triggering, stopping and restarting.

D The analyst can override the parameter on a manual run; a default parameter value is not a security control

Correct. The only protection is restricting who holds CAN MANAGE RUN on the job.

THE SIGNAL

The team built a security control out of a convenience feature. Defaults exist so a job runs sensibly, not to stop anyone changing them.

A team keeps all pipeline notebooks in a Git folder. They promote to production by pointing a production-workspace Git folder at main, and syncing after each merge.

They now need production jobs to use a different catalog, larger clusters and a nightly schedule that development does not have.

What should they move to?

A Environment-detection logic inside each notebook

B A separate Git repository per environment

C A separate branch per environment, each with its own hard-coded settings

D Declarative Automation Bundles, which package jobs and configuration with the code and support per-target variables

◆ PAUSE AND DECIDE

What should they move to?

A Environment-detection logic inside each notebook

Scatters configuration across notebooks, and cannot set cluster size or a job schedule, which live outside the notebook.

B A separate Git repository per environment

The same divergence problem with more ceremony. The code is identical; only the configuration differs.

C A separate branch per environment, each with its own hard-coded settings

Guarantees divergence. Branches never merged drift apart, and every fix must be cherry-picked into each.

D Declarative Automation Bundles, which package jobs and configuration with the code and support per-target variables

Correct. Bundles version the project rather than only the code, with targets and overrides for each environment.

THE SIGNAL

Git folders are a development tool, not a deployment mechanism. They version code and nothing around it.

A release engineer runs databricks bundle deploy -t prod from a hotfix branch. The deployment fails, reporting that the current Git branch does not match the branch specified in the target.

A colleague suggests adding --force to the pipeline's production deploy step so this can never block a release again.

What should the team do?

A Remove the git property from the production target

B Merge the hotfix to main and deploy from there, keeping the branch validation

C Add --force permanently, since urgent fixes should never be blocked

D Point the production target's branch at the hotfix branch

◆ PAUSE AND DECIDE

What should the team do?

A Remove the git property from the production target

Removes the check silently rather than overriding it, which is worse because nobody notices afterwards.

B Merge the hotfix to main and deploy from there, keeping the branch validation

Correct. The check exists to stop production being deployed from a branch that has not been reviewed or merged.

C Add --force permanently, since urgent fixes should never be blocked

Turns a safety check into decoration. It still runs, still reports, and can no longer stop anything.

D Point the production target's branch at the hotfix branch

Production would then track a temporary branch, whose deletion leaves production's declared source non-existent.

THE SIGNAL

Urgency is the argument every safety check eventually faces. The question is whether the check was wrong, and here it was right.

> A GitHub Actions workflow deploys a bundle to production. It authenticates using a personal access token generated from the lead engineer's account and stored as a repository secret.
>
> The team asks whether this is the right long-term arrangement.

What should they change it to?

A Store the token in a Databricks secret scope instead of a repository secret

B OAuth U2M, so the workflow logs in interactively at deploy time

C OAuth M2M with a dedicated service principal, supplied to the workflow as environment variables

D Nothing; a PAT in a repository secret is the current recommendation

◆ PAUSE AND DECIDE

What should they change it to?

A Store the token in a Databricks secret scope instead of a repository secret

The runner needs credentials before it can reach Databricks, so the scope cannot be read yet. It also keeps the person-owned token.

B OAuth U2M, so the workflow logs in interactively at deploy time

U2M requires a browser sign-in. There is nobody in a CI runner to complete it.

C OAuth M2M with a dedicated service principal, supplied to the workflow as environment variables

Correct. No interactive sign-in, short-lived tokens, and production deployment belongs to a service principal rather than a person.

D Nothing; a PAT in a repository secret is the current recommendation

It works, and it ties production deployment to one person's account and uses the method Databricks now labels legacy.

THE SIGNAL

Two problems, not one: the method is now labelled legacy, and the identity is an individual.

> A job's longest stage takes 52 minutes across 400 tasks. Summary Metrics show a median task duration of 31 seconds, a 75th percentile of 38 seconds, and a maximum of 47 minutes.
>
> No spill statistics appear on the stage page.

What is the bottleneck, and what should be tried first?

A Insufficient memory; move to a memory-optimised node type

B Data skew; confirm AQE skew join handling is active before writing any salting logic

C Under-parallelism; raise spark.sql.shuffle.partitions

D The cluster is undersized; add executors so the slow task finishes faster

◆ PAUSE AND DECIDE

What is the bottleneck, and what should be tried first?

A Insufficient memory; move to a memory-optimised node type

The scenario states no spill, which is the symptom memory pressure produces.

B Data skew; confirm AQE skew join handling is active before writing any salting logic

Correct. 47 minutes against a 38-second 75th percentile is vastly past the 50% threshold. AQE splits skewed partitions at runtime and is on by default.

C Under-parallelism; raise spark.sql.shuffle.partitions

400 tasks with 399 finishing in seconds is not an under-parallelism problem, and more partitions do not split one hot key.

D The cluster is undersized; add executors so the slow task finishes faster

Executors are idle, not struggling. One task holds the stage open while the rest have nothing to do.

THE SIGNAL

Databricks' own threshold: if Max duration is 50% more than the 75th percentile, suspect skew. Here it is roughly 74 times.

A nightly job fails with an out-of-memory error. The team doubles worker memory twice, at considerable cost, and it still fails. Reading the stack trace properly, they find it names the driver.

The job ends by collecting the result to build a formatted report.

What should they do?

A The failure is on the driver; aggregate to report size before collecting, or write the file without collecting

B Continue increasing worker memory until it succeeds

C Enable AQE to handle the skew causing the failure

D Raise spark.sql.shuffle.partitions to reduce partition size

◆ PAUSE AND DECIDE

What should they do?

A The failure is on the driver; aggregate to report size before collecting, or write the file without collecting

Correct. collect and toPandas pull the whole result to a single machine. The driver does not process partitions.

B Continue increasing worker memory until it succeeds

Two doublings have already failed, which is the evidence that workers are not the constraint.

C Enable AQE to handle the skew causing the failure

AQE optimises distributed execution. It has no effect on a single-machine collection to the driver.

D Raise spark.sql.shuffle.partitions to reduce partition size

Shuffle partitions govern executor-side partition size. The driver does not process partitions.

THE SIGNAL

Two failed doublings were already the answer. If more executor memory does not help, the executors were never the problem.

> A team drops nine tables during a schema clean-up. Two turn out to be needed. One is restored in minutes by running CREATE TABLE against its original storage path. The other cannot be restored that way.
>
> Both were created by the same engineer, in the same schema, on the same day.

What is the difference between them?

A The restorable one was external, so DROP removed only its metadata; the other was managed and its files were deleted

B DROP behaves differently depending on how recently a table was queried

C The restorable one had been backed up and the other had not

D The unrestorable one had predictive optimisation enabled, which vacuumed its files

◆ PAUSE AND DECIDE

What is the difference between them?

A The restorable one was external, so DROP removed only its metadata; the other was managed and its files were deleted

Correct. Pointing CREATE TABLE at the original path only works if the files are still there, which is external behaviour.

B DROP behaves differently depending on how recently a table was queried

Invented behaviour. DROP does not consider query history.

C The restorable one had been backed up and the other had not

The described recovery was recreating a table over an existing path, not restoring from a backup.

D The unrestorable one had predictive optimisation enabled, which vacuumed its files

VACUUM removes files not referenced by any table version, on an existing table. DROP deleted these.

THE SIGNAL

The recovery method is the evidence. Pointing CREATE TABLE at a path only works if the files survived the DROP.

A new analyst is added to the reporting group, which holds SELECT on twelve tables in the finance catalog. SHOW GRANTS confirms the grants and names her group correctly.

Every query she runs fails with a permissions error. Colleagues in the same group query the same tables without difficulty.

What is the most likely explanation?

A She lacks USE CATALOG on finance or USE SCHEMA on the schema, which her colleagues hold via another group

B She needs ALL PRIVILEGES on the tables rather than SELECT

C Grants take up to 24 hours to propagate to new group members

D A DENY on the catalog is overriding her SELECT

◆ PAUSE AND DECIDE

What is the most likely explanation?

A She lacks USE CATALOG on finance or USE SCHEMA on the schema, which her colleagues hold via another group

Correct. Reaching a table requires traversal privileges as well as SELECT. Same group, same grant, different outcome means something held elsewhere.

B She needs ALL PRIVILEGES on the tables rather than SELECT

SELECT is the correct privilege for reading. ALL PRIVILEGES is far broader and still would not supply traversal on its own.

C Grants take up to 24 hours to propagate to new group members

Invented delay, and it would not explain colleagues succeeding right now.

D A DENY on the catalog is overriding her SELECT

Unity Catalog does not support DENY. Its model is least privilege: ungranted is implicitly denied.

THE SIGNAL

Same group, same grant, different outcome. The difference must come from a privilege held outside that group.

> A governance team tags every table in a catalog with pii:true, then creates an ABAC column mask policy on that catalog using MATCH COLUMNS has_tag_value('pii','true').
>
> The policy is created without error and SHOW POLICIES confirms it. Nothing is masked for anyone.

Why?

A. A WHEN clause is required for a column mask policy to take effect

B. The policy must be attached to each table individually

C. The tags were applied to tables; column tags do not inherit from the parent table and must be applied directly

D. has_tag_value is not supported inside MATCH COLUMNS

◆ PAUSE AND DECIDE

Why?

A A WHEN clause is required for a column mask policy to take effect

WHEN is optional. It narrows which securables match; omitting it considers all of them.

B The policy must be attached to each table individually

Catalog-level attachment is normal and intended. Scope is not the problem.

C The tags were applied to tables; column tags do not inherit from the parent table and must be applied directly

Correct. Tag inheritance can be overridden at every level except columns. MATCH COLUMNS finds no column tags, so nothing matches.

D has_tag_value is not supported inside MATCH COLUMNS

has_tag and has_tag_value are the only two functions supported in policy conditions, and both work there.

THE SIGNAL

The policy is correct and matches nothing. MATCH COLUMNS looks at column tags, and only tables were tagged.

What this paper covered

DOMAIN	QUESTIONS
Domain 1 — Platform	2
Domain 2 — Ingest	5
Domain 3 — Transform	6
Domain 4 — Jobs	4
Domain 5 — CI/CD	3
Domain 6 — Troubleshoot	2
Domain 7 — Govern	3
Total	25

ON THE EXAM

Weighted across all three papers rather than within each one, so a single paper will not match the exam's weights exactly.

MODULE 35 · EXAM STRATEGY - ALL DOMAINS

Practice exam 2

25 further questions across all seven domains. No topic repeated from paper 1.

QUESTIONS 25 FORMAT 4 options, A–D MODULE 35 of 36

admin@sthithapragna.com

How to take this paper

- **Each question is read aloud, then the slide holds** while you decide.
- **Pause if you need longer.** The hold is a prompt, not a limit.
- **Answer every question**, even when unsure. There is no penalty for a wrong answer.
- **The worked answer follows immediately**, with a reason for each wrong option.

> ON THE EXAM
>
> The real exam is 45 questions in 90 minutes — two minutes each. Try to keep near that pace.

A data science team needs a cluster running Databricks Runtime for Machine Learning, with GPU instances, to train a model. They also want Unity Catalog governance on the data they read.

A platform engineer configures the cluster with standard access mode and the team reports that it will not start with the ML runtime.

What is required?

A Standard access mode with the ML libraries installed manually

B Dedicated access mode, which is required for Runtime ML, GPU workloads, the RDD API and R

C A serverless SQL warehouse, which supports GPU workloads

D No isolation shared access mode, which supports all runtimes

◆ PAUSE AND DECIDE

What is required?

A Standard access mode with the ML libraries installed manually

The requirement is the ML runtime itself, not a set of libraries. Access mode is what blocks it.

B Dedicated access mode, which is required for Runtime ML, GPU workloads, the RDD API and R

Correct. Dedicated was formerly called single user, and it is the mode those workloads require.

C A serverless SQL warehouse, which supports GPU workloads

SQL warehouses run SQL. They do not run training workloads or GPU instances.

D No isolation shared access mode, which supports all runtimes

It bypasses Unity Catalog entirely, so the governance requirement is lost. It is also legacy.

THE SIGNAL

Four workloads need dedicated: Runtime ML, GPU, the RDD API and R. Dedicated still supports Unity Catalog.

> A COPY INTO job loads CSV files from a supplier. Some files carry an extra column that others do not. The load fails on schema mismatch between the source files themselves.
>
> An engineer adds mergeSchema to COPY_OPTIONS. The same failure occurs.

What is wrong?

A mergeSchema is not supported by COPY INTO at all

B COPY INTO cannot load CSV files with differing schemas under any configuration

C The FILES clause must be used instead of PATTERN when schemas differ

D mergeSchema in COPY_OPTIONS evolves the target table; reconciling differing source files needs it in FORMAT_OPTIONS

◆ PAUSE AND DECIDE

What is wrong?

A mergeSchema is not supported by COPY INTO at all

It is supported, in both option blocks. Which block you put it in determines what it reconciles.

B COPY INTO cannot load CSV files with differing schemas under any configuration

It can, with FORMAT_OPTIONS mergeSchema, which is exactly the fix.

C The FILES clause must be used instead of PATTERN when schemas differ

FILES and PATTERN control which files are selected, not how their schemas are reconciled.

D mergeSchema in COPY_OPTIONS evolves the target table; reconciling differing source files needs it in FORMAT_OPTIONS

Correct. The same option name appears in both blocks and does two different jobs.

THE SIGNAL

One option name, two blocks, two jobs. FORMAT_OPTIONS reconciles source files; COPY_OPTIONS evolves the target.

A team runs two Auto Loader streams, one over an orders directory and one over a payments directory. To simplify configuration they point both at the same checkpoint location.

Both streams run without error. Weeks later, reconciliation shows that a substantial number of files from both sources were never processed.

What happened?

A Auto Loader cannot read two directories in the same workspace concurrently

B Each source location requires its own checkpoint; sharing one causes silent data loss

C The schema location must also be shared for the checkpoint to work

D The streams need cloudFiles.includeExistingFiles set to true

◆ PAUSE AND DECIDE

What happened?

A Auto Loader cannot read two directories in the same workspace concurrently

It can. Many concurrent streams are normal; each simply needs its own checkpoint.

B Each source location requires its own checkpoint; sharing one causes silent data loss

Correct. The checkpoint records processing progress, so two streams sharing it overwrite each other's position.

C The schema location must also be shared for the checkpoint to work

Sharing more state makes the problem worse. Both must be per stream.

D The streams need cloudFiles.includeExistingFiles set to true

That option controls whether pre-existing files are processed on first start. It does not fix a shared checkpoint.

THE SIGNAL

No error is raised, which is what makes it dangerous. The checkpoint is progress state, and it is per stream.

> A regulated ingestion stream must never write a record whose schema it does not fully understand. If an unexpected column appears, processing must stop and stay stopped until an engineer has reviewed and approved the change.
>
> The team is choosing an Auto Loader schema evolution mode.

Which mode fits?

A rescue, which captures unexpected columns without failing

B none, which ignores new columns entirely

C failOnNewColumns, which fails and will not restart until the schema is updated or the file removed

D addNewColumns, which fails the stream and then resumes on restart

◆ PAUSE AND DECIDE

Which mode fits?

A rescue, which captures unexpected columns without failing

The stream never stops, so records with unreviewed schemas are written. Wrong for this requirement.

B none, which ignores new columns entirely

Processing continues and the new column is silently discarded. The opposite of what is required.

C failOnNewColumns, which fails and will not restart until the schema is updated or the file removed

Correct. It is the only mode that stops and stays stopped, which is what the review requirement demands.

D addNewColumns, which fails the stream and then resumes on restart

It resumes automatically on restart, so processing continues without anybody approving the change.

THE SIGNAL

'Stop and stay stopped' is the deciding phrase. addNewColumns stops and then starts again by itself.

> A team lands deeply nested JSON in a VARIANT column, which works well. They then try to improve query performance by adding the VARIANT column as a clustering key on the table.
>
> The statement is rejected.

Why, and what should they do?

A VARIANT requires liquid clustering to be disabled on the table

B Clustering keys must be declared at table creation and cannot be added later

C The VARIANT column must be cast to STRING before clustering

D VARIANT columns cannot be clustering keys, partition columns or Z-order keys; extract frequently queried fields into typed columns

◆ PAUSE AND DECIDE

Why, and what should they do?

A VARIANT requires liquid clustering to be disabled on the table

Liquid clustering works on the table. The restriction is on VARIANT columns being used as keys.

B Clustering keys must be declared at table creation and cannot be added later

Liquid clustering keys can be changed without rewriting the table. That is one of its main advantages.

C The VARIANT column must be cast to STRING before clustering

That discards the structure VARIANT exists to preserve, and reintroduces JSON string parsing.

D VARIANT columns cannot be clustering keys, partition columns or Z-order keys; extract frequently queried fields into typed columns

Correct. The documented pattern is to extract hot fields alongside the VARIANT, keeping both.

THE SIGNAL

VARIANT also cannot be used for comparisons, grouping, ordering or set operations. Extract what you query often.

> A silver build explodes an order_lines array from a bronze orders table, producing one row per line item. Reconciliation shows the line item count is correct, but 3% of orders have disappeared entirely.
>
> Those orders are ones placed with no line items, where the array is empty.

What should be changed?

A Use explode_outer, which keeps rows whose array is empty or null

B Filter out empty arrays before exploding so the count reconciles

C Use a left join back to bronze after exploding

D Use posexplode, which returns the position as well as the value

◆ PAUSE AND DECIDE

What should be changed?

A Use explode_outer, which keeps rows whose array is empty or null

Correct. explode silently drops those rows; explode_outer keeps them with null in the exploded column.

B Filter out empty arrays before exploding so the count reconciles

That removes the rows deliberately, which is the current behaviour. The requirement is to keep them.

C Use a left join back to bronze after exploding

It would restore the rows and is far more work than changing one function name.

D Use posexplode, which returns the position as well as the value

posexplode adds position information. It drops empty arrays just as explode does.

THE SIGNAL

Explode changes the grain and silently drops empty arrays. That is a documented behaviour, not a bug.

> A team is designing bronze. To save storage they propose deduplicating, dropping malformed records and standardising timestamps as the data is written into bronze.
>
> A colleague objects.

What is the strongest objection?

A Deduplication is computationally expensive and would slow ingestion

B Bronze tables cannot have transformations applied to them

C Storage is cheap, so the saving is not worth pursuing

D Bronze should be append-only and hold data as it arrived, so that silver can be rebuilt when the cleaning logic is found to be wrong

◆ PAUSE AND DECIDE

What is the strongest objection?

A Deduplication is computationally expensive and would slow ingestion

It costs something, and cost is not the reason. The reason is losing the ability to reprocess.

B Bronze tables cannot have transformations applied to them

There is no such restriction. It is a design convention, and a well-motivated one.

C Storage is cheap, so the saving is not worth pursuing

True and beside the point. The objection is about reprocessing, not economics.

D Bronze should be append-only and hold data as it arrived, so that silver can be rebuilt when the cleaning logic is found to be wrong

Correct. Cleaning on the way in destroys the original, and the original is what makes reprocessing possible.

THE SIGNAL

Every cleaning decision is a decision that might be wrong. Bronze is what lets you make it again.

> A DataFrame with columns (txn_id, merchant_ref, terminal_ref) is written into an existing table whose columns are (txn_id, terminal_ref, merchant_ref). All three columns are strings.
>
> The write succeeds with no error. Two months later, fraud analysis is producing nonsense because merchant and terminal references are transposed.

What was used, and what should have been?

A insertInto was used; saveAsTable matches by name and would have aligned them

B append mode was used; overwrite would have corrected the ordering

C saveAsTable was used; insertInto matches by name

D The table schema must be altered to match the DataFrame's column order

◆ PAUSE AND DECIDE

What was used, and what should have been?

A insertInto was used; saveAsTable matches by name and would have aligned them

Correct. insertInto matches by position, and three string columns meant no type error surfaced.

B append mode was used; overwrite would have corrected the ordering

Write mode governs what happens to existing rows, not how columns are matched.

C saveAsTable was used; insertInto matches by name

The pair stated backwards. saveAsTable matches by name; insertInto matches by position.

D The table schema must be altered to match the DataFrame's column order

Reordering a table to suit one writer is fragile. Matching by name solves it properly.

THE SIGNAL

'All three are strings' is doing the work. A type mismatch would have failed the write immediately.

A team migrates a SQL pipeline to PySpark. The original combined four regional tables with UNION. The engineer translates it as df_a.union(df_b).union(df_c).union(df_d).

The PySpark output has more rows than the SQL version. A small number of customers trade in two regions and appear in two source tables.

What is the fix?

A Use unionByName(), which removes duplicate rows

B DataFrame union() does not deduplicate; append .distinct() to match SQL UNION

C Deduplicate each regional table before combining them

D Use unionAll(), which deduplicates correctly

◆ PAUSE AND DECIDE

What is the fix?

A Use unionByName(), which removes duplicate rows

unionByName changes how columns are matched, from position to name. It says nothing about duplicates.

B DataFrame union() does not deduplicate; append .distinct() to match SQL UNION

Correct. The docs state it performs the union with no automatic deduplication.

C Deduplicate each regional table before combining them

The duplicates are across sources, not within them. Each table is internally correct.

D Use unionAll(), which deduplicates correctly

unionAll is an alias for union. Identical behaviour, so this changes nothing at all.

THE SIGNAL

SQL UNION deduplicates. The DataFrame method does not, and neither does unionAll.

> A report should list every customer whose total spend across the year exceeds fifty thousand. An engineer filters where amount is greater than fifty thousand, then groups by customer, then sums.
>
> The report returns very few customers, and their totals are far below their real annual spend.

What is wrong?

A The comparison should be greater than or equal to

B sum ignores nulls, so some amounts were excluded

C The filter runs before the grouping, so only individual transactions over fifty thousand survive to be summed

D The grouping should have been a window function

◆ PAUSE AND DECIDE

What is wrong?

A The comparison should be greater than or equal to

A boundary difference would change the result by a handful of customers, not produce systematically low totals.

B sum ignores nulls, so some amounts were excluded

True of sum, and it would not produce this pattern, nor explain the missing customers.

C The filter runs before the grouping, so only individual transactions over fifty thousand survive to be summed

Correct. Filtering first changes which rows are aggregated, so both the customer list and the totals are wrong.

D The grouping should have been a window function

A window keeps every row, which is not what a customer-level report wants. The filter's position is the problem.

THE SIGNAL

Two symptoms together: too few customers AND wrong totals. Filtering after grouping would change only the first.

A team ingests events from a Kafka topic that retains messages for three days. They want a gold object holding a running total per product, covering the last eighteen months.

An engineer proposes a materialised view defined directly over the Kafka source, refreshed every fifteen minutes.

What is wrong with the proposal?

A Materialised views cannot compute running totals

B Materialised views cannot be refreshed more often than hourly

C Fifteen minutes is too slow for clickstream data; it needs continuous processing

D A materialised view may recompute from source, and Kafka retains only three days, so older history would be lost

◆ PAUSE AND DECIDE

What is wrong with the proposal?

A Materialised views cannot compute running totals

Aggregation is one of the main reasons to use one.

B Materialised views cannot be refreshed more often than hourly

Invented limit. The refresh schedule is configurable.

C Fifteen minutes is too slow for clickstream data; it needs continuous processing

Latency was never stated as a requirement. The problem is data loss, not freshness.

D A materialised view may recompute from source, and Kafka retains only three days, so older history would be lost

Correct. Databricks documents this: do not define a materialised view against a source where records should only be processed once.

THE SIGNAL

It would appear to work for three days, then silently lose everything older on the first full recompute.

A regulated silver table must never contain a row with a null policy_number. It is currently written by one Lakeflow pipeline that has an expectation with ON VIOLATION FAIL UPDATE.

The platform team is concerned that another team may later write to this table from a notebook, bypassing the pipeline.

What should they add?

A. A NOT NULL constraint on the table, which Delta enforces against every writer

B. A scheduled query that checks for nulls and alerts the team

C. A second expectation with ON VIOLATION DROP ROW as a fallback

D. Revoke MODIFY on the table from everyone except the pipeline's service principal

◆ PAUSE AND DECIDE

What should they add?

A A NOT NULL constraint on the table, which Delta enforces against every writer

Correct. A constraint is a property of the table, enforced at write time regardless of which tool performs the write.

B A scheduled query that checks for nulls and alerts the team

Detection after the fact. The requirement is that the row must never exist.

C A second expectation with ON VIOLATION DROP ROW as a fallback

Expectations only run inside the pipeline. Two of them protect the same single path.

D Revoke MODIFY on the table from everyone except the pipeline's service principal

Reasonable defence in depth, and it does not prevent the pipeline's own identity writing a null.

THE SIGNAL

'Regardless of the writer' points at the table, not at any pipeline filling it.

A platform team reviews the previous week's job runs by filtering the Runs tab to Failed. Nothing appears and they report a clean week.

An analyst separately reports that a reconciliation table has been missing two regions since Tuesday. The loading job shows Succeeded with failures on every run since then.

What is happening?

A The analyst is looking at a different job from the one the team reviewed

B The job status is incorrect and should be raised with support

C Succeeded with failures means some tasks failed but all leaf tasks succeeded; a Failed filter does not show it

D The runs are older than the 60-day retention window and have been removed

◆ PAUSE AND DECIDE

What is happening?

A The analyst is looking at a different job from the one the team reviewed

The scenario states the loading job's status directly.

B The job status is incorrect and should be raised with support

It is exactly correct. Job outcome comes from leaf tasks, and the leaf tasks succeeded.

C Succeeded with failures means some tasks failed but all leaf tasks succeeded; a Failed filter does not show it

Correct. It is a distinct outcome, and filtering by Failed hides every handled failure in the estate.

D The runs are older than the 60-day retention window and have been removed

They are from this week, and their status is visible, so they are plainly still in history.

THE SIGNAL

The filter created the blind spot, not the platform. Job status is determined by leaf tasks.

A job's fourth task appends the day's records to a fact table. One night it fails part way through, having written roughly 70% of the batch. An engineer uses Repair run, which succeeds.

The next morning the fact table holds about 170% of the expected rows for that date.

What happened?

A Repair re-runs each unsuccessful task from the beginning, and because the task appends, the partial output was duplicated

B Repair re-ran every task in the job, including the successful ones

C A retry policy fired concurrently with the repair, running the task twice

D Delta failed to roll back the partial write, which is a known defect

◆ PAUSE AND DECIDE

What happened?

A Repair re-runs each unsuccessful task from the beginning, and because the task appends, the partial output was duplicated

Correct. Lakeflow Jobs does not make tasks idempotent, so a partial append is added to twice.

B Repair re-ran every task in the job, including the successful ones

The opposite of what repair does. Successful tasks and their dependents are deliberately not re-run.

C A retry policy fired concurrently with the repair, running the task twice

Invents a concurrent execution. 170% matches one partial write plus one full write.

D Delta failed to roll back the partial write, which is a known defect

Delta writes are atomic per write. Batches that committed before the failure are correctly committed.

THE SIGNAL

70% written, then 100% written again, gives 170%. The arithmetic identifies the cause.

A partner delivers around 300 small files in a burst lasting about five minutes, three times a day. A file arrival trigger is configured on the volume with no advanced options set.

The job runs close to 300 times per delivery, mostly against partial data, and compute cost has become the team's largest line item.

What should be configured?

A Minimum time between triggers, set to a few minutes

B Switch to a scheduled trigger three times a day

C Wait after last change, set longer than the gap between files within a burst

D Maximum concurrent runs, set to 1

◆ PAUSE AND DECIDE

What should be configured?

A Minimum time between triggers, set to a few minutes

A cooldown caps how often runs start, which helps cost. Runs would still fire against partial data.

B Switch to a scheduled trigger three times a day

Abandons the data-driven approach and reintroduces the margin problem, assuming punctual deliveries.

C Wait after last change, set longer than the gap between files within a burst

Correct. Debouncing waits for quiet, and each new file resets the timer, so one run fires after the whole burst lands.

D Maximum concurrent runs, set to 1

Already the default. It prevents overlapping runs, not repeated ones, so 300 runs would simply queue.

THE SIGNAL

Two symptoms: too many runs, and partial data. A cooldown addresses the first only; debouncing addresses both.

A Lakeflow declarative pipeline runs in continuous mode, ingesting from a message bus. A team wants a notebook task to run after the pipeline finishes processing each batch.

They configure a job with a pipeline task pointing at that pipeline, then a notebook task depending on it.

What is the problem?

A. A notebook task cannot depend on a pipeline task

B. Continuous pipelines are not supported as job tasks; the pipeline must be triggered

C. The pipeline task must be the last task in the job

D. The notebook task needs Run if set to All done

◆ PAUSE AND DECIDE

What is the problem?

A. A notebook task cannot depend on a pipeline task

It can. Any task may depend on any other; that is what the DAG is for.

B. Continuous pipelines are not supported as job tasks; the pipeline must be triggered

Correct. A continuous pipeline is already running, so there is nothing to trigger and no completion to wait on.

C. The pipeline task must be the last task in the job

Invented ordering rule. Task order comes entirely from declared dependencies.

D. The notebook task needs Run if set to All done

Run if governs behaviour once dependencies finish. A continuous pipeline never finishes.

THE SIGNAL

A continuous pipeline never completes, so there is no completion event for a downstream task to wait on.

> An engineer hard-codes a storage account key into a notebook while debugging and pushes it to the team's repository. She notices an hour later.
>
> She removes the key, commits again with the message 'remove key', pushes, and reports the issue resolved.

Is it resolved?

A Yes; the current version of the file no longer contains the key

B Yes, provided the repository is private

C No; she should delete the notebook from the repository instead

D No; the key remains in the commit history and must be rotated. Rewriting history is a separate question.

◆ PAUSE AND DECIDE

Is it resolved?

A Yes; the current version of the file no longer contains the key

Git stores history. The previous commit still contains the key and is readable by anyone with access.

B Yes, provided the repository is private

Private limits who can read it and does not un-expose it. Everyone with access, now or later, can read that commit.

C No; she should delete the notebook from the repository instead

Deleting a file removes it from the current version and not from history. Same mistake, more disruption.

D No; the key remains in the commit history and must be rotated. Rewriting history is a separate question.

Correct. The credential is compromised the moment it is pushed, and rotation is the only action that invalidates it.

THE SIGNAL

Git stores history, not just current state. Rotate first; history rewriting is legitimate and secondary.

> Five engineers share a development workspace and each deploys the same bundle from their own branch to test changes. The target is configured with mode: production, chosen because it sounded more rigorous.
>
> They keep overwriting each other's jobs, and whoever deployed most recently owns every job in the workspace.

What should they change?

A Give each engineer their own workspace

B Have one engineer deploy on behalf of the others

C Add a variable for the job name and have each engineer override it

D Set mode: development on that target, which prefixes resource names with the deploying user

◆ PAUSE AND DECIDE

What should they change?

A Give each engineer their own workspace

Would work, at considerable cost and administration, when a feature exists that solves it without either.

B Have one engineer deploy on behalf of the others

Solves collisions by removing the ability to test independently, which is what a dev environment is for.

C Add a variable for the job name and have each engineer override it

Reimplements development mode by hand, and depends on every engineer setting it correctly every time.

D Set mode: development on that target, which prefixes resource names with the deploying user

Correct. Development mode prefixes resources not deployed as files or notebooks, so five engineers get five sets of jobs.

THE SIGNAL

Modes are not a quality dial. They select behaviours suited to a purpose, and prefixing is development mode's.

A bundle deploys correctly from an engineer's laptop with databricks bundle deploy -t dev. The same command in a CI job fails immediately, reporting that the CLI is not configured.

The CI secret store holds a valid service principal client id and client secret.

What is missing?

A The CI runner needs the Databricks CLI reinstalled

B The credentials are in the secret store but are not exposed as environment variables for the step

C The bundle needs a profile name added to the target definition

D The service principal has not been granted CAN MANAGE on the bundle

◆ PAUSE AND DECIDE

What is missing?

A The CI runner needs the Databricks CLI reinstalled

The CLI ran and produced a configuration error, so it is installed and executing.

B The credentials are in the secret store but are not exposed as environment variables for the step

Correct. A fresh CI container has no .databrickscfg, and environment variables take precedence for OAuth M2M.

C The bundle needs a profile name added to the target definition

A profile name refers to an entry in .databrickscfg, and the runner has no such file.

D The service principal has not been granted CAN MANAGE on the bundle

A permissions problem produces an authorisation error, not 'not configured'.

THE SIGNAL

'Works on my laptop' is the clue. The laptop has a .databrickscfg; a container created 30 seconds ago has nothing.

> A stage takes 38 minutes across 200 tasks. Summary Metrics show a median task duration of 10.2 minutes, a 75th percentile of 10.9 minutes, and a maximum of 11.4 minutes.
>
> The cluster has 24 cores. No spill is reported.

What is the bottleneck?

A Too many shuffle partitions, creating scheduling overhead

B Memory pressure, indicated by the long task durations

C Not skew; the distribution is even and 200 tasks on 24 cores means the cluster is under-resourced for the work

D Data skew, since the maximum exceeds the median

◆ PAUSE AND DECIDE

What is the bottleneck?

A Too many shuffle partitions, creating scheduling overhead

Excessive partitions produce many tiny tasks. These take over ten minutes each, the opposite symptom.

B Memory pressure, indicated by the long task durations

Memory pressure shows as spill or GC time, and the scenario states there is no spill.

C Not skew; the distribution is even and 200 tasks on 24 cores means the cluster is under-resourced for the work

Correct. Every task is slow, evenly, which is the signature of capacity rather than skew.

D Data skew, since the maximum exceeds the median

11.4 against 10.9 minutes is about 5% above the 75th percentile, far below the 50% threshold.

THE SIGNAL

A few slow tasks is skew. All tasks slow is not, and the fixes point in opposite directions.

> A stage reports shuffle spill of 52 GB in memory and 8 GB on disk. Task durations are fairly even, with the maximum about 15% above the 75th percentile.
>
> An engineer proposes moving to a node type with four times the memory.

What should be tried first?

A The four-times-memory node type, since spill means insufficient memory

B Increase spark.sql.shuffle.partitions so each partition is smaller and fits the memory available

C Nothing; 52 GB against 8 GB shows the metrics are inconsistent

D Enable AQE to split the skewed partitions

◆ PAUSE AND DECIDE

What should be tried first?

A The four-times-memory node type, since spill means insufficient memory

It would help, at four times the cost, permanently. Spill is as often partitions being too large.

B Increase spark.sql.shuffle.partitions so each partition is smaller and fits the memory available

Correct. Smaller partitions remove the spill without changing the cluster, and it is the cheaper fix.

C Nothing; 52 GB against 8 GB shows the metrics are inconsistent

They are consistent. Memory spill measures deserialised data; disk spill measures the serialised form.

D Enable AQE to split the skewed partitions

There is no skew. 15% above the 75th percentile is under the 50% threshold.

THE SIGNAL

Memory spill much larger than disk spill is normal. Deserialised data is bigger than serialised.

> An engineer adds a cluster-scoped init script that installs a package from an internal repository. On his two-node development cluster it works every time.
>
> On the sixty-node production cluster the launch fails with INIT_SCRIPT_FAILURE, but not always — roughly one launch in four succeeds.

What is the most likely explanation?

A Init scripts run on every node, so a transient failure reaching the repository on any one of sixty nodes terminates the cluster

B The production cluster has an incompatible runtime version

C The script should be converted to a global init script

D Init scripts are not supported on clusters above a certain size

◆ PAUSE AND DECIDE

What is the most likely explanation?

A Init scripts run on every node, so a transient failure reaching the repository on any one of sixty nodes terminates the cluster

Correct. A per-node success rate that is fine on two nodes becomes unreliable on sixty, because all sixty must succeed.

B The production cluster has an incompatible runtime version

A version incompatibility fails consistently, not one launch in four. Intermittency points at something probabilistic.

C The script should be converted to a global init script

Changes who manages it and applies it workspace-wide. It does not change the per-node execution that is failing.

D Init scripts are not supported on clusters above a certain size

Invented limit. Init scripts run on clusters of any size.

THE SIGNAL

Two nodes and sixty nodes are not the same test. Thirty times the nodes is thirty times the chances of one failing.

A platform team owns every catalog and has become a bottleneck for access requests. They want team leads to grant access to their own team's tables, without being able to drop those tables or transfer ownership.

An engineer proposes granting ALL PRIVILEGES on the relevant schemas.

What should they grant instead?

A MANAGE on the relevant schemas

B Metastore admin, so they can administer their own data

C ALL PRIVILEGES as proposed

D Ownership of each schema, transferred to the relevant team lead

◆ PAUSE AND DECIDE

What should they grant instead?

A MANAGE on the relevant schemas

Correct. MANAGE confers exactly the ability to grant and revoke privileges, without ownership's other capabilities.

B Metastore admin, so they can administer their own data

Grants administrative reach over the entire metastore to solve a per-schema problem.

C ALL PRIVILEGES as proposed

ALL PRIVILEGES deliberately excludes MANAGE, so it would not let them grant to others — the one thing they need.

D Ownership of each schema, transferred to the relevant team lead

Owners can also drop the object and transfer ownership onward, which the requirement excludes.

THE SIGNAL

ALL PRIVILEGES sounds like more than MANAGE and specifically excludes it, to prevent privilege escalation.

> A sales table is queried by fourteen regional teams. Each team must see only rows for its own region, and a global team must see every row. Every team must be able to query the same table by name.
>
> Row counts per team will naturally differ, and that is expected.

What should be applied?

A. A row filter on the table, using a function that returns a boolean based on group membership

B. Revoke SELECT and grant it per region on separate copies of the table

C. Fourteen dynamic views, one per region

D. A column mask on the region column

◆ PAUSE AND DECIDE

What should be applied?

A A row filter on the table, using a function that returns a boolean based on group membership

Correct. A row filter returns a boolean and removes rows the user should not see, evaluated per row at query time.

B Revoke SELECT and grant it per region on separate copies of the table

Fourteen copies to keep in sync, which is what query-time filtering exists to avoid.

C Fourteen dynamic views, one per region

Would work, and teams must then query different objects, which the requirement forbids.

D A column mask on the region column

A mask transforms a value and keeps every row. It would hide the region name while still returning all rows.

THE SIGNAL

Hide rows: row filter, returns boolean. Hide a value but keep the row: column mask, returns the column's type.

> A team is migrating from table-level column masks to ABAC policies. On one table they add an ABAC column mask policy while the existing ALTER TABLE mask is still in place. The two use different functions, though both redact to the same fixed string.
>
> Queries against that table now fail with an error rather than returning masked data.

What is happening?

A. ABAC requires the table-level mask to be dropped before the policy is created

B. The two masks are applied in sequence, double-masking the value

C. When both mechanisms apply different functions, access is blocked — even if the output would be identical

D. ABAC policies cannot be applied to tables that already have masks, so the policy was never created

◆ PAUSE AND DECIDE

What is happening?

A ABAC requires the table-level mask to be dropped before the policy is created

Creation succeeds either way. The conflict surfaces at query time, not at creation.

B The two masks are applied in sequence, double-masking the value

The system does not chain them. It refuses rather than composing two protections it cannot reconcile.

C When both mechanisms apply different functions, access is blocked — even if the output would be identical

Correct. The same function on both sides is allowed; different functions are refused, and the system does not compare outputs.

D ABAC policies cannot be applied to tables that already have masks, so the policy was never created

Both can coexist. The same function on both sides is explicitly allowed, so creation is not what failed.

THE SIGNAL

'Both redact to the same fixed string' is placed to tempt you. The system compares functions, not outputs.

What this paper covered

DOMAIN	QUESTIONS
Domain 1 — Platform	1
Domain 2 — Ingest	6
Domain 3 — Transform	5
Domain 4 — Jobs	4
Domain 5 — CI/CD	3
Domain 6 — Troubleshoot	3
Domain 7 — Govern	3
Total	25

ON THE EXAM

Weighted across all three papers rather than within each one, so a single paper will not match the exam's weights exactly.

MODULE 36 · EXAM STRATEGY - ALL DOMAINS

Practice exam 3

The final 25 questions. No topic repeated from papers 1 or 2.

QUESTIONS 25 FORMAT 4 options, A–D MODULE 36 of 36

admin@sthithapragna.com

How to take this paper

- **Each question is read aloud, then the slide holds** while you decide.
- **Pause if you need longer.** The hold is a prompt, not a limit.
- **Answer every question**, even when unsure. There is no penalty for a wrong answer.
- **The worked answer follows immediately**, with a reason for each wrong option.

> ON THE EXAM
>
> The real exam is 45 questions in 90 minutes — two minutes each. Try to keep near that pace.

A BI team's dashboards are queried in short bursts through the morning, with long idle gaps between them. They currently use a classic SQL warehouse, which is stopped overnight to save cost.

Analysts complain that the first query each morning takes several minutes before returning anything.

What should the team change?

A Increase the classic warehouse's cluster size

B Move to a serverless SQL warehouse, which starts in seconds rather than minutes

C Enable Photon on the classic warehouse

D Leave the classic warehouse running overnight so it is always warm

◆ PAUSE AND DECIDE

What should the team change?

A Increase the classic warehouse's cluster size

A bigger warehouse takes just as long to start. Size governs query throughput, not startup.

B Move to a serverless SQL warehouse, which starts in seconds rather than minutes

Correct. Classic and pro warehouses cold-start in around four minutes; serverless starts in seconds.

C Enable Photon on the classic warehouse

Photon accelerates query execution. It does nothing to the cold-start delay before the warehouse exists.

D Leave the classic warehouse running overnight so it is always warm

Removes the delay by paying for idle compute all night, which is the cost they were avoiding.

THE SIGNAL

'The first query each morning' is a startup symptom, not a query performance one.

A COPY INTO statement is written with both a FILES clause listing 40 specific files and a PATTERN clause matching a glob. The intention is to load only files matching the pattern from within that list.

The statement is rejected.

Why?

A FILES is limited to 40 entries, and the limit was exceeded

B PATTERN may only be used with Parquet sources

C VALIDATE must be run before a statement using PATTERN

D FILES and PATTERN are mutually exclusive; only one may be specified

◆ PAUSE AND DECIDE

Why?

A FILES is limited to 40 entries, and the limit was exceeded

The FILES limit is 1000, and 40 is well inside it.

B PATTERN may only be used with Parquet sources

PATTERN works across the supported formats. It is not format-restricted.

C VALIDATE must be run before a statement using PATTERN

VALIDATE is an optional dry run. It is not a prerequisite.

D FILES and PATTERN are mutually exclusive; only one may be specified

Correct. They are alternative ways of selecting files, not filters that combine.

THE SIGNAL

Two ways to select files, not two filters to intersect. FILES caps at 1000.

> An Auto Loader stream reads JSON with schema inference enabled. Downstream, a comparison of order_total against a numeric threshold returns no rows, even though large orders plainly exist in the source.
>
> Inspecting the silver table shows order_total holds values such as 1499.99 as expected.

What is the most likely cause?

A JSON cannot represent decimal values accurately, so comparisons fail

B Auto Loader infers JSON and CSV columns as STRING by default; the comparison is comparing strings

C The threshold must be cast to STRING to match the column

D The values were rescued into _rescued_data and are not really in the column

◆ PAUSE AND DECIDE

What is the most likely cause?

A JSON cannot represent decimal values accurately, so comparisons fail

JSON numbers parse fine. The issue is what type Auto Loader assigned on inference.

B Auto Loader infers JSON and CSV columns as STRING by default; the comparison is comparing strings

Correct. cloudFiles.inferColumnTypes is false by default. Use schema hints, or enable inference of types.

C The threshold must be cast to STRING to match the column

That would make the comparison lexicographic and still wrong. Fix the column's type instead.

D The values were rescued into _rescued_data and are not really in the column

The scenario states the column holds the expected values, so they were not rescued.

THE SIGNAL

The values look right because they are right — as strings. String comparison orders '9' above '1499.99'.

A team is automating the creation of Lakeflow Connect ingestion pipelines through their CI system. Connections for their database sources are created successfully by the pipeline.

The connection for a SaaS source that authenticates with browser-based OAuth cannot be created the same way.

Why?

A Connections cannot be created programmatically at all; only through Catalog Explorer

B The CI principal needs CREATE CONNECTION on the metastore

C OAuth U2M connectors require an interactive browser sign-in, so they cannot be created programmatically

D SaaS connectors must be created by a metastore admin, and the CI principal is not one

◆ PAUSE AND DECIDE

Why?

A Connections cannot be created programmatically at all; only through Catalog Explorer

They can, using the CLI or the API. The exception is browser-based OAuth.

B The CI principal needs CREATE CONNECTION on the metastore

A sensible prerequisite, and it would not enable an interactive sign-in either.

C OAuth U2M connectors require an interactive browser sign-in, so they cannot be created programmatically

Correct. Somebody must complete the sign-in, which no CI system can do.

D SaaS connectors must be created by a metastore admin, and the CI principal is not one

Admin rights would not supply an interactive browser session, which is what is missing.

THE SIGNAL

The same constraint as CI authentication: nobody is present to complete a browser flow.

A bronze table holds a payload column containing JSON strings of varying shape. An analyst needs to read a single field from within the payload for an exploratory query, and does not want to define a schema.

The payload structure changes occasionally, and defining a schema would need maintaining.

What should they use?

A from_json with a schema derived by schema_of_json

B Cast the payload column to STRUCT

C explode on the payload column

D The colon operator, which extracts a field from a JSON string without requiring a schema

◆ PAUSE AND DECIDE

What should they use?

A from_json with a schema derived by schema_of_json

Works, and it fixes a schema that then needs maintaining, which the requirement rules out.

B Cast the payload column to STRUCT

Casting to STRUCT requires stating the struct's fields, which is defining a schema.

C explode on the payload column

explode operates on arrays. The column holds a JSON string.

D The colon operator, which extracts a field from a JSON string without requiring a schema

Correct. It is the lightest option for exploratory access and tolerates shape changes.

THE SIGNAL

'Does not want to define a schema' rules out from_json and STRUCT alike. VARIANT is the durable answer at scale.

> A team must ingest from a niche vertical SaaS product used only in their industry. There is no managed connector for it in Lakeflow Connect.
>
> An engineer proposes writing and maintaining a custom Structured Streaming source against the vendor's API.

What should they check first?

A Whether a partner connector exists, since partners cover sources Databricks has no managed connector for

B Whether Lakehouse Federation can query the SaaS product in place

C Nothing; a custom source is the only option when no managed connector exists

D Whether the vendor's API supports Auto Loader

◆ PAUSE AND DECIDE

What should they check first?

A Whether a partner connector exists, since partners cover sources Databricks has no managed connector for

Correct. Checking Partner Connect costs minutes and can remove the whole maintenance burden.

B Whether Lakehouse Federation can query the SaaS product in place

Federation covers supported database sources. It is also not an ingestion mechanism.

C Nothing; a custom source is the only option when no managed connector exists

Skips the layer between managed connectors and building your own, which is exactly where partners sit.

D Whether the vendor's API supports Auto Loader

Auto Loader reads files in cloud object storage. An API is the wrong source type for it.

THE SIGNAL

The layer people forget. Between a managed connector and building it yourself sits the partner ecosystem.

A satisfaction_score column is populated only when a customer chooses to leave a rating. About half the rows are null. A dashboard has reported an average of 4.1 for two years.

An engineer adds fillna(0) across all numeric columns while tidying the pipeline. The dashboard now shows 2.0.

What happened?

A fillna overwrote real scores with zero

B The dashboard is caching an old value and needs refreshing

C fillna changed the column type, causing an incorrect calculation

D Averages skip nulls, so those rows were previously excluded; filling them with 0 made them count as scores of zero

◆ PAUSE AND DECIDE

What happened?

A fillna overwrote real scores with zero

fillna only replaces nulls. Existing scores are untouched.

B The dashboard is caching an old value and needs refreshing

A cache would show the old number, not a new and lower one.

C fillna changed the column type, causing an incorrect calculation

Filling an integer column with 0 does not change its type, and a type change would not give a plausible but wrong mean.

D Averages skip nulls, so those rows were previously excluded; filling them with 0 made them count as scores of zero

Correct. A null means 'did not rate'. A zero means 'rated it zero'. Half the data changed meaning.

THE SIGNAL

Filling a null always changes a meaning. Here it changed the meaning of half the data.

A customer table has 52 columns. Fifteen are genuinely optional: a second address line, a middle name, a mobile number and similar. Two are essential: customer_id and email.

A cleaning step calls df.dropna() with no arguments. The silver table ends up with about 4% of the expected rows.

What should the call be?

A df.dropna(subset=['customer_id', 'email'])

B df.dropna(thresh=52)

C df.dropna(how='all')

D df.fillna('') followed by df.dropna()

◆ PAUSE AND DECIDE

What should the call be?

A df.dropna(subset=['customer_id', 'email'])

Correct. It drops rows missing something essential and ignores nulls in the fifteen optional columns.

B df.dropna(thresh=52)

Requires all 52 columns to be non-null, which is stricter than the default and keeps even fewer rows.

C df.dropna(how='all')

Drops only rows where all 52 columns are null, so records missing a customer_id survive.

D df.fillna('') followed by df.dropna()

Keeps every row by turning optional nulls into empty strings, and disguises missing customer_ids as empty strings.

THE SIGNAL

The default is how='any' across all 52 columns. With fifteen optional fields, almost every real customer has a null.

> A gold table must be fully replaced each night from a silver source whose column set occasionally changes. The team wants the gold table's schema to follow the source automatically.
>
> An engineer currently uses INSERT OVERWRITE and finds that new source columns never appear in gold.

What should be used instead?

A TRUNCATE followed by INSERT INTO

B CREATE OR REPLACE TABLE, which is DDL and replaces the schema as well as the data

C DROP TABLE followed by CREATE TABLE AS SELECT

D INSERT OVERWRITE with mergeSchema enabled on the write

◆ PAUSE AND DECIDE

What should be used instead?

A TRUNCATE followed by INSERT INTO

Truncate removes rows and leaves the schema, which is the same limitation as INSERT OVERWRITE.

B CREATE OR REPLACE TABLE, which is DDL and replaces the schema as well as the data

Correct. INSERT OVERWRITE is DML and preserves the existing schema; CREATE OR REPLACE TABLE redefines it.

C DROP TABLE followed by CREATE TABLE AS SELECT

Achieves the same result and discards grants, table history and any downstream references in between.

D INSERT OVERWRITE with mergeSchema enabled on the write

Closer, and the requirement is for the schema to follow the source exactly, including removals, which replacement gives.

THE SIGNAL

DDL replaces the schema. DML keeps it. That single distinction answers the question.

A dashboard shows daily unique devices across 600 million events. It uses count_distinct on device_id grouped by day, takes fourteen minutes, and times out.

The dashboard exists to spot week-on-week trends. A separate regulatory return also counts unique devices and must be exact.

What should the team do?

A Keep count_distinct for both and increase the warehouse size

B Use approx_count_distinct for both, since it is far faster

C Use approx_count_distinct for the dashboard and keep count_distinct for the regulatory return

D Use approx_count_distinct with rsd set to 0.001 for both

◆ PAUSE AND DECIDE

What should the team do?

A Keep count_distinct for both and increase the warehouse size

Buys time at permanent cost, and the exact count still scales badly as events grow.

B Use approx_count_distinct for both, since it is far faster

Makes the regulatory figure an estimate. 'Must be exact' is a stated requirement.

C Use approx_count_distinct for the dashboard and keep count_distinct for the regulatory return

Correct. Two consumers with different requirements, so two different answers.

D Use approx_count_distinct with rsd set to 0.001 for both

Below rsd 0.01 it is more efficient to use count_distinct, so this pays for precision and loses the speed.

THE SIGNAL

Nothing says one query must serve both consumers. The exam rewards noticing when it need not.

> A nightly join between a 1.2 TB fact table and a currency_rates lookup has completed in twelve minutes for over a year. In the last fortnight it has slowed to nearly ninety minutes.
>
> No code has changed. The Spark UI now shows a large shuffle on both sides where previously there was none.

What is the most likely explanation?

A Nulls appeared in the join key, forcing a different strategy

B The fact table has grown, so the shuffle takes longer

C Adaptive query execution was disabled by a platform change

D The lookup grew past the broadcast threshold, so Spark switched to a sort-merge join

◆ PAUSE AND DECIDE

What is the most likely explanation?

A Nulls appeared in the join key, forcing a different strategy

Nulls affect which rows match, not which strategy is chosen. They would change the row count.

B The fact table has grown, so the shuffle takes longer

Gradual growth does not create a shuffle that was not there before. The shuffle is new.

C Adaptive query execution was disabled by a platform change

Possible, and it would not by itself stop a broadcast if the lookup were still small enough at plan time.

D The lookup grew past the broadcast threshold, so Spark switched to a sort-merge join

Correct. Below 10 MB it was broadcast with no shuffle. Once past it, broadcasting stops and both sides shuffle.

THE SIGNAL

'A shuffle where previously there was none' is a strategy change, not a volume change.

> A job's first task loads new records and must tell later tasks how many arrived. An expensive rebuild can then be skipped when nothing new came in.
>
> An engineer proposes writing the count into a small Delta table that the downstream task reads.

What is the more idiomatic approach?

A. Set a task value with dbutils.jobs.taskValues, and branch on it with an If/else task

B. A job parameter, set by the first task at runtime

C. The Delta table approach, since it is the only way to share state between tasks

D. Combine both tasks into one notebook so the count stays in memory

◆ PAUSE AND DECIDE

What is the more idiomatic approach?

A Set a task value with dbutils.jobs.taskValues, and branch on it with an If/else task

Correct. Task values exist to return a result into the job environment, and If/else conditions test them directly.

B A job parameter, set by the first task at runtime

Job parameters are supplied when the run starts, not written by a task mid-run.

C The Delta table approach, since it is the only way to share state between tasks

It works, and 'only way' is wrong. It also leaves a table whose sole purpose is carrying one number.

D Combine both tasks into one notebook so the count stays in memory

Removes the ability to skip the expensive work as a separate task, which was the requirement.

THE SIGNAL

Tables pass data. Task values pass a decision. A single row count is a decision.

A production job has run nightly for two years. It was created by an engineer who left the company last month, and her account was deprovisioned yesterday.

The job now fails immediately with a permissions error on its source tables. The on-call engineer can query those tables without difficulty.

What is the cause and the fix?

A Re-grant SELECT on the source tables to the departed engineer's account

B Grant the on-call engineer CAN MANAGE on the job

C The job runs as the departed engineer; change the run-as identity to a service principal

D The tables were moved when the account was deprovisioned

◆ PAUSE AND DECIDE

What is the cause and the fix?

A Re-grant SELECT on the source tables to the departed engineer's account

Reviving a deprovisioned account to keep production running is the opposite of correct governance.

B Grant the on-call engineer CAN MANAGE on the job

Job privileges are a separate system from run-as. His access changes who can edit the job, not what it can read.

C The job runs as the departed engineer; change the run-as identity to a service principal

Correct. The run-as identity determines data access, and deprovisioning removed the job's ability to read.

D The tables were moved when the account was deprovisioned

Deprovisioning an account does not move tables. It invents a cause when the timing already points at the account.

THE SIGNAL

The on-call engineer's own access is a deliberate red herring. His permissions are unrelated to the job's.

A job's first task is a notebook that reads a job parameter called batch_date and works correctly. Its second task is a Python script task that needs the same value.

The script receives nothing and fails on a missing argument. The job parameter is definitely set, as the notebook task proves.

Why, and what is needed?

A. A Python script task receives an ordered JSON array, so it must reference the job parameter explicitly with a dynamic value reference

B. Job parameters apply only to the first task in a job

C. The parameter must be redefined as a task parameter on every task that needs it

D. The script must call dbutils.widgets.get, as the notebook does

◆ PAUSE AND DECIDE

Why, and what is needed?

A A Python script task receives an ordered JSON array, so it must reference the job parameter explicitly with a dynamic value reference

Correct. Job parameters are pushed down to key-value tasks. Array tasks must ask for the value by name.

B Job parameters apply only to the first task in a job

Invented rule. They are pushed to every task that uses key-value parameters, regardless of position.

C The parameter must be redefined as a task parameter on every task that needs it

Would work, and it duplicates the value everywhere, defeating the point of defining it once.

D The script must call dbutils.widgets.get, as the notebook does

Widgets are the notebook interface for reading a delivered value. They do not deliver anything to a script.

THE SIGNAL

The difference is the task type, not the parameter. Key-value tasks get the push-down; array tasks must ask.

A job is scheduled every twenty minutes. Reviewing the Runs tab, an engineer sees that around a third of runs show a status of Skipped, coloured pink. The runs that do execute succeed normally.

Recent successful runs have been taking between 24 and 31 minutes.

What is happening?

A The job is failing intermittently and failures are reported as Skipped

B The schedule's time zone is misconfigured

C Runs are being skipped because a previous run is still in flight and maximum concurrent runs is 1

D The cluster fails to start on those runs

◆ PAUSE AND DECIDE

What is happening?

A The job is failing intermittently and failures are reported as Skipped

Skipped means the run never executed. A failure would be red and would carry an error.

B The schedule's time zone is misconfigured

A time zone offset shifts when runs happen. It does not stop a third of them happening at all.

C Runs are being skipped because a previous run is still in flight and maximum concurrent runs is 1

Correct. Runs take longer than the interval, so triggers land mid-run and the default of 1 causes them to be skipped.

D The cluster fails to start on those runs

That produces a cluster error and a failed run, not a skipped one. The run would have started.

THE SIGNAL

The arithmetic gives it away: a job taking 24–31 minutes cannot run every 20. It has outgrown its schedule.

A team's pipeline deploys to production whenever main is updated. There is no validate step. A pull request renames a variable in databricks.yml but misses one reference elsewhere in the file.

The change is reviewed, approved and merged. The production deployment fails part way, leaving some resources updated and others not.

What would have prevented this?

A A more thorough code review

B A databricks bundle validate step on the pull request, failing the check before merge

C Deploying to production with --force

D Running databricks bundle destroy before each deployment

◆ PAUSE AND DECIDE

What would have prevented this?

A. A more thorough code review

A dangling reference in a long YAML file is what humans skim past and machines catch instantly. Asking people to be careful is not a control.

B. A databricks bundle validate step on the pull request, failing the check before merge

Correct. Validate resolves the configuration without touching a workspace, so an unresolved reference fails the check.

C. Deploying to production with --force

Overrides validations, making the pipeline more willing to deploy broken configuration.

D. Running databricks bundle destroy before each deployment

Tears down production before deploying to it, converting a partial failure into a total outage.

THE SIGNAL

The exam objective names validate before deploy and manage. This is what that ordering is for.

> A platform team keeps all code in one repository covering fourteen projects and several thousand files. Engineers report that creating a Git folder takes a very long time, and some Git operations fail outright.
>
> Each engineer works on only one or two of the fourteen projects.

What should they do?

A Add the unused projects to .gitignore

B Have all engineers share one Git folder to avoid cloning it repeatedly

C Increase the Git folder size limit in workspace settings

D Use sparse checkout with cone patterns so each engineer clones only the projects they work on

◆ PAUSE AND DECIDE

What should they do?

A Add the unused projects to .gitignore

Prevents new files being tracked. It does not remove committed content or reduce what is cloned.

B Have all engineers share one Git folder to avoid cloning it repeatedly

Solves the cloning cost by creating a worse problem: one working copy, so a colleague switching branch switches it for everybody.

C Increase the Git folder size limit in workspace settings

The limits are platform limits, not a workspace setting you can raise.

D Use sparse checkout with cone patterns so each engineer clones only the projects they work on

Correct. It is the documented answer for monorepos. Splitting the repository is the other legitimate option.

THE SIGNAL

Databricks recommends against monorepo-backed Git folders. Sparse checkout is the mitigation; splitting is the cure.

A platform team enables predictive optimisation across the account. A month later they audit results. Most tables are being optimised and vacuumed automatically.

Three groups receive nothing: tables in a partner-shared external location, their Databricks SQL materialized views, and the streaming tables in their ingestion pipelines.

Why?

A Those tables are too small to benefit, so the heuristics skipped them

B The account rollout has not yet reached those tables

C Predictive optimisation must also be enabled per table

D It runs only on Unity Catalog managed tables, and does not run on streaming tables or materialized views

◆ PAUSE AND DECIDE

Why?

A Those tables are too small to benefit, so the heuristics skipped them

The heuristics do skip tables, and that would not correlate so neatly with three distinct object types.

B The account rollout has not yet reached those tables

The rollout is per account, not per table. Everything else in the account is working.

C Predictive optimisation must also be enabled per table

Does not explain why exactly these three categories were affected while everything else worked.

D It runs only on Unity Catalog managed tables, and does not run on streaming tables or materialized views

Correct. Two documented exclusions cover all three groups.

THE SIGNAL

When affected objects share a type rather than a size, look for a categorical rule rather than a heuristic.

> For years, a finance team has queried a regulatory table as it stood on any day in the previous quarter, using Delta time travel. Nobody has ever run VACUUM on it.
>
> Predictive optimisation is enabled on the account. Six weeks later an auditor requests the table as at 40 days ago, and the query fails.

What happened, and what should have been done?

A OPTIMIZE rewrote the files, which invalidates all previous versions

B VACUUM ran at the default 7-day retention; delta.deletedFileRetentionDuration should have been set before enabling

C Time travel is not supported once predictive optimisation is enabled

D The table should have been converted to Iceberg to retain long-range time travel

◆ PAUSE AND DECIDE

What happened, and what should have been done?

A OPTIMIZE rewrote the files, which invalidates all previous versions

OPTIMIZE rewrites files and the old ones stay referenced by earlier versions until VACUUM removes them.

B VACUUM ran at the default 7-day retention; delta.deletedFileRetentionDuration should have been set before enabling

Correct. Databricks flags this as a warning, because vacuumed files are gone and no setting brings them back.

C Time travel is not supported once predictive optimisation is enabled

Time travel still works. What changed is how far back the files exist.

D The table should have been converted to Iceberg to retain long-range time travel

PO covers managed Delta and Iceberg alike, with the same VACUUM retention behaviour.

THE SIGNAL

Nobody ever ran VACUUM, so long time travel worked by accident. Enabling PO removed the accident.

> A nightly job has begun missing its 07:00 deadline about twice a week. Nobody can say when it started. It has never failed, so no notification has ever fired.
>
> An engineer opens the job's Runs tab and looks at the matrix view for the last month.

What will the matrix view show her most directly?

A Nothing useful; she needs the system tables instead

B The cost of each run, so she can find the expensive one

C Whether the runtime has been increasing, because bar height indicates duration

D Which task is failing, indicated in red

◆ PAUSE AND DECIDE

What will the matrix view show her most directly?

A Nothing useful; she needs the system tables instead

System tables suit a longer window or many jobs. For one job over one month the matrix answers it immediately.

B The cost of each run, so she can find the expensive one

The matrix shows duration and status, not cost. Cost attribution comes from the billing system tables.

C Whether the runtime has been increasing, because bar height indicates duration

Correct. A rising staircase across the columns is a visible trend, per task and for the run overall.

D Which task is failing, indicated in red

Nothing is failing. The scenario says so, which is why no notification fired.

THE SIGNAL

'It has never failed' rules out looking for red. The information is in the height of the bars, not their colour.

A team has thirty external Delta tables they want to convert to Unity Catalog managed tables, to gain predictive optimisation. They must keep table history, keep existing grants and views working, and minimise downtime for streaming writers.

An engineer proposes CREATE TABLE AS SELECT into new managed tables, then renaming.

What should they use instead?

A The CTAS approach as proposed, which is the documented method

B ALTER TABLE … SET MANAGED, which retains history, name, settings, permissions and views

C DEEP CLONE into a managed table, then drop the originals

D Move the files into the managed storage location manually

◆ PAUSE AND DECIDE

What should they use instead?

A The CTAS approach as proposed, which is the documented method

It works, and it loses table history, creates a new object needing grants reapplied, and requires downtime.

B ALTER TABLE ... SET MANAGED, which retains history, name, settings, permissions and views

Correct. Databricks recommends it over CTAS precisely for these reasons, and it minimises downtime.

C DEEP CLONE into a managed table, then drop the originals

A real option Databricks names, and SET MANAGED minimises or eliminates downtime compared with it.

D Move the files into the managed storage location manually

Unity Catalog manages the storage layout of managed tables. Hand-moving files leaves metastore and storage disagreeing.

THE SIGNAL

The team's three requirements match the three documented benefits of SET MANAGED one for one.

> Three weeks after converting a large external table with SET MANAGED, a data science team reports that a Python script has broken. It reads the table by its old storage path rather than by name.
>
> The cluster runs Databricks Runtime 16.4 LTS. Predictive optimisation is enabled.

What happened?

A With predictive optimisation on, the old external data is deleted 14 days after conversion; the script needs name-based access

B The conversion failed silently and should be re-run

C Path-based reads always fail against managed tables from the moment of conversion

D The table was rolled back with UNSET MANAGED by someone else

◆ PAUSE AND DECIDE

What happened?

A With predictive optimisation on, the old external data is deleted 14 days after conversion; the script needs name-based access

Correct. Three weeks is past 14 days, and path-based redirect requires DBR 18.1 or above.

B The conversion failed silently and should be re-run

It plainly succeeded — the table worked by name for three weeks. Only the old path stopped working.

C Path-based reads always fail against managed tables from the moment of conversion

They worked for roughly two weeks after conversion, which is why the failure was delayed.

D The table was rolled back with UNSET MANAGED by someone else

A rollback deletes the managed data after 7 days, a different symptom on a different timer.

THE SIGNAL

The delay is the clue. Nothing broke at conversion and something broke three weeks later, which points at a timer.

A team must share a curated dataset with an external analyst group. It joins four tables, renames columns, and exposes only aggregated figures. The analysts must not be able to reach any of the four base tables.

A colleague suggests applying row filters and column masks to all four tables instead.

Which approach fits?

A. A dynamic view over the four tables, with SELECT granted on the view only

B. Grant the analysts BROWSE on the four tables

C. Row filters and column masks on the four base tables, then grant SELECT on them

D. Copy the joined result into a new table on a schedule

◆ PAUSE AND DECIDE

Which approach fits?

A A dynamic view over the four tables, with SELECT granted on the view only

Correct. Views span multiple tables, reshape and aggregate, and users need no access to the base tables.

B Grant the analysts BROWSE on the four tables

BROWSE shows metadata without data. The analysts need the data, in a specific shape.

C Row filters and column masks on the four base tables, then grant SELECT on them

Requires granting SELECT on all four, which the requirement forbids. It also cannot join or aggregate.

D Copy the joined result into a new table on a schedule

Works, and creates a second copy to maintain and secure, plus staleness between refreshes.

THE SIGNAL

'Must not reach the base tables' plus 'joins four tables' is the dynamic view signature.

> A catalog holds sixty schemas. A group has been granted SELECT at catalog level, which they need for fifty-nine of them. One schema holds compensation data this group must not see.
>
> An engineer proposes DENY SELECT on that one schema so the catalog-level grant can stay.

What should they do instead?

A Unity Catalog has no DENY; revoke the catalog grant and grant on the 59 schemas instead

B Grant ALL PRIVILEGES on the catalog and rely on ownership to exclude the schema

C Move the compensation schema into the hive_metastore catalog so DENY can be used

D Use DENY as proposed, which is the standard way to carve out an exception

◆ PAUSE AND DECIDE

What should they do instead?

A Unity Catalog has no DENY; revoke the catalog grant and grant on the 59 schemas instead

Correct. Inheritance flows downward with no way to interrupt it, so the grant must start below the exclusion.

B Grant ALL PRIVILEGES on the catalog and rely on ownership to exclude the schema

Makes the exposure broader, not narrower. Ownership does not exclude anyone from an inherited grant.

C Move the compensation schema into the hive_metastore catalog so DENY can be used

Moves the most sensitive data into a legacy model Databricks recommends migrating away from.

D Use DENY as proposed, which is the standard way to carve out an exception

Standard in many database systems, and Unity Catalog does not support it. The proposal would simply fail.

THE SIGNAL

Unity Catalog is least privilege: ungranted is implicitly denied, so there is nothing to deny.

A governance team is designing a governed tag vocabulary. To make classification searchable, an engineer proposes tags whose values carry an example of the data, such as a sample national identifier from each column.

A colleague objects.

What is the objection?

A ABAC policy conditions cannot read tag values, only tag keys

B Tags cannot be applied to columns, only to tables and schemas

C Tag data can be replicated globally, so tag names and values must never contain personal or sensitive information

D Governed tags support keys only, not values

◆ PAUSE AND DECIDE

What is the objection?

A ABAC policy conditions cannot read tag values, only tag keys

has_tag_value reads values. It is one of the two functions supported in policy conditions.

B Tags cannot be applied to columns, only to tables and schemas

Tags apply to catalogs, schemas, tables, columns, models and volumes.

C Tag data can be replicated globally, so tag names and values must never contain personal or sensitive information

Correct. Databricks flags this explicitly. A tag describes the data; it must not contain any of it.

D Governed tags support keys only, not values

They are key-value pairs. Values are supported and widely used, such as pii:ssn.

THE SIGNAL

A tag describes the data. It is not the data. pii:ssn is correct; a tag holding an actual identifier is not.

What this paper covered

DOMAIN	QUESTIONS
Domain 1 — Platform	1
Domain 2 — Ingest	5
Domain 3 — Transform	5
Domain 4 — Jobs	4
Domain 5 — CI/CD	2
Domain 6 — Troubleshoot	3
Domain 7 — Govern	5
Total	25

ON THE EXAM

Weighted across all three papers rather than within each one, so a single paper will not match the exam's weights exactly.

Thirty-six modules, seven domains and seventy-five practice questions. **You have covered every objective on the current exam guide.**